TABLE OF CONTENTS

Table of Contents i

Chapter 1 为什么「预测下一个词」能产生智能? 1
1.1 一个看似简单的任务 1
1.2 压缩即理解 2
1.2.1 信息熵与交叉熵损失 2
1.2.2 困惑度：压缩效率的直觉度量 2
1.2.3 Sutskever 的压缩哲学 3
1.3 苦涩的教训 3
1.3.1 历史佐证 3
1.3.2 反面论证：人类知识的价值 4
1.3.3 LLM 作为 Bitter Lesson 的极致体现 4
1.4 涌现：大就是不同 5
1.4.1 经典涌现案例 5
1.4.2 涌现是真实的还是度量假象? 5
1.5 从 GPT 到 DeepSeek：一部效率进化史 6
1.5.1 前传：词向量与上下文表示（2013–2018） 6
1.5.2 架构探索期（2017–2019） 6
1.5.3 规模竞赛期（2020–2022） 7
1.5.4 产品化爆发期（2023–2024） 7
1.5.5 效率革命期（2025–2026） 8
1.5.6 Scaling Hypothesis 之争 9
1.6 2026 年的格局 10
1.6.1 闭源阵营 10
1.6.2 开源阵营的历史性追赶 11
1.6.3 关键趋势 11

Chapter 2 Transformer——注意力就是一切 12
2.1 自注意力：学会「看哪里」 12
2.1.1 直觉：图书馆的类比 12
2.1.2 数学公式 12
2.1.3 计算复杂度分析 13
2.1.4 手工计算示例 13
2.1.5 代码实现 14
2.2 多头注意力：同时关注不同关系 15
2.2.1 为什么多头比单头好? 16
2.2.2 从 MHA 到 MQA 到 GQA：效率的演进 16
2.3 位置编码：让模型知道词的位置 17
2.3.1 从绝对到相对位置编码 17
2.3.2 RoPE：旋转位置编码 18
2.4 前馈网络与激活函数 19

2.4.1 激活函数的演进 19
2.4.2 为什么门控机制有效? 20
2.5 层归一化：训练的稳定器 20
2.5.1 Pre-Norm vs Post-Norm 20
2.5.2 RMSNorm：去掉均值，只留尺度 21
2.6 Flash Attention：让注意力更快 21
2.6.1 GPU 内存层次结构 22
2.6.2 核心算法：分块计算 22
2.6.3 Flash Attention 的演进 23
2.7 MLA：DeepSeek 的注意力压缩 24
2.7.1 动机与设计思想 24
2.7.2 数学形式 24
2.7.3 压缩效果 25
2.8 完整的 Transformer 块 25
2.8.1 参数量分析 26
2.8.2 三种架构风格 27
2.9 Transformer 成功的根本原因 28
2.9.1 RNN/LSTM 的根本瓶颈 28
2.9.2 CNN 用于序列建模的局限 29
2.9.3 Transformer 的优势总结 29
2.9.4 Attention is All You Need 的深远影响 29
2.9.5 挑战者：状态空间模型 30
2.9.6 新型注意力变体 33
2.10 词嵌入：让词进入向量空间 34

Chapter 3 分词——教机器「认字」 36
3.1 从词向量到子词：一段简史 36
3.1.1 词向量的黄金时代 36
3.1.2 上下文嵌入的崛起 36
3.1.3 未登录词问题 37
3.2 为什么不能直接用字或词? 37
3.3 BPE：折中的智慧 38
3.3.1 从数据压缩到自然语言 38
3.3.2 算法详解 38
3.3.3 完整的合并示例 38
3.3.4 代码实现 39
3.3.5 时间复杂度分析 40
3.4 WordPiece 与 Unigram：其他选择 40
3.4.1 WordPiece：基于似然的合并 40
3.4.2 Unigram 模型：从大到小的剪枝 41
3.4.3 三种方法的对比 41
3.5 词表大小：一个微妙的权衡 41
3.6 SentencePiece：工业级实现 43
3.6.1 语言无关性 43
3.6.2 字节回退机制 44
3.6.3 确定性与可复现性 44
3.6.4 训练与使用 44
3.7 特殊 Token 的设计 45

3.7.1 基础特殊 Token 45
3.7.2 对话模板 Token 45
3.7.3 推理与工具调用 Token 46
3.8 分词的陷阱与挑战 46
3.8.1 Glitch Token：分词器的「幻觉」 46
3.8.2 数学与数字的困境 47
3.8.3 代码中的空白问题 48
3.8.4 新编程语言的分词挑战 48
3.9 前沿探索：超越传统分词 49
3.9.1 Tiktoken：速度的极致 49
3.9.2 无分词器模型：字节级别的终极方案? 49
3.9.3 多模态分词：图像也是 Token 50

Chapter 4 数据工程——万亿 Token 的炼金术 52
4.1 数据来源 52
4.1.1 网页数据：最大的宝藏与垃圾场 52
4.1.2 书籍：长文本的稀缺宝藏 53
4.1.3 代码：推理能力的秘密武器 54
4.1.4 学术论文 54
4.1.5 对话与社交媒体 54
4.1.6 策划型数据集 55
4.2 数据清洗流水线 55
4.2.1 文本提取与规范化 55
4.2.2 语言识别 55
4.2.3 去重 56
4.2.4 质量过滤 58
4.3 数据配比：一门精细的艺术 60
4.3.1 经验法则与实验结果 60
4.3.2 真实模型的配比参考 60
4.3.3 自动数据配比：DoReMi 与后继者 61
4.3.4 课程学习 62
4.3.5 数据重复（Multi-Epoch Training） 62
4.4 合成数据：模型教模型 63
4.4.1 教科书级合成数据：Phi 的启示 63
4.4.2 自指令与进化指令 63
4.4.3 大规模合成预训练数据 63
4.4.4 数学与代码的合成数据 64
4.4.5 合成数据在 frontier 模型中的占比 64
4.4.6 合成数据的风险：模型坍缩 65
4.4.7 Constitutional AI 与 AI 反馈 66
4.5 数据处理工具生态 66
4.5.1 Datatrove 66
4.5.2 CCNet 66
4.5.3 Data-Juicer 67
4.5.4 其他重要工具 67
4.5.5 数据处理流水线示例 67
4.5.6 规模化的工程挑战 68
4.6 数据污染检测 69

4.6.1 数据污染的定义与危害 69
4.6.2 检测方法 69
4.7 版权与法律：数据的法律边界 70
4.7.1 《纽约时报》诉 OpenAI 案 70
4.7.2 合理使用（Fair Use）之争 70
4.7.3 欧盟 AI 法案与全球监管 70
4.7.4 行业的应对策略 71

Chapter 5 预训练——从随机噪声到语言理解 73
5.1 训练目标：下一个 Token 预测 73
5.1.1 交叉熵、似然与 KL 散度的三位一体 73
5.1.2 Teacher Forcing 74
5.1.3 因果遮盖 vs 双向遮盖 74
5.1.4 Prefix Language Modeling 75
5.2 Scaling Laws：计算的经济学 76
5.2.1 Kaplan Scaling Laws（2020） 76
5.2.2 Chinchilla 定律（2022） 76
5.2.3 后 Chinchilla 时代：过训练策略 77
5.2.4 “Chinchilla is Dead”：推理感知的 Scaling Laws 78
5.2.5 数据受限的 Scaling Laws 与合成数据 78
5.2.6 Scaling Laws 的新方向（2025–2026） 79
5.2.7 Scaling Laws 的局限性 79
5.3 分布式训练：为什么需要数千块 GPU? 79
5.3.1 数据并行（Data Parallelism） 80
5.3.2 ZeRO 与 FSDP：分片数据并行 81
5.3.3 张量并行（Tensor Parallelism） 81
5.3.4 流水线并行（Pipeline Parallelism） 82
5.3.5 专家并行（Expert Parallelism） 82
5.3.6 上下文并行（Context Parallelism） 83
5.3.7 多维并行组合 83
5.3.8 通信–计算重叠 84
5.3.9 训练框架演进（2025–2026） 85
5.3.10 十万卡级别的训练集群 85
5.3.11 Blackwell 架构的训练表现 86
5.4 混合精度训练 86
5.4.1 数值格式详解 86
5.4.2 FP16 的 Loss Scaling 87
5.4.3 AMP：自动混合精度 87
5.4.4 FP8：DeepSeek 的突破 87
5.4.5 FP4：下一个精度前沿 88
5.5 完整的训练循环 89
5.6 优化器的演进：超越 AdamW 92
5.6.1 Muon：矩阵正交化的动量 92
5.6.2 其他值得关注的优化器 92
5.7 多 Token 预测 93
5.7.1 为什么多 Token 预测有用? 93
5.7.2 DeepSeek-V3 的 MTP 实现 94
5.7.3 MTP 训练课程 94

5.8 训练稳定性 94
5.8.1 Loss Spike：原因与应对 94
5.8.2 梯度裁剪 95
5.8.3 权重衰减与 AdamW 95
5.8.4 真实案例：大规模训练的挑战 96
5.9 硬件基础设施 96
5.9.1 GPU 选择与代际跃升 96
5.9.2 互联网络：数据中心的「神经系统」 97
5.9.3 集群设计与成本 97
5.9.4 GPU 利用率：最重要的效率指标 98

Chapter 6 Mixture of Experts——参数效率的极致 99
6.1 核心思想：不是每个神经元都需要工作 99
6.1.1 历史脉络 99
6.1.2 稠密模型 vs 稀疏模型 99
6.2 路由机制 100
6.2.1 Top-K 路由 100
6.2.2 Token Choice vs Expert Choice 100
6.2.3 其他路由策略 101
6.2.4 动态路由与可微路由（2025–2026 前沿） 101
6.3 负载均衡：MoE 训练的核心挑战 102
6.3.1 为什么负载不均衡是灾难性的 102
6.3.2 传统方案：辅助损失 103
6.3.3 DeepSeek 的无辅助损失方案 103
6.4 DeepSeek 的三大创新 104
6.4.1 细粒度专家 104
6.4.2 共享专家 105
6.4.3 DeepEP：专用通信库 105
6.5 MoE 在实践中 105
6.5.1 Mixtral 8x7B：MoE 的开源先驱 105
6.5.2 DeepSeek-V3：MoE 的巅峰 106
6.5.3 LLaMA 4：Meta 的 MoE 转型 106
6.5.4 Qwen3-235B：阿里巴巴的 MoE 实践 107
6.5.5 Kimi K2：万亿参数的 MoE 107
6.6 MoE 推理的工程挑战 108
6.6.1 内存瓶颈与专家卸载 108
6.6.2 MoE 模型的量化 108
6.6.3 批处理的矛盾 108
6.6.4 MoE 的优势与挑战 109

Chapter 7 长上下文——让模型阅读整本书 111
7.1 为什么长上下文很难? 111
7.1.1 计算复杂度：$O(n^2)$ 的诅咒 112
7.1.2 内存瓶颈：KV Cache 的爆炸 112
7.1.3 训练–推理的长度失配 113
7.1.4 Lost in the Middle：中间信息的遗忘 113
7.2 位置编码的扩展方法 113
7.2.1 线性位置插值 113

7.2.2 NTK-aware 插值 114
7.2.3 YaRN：优雅的频率操控 114
7.2.4 Dynamic NTK：推理时自适应 115
7.3 iRoPE：Meta 的交替架构 115
7.3.1 核心设计：交替堆叠 115
7.3.2 LLaMA 4 的 iRoPE 实现细节 116
7.3.3 iRoPE 的推理优化 117
7.4 Ring Attention：跨 GPU 处理超长序列 117
7.4.1 核心思想：环形传递 117
7.4.2 内存与扩展性分析 118
7.4.3 Zig-Zag Ring Attention：负载均衡 118
7.5 长上下文训练的实践 119
7.5.1 多阶段扩展训练 119
7.5.2 长上下文训练数据 119
7.5.3 长上下文 SFT 数据 120
7.5.4 推理工程优化 120
7.6 长上下文的实际应用场景 120
7.7 Context Rot 与 NIAH：长上下文的质量评估 121
7.7.1 Needle in a Haystack 测试 121
7.7.2 NIAH 的局限与更好的评估 121
7.7.3 Context Rot：系统性退化 122
7.7.4 Context Engineering：从「更长」到「更智能」 122

Chapter 8 后训练——从语言模型到 AI 助手 124
8.1 监督微调（SFT）：学会对话 124
8.1.1 为什么需要 SFT? 124
8.1.2 Stanford Alpaca：SFT 的分水岭 125
8.1.3 数据质量的悖论 125
8.1.4 SFT 的数据来源 126
8.1.5 Chat 模板与训练格式 127
8.1.6 SFT 的训练细节 127
8.1.7 LoRA：高效微调 128
8.2 RLHF：学会人类偏好 130
8.2.1 第一步：收集偏好数据 130
8.2.2 第二步：训练奖励模型 130
8.2.3 第三步：PPO 优化 133
8.2.4 PPO 的工程挑战 134
8.2.5 PPO 的工程实践 134
8.3 DPO：去掉奖励模型 136
8.3.1 从 RLHF 到 DPO 的完整推导 137
8.3.2 DPO 损失函数的直觉 138
8.3.3 DPO vs RLHF：简单性的代价 138
8.3.4 DPO 的训练代码 139
8.3.5 DPO 的变体与演进 140
8.3.6 前沿实验室的选择：2025–2026 142
8.4 GRPO：推理模型的训练 143
8.4.1 PPO 的瓶颈 143
8.4.2 GRPO 的核心算法 143

8.4.3 GRPO 的广泛采用 145
8.4.4 GRPO 的训练代码 146
8.4.5 奖励信号的设计 147
8.4.6 DeepSeek-R1：涌现的推理能力 147
8.5 Constitutional AI：让 AI 自我改进 149
8.5.1 核心思想：RLAIF 149
8.5.2 两阶段流程 149
8.5.3 宪法的设计 150
8.5.4 Constitutional AI 的意义 150
8.5.5 Claude 的新宪法（2026） 150
8.5.6 Collective Constitutional AI：民主化对齐 151
8.5.7 RLAIF 的数学框架 151
8.6 推理能力的后训练 152
8.6.1 推理模型的训练范式 152
8.6.2 审议式对齐：OpenAI 的安全推理 152
8.6.3 过程奖励模型 vs 结果奖励模型 153
8.6.4 推理蒸馏：让小模型也能推理 154
8.7 安全对齐的深层技术 155
8.7.1 红队测试：攻击驱动的防御 155
8.7.2 表示工程：激活层面的安全控制 155
8.7.3 越狱攻击的演化 156
8.7.4 多语言安全挑战 156
8.8 SFT 数据的质量工程 157
8.8.1 前沿模型的 SFT 数据规模 157
8.8.2 LIMA vs 规模路线的辩论 157
8.8.3 SFT 数据的质量过滤 158
8.9 后训练的综合流水线 158
8.9.1 典型流水线 159
8.9.2 流水线中的关键决策 159

Chapter 9 推理优化——让模型跑得更快 161
9.1 推理的根本瓶颈 161
9.2 量化：精度换速度 162
9.2.1 量化基础 162
9.2.2 Post-Training Quantization（PTQ） 163
9.2.3 Quantization-Aware Training（QAT） 164
9.2.4 量化的质量–大小权衡 165
9.2.5 量化方法的选择决策 165
9.3 GGUF K-Quant：层级量化的艺术 166
9.3.1 为什么需要分块量化? 166
9.3.2 K-Quant 的两级结构 166
9.3.3 K-Quant 变体 166
9.3.4 混合精度策略 167
9.3.5 llama.cpp 与 GGUF 格式 167
9.4 Flash Attention 与内存高效注意力 168
9.5 KV Cache：空间换时间 168
9.5.1 为什么需要 KV Cache? 168
9.5.2 KV Cache 的内存开销 168

9.5.3 PagedAttention 169
9.5.4 KV Cache 压缩 169
9.5.5 Prefix Caching 170
9.6 推测解码：小模型辅助大模型 171
9.6.1 核心洞察 171
9.6.2 算法流程 171
9.6.3 数学保证 172
9.6.4 加速比分析 172
9.6.5 草稿模型选择与进化 173
9.7 Continuous Batching：提升服务吞吐 174
9.7.1 Static Batching 的问题 174
9.7.2 Iteration-Level Scheduling 175
9.7.3 Chunked Prefill 175
9.7.4 Prefix-Aware Batching 175
9.8 推理服务架构 176
9.8.1 推理框架对比 176
9.8.2 Disaggregated Serving：分离 Prefill 和 Decode 176
9.8.3 Dual-Batch 与异步调度 177
9.9 模型蒸馏：大模型教小模型 178
9.9.1 知识蒸馏的基本原理 178
9.9.2 LLM 蒸馏的实践 178
9.9.3 DeepSeek-R1 的蒸馏实验 179
9.9.4 蒸馏的局限性 179
9.10 推理优化的整体思路 180

Chapter 10 推理时计算缩放——用更多思考换更好答案 181
10.1 核心洞察 181
10.2 推理时缩放的数学框架 182
10.2.1 Snell et al. 的计算最优缩放理论 182
10.2.2 三种 Scaling Laws 的统一视角 183
10.3 三种策略 183
10.3.1 策略一：扩展推理链（Sequential Scaling） 184
10.3.2 策略二：并行采样与 Best-of-N（Parallel Scaling） 184
10.3.3 策略三：搜索（Search / Tree of Thought） 186
10.3.4 三种策略的对比 187
10.3.5 混合缩放（Hybrid Scaling） 187
10.4 DeepSeek-R1：从零开始学会推理 188
10.4.1 R1-Zero：从 RL 中涌现的推理 188
10.4.2 “Aha Moment”——模型的顿悟时刻 188
10.4.3 R1 的完整训练流程 189
10.4.4 R1 vs o1/o3：两条不同的路径 189
10.5 工具增强推理 190
10.5.1 代码执行作为推理工具 190
10.5.2 搜索与检索增强推理 191
10.5.3 多步 Agent 推理 191
10.6 推理模型的部署经济学 192
10.6.1 何时使用推理时缩放 192
10.6.2 路由与预算管理 192

10.6.3 共享推理的可能性 193
10.6.4 成本模型 193
10.7 推理模型的前沿与未来 193
10.7.1 从推理到行动：Agent 范式 193
10.7.2 效率缩放：做更多、花更少 194
10.7.3 开放的问题 194

Chapter 11 评估与安全——如何知道模型足够好? 195
11.1 评估的困境 195
11.1.1 Goodhart 定律 195
11.1.2 数据污染 195
11.1.3 基准饱和 197
11.2 基准测试全景 197
11.2.1 知识与推理 197
11.2.2 数学 199
11.2.3 代码 199
11.2.4 长上下文 200
11.2.5 指令跟随与对话 201
11.2.6 动态基准：LiveBench 201
11.2.7 Chatbot Arena：人类评估的金标准 201
11.3 推理能力评估 204
11.3.1 推理模型 vs 标准模型：评估差异 204
11.3.2 过程奖励模型 vs 结果奖励模型 204
11.3.3 推理模型的专项基准 206
11.4 前沿模型基准对比 207
11.5 安全对齐 208
11.5.1 有害内容生成 208
11.5.2 越狱攻击 208
11.5.3 幻觉 208
11.5.4 RAG：用检索对抗幻觉 209
11.5.5 隐私与安全评估 209
11.5.6 对齐税 210
11.6 评估的实践智慧 210
11.6.1 持续评估 210
11.6.2 能力维度分析 210
11.6.3 数据泄露防范 210
11.6.4 评估金字塔 211
11.6.5 建立针对性评估 211
11.6.6 A/B 测试 211
11.6.7 回归测试 211

Chapter 12 团队与组织——谁在训练这些模型? 213
12.1 核心团队 213
12.1.1 数据团队：被低估的英雄 213
12.1.2 基础设施团队：让数千块 GPU 协同工作 214
12.1.3 预训练研究团队：训练的「配方师」 215
12.1.4 后训练/对齐团队：定义「好」的含义 215
12.1.5 评估团队：度量你所关心的 216

12.1.6 推理/部署团队：连接模型与用户 217
12.2 组织模式：从实验室到公司 217
12.2.1 主要玩家深度解析 217
12.3 训练成本的演变：2024–2026 219
12.3.1 成本的量级 220
12.3.2 成本的构成 220
12.3.3 云端 vs. 自建集群的经济学 221
12.4 开源训练框架生态 222
12.4.1 预训练框架 222
12.4.2 后训练/微调框架 223
12.5 开源社区生态：从工具到运动 223
12.5.1 HuggingFace：ML 领域的 GitHub 223
12.5.2 本地推理生态 224
12.5.3 社区模型：百花齐放 224
12.5.4 Chatbot Arena：社区驱动的评估 225
12.6 从论文到产品：被低估的工程挑战 225
12.6.1 训练基础设施的可靠性 225
12.6.2 调试分布式训练 225
12.6.3 数据管道的效率 226
12.6.4 评估的自动化 226
12.6.5 人的因素 226

Chapter 13 前沿展望——LLM 的下一步 227
13.1 推理模型的进化：从技巧到系统能力 227
13.1.1 推理模型的技术路线分化 227
13.2 多模态：走向统一理解 228
13.2.1 视觉-语言模型的架构演进 228
13.2.2 音频：从转录到原生理解 229
13.2.3 视频理解与世界模型 229
13.2.4 Omni-模型：全模态输入与输出 229
13.3 Agent：从对话到行动 230
13.3.1 Agent 的核心架构 230
13.3.2 工具使用与标准化 230
13.3.3 代码 Agent：最成熟的落地场景 231
13.3.4 多 Agent 系统 231
13.3.5 Agent 的可靠性挑战 232
13.4 硬件路线图：算力的下一个台阶 232
13.4.1 NVIDIA：从 Blackwell 到 Vera Rubin 232
13.4.2 AMD：从追赶者到竞争者 233
13.4.3 Google TPU：垂直整合的典范 233
13.4.4 AWS Trainium：云厂商的自研芯片之路 233
13.5 端侧模型：让 AI 无处不在 234
13.5.1 为什么端侧模型重要? 234
13.5.2 NPU 硬件的快速演进 234
13.5.3 小模型家族与端侧能力 235
13.5.4 端云协作：混合推理 235
13.6 AI 监管：规则正在形成 236
13.6.1 EU AI Act：全球最严格的 AI 法规 236

13.6.2 美国：碎片化但加速中的监管 237
13.6.3 中国：审批制 + 内容管控 237
13.7 开合之争：开源 vs. 闭源的哲学与实践 238
13.7.1 闭源的论据 238
13.7.2 开源的论据 238
13.7.3 “开放权重” 不等于真正的开源 239
13.7.4 Meta 的开源战略 239
13.7.5 DeepSeek-R1 的冲击 239
13.8 总结与展望 240

Appendix A 附录：完整训练流水线速查 242
A.1 阶段一：数据准备（1–3 个月） 242
A.2 阶段二：预训练（1–3 个月） 243
A.3 阶段三：后训练（2–6 周） 244
A.4 阶段四：部署与推理优化（持续） 245
A.5 资源需求总览 246
A.6 术语表 247
A.7 推荐阅读 247

References 249

CHAPTER 1

为什么「预测下一个词」能产生智能?

1.1 一个看似简单的任务

假设你看到这样一个句子的开头:「法国的首都是 ____」。你会毫不犹豫地填入「巴黎」。这个填空题看起来极其简单，但请思考一下：要正确完成这个填空，你需要具备什么?你需要知道法国是一个国家，首都是一个政治概念，以及巴黎与法国之间的具体关系。换句话说，你需要拥有**世界知识**。

再看一个稍微复杂的例子:「她在得知祖母去世后感到 ____」。要预测这里的词，你需要理解人类情感——失去亲人通常会带来悲伤。这不仅仅是语言知识，更是对人类心理和社会关系的理解。

再来一个:「如果所有的猫都是动物，而 Tom 是一只猫，那么 Tom 是 ____」。这里需要的是逻辑推理能力——三段论的基本形式。

还有一个更微妙的例子:「他把钥匙放在桌子上，然后走进了厨房。几分钟后他回来拿 ____」。要预测「钥匙」，模型需要维持跨越多个句子的**工作记忆**——记住「钥匙」这个对象及其位置，并在上下文发生变化后依然能够追溯引用它。

最后一个:「$\int_0^1 x^2\,dx =$____」。这里需要的是纯粹的数学计算能力——理解积分符号的含义、掌握幂函数积分的规则、执行代入求值。

这就是大语言模型（Large Language Model, LLM）训练的核心目标：**准确预测下一个词**。这个目标看似简单，实则蕴含着一个深刻的洞察——要在所有可能的上下文中都准确预测下一个词，模型必须隐式地学会语法、语义、逻辑推理、世界知识、情感理解，乃至数学和编程。

预测下一个词的数学形式

给定前 t 个 token $x_1, x_2, \ldots, x_t$，模型学习条件概率分布:

$$P(x_{t+1} \mid x_1, x_2, \ldots, x_t)$$

训练目标是最大化整个训练语料库上所有位置的对数似然:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(x_t \mid x_1, \ldots, x_{t-1})$$

这个看似简单的损失函数，驱动了从 GPT-1 到 GPT-5 的全部进化。

1.2 压缩即理解

信息论的创始人 Claude Shannon 在 1948 年就揭示了这个联系[1]：**预测和压缩是同一件事的两面**。如果你能完美预测一段文字的下一个字符，你就可以用最少的比特来编码它——这就是最优压缩。反过来，最优压缩器必然是最优预测器。

1.2.1 信息熵与交叉熵损失

Shannon 定义了信息熵（entropy）来度量信息源的不确定性：

$$H(X) = -\sum_{x} p(x) \log_2 p(x) \tag{1.1}$$

其中 $p(x)$ 是符号 x 的真实概率。熵越高，信息源越不可预测，需要的编码位数越多。

Shannon 在 1950 年的论文《Prediction and Entropy of Printed English》[2]中通过实验估算，英语的熵大约在 **0.6–1.3 bits/字符**之间。这意味着如果你有一个完美的英语预测器，每个字符平均只需要约 1 bit 来编码——远少于使用 ASCII 编码的 7 bits/字符。换句话说，英语文本有大约 75–85% 的**冗余度**，而这些冗余恰恰来自语言的规律性——语法、语义、常识，正是模型需要学习的知识。

当我们用模型 q 来近似真实分布 p 时，编码效率由**交叉熵**（cross-entropy）衡量：

$$H(p, q) = -\sum_{x} p(x) \log_2 q(x) \geq H(p) \tag{1.2}$$

交叉熵永远大于等于真实熵——差值就是 KL 散度，衡量模型 q 与真实分布 p 之间的差距。LLM 训练时最小化的损失函数正是交叉熵——**训练 LLM 本质上是在训练一个压缩器**。

1.2.2 困惑度：压缩效率的直觉度量

困惑度（perplexity）是交叉熵的指数形式：

$$\mathrm{PPL} = 2^{H(p,q)} \tag{1.3}$$

直觉上，困惑度表示模型在预测下一个 token 时的「平均选项数」。PPL = 10 意味着模型在每一步平均面临 10 个等可能的选择。如果模型对语言有更好的理解（更好的压缩），选项数更少，PPL 更低。

一个具体的数字感受：GPT-2（2019, 1.5B）在 WikiText-103 上的 zero-shot PPL 约为

[1] C. E. Shannon, “A mathematical theory of communication,” *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[2] C. E. Shannon, “Prediction and entropy of printed English,” *Bell System Technical Journal*, vol. 30, no. 1, pp. 50–64, 1951.

37.5，而 GPT-3（2020）约为 20，GPT-4 级别的模型据推测可能低于 10（OpenAI 未公开具体数字）。PPL 的下降直接对应着模型对语言理解的提升。

1.2.3 Sutskever 的压缩哲学

OpenAI 的联合创始人 Ilya Sutskever 将这个思想推向了极致。他在多次演讲中强调：「预测下一个词的过程，就是压缩人类知识的过程。而足够好的压缩，本质上就是理解。」

这个论断并非修辞。考虑一段关于量子力学的论文——如果模型能准确预测下一个词，它必须在某种程度上「理解」量子力学的概念和推理链。一个仅仅记忆了表面统计模式的模型（比如「量子」后面常跟「力学」）在面对新的推理组合时会失败。只有真正建立了概念之间因果关系的模型 才能在新颖的上下文中做出正确预测。

Deepmind 在 2023 年发表的研究[3] 将这一直觉形式化：他们证明足够强大的语言模型本质上是通用的数据压缩器，因此 next-token prediction 在理论上蕴含了强大的表示能力。当然，「足够强大」和「实际训练出来」之间还有巨大的鸿沟——但这至少为「压缩即理解」提供了理论基础。

这就是整个 LLM 训练范式的哲学基础：我们不直接教模型「什么是知识」，而是通过让它预测海量文本中的下一个词，迫使它自发地组织和内化人类知识。

压缩–预测–理解 三位一体

- **压缩**：用更少的 bit 表示信息 ⇔ 找到数据中的规律
- **预测**：准确预测下一个 token ⇔ 理解上下文的结构
- **理解**：建立概念之间的关系 ⇔ 实现高效压缩
- 三者在数学上等价：最小化交叉熵 = 最大化压缩率 = 最优预测

1.3 苦涩的教训

强化学习之父 Rich Sutton 在 2019 年写了一篇影响深远的短文，题为「苦涩的教训」(The Bitter Lesson)[4]。他的核心观点是：在 AI 研究的 70 年历史中，**利用计算能力的通用方法，最终总是击败人工设计的巧妙方法**。

1.3.1 历史佐证

国际象棋。1997 年，IBM 的 Deep Blue 击败了世界冠军 Kasparov。Deep Blue 的核心并非对国际象棋有多深刻的「理解」，而是每秒评估 2 亿个棋局的暴力搜索能力，辅以相对简单的评估函数。棋手们花了几十年精心设计的开局库和战术知识库，最终败给了摩尔定律驱动的算力增长。

[3] G. Delétang et al., "Language modeling is compression," in *International Conference on Learning Representations*, 2024.

[4] R. Sutton, *The bitter lesson*, `http://www.incompleteideas.net/IncIdeas/BitterLesson.html`, 2019.

围棋。围棋的搜索空间比国际象棋大数十个数量级（$\sim 10^{170}$ 种合法局面），暴力搜索不再可行。2016 年，DeepMind 的 AlphaGo 通过深度神经网络 + 蒙特卡洛树搜索击败了李世乭。更惊人的是 AlphaGo Zero（2017）——它完全不使用人类棋谱，仅通过自我对弈学习，40 小时内就超越了所有使用人类知识训练的版本。这是对 Bitter Lesson 最戏剧化的诠释：**人类知识不仅不必要，甚至是一种限制**。

语音识别。从 1970 年代到 2010 年代，语音识别的主流方法是隐马尔可夫模型（HMM）加上 精心设计的声学特征（MFCC）和发音词典。这套系统凝聚了数十年的语言学知识。2012 年后，端到端的深度学习方法（先是 DNN-HMM 混合系统，后是纯端到端的 CTC 和 attention 模型）逐步取代了传统方法。到 2023 年，OpenAI 的 Whisper 模型 在 68 万小时的弱标注数据上训练，无需任何语言学先验知识就达到了人类水平的识别精度。

机器翻译。1990 年代，基于规则的机器翻译（如 Systran）需要语言学家手工编写数千条转换规则。2000 年代，统计机器翻译（如 Moses）用双语平行语料自动学习翻译概率。2014 年后，神经机器翻译（seq2seq + attention）再次颠覆了整个领域。到 2017 年 Transformer 出现时，机器翻译已经完全是一个端到端学习问题，不需要任何语言学知识。

计算机视觉。2012 年之前，图像分类的主流方法是手工设计特征（SIFT、HOG、LBP）加上经典分类器（SVM）。AlexNet 在 ImageNet 上的胜利标志着这套范式的终结。此后，更深的网络（VGG、ResNet）和更多的数据 不断推高性能，手工特征被彻底淘汰。

1.3.2 反面论证：人类知识的价值

Bitter Lesson 并非没有争议。一个重要的反面论证是：**正确的归纳偏置（inductive bias）能大幅提升学习效率**。

卷积神经网络（CNN）内置了平移不变性和局部性的先验——这是人类对视觉世界的知识编码。正是这个先验使得 CNN 在相同数据量下 远优于全连接网络。Transformer 的注意力机制本身也是一种归纳偏置——它假设序列中的关系可以通过内容相似度来发现。

更广泛地说，选择正确的**架构**（CNN 用于视觉、Transformer 用于序列）就是一种人类知识的注入。Bitter Lesson 的真正含义可能不是「人类知识无用」，而是「**手工编码的领域规则**不如**通用学习算法 + 计算规模**，但**结构性先验**（如架构选择）仍然至关重要」。

1.3.3 LLM 作为 Bitter Lesson 的极致体现

LLM 的成功正是这个教训的最新、最戏剧化的例证。我们没有手动编程语法规则、知识库或逻辑推理引擎，而是选择了最简单的目标函数（预测下一个词），然后用数万亿的文本和数千块 GPU 进行训练。结果是：模型涌现出了令人惊叹的能力。

但值得注意的是，Transformer 架构本身——注意力机制、位置编码、残差连接——这些设计并非「无知识」的，而是凝聚了深度学习社区多年的经验和直觉。从这个角度看，LLM 的成功是 Bitter Lesson 和人类巧思的**结合**，而非简单的「规模碾压一切」。

1.4 涌现：大就是不同

LLM 的一个最引人入胜的现象是**涌现**（emergence）。当模型规模超过某个阈值时，会突然展现出在较小规模上完全不存在的能力。

1.4.1 经典涌现案例

例如，思维链推理（Chain-of-Thought, CoT）[5]：小模型在被提示「让我们一步一步思考」时不会产生任何有意义的改进，但当模型参数超过约 60B 时，这个简单的提示会带来巨大的推理能力提升。Wei 等人在 2022 年的研究表明，CoT 推理能力在模型规模低于 ~10B 时几乎不存在，在 10–100B 之间开始出现，在 100B 以上则表现出稳定的强大效果。

算术能力的涌现更为戏剧化。GPT-3（175B）能做三位数加法但经常出错，GPT-4 则能可靠地完成多步骤数学推理。类比推理、代码生成、多语言翻译——这些能力都不是线性增长的，而是在模型跨越某个规模门槛时「突然出现」。

这种涌现现象在物理学中有一个贴切的类比：**相变**（phase transition）。水在 99°C 和 100°C 之间发生的不是渐进变化，而是质变——从液态到气态。类似地，语言模型在某个参数规模或训练数据量的临界点上，可能经历从「不具备某能力」到「具备该能力」的跃迁。这个类比虽然直觉上吸引人，但严格性有限——神经网络并非热力学系统，「相变」是否有精确的数学对应仍是开放问题。

1.4.2 涌现是真实的还是度量假象？

2023 年，Schaeffer 等人[6] 发表了一篇引人深思的论文，题为「Are Emergent Abilities of Large Language Models a Mirage?」。他们的核心论点是：许多所谓的涌现可能是**评估方式的假象**。

具体来说，当使用**离散**的评估指标（如准确率——答案要么全对要么全错）时，能力增长的曲线会呈现出阶梯状的「突变」。但如果改用**连续**的评估指标（如 token-level 的对数似然、Brier 分数），同一个模型的能力增长曲线就变得平滑了——不存在明显的「突变点」。

这就好比考试成绩从 59 分跳到 60 分，在「是否及格」这个离散指标上是一个突变，但在连续分数上只是 1 分的平滑进步。

[5] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[6] R. Schaeffer et al., "Are emergent abilities of large language models a mirage?" *Advances in Neural Information Processing Systems*, vol. 36, 2023.

涌现之争的现状

对涌现现象的正确理解是：

- 在**离散指标**（如 exact-match accuracy）下，涌现的阶跃特征确实存在
- 在**连续指标**（如 perplexity）下，能力增长通常是平滑的
- 但**实际应用**往往关心的就是离散指标（答案对不对），所以涌现在应用层面是「真实」的
- 大模型能做到小模型做不到的事——这一事实无可争议，争议仅在于「过渡」是突变还是平滑的

从工程角度来说，无论涌现的「阶跃」是真实的还是度量假象，规模仍然是最可靠的能力提升路径——尽管 Scaling Laws 的回报率可能在递减。

1.5 从 GPT 到 DeepSeek：一部效率进化史

LLM 的历史可以粗略地分为几个阶段，但在进入时间线之前，让我们先理解一个贯穿整个历史的核心转变：从**特征工程**到**上下文学习**，从**任务专用**到**通用基础模型**。

1.5.1 前传：词向量与上下文表示（2013–2018）

在 Transformer 之前，自然语言处理经历了两次重要的范式转变。

Word2Vec[7]（2013）和 GloVe（2014）证明了一个惊人的事实：通过在大规模语料上训练词向量，语义关系可以被编码为向量空间中的方向。但这些是**静态**词向量——无论上下文如何，「bank」这个词只有一个表示，无法区分「银行」和「河岸」。

ELMo[8]（2018）解决了这个问题：通过双向 LSTM 生成**上下文相关**的词表示。同一个词在不同句子中会有不同的向量。ELMo 的出现标志着 NLP 领域从静态表示走向动态表示，为后来的预训练范式铺平了道路。

1.5.2 架构探索期（2017–2019）

2017 年，Transformer[9] 被提出，最初只是一个机器翻译模型。但它的自注意力机制带来的并行化优势 很快引起了整个领域的注意。

2018 年发生了两件大事：

GPT-1（2018, 117M 参数）选择了 **decoder-only** 路线——单向语言模型，通过预测下一个 token 来预训练，再在下游任务上微调。它首次证明了「预训练 + 微调」范式的威力。

[7] T. Mikolov et al., "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[8] M. E. Peters et al., "Deep contextualized word representations," in *Proceedings of NAACL-HLT*, 2018.

[9] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

BERT-Large[10]（2018, 110M/340M 参数）选择了 **encoder-only** 路线——双向语言模型，通过 Masked Language Model（随机遮盖 15% 的 token 并预测）来预训练。BERT 在 11 项 NLP 基准上刷新了记录，震动了整个学界。

这一阶段的模型较小，训练成本以千美元计。两条路线的竞争刚刚开始。

GPT-2（2019, 1.5B 参数）是一个转折点。OpenAI 声称它「太危险而不能公开发布」——这在当时引发了激烈争议，但也将公众注意力吸引到了语言模型上。更重要的是，GPT-2 展示了零样本能力的雏形：不需要微调，仅通过提示就能完成多种任务。

1.5.3 规模竞赛期（2020–2022）

GPT-3[11]（2020, 175B 参数）是一个划时代的模型。它的训练成本约 460 万美元，参数量比 GPT-2 大 100 倍。GPT-3 最重要的贡献是发现了 **in-context learning**（上下文学习）——你不需要微调模型，只需在提示中给出几个示例，模型就能学会新任务。这从根本上改变了 NLP 的使用范式：从「训练一个任务专用模型」变成「设计好的提示给通用模型」。

Google 在这一时期也没有闲着。**T5**（2020, 11B 参数）提出了「将所有 NLP 任务统一为 text-to-text」的框架——无论是分类、翻译还是摘要，都是输入一段文本、输出一段文本。**PaLM**[12]（2022, 540B 参数）是当时最大的密集模型，展示了规模带来的能力跃升，特别是在推理和多语言任务上。

DeepMind 的 **Chinchilla**[13]（2022, 70B 参数）带来了一个关键发现：之前的模型普遍**训练不足**。Chinchilla 定律指出，模型参数和训练数据应该等比例增长——一个 70B 的模型如果用足够的数据训练（1.4T tokens），可以超越参数量大 4 倍但数据不足的 280B 模型。这深刻地影响了后续所有模型的训练策略。

训练成本在这一时期上升到数千万美元级别。

1.5.4 产品化爆发期（2023–2024）

2022 年 11 月，**ChatGPT** 的发布引发了全球性的 AI 浪潮。ChatGPT 本质上是 GPT-3.5 经过 RLHF（基于人类反馈的强化学习）[14]微调的产物——技术上并非革命性突破，但**对话式交互**的产品形态让普通人第一次直观感受到了 AI 的能力。它在两个月内获得了 1 亿用户，创造了消费者应用增长最快的历史记录。

[10] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019.

[11] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[12] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[13] J. Hoffmann et al., "Training compute-optimal large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[14] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

GPT-4（2023）将能力推向了新高度。虽然 OpenAI 没有公开其架构细节，但广泛推测它采用了 MoE（混合专家）架构，总参数量可能超过 1T，但每次推理只激活其中一部分。GPT-4 在律师资格考试中排名前 10%，能够理解图片，生成复杂代码。

与此同时，开源运动蓬勃兴起。Meta 在 2023 年发布 **LLaMA**[15]（7B–65B），展示了较小模型经过充分训练后也能达到优秀性能。LLaMA 的开源引爆了整个社区——数百个微调版本在几周内涌现，从 Alpaca 到 Vicuna 到 WizardLM。随后的 **Llama 2**（2023）和 **Llama 3**[16]（2024）持续提升了开源模型的水平。

Google 发布了 **Gemini**[17]（2023），这是一个原生多模态模型。Anthropic 发布了 Claude 系列模型，以安全性和长上下文能力著称。

中国的大模型生态也在这一时期快速发展。清华的 **GLM**（ChatGLM）是最早对标 ChatGPT 的中文模型之一。阿里巴巴的 **Qwen** 系列逐步追赶国际水平。**DeepSeek** 以其开放的技术报告和创新的架构设计 赢得了全球社区的关注。

2024 年 9 月，OpenAI 发布了 **o1** 预览版——这是第一个专用推理模型，通过在回答前进行长时间内部「思考」，在数学、编程和科学推理上取得了质的飞跃。o1 开创了 **test-time compute scaling** 的新范式，为后续的 o3、DeepSeek-R1 以及各家模型的推理增强模式奠定了基础。

训练成本达到数亿美元级别。

1.5.5 效率革命期（2025–2026）

2024 年末至 2025 年初，效率革命的序幕由中国团队拉开。**DeepSeek-V3**[18]（2024 年 12 月）以 560 万美元的训练成本 达到了 GPT-4 级别的性能——这是训练成本的**两个数量级下降**。MoE 架构、FP8 训练、Multi-head Latent Attention（MLA）、高效的 Multi-Token Prediction 以及极致的工程优化 共同推动了这场效率革命。

2025 年 1 月，**DeepSeek-R1**[19] 在推理模型领域掀起了另一场革命。它证明了通过纯强化学习（不依赖于人类标注的推理数据），模型可以自发地学会思维链推理。DeepSeek-R1 在数学、代码和推理任务上达到了与 OpenAI o1 相当的水平，且完全开源（MIT 许可证）。其蒸馏版本 R1-Distill-Qwen-32B 在多个推理基准上 甚至超越了 OpenAI o1-mini，以极低的参数量实现了令人瞩目的推理能力。

2025 年春季迎来了一波密集发布。4 月，OpenAI 同时推出了 **o3** 和 **o4-mini**——第二代推理模型，首次实现了推理模型的工具调用能力，包括网页搜索、Python 代码执行和视觉推理。同期发布的 **GPT-4.1** 以百万 token 上下文窗口 重新定义了长上下文处理的标杆。Meta 发布了 **Llama 4** 系列（Scout 和 Maverick），首次采用 MoE 架构并原生支持

[15] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[16] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[17] G. Team et al., "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[18] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[19] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

多模态，Scout 甚至支持 1000 万 token 的上下文窗口。阿里巴巴发布了 **Qwen3**[20]，首创「混合推理」模式——同一模型可在深度思考与快速响应之间无缝切换。

5 月，Anthropic 发布了 **Claude 4**（Opus 4 和 Sonnet 4），在 agentic coding 和长时间自主任务上树立了新标杆。8 月，OpenAI 推出了划时代的 **GPT-5**——一个统一的系统，内置了智能路由器，能自动判断何时快速响应、何时调用深度推理，实质上将 GPT 系列和 o 系列合二为一。

下半年的节奏更加紧凑：Anthropic 的 **Claude Sonnet 4.5**（9 月）和 **Claude Opus 4.5**（11 月）将 agentic coding 推向新高度；Google 发布 **Gemini 3**（11 月），以 Deep Think 模式和原生多模态能力重新加入竞争；DeepSeek 发布 **V3.2**（12 月），引入 Sparse Attention 机制，性能达到 GPT-5 水平，高计算变体 V3.2-Speciale 在 2025 年 IMO 和 IOI 中取得金牌级表现。

进入 2026 年，迭代速度进一步加快。Anthropic 在 2 月连续发布 **Claude Opus 4.6** 和 **Sonnet 4.6**，首次为 Opus 级模型提供百万 token 上下文窗口（beta）。Google 推出 **Gemini 3.1 Pro**（2 月），阿里巴巴发布 **Qwen3.5**（2 月），以 397B 旗舰模型和全系列小模型（0.8B–9B）覆盖从云端到边缘的全场景。OpenAI 则以 **GPT-5.3-Codex**（2 月）和 **GPT-5.4**（3 月）持续推进 agentic coding 的边界——GPT-5.3-Codex 甚至参与了自身的训练调试和部署。DeepSeek-V4 也预计将于 2026 年 3 月发布，以约 1T 总参数（32B 激活）和百万 token 上下文 向闭源模型发起最强挑战。

这场效率革命的意义远超技术本身。当训练一个 frontier 模型的成本从数亿美元降至数百万美元，AI 研究的参与门槛大幅降低。大学实验室、初创企业甚至独立研究者 都有可能做出有意义的贡献。这对整个领域的健康发展至关重要——创新不应该被垄断在少数拥有万卡集群的机构手中。

1.5.6 Scaling Hypothesis 之争

贯穿这段历史的是一个核心假说：**Scaling Hypothesis**——持续增加模型规模、数据量和计算量，就能持续获得更强的能力。Kaplan 等人[21] 在 2020 年的研究 给出了精确的幂律关系（Scaling Laws），为这一假说提供了实证基础。

但到了 2025–2026 年，这一假说面临深刻的重构。

预训练 scaling 遭遇瓶颈。Ilya Sutskever 在 NeurIPS 2024 的演讲中明确指出：「我们已知的预训练方式将会终结，因为我们只有一个互联网。」高质量文本数据正在接近耗尽——据估计，公开可用的高质量文本数据总量约为 10–15T tokens，而 frontier 模型的训练数据量已逼近这一上限。合成数据（synthetic data）虽然是一条出路，但如何避免模型在自身生成的数据上训练导致的退化（所谓「模型坍缩」）仍是开放问题。

Test-time compute 开辟新维度。OpenAI 的 o 系列模型（o1/o3）和 DeepSeek-R1 证明了一条新路径：不增大模型本身，而是让模型在推理时花更多计算来「思考」。这本质上是将 scaling 从训练阶段转移到推理阶段。2025 年 NeurIPS 的教程「Scale Test-Time

[20] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
[21] J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

Compute on Modern Hardware」系统总结了这一范式：test-time compute 的 scaling law 与训练阶段有本质不同——推理负载具有低并行度、不规则工作量和动态执行路径的特点，在现代硬件上的高效部署是一个独立的技术挑战。

从「规模时代」到「研究时代」。一个越来越强的共识是：AI 正在从「规模时代」（Age of Scaling）过渡到「研究时代」（Age of Research）——单纯增加 GPU 集群和数据量的策略回报递减，突破需要来自架构创新（如 DeepSeek 的 MLA 和 Sparse Attention）、训练方法创新（如纯 RL 训练推理能力）、以及数据工程创新（如 agentic task synthesis pipeline）。Meta 的 Llama 4 Behemoth（2T 参数）的延期发布 就是「更大不一定更好」的一个注脚——内部评估发现它未必显著优于前代模型，促使 Meta 转向更注重架构效率的策略。

但 scaling 并未「死亡」——它在**变形**。DeepSeek-V3.2 通过 scalable RL 框架 在后训练阶段 scale compute 至 GPT-5 水平；GPT-5 将推理和生成统一在一个模型中自动路由计算资源；Qwen3 的混合推理模式让用户动态控制思考深度。「规模」的含义已经从「更大的模型」扩展到了一个多维空间：更多的训练数据、更好的数据质量、更高效的架构、更智能的后训练、更多的推理时计算——以及这些维度之间的最优分配。

1.6 2026 年的格局

截至 2026 年 3 月，LLM 领域已经形成了多极竞争的格局，闭源与开源的边界日益模糊。

1.6.1 闭源阵营

OpenAI 在 2025 年完成了从 GPT 系列到统一模型的转型。GPT-5（2025 年 8 月）将生成模型（GPT 系列）和推理模型（o 系列）统一为单一系统，内置智能路由器自动分配计算资源。后续的 GPT-5.1、5.2、5.3-Codex（2026 年 2 月）和 GPT-5.4（2026 年 3 月）以每隔数周的节奏迭代，GPT-5.3-Codex 甚至参与了自身的训练调试。此前的 o3/o4-mini（2025 年 4 月）开创了推理模型的工具调用能力，已在 GPT-5 发布后从 ChatGPT 主界面退役，但仍可通过 API 访问。

Anthropic 以 Claude 系列在 agentic coding 领域建立了强势地位。Claude 4（2025 年 5 月）率先展示了长时间自主编程能力，Claude Sonnet 4.5（2025 年 9 月）和 Opus 4.5（2025 年 11 月）持续领先，2026 年 2 月的 **Claude Opus 4.6** 和 **Sonnet 4.6** 在 Terminal-Bench 2.0 和 Humanity’s Last Exam 上取得最高分，首次为 Opus 级模型提供百万 token 上下文窗口。Claude 的 extended thinking 模式提供了一种不同于 o 系列的推理增强路径——在通用模型中按需激活深度推理，而非训练独立的推理专用模型。

Google 以 Gemini 2.5 Pro（2025 年 3 月）重新加入竞争，Gemini 3（2025 年 11 月）引入 Deep Think 模式和原生多模态，**Gemini 3.1 Pro**（2026 年 2 月）进一步提升了复杂推理和 agentic 能力，以 1M token 上下文窗口覆盖代码库级别的分析任务。

1.6.2 开源阵营的历史性追赶

2025–2026 年，开源模型首次在质量上真正逼近甚至超越闭源模型。

DeepSeek 是这场追赶的先锋。V3（2024 年 12 月）以 560 万美元达到 GPT-4 水平[22]；R1（2025 年 1 月）以纯 RL 训练达到 o1 水平[23]；V3.2（2025 年 12 月）通过 Sparse Attention 和 scalable RL 达到 GPT-5 水平，其高计算变体在 2025 年 IMO 和 IOI 中 取得金牌级表现。即将发布的 **V4**（预计 2026 年 3 月）以约 1T 总参数、32B 激活参数和百万 token 上下文，目标是成为开源模型对闭源的最强挑战。

Meta 的 Llama 4（2025 年 4 月）发布了 Scout 和 Maverick 两款 MoE 多模态模型，其中 Scout 支持高达 1000 万 token 的上下文窗口。但旗舰级 Behemoth（2T 参数）因内部评估显示其 未必显著优于前代而反复延期，至今未发布，成为「更大不一定更好」的典型案例。

阿里巴巴 的 Qwen3[24]（2025 年 4 月）首创混合推理模式，旗舰 Qwen3-235B-A22B 在多项基准上与 DeepSeek-R1 持平。2026 年 2 月发布的 **Qwen3.5** 以 397B 旗舰模型为核心，采用 Gated DeltaNet 混合架构（3:1 线性/全注意力比例），支持 201 种语言和 262K 上下文，并推出 0.8B–9B 全系列小模型覆盖边缘场景——其中 Qwen3.5-9B 在多项基准上超越前代 Qwen3-30B。

1.6.3 关键趋势

推理模型的普及与融合。从 OpenAI o1/o3 到 DeepSeek-R1，再到 GPT-5 的内置推理路由和 Claude 的 extended thinking，「慢速思考」已从独立的推理模型范式 演变为通用模型的标准能力。Qwen3 的 `/think` 和 `/no_think` 切换，以及 Gemini 3 的 Deep Think 模式，都体现了同一趋势：用户可按需控制推理深度，在响应速度与推理质量之间灵活权衡。

多模态的原生化。当前的 frontier 模型普遍支持文本、图片、音频甚至视频的理解和生成。Llama 4 和 Qwen3.5 从架构层面原生支持多模态，不再是在文本模型上「外挂」视觉编码器。纯文本 LLM 正在被多模态大模型取代——但 Transformer 架构和预训练范式的核心不变。

Agentic AI 的成熟。2026 年的模型不再仅仅是问答工具，而是能够自主执行复杂工作流的 agent。从 Claude Code 的多小时自主编程到 GPT-5.3-Codex 参与自身开发，从 Gemini 3.1 Pro 的高级工具使用到 Qwen3.5 的自主 agent 能力，AI 正在从「助手」演进为「同事」。

这些成就背后有一个共同的故事：更聪明的训练方法、更高效的架构设计、更精细的数据工程。本文将完整地讲述这个故事——从数据采集到模型部署，从 Transformer 的数学原理到分布式训练的工程实践，从预训练的计算规模定律到后训练的强化学习技巧。

我们的旅程从最基础的构件开始：Transformer 架构。

---

[22] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[23] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[24] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

CHAPTER 2

TRANSFORMER——注意力就是一切

2017 年，Google 的研究团队发表了一篇改变历史进程的论文：《Attention is All You Need》[1]。这篇论文提出的 Transformer 架构，在不到十年的时间里，从一个机器翻译模型发展成了当今所有 AI 系统的核心引擎。为什么 Transformer 如此成功？让我们从最核心的组件——注意力机制——开始理解。

2.1 自注意力：学会「看哪里」

2.1.1 直觉：图书馆的类比

想象你走进一座图书馆，你有一个问题想要回答。你的问题就是**查询**（Query）：「我想找关于法国历史的信息。」图书馆里每本书的标题是**键**（Key）：「法国大革命」「量子物理导论」「巴黎美食指南」……每本书的内容是**值**（Value）。

你会把自己的查询与每本书的标题做比较，与「法国大革命」的匹配度最高，与「巴黎美食指南」也有一定相关性，与「量子物理导论」则几乎不相关。然后你根据这些相关性，加权地阅读各本书的内容。

这个类比可以更精确地对应到信息检索领域。在搜索引擎中，用户输入查询 q，系统计算 q 与每个文档键 k_i 的相关性分数 $\text{score}(q, k_i)$，然后按分数排序返回文档内容 v_i。注意力机制做的事情完全一样，只不过它不是返回 top-k 文档，而是对**所有**文档做**软性**加权平均——相关性越高的文档权重越大。

自注意力（Self-Attention）做的就是这件事。在一个句子中，**每个词同时扮演查询、键和值三个角色**。每个词都在问：「句子里的其他词中，哪些与我最相关?」然后根据相关性，加权地整合其他词的信息。

2.1.2 数学公式

给定输入序列中某个位置的向量 $\mathbf{x}$，我们用三个可学习的线性变换将其映射为查询、键、值：

$$\mathbf{q} = W_Q\mathbf{x}, \quad \mathbf{k} = W_K\mathbf{x}, \quad \mathbf{v} = W_V\mathbf{x} \tag{2.1}$$

其中 $W_Q, W_K \in \mathbb{R}^{d_k \times d}$，$W_V \in \mathbb{R}^{d_v \times d}$ 是参数矩阵。

[1] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

对整个序列做矩阵运算，注意力的计算公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.2}$$

这个公式分为三步。第一步，$QK^\top$ 计算所有查询与所有键的点积，得到一个 $n \times n$ 的相关性矩阵（n 是序列长度）。第二步，除以 $\sqrt{d_k}$ 进行缩放——这一步至关重要，因为当维度 d_k 较大时，点积的值会变得很大，导致 softmax 的梯度趋近于零（饱和），模型无法有效学习。第三步，softmax 将相关性分数转换为概率分布，然后与 V 相乘得到加权输出。

2.1.3 计算复杂度分析

注意力的计算复杂度值得仔细分析，因为它决定了 Transformer 能处理多长的序列。

$QK^\top$ 的计算：$Q \in \mathbb{R}^{n \times d_k}$，$K \in \mathbb{R}^{n \times d_k}$，矩阵乘法的复杂度为 $O(n^2 d_k)$。softmax 的复杂度为 $O(n^2)$（对 $n \times n$ 矩阵逐行操作）。最终 $\text{softmax}(\cdot) \times V$ 的复杂度为 $O(n^2 d_v)$。总计算复杂度为 $O(n^2 d)$，其中 $d = d_k = d_v$。

内存方面，需要存储 $n \times n$ 的注意力矩阵。对于 $n = 128{,}000$（128K 上下文），这个矩阵有 $128K \times 128K \approx 160$ 亿个元素——仅以 FP16 存储就需要约 30 GB。这就是为什么长上下文是 LLM 的核心挑战之一，也是 Flash Attention 如此重要的原因。

2.1.4 手工计算示例

为了让注意力的计算过程完全透明，让我们用一个极简的例子手动走一遍。假设序列长度 $n = 3$，每个 token 的维度 $d = 4$，查询/键/值的维度 $d_k = d_v = 2$。

输入矩阵（3 个 token，每个 4 维）：

$$X = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

投影矩阵（简化为固定值）：

$$W_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad W_V = \begin{pmatrix} 1 & 1 \\ 0 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Step 1: 计算 Q, K, V：

$$Q = XW_Q = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}, \quad K = XW_K = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}, \quad V = XW_V = \begin{pmatrix} 1 & 2 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$$

Step 2: 计算 $QK^\top/\sqrt{d_k}$（$\sqrt{d_k} = \sqrt{2} \approx 1.41$）：

$$QK^\top = \begin{pmatrix} 0 & 4 & 2 \\ 4 & 0 & 2 \\ 2 & 2 & 2 \end{pmatrix} \quad \Rightarrow \quad \frac{QK^\top}{\sqrt{2}} \approx \begin{pmatrix} 0 & 2.83 & 1.41 \\ 2.83 & 0 & 1.41 \\ 1.41 & 1.41 & 1.41 \end{pmatrix}$$

Step 3: 对每一行做 softmax，得到注意力权重矩阵 A：

$$A \approx \begin{pmatrix} 0.05 & 0.77 & 0.19 \\ 0.77 & 0.05 & 0.19 \\ 0.33 & 0.33 & 0.33 \end{pmatrix}$$

Step 4: 加权求和 Output $= AV$：

$$\text{Output} \approx \begin{pmatrix} 1.00 & 0.28 \\ 1.00 & 1.72 \\ 1.00 & 1.00 \end{pmatrix}$$

观察结果：token 1 的输出主要由 token 2 的值决定（权重 0.77），token 2 的输出主要由 token 1 的值决定（权重 0.77），而 token 3 均匀地关注所有 token（权重各约 0.33）。这就是注意力的本质——让每个 token 选择性地聚焦于最相关的其他 token。

2.1.5 代码实现

用 PyTorch 实现自注意力仅需几行代码：

Listing 2.1: Self-Attention in PyTorch

```python
import torch
import torch.nn.functional as F
import math

def self_attention(x, W_q, W_k, W_v):
    """
    x: (batch, seq_len, d_model)
    W_q, W_k, W_v: (d_model, d_k) or (d_model, d_v)
    """
    Q = x @ W_q # (batch, seq_len, d_k)
```

```
    K = x @ W_k # (batch, seq_len, d_k)
    V = x @ W_v # (batch, seq_len, d_v)

    d_k = Q.shape[-1]
    # Step 1+2: scaled dot-product
    scores = (Q @ K.transpose(-2, -1)) / math.sqrt(d_k)
    # Step 3: softmax + weighted sum
    weights = F.softmax(scores, dim=-1)
    output = weights @ V
    return output
```

这段代码对应了公式 (2.2)的三个步骤。`Q @ K.transpose(-2, -1)` 是 $QK^\top$（矩阵乘法），除以 `math.sqrt(d_k)` 是缩放，`F.softmax` 是 softmax 归一化，最后 `weights @ V` 是加权求和。

为什么用点积而不是其他相似度度量?

Bahdanau 等人在 2015 年提出的注意力使用加法形式 score $= v^\top \tanh\big(W_1 q + W_2 k\big)$，需要额外的参数和非线性操作。点积注意力 $q^\top k$ 没有额外参数，而且可以直接用高度优化的矩阵乘法实现——这在 GPU 上的效率远高于逐元素操作。在实践中，缩放点积注意力的效果与加法注意力相当，但速度快得多。此外，点积在几何上有清晰的含义：$q^\top k = \|q\|\|k\|\cos\theta$，即两个向量的相似度由它们之间的夹角决定。这使得注意力权重具有直觉上的可解释性——内容越相似（夹角越小），注意力越高。

2.2 多头注意力：同时关注不同关系

单个注意力头只能学习一种关注模式。但在语言中，词与词之间存在多种关系：语法关系（主语–谓语）、语义关系（同义词、反义词）、指代关系（代词指向哪个实体）、位置关系（相邻的词往往相关）。

多头注意力（Multi-Head Attention, MHA）的解决方案是：**并行运行多个注意力头，每个头独立学习不同的关注模式**。

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W^O \tag{2.3}$$

其中 $\mathrm{head}_i = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$。

每个头使用自己的投影矩阵（W_i^Q, W_i^K, W_i^V），在不同的子空间中计算注意力。最后将所有头的输出拼接起来，通过一个线性层 W^O 融合。

2.2.1 为什么多头比单头好?

直觉上，多头注意力的好处是**分工**——不同的头可以专注于不同类型的语言关系。Clark 等人[2] 在对 BERT 的注意力模式进行可视化分析后发现，不同的头确实学到了截然不同的模式:

- 某些头专注于**局部上下文**（如相邻词），类似于 n-gram 模型
- 某些头关注**语法依赖**，如主语和谓语之间的一致性关系，即使它们相距很远
- 某些头追踪**指代关系**——「他」指向哪个人物，「它」指向哪个对象
- 某些头负责**分隔符注意力**——大量注意力集中在句号、逗号等标点符号上，可能用于分割句子结构

从数学角度看，多头注意力的总参数量与单头注意力相同。如果模型维度为 d，头数为 h，那么每个头的维度为 $d_k = d/h$。单头注意力的投影矩阵大小为 $d \times d$，多头注意力中每个头的投影矩阵大小为 $d \times d/h$，h 个头加起来仍然是 $d \times d$。因此，多头注意力**不增加参数量**，但通过在不同子空间中并行计算，获得了更丰富的表示能力。

现代 LLM（如 LLaMA 3、DeepSeek-V3）通常使用 32–128 个注意力头。

2.2.2 从 MHA 到 MQA 到 GQA：效率的演进

MHA 的问题在推理时暴露无遗。在自回归生成中，每生成一个新 token，都需要用到之前所有 token 的 K 和 V。为了避免重复计算，这些 K, V 会被缓存——这就是 **KV cache**。

对于标准 MHA，每个头都有独立的 K 和 V 矩阵。一个 70B 参数模型处理 128K token 的上下文时，KV cache 的大小为:

$$2 \times n_{\text{layers}} \times n_{\text{heads}} \times \text{seq_len} \times d_{\text{head}} \times \text{bytes}$$

代入典型值（80 层、64 头、128K 序列、128 维、FP16），$2 \times 80 \times 64 \times 128{,}000 \times 128 \times 2 \approx 335\,\text{GB}$——远超单块 GPU 的显存。

Multi-Query Attention（MQA）[3] 是 Shazeer 在 2019 年提出的激进方案：**所有查询头共享一套 K, V**。KV cache 从 h 套降到 1 套，内存减少到 $1/h$。代价是质量下降——所有头被迫使用相同的 KV 信息，表达能力受限。

分组查询注意力（Grouped-Query Attention, GQA）是 Ainslie 等人（2023）提出的折中方案[4]，被 LLaMA 2 广泛采用[5]：将多个查询头分成若干组，每组共享一套 KV。例如，32 个查询头分成 8 组，只需要 8 套 KV——KV cache 减少到原来的 1/4，而质量几乎没有损失。

[2] K. Clark et al., "What does BERT look at? an analysis of BERT's attention," in *Proceedings of the ACL Workshop BlackboxNLP*, 2019.

[3] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[4] J. Ainslie et al., "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[5] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

GQA 处于两个极端之间：

方法	KV 头数	KV cache 大小	质量
MHA	h（全部独立）	1×	最高
GQA	h/g（分组共享）	$1/g$	接近 MHA
MQA	1（全部共享）	$1/h$	可能下降

GQA 在质量和效率之间找到了最优平衡，是目前几乎所有主流 LLM 的标准选择。LLaMA 3 使用 $h = 32, g = 4$（即 8 组 KV 头），DeepSeek-V2 则走得更远，采用了 MLA（下文详述）。

2.3 位置编码：让模型知道词的位置

注意力机制有一个本质特性：**它对输入的排列是不变的**。也就是说，如果你打乱输入词的顺序，注意力的计算结果不会改变——因为点积只关心内容，不关心位置。

但语言显然是有顺序的。「狗咬了人」和「人咬了狗」包含完全相同的词，但意思截然相反。模型必须知道每个词在句子中的位置。

2.3.1 从绝对到相对位置编码

最初的 Transformer[6] 使用正弦函数生成固定的位置编码，直接加到词嵌入上：

$$\mathrm{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right) \tag{2.4}$$

$$\mathrm{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{2.5}$$

这种方法有两个问题：一是模型只在有限长度上训练，无法自然泛化到更长的序列；二是绝对位置并不是最有用的信息——对于语言理解来说，「这两个词相距 3 个位置」（相对位置）比「这个词在第 47 个位置」（绝对位置）更重要。

BERT 使用了可学习的绝对位置编码——为每个位置学习一个独立的向量。这比正弦编码更灵活，但仍然是绝对位置，且不能泛化到训练时未见过的长度。

ALiBi（Attention with Linear Biases）[7] 提出了另一种方案：不修改嵌入，而是在注意力分数上直接加一个与距离成正比的偏置。距离越远的 token 对，偏置越大（越负），注意力越低。这自然编码了「近距离优先」的归纳偏置。

但最终胜出的是 RoPE。

[6] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[7] O. Press et al., "Train short, test long: Attention with linear biases enables input length generalization," in *International Conference on Learning Representations*, 2022.

2.3.2 RoPE：旋转位置编码

RoPE（Rotary Position Embedding）[8] 是当前所有主流 LLM 的标准位置编码方案。它的核心思想优雅而巧妙：**通过在复数空间中旋转向量来编码位置**。

核心直觉

为什么用「旋转」来编码位置？考虑二维平面上的两个向量 $\mathbf{q}$ 和 $\mathbf{k}$。如果我们分别将它们旋转角度 $m\theta$ 和 $n\theta$，它们的点积变成：

$$\tilde{\mathbf{q}}^\top \tilde{\mathbf{k}} = \|\mathbf{q}\|\|\mathbf{k}\| \cos(\alpha + (m-n)\theta)$$

其中 α 是原始夹角。注意，结果只依赖于**相对位置** $m-n$，而不是绝对位置 m 或 n！这正是我们想要的性质。

而且，旋转角度 $(m-n)\theta$ 意味着 相对距离越大，注意力分数的衰减越快——这提供了一种自然的**远程衰减**（long-range decay）效应，让模型更关注近距离的 token，同时仍保留对远距离 token 的感知能力。

数学公式

对于位置 m 的查询向量和位置 n 的键向量，RoPE 分别对它们施加旋转：

$$\tilde{q}_m = R(\theta, m)\, q_m, \quad \tilde{k}_n = R(\theta, n)\, k_n \tag{2.6}$$

其中 $R(\theta, m)$ 是一个旋转矩阵，每对相邻维度 $(2i, 2i+1)$ 按角度 $m\theta_i$ 旋转：

$$R(\theta_i, m) = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}, \quad \theta_i = 10000^{-2i/d} \tag{2.7}$$

关键在于：旋转后的查询和键做点积时，$\tilde{q}_m^\top \tilde{k}_n$ 只依赖于**相对位置** $m-n$，而不是绝对位置 m 或 n。这是因为旋转矩阵满足 $R(m)^\top R(n) = R(n-m)$。

θ_i 的设计也很有讲究：低维度（i 小）对应较大的 θ_i，旋转速度快，捕捉短距离的位置模式（如相邻词的语法关系）。高维度（i 大）对应较小的 θ_i，旋转速度慢，捕捉长距离的位置模式（如段落间的主题关联）。这种多尺度编码使得模型能同时捕捉局部和全局的位置关系。

上下文长度外推

RoPE 的一个实际挑战是**上下文长度外推**——模型在 4K 长度上训练后，能否推广到 32K 或 128K？

[8] J. Su et al., "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127 063, 2024.

直接外推通常会失败，因为模型没有见过高位置的旋转角度。几种主流的解决方案：

位置插值（Position Interpolation）[9]：将长序列的位置缩放到训练时的范围。例如，32K 的位置映射到 $[0, 4K)$，相当于压缩了位置空间。需要少量微调来适应压缩后的位置表示。

NTK-aware 插值[10]：不是均匀缩放所有维度的频率，而是按频率的高低差异化缩放——高频维度（短距离模式）少缩放，低频维度（长距离模式）多缩放。这保留了局部位置精度，同时扩展了全局位置范围。NTK 名称来源于神经正切核（Neural Tangent Kernel）理论的类比。

YaRN（Yet another RoPE extensioN）进一步改进了 NTK-aware 插值，结合了温度缩放和频率分段调整。

LLaMA 3 和 DeepSeek-V3 都使用了某种形式的 RoPE 频率调整 来支持 128K+ 的上下文长度。

2.4 前馈网络与激活函数

Transformer 的每一层由两个子层组成：注意力层和前馈网络（FFN）层。如果说注意力层负责「让词与词交流」，那么 FFN 层则负责「在每个位置上独立地处理信息」。

2.4.1 激活函数的演进

激活函数的演进历史反映了深度学习社区对非线性建模的认知深化。

ReLU（2010s 主流）：$\mathrm{ReLU}(x) = \max(0, x)$。计算简单、训练快，但存在「死亡 ReLU」问题——一旦某个神经元的输入为负，梯度为零，该神经元永久失活。原始 Transformer 的 FFN 就使用 ReLU：$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$。

GELU（Gaussian Error Linear Unit, 2016）[11]：$\mathrm{GELU}(x) = x \cdot \Phi(x)$，其中 $\Phi(x)$ 是标准正态分布的累积分布函数。GELU 对负值不是硬截断，而是软性抑制，提供了更平滑的梯度。BERT 和 GPT-2 使用了 GELU。

SiLU/Swish（2017）[12]：$\mathrm{Swish}(x) = x \cdot \sigma(\beta x)$，其中 σ 是 sigmoid 函数，β 通常为 1。Swish 在数学上与 GELU 非常接近，但形式更简洁。Google Brain 通过自动搜索发现了这个激活函数。

SwiGLU（2020）[13]：将 Swish 与门控线性单元（GLU）结合：

$$\mathrm{SwiGLU}(x) = \big(\mathrm{Swish}(xW_1) \odot (xW_3)\big)\, W_2 \tag{2.8}$$

[9] S. Chen et al., "Extending context window of large language models via positional interpolation," *arXiv preprint arXiv:2306.15595*, 2023.

[10] bloc97, *NTK-aware scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and target length training*, Reddit r/LocalLLaMA, 2023.

[11] D. Hendrycks et al., "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[12] P. Ramachandran et al., "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.

[13] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

其中 $\text{Swish}(x) = x \cdot \sigma(x)$，$\odot$ 是逐元素乘法。SwiGLU 引入了一个门控机制——xW_3 控制哪些信息被通过，$\text{Swish}(xW_1)$ 提供非线性变换。

2.4.2 为什么门控机制有效?

门控机制的核心思想是**信息过滤**。在标准 FFN 中，所有信息经过相同的非线性变换。在门控 FFN 中，模型学习了一个「开关」——决定哪些维度的信息应该通过，哪些应该被抑制。这类似于 LSTM 中的遗忘门和输入门。

直觉上，语言模型在每个位置上需要做两件事：(1) 提取有用的特征，(2) 过滤无关的噪声。门控机制让模型显式地学习这两个步骤，而不是将它们混在一起。

Shazeer[14] 的实验表明，在控制参数量相同的条件下，SwiGLU 在多个基准上一致优于 ReLU 和 GELU。这个优势虽然不大（通常 1–3 个百分点），但在大规模训练中累积起来就是显著的差异。

为什么 FFN 的隐藏维度是 $8d/3$?

原始 Transformer 中 FFN 的隐藏维度是 $4d$（d 是模型维度）。FFN 有两个矩阵 $W_1 \in \mathbb{R}^{d\times 4d}$ 和 $W_2 \in \mathbb{R}^{4d\times d}$，总参数量为 $2 \times 4d^2 = 8d^2$。SwiGLU 引入了第三个权重矩阵 W_3，增加了约 50% 的参数。为了保持总参数量不变（$3 \times d \times d_{\text{ffn}} = 8d^2$），隐藏维度从 $4d$ 缩小到 $d_{\text{ffn}} = 8d/3 \approx 2.67d$。这样 SwiGLU 在相同参数预算下，性能优于更大的 ReLU FFN。实践中，这个值通常被取整到 256 的倍数 以便于 GPU tensor core 对齐，例如 LLaMA 3-8B 使用 $d = 4096, d_{\text{ffn}} = 14336$（恰好是 $3.5d$，略大于 $8d/3$）。

2.5 层归一化：训练的稳定器

深层神经网络的训练面临一个根本挑战：随着层数增加，各层的输入分布可能剧烈变化，导致训练不稳定甚至发散。

层归一化（Layer Normalization）通过对每一层的输入做标准化来缓解这个问题。现代 LLM 做了两个关键改进。

2.5.1 Pre-Norm vs Post-Norm

原始 Transformer 在注意力/FFN **之后**做归一化（Post-Norm）：

$$x' = \text{Norm}(x + \text{SubLayer}(x))$$

现代模型在**之前**做归一化（Pre-Norm）：

$$x' = x + \text{SubLayer}(\text{Norm}(x))$$

[14] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

这两种位置看似只是微小的差异，但影响深远。

Pre-Norm 的优势在于残差连接的梯度流。在 Pre-Norm 中，残差连接形成了一条从最后一层到第一层的「梯度高速公路」——$x' = x + f(\text{Norm}(x))$，其中 $\partial x'/\partial x = \boldsymbol{I} + \partial f/\partial x$。即使 $\partial f/\partial x$ 很小或为零，梯度也能通过恒等映射 $\boldsymbol{I}$ 畅通无阻地反向传播。这使得训练 100+ 层的深层模型成为可能。

Post-Norm 的优势在于归一化后的表示更稳定。由于归一化在残差连接之后，每一层的输出都被归一化到统一尺度，理论上表示能力更强。但代价是深层模型容易出现梯度消失或训练不稳定。

实践中，几乎所有现代 LLM 都选择 Pre-Norm——训练稳定性比微小的表示能力差异更重要。DeepSeek-V3 在 Pre-Norm 基础上进一步引入了 额外的辅助损失来防止训练崩溃。

2.5.2 RMSNorm：去掉均值，只留尺度

标准 LayerNorm 的计算过程包括两步：均值平移（center）和尺度归一化（scale）：

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta \tag{2.9}$$

其中 $\mu = \frac{1}{d}\sum_i x_i$ 是均值，$\sigma^2 = \frac{1}{d}\sum_i (x_i - \mu)^2$ 是方差，γ 和 β 是可学习的缩放和偏移参数。

RMSNorm[15] 去掉了均值平移操作，只做尺度归一化：

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \cdot \gamma \tag{2.10}$$

从数学上分析，LayerNorm 的均值平移操作 相当于将向量投影到垂直于全 1 向量 $\mathbf{1}$ 的超平面上。Zhang 和 Sennrich[16] 的研究表明，**归一化的主要贡献来自尺度不变性，而非均值平移**。去掉均值平移不影响模型质量，但节省了计算均值和减去均值的操作——在大规模模型中，这带来了约 10–15% 的归一化层加速。RMSNorm 还去掉了偏移参数 β，进一步简化了模型。

2.6 Flash Attention：让注意力更快

注意力机制的计算复杂度是 $O(n^2)$——对于长序列，这既是计算瓶颈，也是内存瓶颈。标准实现需要在 GPU 的高带宽内存（HBM）中存储完整的 $n \times n$ 注意力矩阵。

[15] B. Zhang et al., "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[16] B. Zhang et al., "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

2.6.1 GPU 内存层次结构

要理解 Flash Attention 为什么有效，必须先理解 GPU 的内存层次：

层级	容量	带宽
寄存器（Register）	~KB / SM	~TB/s
SRAM（片上缓存）	20 MB（A100）	~19 TB/s
HBM（高带宽内存）	80 GB（A100）	~2 TB/s

SRAM 的带宽是 HBM 的约 **10 倍**，但容量只有 HBM 的约 **1/4000**。标准注意力实现将 Q, K, V 和 $n \times n$ 注意力矩阵都存在 HBM 中，计算过程不断在 HBM 和 SRAM 之间搬运数据，而 GPU 的算术单元大部分时间在等待数据传输。这就是为什么标准注意力是 **memory-bound**（内存带宽受限）而非 compute-bound（计算受限）的。

2.6.2 核心算法：分块计算

Flash Attention[17] 的核心洞察是：**注意力计算是内存带宽瓶颈的（memory-bound），而非计算瓶颈的**。GPU 的算术运算速度远快于内存读写速度。标准实现在 HBM 和 SRAM 之间来回搬运数据的时间，远超实际的矩阵运算时间。

Flash Attention 的解决方案是**分块计算（tiling）**：

1. 将 Q 分成块 $Q_1, Q_2, \ldots$，将 K, V 也分成对应的块
2. 对于每个 Q 块，依次加载 K, V 的块到 SRAM
3. 在 SRAM 中完成 $Q_i K_j^\top$ 的计算、softmax、以及与 V_j 的乘法
4. 使用**在线 softmax**（online softmax）算法 增量地更新输出——不需要先计算整行的注意力分数再做 softmax，而是在处理每个 K, V 块时动态更新 softmax 的分母
5. 最终结果直接写回 HBM，中间的 $n \times n$ 注意力矩阵**从不存储**

结果是：Flash Attention 在数学上**完全等价**于标准注意力（不是近似！），但内存占用从 $O(n^2)$ 降至 $O(n)$，速度提升 2–4×。

在线 softmax 的关键

标准 softmax 需要两遍扫描：第一遍求分母 $\sum_j e^{s_j}$，第二遍归一化。在线 softmax 在一遍扫描中完成：维护一个运行中的最大值 m 和分母 l，每处理一个新块时更新：

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{block}}), \quad l_{\text{new}} = l_{\text{old}} \cdot e^{m_{\text{old}} - m_{\text{new}}} + l_{\text{block}} \cdot e^{m_{\text{block}} - m_{\text{new}}}$$

这使得分块计算在数学上精确等价于一次性计算。

[17] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

2.6.3 Flash Attention 的演进

Flash Attention 1[18]（2022）引入了基本的分块算法和 IO-awareness 概念。在 A100 GPU 上实现了 2–4× 的加速和显著的内存节省。

Flash Attention 2[19]（2023）在算法层面做了优化：

- 减少了非矩阵乘法操作的数量（这些操作在 GPU 上效率较低）
- 改进了 GPU 线程块和 warp 之间的并行化策略
- 将序列长度维度的并行化改为在注意力头和 batch 维度上并行
- 在 A100 上达到了理论峰值 FLOPS 的 50–73%，而 Flash Attention 1 仅达到 25–40%

Flash Attention 3（2024)[20] 针对 H100 GPU（Hopper 架构）的新特性做了深度优化。Flash Attention 2 的设计面向 Ampere 架构（A100），在 H100 上仅能达到约 35% 的利用率——远未充分利用 Hopper 的新硬件能力。Flash Attention 3 通过三项关键技术释放了 Hopper 的潜力：

（1）Warp 专业化与异步流水线。Hopper 架构引入了两个关键新特性：**WGMMA**（Warp Group Matrix Multiply Accumulate）指令 可以直接从共享内存执行矩阵乘法，**TMA**（Tensor Memory Accelerator）可以异步地在 HBM 和共享内存之间搬运数据。Flash Attention 3 将 warp 分为**生产者**和**消费者**两组：生产者 warp 使用 TMA 异步加载下一个数据块到共享内存，消费者 warp 使用 WGMMA 对当前数据块执行矩阵乘法。通过环形共享内存缓冲区，数据加载和计算可以完全**重叠**——GPU 不再有任何空闲周期等待数据。

（2）两阶段 GEMM-softmax 流水线（Ping-Pong 调度）。注意力计算中有两次矩阵乘法：$QK^\top$ 和 softmax $\times V$。传统实现必须等 $QK^\top$ 完成后才能开始 softmax，再等 softmax 完成后才能执行 $\times V$。Flash Attention 3 将相邻迭代的操作交错执行——在第 $j+1$ 次迭代的 $QK^\top$ 计算同时，执行第 j 次迭代的 softmax。这种 ping-pong 调度进一步隐藏了非矩阵乘法操作的延迟。

（3）FP8 块量化与不相干处理。H100 的 FP8 tensor core 提供高达 **1.98 PFLOPS** 的理论峰值。Flash Attention 3 利用**块量化**（block quantization）将 FP16 的 Q, K 在块级别转换为 FP8，配合**不相干处理**（incoherent processing）——通过随机正交变换使量化误差分布更均匀。结果是 FP8 Flash Attention 3 的数值误差 比 baseline FP8 注意力低 **2.6**×，吞吐量接近 **1.2 PFLOPS**。

---

[18] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[19] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[20] J. Shah et al., "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," *arXiv preprint arXiv:2407.08608*, 2024.

版本	GPU	精度	吞吐量	峰值利用率
标准注意力	A100	FP16	—	<20%
Flash Attention 1	A100	FP16	—	25–40%
Flash Attention 2	A100	FP16	—	50–73%
Flash Attention 3	H100	FP16	740 TFLOPS	~75%
Flash Attention 3	H100	FP8	~1.2 PFLOPS	~60%

Flash Attention 3 使得在 H100 上处理超长上下文变得实际可行——740 TFLOPS 的 FP16 吞吐量意味着 处理 128K 上下文的注意力计算时间 比 Flash Attention 2 在 A100 上快约 3–4×（考虑 H100 本身的硬件提升）。FP8 模式更是将吞吐量推到 PFLOPS 级别，为 1M+ token 的上下文处理开辟了道路。

截至 2026 年初，尚未有官方的 “Flash Attention 4” 发布，但 Flash Attention 3 已在 GTC 2025 上 被 Tri Dao 演示了面向 Blackwell（B200）架构的优化方向——包括利用 B200 的第五代 Tensor Core（FP4 支持）和更大的 SRAM（每 SM 提升至 32 MB 级别）。

Flash Attention 已经成为所有现代 LLM 训练和推理的基础设施，集成在 PyTorch 2.0+、Hugging Face Transformers、以及几乎所有主流训练框架中。值得注意的是，Flash Attention 的 IO-awareness 理念 也被应用到了 SSM 领域——Mamba-2 的 SSD 算法就采用了类似的分块策略 来最大化 GPU 硬件利用率。

2.7 MLA：DeepSeek 的注意力压缩

DeepSeek-V2 引入了**多头潜在注意力**（Multi-head Latent Attention, MLA），进一步解决 KV cache 的内存瓶颈。

2.7.1 动机与设计思想

GQA 通过共享 KV 减少了 KV 头的数量，但每个 KV 头的维度不变。MLA 的思路不同：**将 KV 投影到低维潜在空间**。

核心观察是：在高维空间中，K 和 V 矩阵存在大量冗余——它们的有效秩（effective rank）远低于矩阵维度。MLA 利用这个低秩结构，将 KV 压缩到一个低维的**潜在向量** c。

2.7.2 数学形式

具体而言，MLA 不直接缓存 K 和 V，而是缓存一个压缩的**潜在向量** c（维度远小于 K, V），在计算注意力时再将 c 投影回 K, V：

$$c = W_{\text{down}}\, x, \quad K = W_K^{\text{up}}\, c, \quad V = W_V^{\text{up}}\, c \tag{2.11}$$

其中 $W_{\text{down}} \in \mathbb{R}^{d_c \times d}$（$d_c \ll d$）是下投影矩阵。

关键的工程细节是：在推理时，W_K^{up} 可以被吸收到查询的投影矩阵中，使得注意力的计算可以直接在低维空间 d_c 中完成，无需将 c 显式展开为 K。这进一步减少了推理时的计算量。

2.7.3 压缩效果

通过将 KV 压缩到 d_c 维（DeepSeek-V2 中 $d_c = 512$，而原始 KV 维度为 $n_h \times d_h = 128 \times 128 = 16{,}384$），MLA 将 KV cache 减少到 GQA 的约 1/6，同时保持与 MHA 相当的质量。

方法	每 token KV cache 大小	相对大小
MHA（64 头）	$2 \times 64 \times 128 = 16{,}384$	1×
GQA（8 KV 头）	$2 \times 8 \times 128 = 2{,}048$	1/8
MQA（1 KV 头）	$2 \times 1 \times 128 = 256$	1/64
MLA（$d_c = 512$）	512	$\sim 1/32$

这在长上下文推理时尤为关键：处理 128K token 的上下文时，MLA 的 KV cache 仅为标准 MHA 的约 3%，使得在单机上处理超长上下文成为可能。DeepSeek-V3 和后续版本都采用了 MLA，这是 DeepSeek 系列模型在推理效率上领先的关键因素之一。

MLA 的限制

MLA 的低秩压缩假设了 KV 空间中的冗余性。对于需要极高注意力精度的任务（如需要精确回忆长文档中特定细节），压缩可能带来信息损失。DeepSeek 的解决方案是对 RoPE 部分单独处理——位置编码对应的 KV 分量不经过压缩，直接存储在 cache 中。这增加了少量额外存储，但确保了位置信息的完整性。

2.8 完整的 Transformer 块

让我们把所有组件组装在一起，看一个完整的现代 Transformer 块（如 LLaMA 3 风格）是什么样的：

Listing 2.2: A complete modern Transformer block

```python
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, n_kv_heads, ffn_dim):
        super().__init__()
        self.attn_norm = RMSNorm(d_model)
        self.attention = GQAttention(d_model, n_heads, n_kv_heads)
        self.ffn_norm = RMSNorm(d_model)
        self.ffn = SwiGLUFFN(d_model, ffn_dim)

    def forward(self, x, freqs_rope, mask=None):
        # Pre-Norm + GQA + Residual
```

```
        h = x + self.attention(
            self.attn_norm(x), freqs_rope, mask
        )
        # Pre-Norm + SwiGLU FFN + Residual
        out = h + self.ffn(self.ffn_norm(h))
        return out

class LLM(nn.Module):
    def __init__(self, vocab_size, d_model, n_layers, ...):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, d_model)
        self.layers = nn.ModuleList([
            TransformerBlock(d_model, ...) for _ in range(n_layers)
        ])
        self.norm = RMSNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False)

    def forward(self, tokens):
        x = self.embed(tokens)
        freqs = precompute_rope_freqs(...)
        for layer in self.layers:
            x = layer(x, freqs)
        x = self.norm(x)
        logits = self.head(x)
        return logits
```

这段代码展示了一个完整 LLM 的结构：词嵌入层将 token ID 映射为向量，经过多个 Transformer 块（每个块包含 RMSNorm + GQA + RMSNorm + SwiGLU FFN），最后通过线性层输出词表维度的 logits。这个 logits 经过 softmax 就是模型预测的下一个 token 的概率分布。

2.8.1 参数量分析

一个典型的 7B 参数模型（如 LLaMA 3-8B，实际 8.03B 参数）的配置为：$d_{\text{model}} = 4096$，$n_{\text{layers}} = 32$，$n_{\text{heads}} = 32$，$n_{\text{kv_heads}} = 8$（GQA），$d_{\text{ffn}} = 14336$（SwiGLU 的 $8d/3$），词表 128K。

将这些数字代入，让我们逐项计算参数量：

嵌入层：$128{,}000 \times 4096 \approx 0.52B$

每个 Transformer 块：

- 注意力层 Q 投影：$4096 \times 4096 = 16.8M$
- 注意力层 K 投影（GQA，8 头）：$4096 \times (8 \times 128) = 4.2M$
- 注意力层 V 投影（同 K）：$4.2M$
- 输出投影 W^O：$4096 \times 4096 = 16.8M$

- FFN W_1：$4096 \times 14336 = 58.7M$
- FFN W_3（SwiGLU gate）：$4096 \times 14336 = 58.7M$
- FFN W_2：$14336 \times 4096 = 58.7M$
- 2 个 RMSNorm（γ 参数）：$2 \times 4096 = 8K$
- 每块合计：$\sim 218M$

32 层合计：$32 \times 218M \approx 6.98B$

输出层：LLaMA 3-8B 使用**非共享**（untied）嵌入，输出投影层有独立的参数：$128{,}000 \times 4096 \approx 0.52B$

总计：$0.52B$（嵌入）$+6.98B$（Transformer 层）$+0.52B$（输出头）$+0.01B$（RMSNorm 等）$\approx 8.03B$

这与 LLaMA 3-8B 公布的 8.03B 参数吻合。注意：如果嵌入层与输出层共享权重（tied embeddings），参数量将减少约 0.5B 至 ~7.5B。LLaMA 3 选择 untied embeddings 以获得更好的模型质量。

2.8.2 三种架构风格

虽然代码展示的是 decoder-only 架构（GPT 风格），但 Transformer 实际上有三种主要变体：

Encoder-only（BERT 风格）：只有 encoder，使用双向注意力——每个 token 可以同时关注前面和后面的 token。适合理解型任务（分类、命名实体识别、问答）。通过 Masked Language Model（MLM）预训练。代表：BERT、RoBERTa、ALBERT。

Encoder-Decoder（T5 风格）：encoder 处理输入序列（双向注意力），decoder 生成输出序列（单向注意力 + 对 encoder 的交叉注意力）。适合序列到序列任务（翻译、摘要）。代表：T5、BART、原始 Transformer。

Decoder-only（GPT 风格）：只有 decoder，使用因果注意力（causal attention）——每个 token 只能关注它自己和前面的 token。通过 next-token prediction 预训练。代表：GPT 系列、LLaMA、DeepSeek、Claude。

为什么 decoder-only 最终胜出?

decoder-only 架构在生成任务上胜出有几个关键原因:

1. **训练效率**:在 next-token prediction 中,序列中的每个位置都是一个训练样本——n 个 token 提供 $n-1$ 个训练信号。而 MLM 只对随机遮盖的 15% token 计算损失。
2. **统一的接口**:所有任务都可以表述为「给定上文,生成下文」——不需要为不同任务设计不同的头(head)。理解和生成使用同一个模型,同一组参数。
3. **涌现能力**:in-context learning、chain-of-thought reasoning 等涌现能力 天然适合自回归生成的范式。
4. **规模优势**:在大规模预训练中,decoder-only 的简单性使得工程实现更高效,更容易扩展到万亿参数。

这并不意味着 encoder 架构消亡了——BERT 风格的模型在嵌入(embedding)和检索任务上仍然活跃。但对于通用语言模型,decoder-only 是 2024–2026 年的唯一选择。

Transformer 层的完整计算流程

一个现代 Transformer 层(如 LLaMA 3)的前向传播:

1. $x_1 = x + \mathrm{GQA}(\mathrm{RMSNorm}(x))$ // Pre-Norm + 分组查询注意力 + 残差连接
2. $x_2 = x_1 + \mathrm{SwiGLU}(\mathrm{RMSNorm}(x_1))$ // Pre-Norm + SwiGLU FFN + 残差连接

将这样的层堆叠 32–128 次,就构成了一个完整的 LLM。

2.9 Transformer 成功的根本原因

回顾完所有组件后,让我们思考一个更深层的问题:**为什么 Transformer 如此成功?** 在它之前,RNN(循环神经网络)和 CNN(卷积神经网络)也可以处理序列数据,为什么被 Transformer 完全取代了?

2.9.1 RNN/LSTM 的根本瓶颈

RNN 的瓶颈在于**顺序性**。RNN 逐步处理序列中的每个位置,第 t 步的计算必须等待第 $t-1$ 步完成。这不仅使得并行化困难(训练慢),更关键的是信息必须通过一个固定维度的**隐状态瓶颈** 在时间步之间传递——对于长序列,早期信息会被后来的信息覆盖。

虽然 LSTM 和 GRU 通过门控机制缓解了梯度消失问题,但信息衰减仍然是根本性的。LSTM 的记忆容量受限于隐状态维度(通常 512–2048),无论序列多长,所有信息都必须被压缩到这个固定大小的向量中。这就好比要求你用一张小纸条传递整本书的内容——必然会丢失大量信息。

更严重的是训练速度问题。在 RNN 中,n 个时间步的计算是**串行**的,无法被 GPU 的数千个核心并行处理。对于一个 100K token 的序列,RNN 需要 100K 步串行计算,

而 Transformer 的注意力可以在一步中完成所有位置的交互。这个并行化优势使得 Transformer 在相同硬件上的训练速度 比 RNN 快**一到两个数量级**。

2.9.2 CNN 用于序列建模的局限

CNN 的瓶颈在于**局部性**。卷积操作天然具有固定的感受野——每个位置只能「看到」前后若干个位置。要捕捉远距离依赖，需要堆叠很多层（每层扩大感受野），效率低下。

一个卷积核大小为 k 的 CNN，需要 $O(n/k)$ 层才能让两个相距 n 的位置直接交互。对于 $n = 128K$、$k = 5$ 的情况，需要约 25K 层——这在实践中完全不可行。

虽然有扩张卷积（dilated convolution）等技巧可以加速感受野增长，但 CNN 始终缺乏在两个任意位置之间建立直接连接的能力。

2.9.3 Transformer 的优势总结

Transformer 的优势在于它直接建模 **任意两个位置之间的关系**——注意力机制允许位置 1 直接与位置 1000 交互，不需要中间位置作为「中继」。而且注意力的计算是完全**并行**的——所有位置同时参与，充分利用 GPU 的并行计算能力。

特性	RNN/LSTM	CNN	Transformer
最大路径长度	$O(n)$	$O(\log_k n)$	$O(1)$
训练并行度	低（串行）	高（并行）	高（并行）
长距离依赖	差（信息衰减）	中（需要深网络）	强（直接注意力）
每层计算量	$O(n \cdot d^2)$	$O(k \cdot n \cdot d^2)$	$O(n^2 \cdot d)$

当然，Transformer 也有代价：$O(n^2)$ 的复杂度。但 Flash Attention 等技术在很大程度上缓解了这个问题。而且在实践中，Transformer 的训练效率（单位计算量学到的知识）远高于 RNN 和 CNN，因此即使单步计算量更大，总体效率仍然更优。

2.9.4 Attention is All You Need 的深远影响

Vaswani 等人[21] 发表的这篇论文影响了几乎所有的 AI 领域，远超其最初的机器翻译应用：

- **NLP**：BERT、GPT 系列、T5 等所有现代语言模型
- **计算机视觉**：Vision Transformer（ViT）证明 Transformer 可以替代 CNN 处理图像
- **语音**：Whisper 等模型用 Transformer 处理音频
- **蛋白质结构**：AlphaFold 2 的核心是 Transformer
- **多模态**：GPT-4V、Gemini 等多模态模型统一处理文本、图像、音频
- **科学计算**：天气预报（GraphCast）、分子设计、代码生成等领域

[21] A. Vaswani et al., “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.

这种跨领域的统治力是前所未有的——没有哪个单一架构曾经在如此多的领域中同时成为最优选择。

2.9.5 挑战者：状态空间模型

2024–2026 年，状态空间模型（State Space Model, SSM）从学术探索走向了工程实践，成为 Transformer 最有力的挑战者。以 Mamba[22]、RWKV[23] 为代表的 SSM 架构在长序列处理上具有 $O(n)$ 的复杂度优势，吸引了广泛的关注和跟进工作。

SSM 的核心原理

SSM 的核心是一个连续时间的线性动力系统：

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) \tag{2.12}$$

离散化后变成递归形式：$h_t = \bar{A}h_{t-1} + \bar{B}x_t$，$y_t = Ch_t$。关键在于：隐状态 h_t 的维度是**固定的**，不随序列长度增长。处理 1K 和 100K token 使用相同大小的隐状态——这就是 $O(n)$ 复杂度的来源。

Mamba（2023）的关键创新是**选择性机制**——矩阵 $\bar{B}$ 和 C 不再是固定参数，而是依赖于当前输入 x_t 动态生成的。这使得模型可以选择性地记住或遗忘信息，类似于 LSTM 的门控机制，但具有更高的并行化潜力。

Mamba-2：结构化状态空间对偶

Mamba-2[24]（2024）提出了 **结构化状态空间对偶**（Structured State Space Duality, SSD）理论，揭示了 SSM 和注意力之间的深层数学联系——两者本质上都是**结构化矩阵**的不同参数化形式。具体而言，SSD 证明了选择性 SSM 等价于一种特殊的半可分矩阵（semiseparable matrix），而标准注意力等价于一种因果掩码矩阵。

这个理论联系带来了直接的工程收益：

- **训练速度提升 2–8×**：Mamba-2 利用 SSD 的矩阵结构，将计算分解为可高效并行的块矩阵乘法，充分利用 GPU tensor core 的硬件特性
- **状态空间扩大 8×**：Mamba-1 的状态维度为 $N = 16$，Mamba-2 扩大到 $N = 128$——更大的状态意味着更强的信息存储能力，部分缓解了「信息瓶颈」问题
- **更优的并行化**：Mamba-2 的 SSD 算法天然支持 tensor parallelism，只需每层 1 次 all-reduce（与 Transformer 相同），而 Mamba-1 需要 2 次

[22] A. Gu et al., "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2024.

[23] B. Peng et al., "RWKV-7 "goose" with expressive dynamic state evolution," *arXiv preprint arXiv:2503.14456*, 2025.

[24] T. Dao et al., "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *International Conference on Machine Learning*, 2024.

RWKV-7：递归的另一条路线

RWKV[25] 是 Linux Foundation 支持的开源项目，走了与 Mamba 不同的技术路线。RWKV-7（代号 “Goose”，2025 年 3 月发布）引入了**广义 delta 规则**——一种具有向量门控和自适应学习率的状态更新机制。

RWKV-7 的关键特性：

- **恒定内存和推理时间**：与 Mamba 相同，每个 token 的推理复杂度为 $O(1)$（不随序列长度增长）
- **状态追踪能力**：RWKV-7 能执行状态追踪（state tracking）并识别所有正则语言——这超越了 Transformer 在标准复杂度猜想下的 TC^0 限制
- **多语言性能**：2.9B 参数的 RWKV-7 在多语言任务上达到 3B 级别 SoTA，尽管训练 token 数量（3.1T）远少于同级别的 Transformer 模型
- **训练可并行化**：尽管是递归架构，RWKV-7 的训练仍然可以通过分块并行化高效进行

混合架构：两者兼得

纯 SSM 模型在需要精确回忆（exact recall）的任务上 仍然弱于 Transformer——例如「在长文档中找到一个特定句子并逐字复制」。这是因为固定维度的隐状态 无法像注意力的 $n \times n$ 矩阵那样 保留对每个 token 的精确访问。

混合架构（Hybrid Architecture）因此成为了更实际的方向——在一个模型中交替使用注意力层和 SSM/线性递归层：

Jamba 1.5（AI21 Labs，2024，ICLR 2025）[26]：Transformer-Mamba-MoE 三者结合的混合架构。Jamba 1.5-Large 有 94B 活跃参数（MoE 总参数更大），Jamba 1.5-Mini 有 12B 活跃参数。两个版本都支持 256K token 的有效上下文长度——在发布时是开源权重模型中最长的。Jamba 的关键设计选择是**不均匀混合**——大约 1:7 的比例交替 Transformer 层和 Mamba 层，Transformer 层负责全局信息整合，Mamba 层负责高效的局部处理。实测吞吐量在长上下文场景下显著优于纯 Transformer 模型。

Zebra-LLaMA（AMD，2025）[27]：将 SSM 层和 MLA（多头潜在注意力）层交替组合，从已有的预训练 Transformer 权重初始化，通过少量后训练（post-training）达到接近 Transformer 的精度，同时获得接近 SSM 的推理效率。这种「从现有 Transformer 转换」的思路，大幅降低了混合架构的训练成本。

OLMo Hybrid（AI2，2026）[28]：用 Gated DeltaNet 层替换标准 Transformer 中的 滑动窗口注意力层，保留部分全局注意力层。7B 参数的 OLMo Hybrid 在标准预训练和中期

[25] B. Peng et al., “RWKV-7 “goose” with expressive dynamic state evolution,” *arXiv preprint arXiv:2503.14456*, 2025.

[26] O. Lieber et al., “Jamba: A hybrid transformer-mamba language model,” *arXiv preprint arXiv:2403.19887*, 2024.

[27] M. Yang et al., “Zebra-Llama: Towards extremely efficient hybrid models,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[28] W. Merrill et al., “OLMo hybrid: From theory to practice,” 2026, Allen Institute for AI technical report.

训练评估中 全面优于同规模的纯 Transformer 模型 OLMo 3。更重要的是，AI2 从理论上证明了 混合架构的表达能力**严格超过** 纯 Transformer 和纯线性 RNN——混合模型可以表达两者都无法单独表达的任务。

架构	推理复杂度	训练并行	精确回忆	代表模型
纯 Transformer	$O(n^2)$	高	强	GPT-4o, LLaMA 3
纯 SSM	$O(n)$	中–高	弱	Mamba-2, RWKV-7
混合架构	$O(n)$–$O(n^2)$	高	中–强	Jamba 1.5, OLMo Hybrid

SSM 在 frontier 模型中的采用现状

截至 2026 年 3 月，所有 frontier 模型（GPT-4.5、Claude、Gemini、DeepSeek-V3、LLaMA 4）仍然使用纯 Transformer 或 Transformer-MoE 架构作为主体。**没有任何 frontier 模型完全采用 SSM 层**。

但混合架构正在快速进入中小规模模型：Jamba 1.5、OLMo Hybrid、Nvidia Nemotron 3 Nano、IBM Granite 4、Qwen 3.5 等模型 都在不同程度上引入了线性递归层。Nathan Lambert 在 2026 年 3 月的分析中指出：混合架构正经历「到处同时被采用」的时刻[29]。

这场竞争的本质不是「SSM 替代 Transformer」，而是**最优的注意力与递归的混合比例是什么？** 一些初步的经验规则开始浮现：

- 大约 10–20% 的层使用全局注意力 足以保持精确回忆能力
- SSM/递归层在处理局部依赖 和长距离的「渐进式」信息整合时效率更高
- 混合架构在长上下文推理中的 KV cache 显著更小（SSM 层不需要 KV cache）

为什么 SSM 至今未能取代 Transformer?

SSM 的 $O(n)$ 复杂度优势是真实的，但面临几个实际障碍：

1. **硬件生态**：7 年来，整个 GPU 软件栈（CUDA 库、编译器、推理引擎）都围绕矩阵乘法和注意力优化。SSM 的扫描操作（scan）缺乏同级别的硬件支持
2. **工程惯性**：训练框架（Megatron、DeepSpeed）、推理引擎（vLLM、TensorRT-LLM）都深度优化了 Transformer 的执行路径。支持新架构需要大量工程投入
3. **精确回忆的差距**：在需要从长上下文中精确检索信息的任务上，注意力的 $n \times n$ 精确匹配 仍然优于 SSM 的压缩隐状态
4. **规模验证不足**：截至 2026 年初，最大的纯 SSM 模型仅有 ~3B 参数，而 frontier Transformer 已到万亿参数规模。SSM 在超大规模下的表现仍是未知数

未来的赢家大概率是混合架构——保留注意力的精确性，同时利用 SSM 的效率优势。

[29] N. Lambert, "OLMo hybrid and future LLM architectures," *Interconnects AI Blog*, 2026.

2.9.6 新型注意力变体

2024–2025 年，研究者们不仅在探索 Transformer 的替代品，也在改进注意力机制本身。两个值得关注的方向是 **差分注意力**（Differential Attention）和**原生稀疏注意力**（Native Sparse Attention）。

差分 Transformer（DIFF Transformer）

Microsoft Research 提出的 DIFF Transformer[30]（ICLR 2025 Oral）瞄准了注意力机制的一个根本缺陷：**标准注意力会将过多的权重分配给无关 token**。

即使在 softmax 归一化后，大量无关 token 的注意力分数虽然单个很小，但加起来仍然构成显著的噪声。这种「注意力噪声」导致模型容易被无关上下文干扰，是幻觉（hallucination）和 in-context learning 对示例顺序敏感等问题的根源之一。

DIFF Transformer 的解决方案在概念上优雅而简洁：**用两个独立的 softmax 注意力图之差来计算注意力**：

$$\mathrm{DiffAttn}(Q, K, V) = \left(\mathrm{softmax}\left(\frac{Q_1 K_1^\top}{\sqrt{d}} \right) - \lambda \cdot \mathrm{softmax}\left(\frac{Q_2 K_2^\top}{\sqrt{d}} \right) \right) V \tag{2.13}$$

其中 Q_1, Q_2, K_1, K_2 分别是查询和键的两个独立投影，λ 是可学习的标量参数。

这种差分操作的物理直觉类似于**降噪**——两个 softmax 注意力图的共同噪声模式在相减时被消除，保留下来的是真正有区分度的注意力信号。结果是注意力分布变得更加稀疏和精确，模型更能聚焦于真正相关的 token。

实验表明 DIFF Transformer 的优势：

- **参数效率**：DIFF Transformer 仅需约 65% 的模型参数 或训练 token 就能达到与标准 Transformer 相当的语言建模性能
- **幻觉缓解**：在问答和摘要任务中显著降低幻觉率——因为模型更少被无关上下文「误导」
- **ICL 鲁棒性**：in-context learning 的准确率更高，且对示例排列顺序的敏感度大幅降低——这曾被认为是注意力机制的顽疾
- **长上下文**：在长上下文建模和关键信息检索任务上表现更优
- **激活异常值减少**：注意力输出中的异常值更少，有利于量化和高效推理

后续工作 DEX[31]（NeurIPS 2025）进一步证明了差分注意力的优势可以**廉价地移植到已有的预训练模型**——通过在输出值矩阵上添加轻量级差分操作，仅需不到 0.01% 的适应数据 即可显著提升预训练 LLM 的性能。

[30] T. Ye et al., "Differential transformer," in *International Conference on Learning Representations*, 2025.

[31] C. Kong et al., "Understanding differential transformer unchains pretrained self-attentions," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

原生稀疏注意力（NSA）

DeepSeek 提出的 原生稀疏注意力（Native Sparse Attention, NSA）[32]（ACL 2025）从另一个角度优化注意力——**不是修改注意力的计算方式，而是减少参与注意力计算的 token 数量**。

传统的稀疏注意力方法（如 Longformer、BigBird）通常在推理时通过固定模式（如滑动窗口 + 全局 token）跳过部分注意力计算。但这些方法有两个根本问题：（1）固定的稀疏模式无法适应不同的输入内容；（2）训练时仍然使用全注意力，训练和推理的不一致导致质量损失。

NSA 的核心创新是**动态分层稀疏策略**，结合了三种互补的注意力分支：

1. **压缩分支**（Compression）：将连续的 token 块压缩为粗粒度的摘要表示，保持全局上下文感知
2. **选择分支**（Selection）：基于重要性评分动态选择关键 token，实现细粒度的信息检索
3. **滑动窗口分支**（Sliding Window）：保留局部上下文的完整注意力，确保短距离依赖不受影响

关键的工程突破是 NSA 可以**端到端训练**——不需要先用全注意力训练再蒸馏到稀疏模型。在 64K 长度的序列上，NSA 在解码、前向传播和反向传播 三个阶段都实现了对全注意力的显著加速，同时在通用基准、长上下文任务和指令推理上 保持或超过全注意力模型的性能。

注意力机制的演化谱系

从 2017 年的标准注意力到 2025 年，注意力机制已经形成了丰富的变体谱系：

- **MHA → GQA → MLA**：减少 KV cache 内存
- **Flash Attention 1 → 2 → 3**：利用 GPU 内存层次加速
- **DIFF Transformer**：差分降噪，提升注意力精度
- **NSA**：动态稀疏，减少参与计算的 token 数量
- **iRoPE**：交替有/无位置编码，改善长上下文外推

这些创新不是互斥的——NSA 可以与 MLA 结合（如 DeepSeek-V3.2 的 MLA + DSA），DIFF Transformer 可以与 Flash Attention 集成。未来的模型很可能会组合多种注意力优化。

2.10 词嵌入：让词进入向量空间

在 Transformer 的所有组件之前，还有一个基础但至关重要的步骤：**将离散的 token ID 转换为连续的向量**。这个过程由**嵌入层**（Embedding Layer）完成。

[32] J. Yuan et al., "Native sparse attention: Hardware-aligned and natively trainable sparse attention," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

嵌入层本质上是一个查找表——一个 $V \times d$ 的矩阵（V 是词表大小，d 是模型维度），每一行是一个 token 的向量表示。给定 token ID t，嵌入向量就是矩阵的第 t 行。

这些嵌入向量在训练过程中通过梯度下降学习。训练结束后，语义相近的词在向量空间中会距离较近——「国王」和「女王」的嵌入向量之间的距离 小于「国王」和「苹果」之间的距离。更有趣的是，这些向量之间会出现有意义的**方向**：经典的例子是 $\vec{\text{King}} - \vec{\text{Man}} + \vec{\text{Woman}} \approx \vec{\text{Queen}}$，说明嵌入空间捕获了「性别」的概念方向。

在一些 LLM 中，嵌入层和最后的输出层（将隐状态映射回词表维度）**共享权重**——这被称为 tied embeddings。直觉上，一个 token 的「输入表示」和「输出表示」应该是一致的。权重共享可以减少参数量（对于 128K 词表、4096 维模型，节省约 0.5B 参数）。但近年来的趋势是使用**非共享**（untied）嵌入——例如 LLaMA 3 系列就使用独立的输出投影层，实验表明这在大词表场景下能获得更好的模型质量。

嵌入层是否需要缩放?

在原始 Transformer 中，嵌入向量会乘以 $\sqrt{d}$。原因是：随机初始化的嵌入向量的模长（L^2 范数）与 $\sqrt{d}$ 成正比，而位置编码的模长固定为 1。如果不缩放，位置信息会在嵌入信息中被淹没。乘以 $\sqrt{d}$ 使两者在同一数量级。现代模型（使用 RoPE 而非加法位置编码）通常不需要这个缩放，因为 RoPE 直接作用于注意力计算而非嵌入空间。

CHAPTER 3

分词——教机器「认字」

在模型能够处理文本之前，必须将自然语言文本转换成模型可以操作的数字序列。这个过程叫做**分词**（Tokenization）。分词看似只是一个预处理步骤，但它对模型的性能、效率和多语言能力有着深远的影响——一个糟糕的分词器可以让一个优秀的模型变得平庸。

3.1 从词向量到子词：一段简史

在讨论现代分词方法之前，有必要回顾一下 NLP 领域是如何一步步走到今天的。这段历史揭示了一个核心矛盾——如何在词汇覆盖率和语义表达力之间找到平衡。

3.1.1 词向量的黄金时代

2013 年，Mikolov 等人提出了 Word2Vec，第一次大规模地证明了 **词的语义可以被编码在低维向量中**。Word2Vec 的核心思想简洁而深刻：「你可以通过一个词的邻居来了解它。」——一个词的含义由它频繁共现的词决定（分布假设）。

Word2Vec 训练出来的向量展现了令人惊叹的代数性质：$\vec{\text{King}} - \vec{\text{Man}} + \vec{\text{Woman}} \approx \vec{\text{Queen}}$。这个结果让整个 NLP 社区为之兴奋——词向量似乎捕获了某种深层的语义结构。

2014 年，Stanford 的 Pennington 等人提出了 GloVe（Global Vectors for Word Representation），通过对全局词共现矩阵做分解来学习词向量。GloVe 的优势在于它同时利用了局部上下文（像 Word2Vec）和全局统计信息（像传统的矩阵分解方法）。

然而，这些**静态词向量**有一个根本性的缺陷：每个词只有一个固定的向量表示，不管它出现在什么上下文中。“bank” 在「river bank」和「investment bank」中 得到完全相同的向量——这显然丢失了关键的语义区分信息。

3.1.2 上下文嵌入的崛起

2018 年，Allen AI 的 Peters 等人提出了 ELMo （Embeddings from Language Models），用双向 LSTM 语言模型为每个词生成 **依赖上下文的**向量表示。同一个 “bank” 在不同句子中会得到不同的嵌入。这是一个范式性的转变——从静态的字典式表示，走向动态的上下文式表示。

随后，BERT[1] 将上下文嵌入推向了新的高度：用 Transformer 替代 LSTM，在大规模

[1] J. Devlin et al., “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019.

语料上预训练，然后微调到下游任务。BERT 的成功不仅仅在于更好的嵌入质量，还在于它证明了一个重要的观点——**分词和嵌入可以被解耦**。模型不需要以完整的词作为输入单元，子词（subword）同样可以工作，甚至工作得更好。

3.1.3 未登录词问题

从 Word2Vec 到 ELMo，所有基于词级别的方法 都面临一个致命的问题：**未登录词**（Out-of-Vocabulary, OOV）。

词表是在训练时固定的。如果测试时遇到了训练语料中没有出现过的词（例如新出现的专有名词 “ChatGPT”、技术术语 “quantization”、或者拼写错误 “langauge”），模型只能将其映射为一个通用的 `<UNK>` token，丢失所有语义信息。

英语的形态变化加剧了这个问题：“play”、“plays”、“played”、“playing”、“player”、“playful” ——这些词共享词根 “play”，语义高度相关，但在词级别的词表中它们是完全独立的条目。词表要么包含所有变体（导致词表巨大），要么遗漏某些变体（导致 OOV）。

对于中文、日文、韩文等语言，问题又有不同的表现形式。中文没有明确的词边界——「研究生命的起源」可以被切分为「研究/生命/的/起源」或「研究生/命/的/起源」——分词歧义本身就是一个经典的 NLP 难题。

子词分词（subword tokenization）正是为解决这些问题而诞生的。它的核心洞察是：不要试图穷举所有的词，而是找到一组合适的**子词单元**，使得任何词都可以由这些子词拼接而成。

3.2 为什么不能直接用字或词?

让我们更系统地分析三种切分粒度的优劣。

字符级切分：每个字母（或汉字）作为一个 token。词表极小（英语仅需 ~256 个字节），永远不会遇到 OOV 问题。但代价是巨大的：字符层面的信息太稀疏了——模型需要从大量的字符序列中学习词的含义，效率极低。而且对于长文本，字符序列会非常长，大幅增加注意力机制的计算成本。以英语为例，一段 1000 词的文本 在字符级别约有 5000 个 token，而在子词级别只有约 1300 个。注意力的 $O(n^2)$ 复杂度意味着字符级的计算量是子词级的约 15 倍。

词级切分：「I love coding」→ [“I”, “love”, “coding”]。问题更严重：英语有数十万个词，而且新词不断出现（如 “ChatGPT”、“DeepSeek”）。按词切分的词表要么巨大无比，要么遇到未登录词就束手无策。对于中文、日文等语言，「什么是一个词」本身就是模糊的。更实际的问题是：巨大的词表意味着巨大的嵌入矩阵——一个 50 万词的词表配上 4096 维嵌入就需要 2B 参数，这比许多小模型的全部参数还多。

子词级切分——即 BPE 等方法所采用的——在两个极端之间找到了最优的平衡：高频词保留为完整的 token，低频词或未见过的词被拆分为子词片段。词表大小可控（32K–150K），同时保证零 OOV。

3.3 BPE：折中的智慧

3.3.1 从数据压缩到自然语言

字节对编码（Byte Pair Encoding, BPE）的历史 比大多数人想象的要久远得多。它最初由 Philip Gage 在 1994 年提出，是一种简单而高效的**数据压缩**算法：反复找到数据中出现最频繁的字节对，用一个新的字节替换它们，从而缩短数据长度。

2016 年，Sennrich 等人[2] 将这个想法引入了自然语言处理，用于解决机器翻译中的稀有词问题。从此，BPE 成为了几乎所有现代 LLM 的标准分词方法。

这个类比值得深思：BPE 本质上是在对语言做**压缩**——将高频模式编码为单个 token，低频模式则由多个 token 组合表示。这与 Shannon 的信息论完美契合：最优编码应该给高频符号分配短码，给低频符号分配长码。BPE 正是在做这件事——只不过它的「码」是 token，它的「符号」是自然语言中的子词片段。

3.3.2 算法详解

BPE 的算法流程如下：

1. 初始词表 = 所有单个字符（或字节）。
2. 在训练语料中统计所有相邻 token 对的频率。
3. 合并频率最高的一对，形成新的 token 加入词表。
4. 重复步骤 2–3，直到词表大小达到目标（通常 32K–150K）。

3.3.3 完整的合并示例

让我们用一个具体的例子来完整演示 BPE 的合并过程。假设训练语料中频繁出现四个词：“low”（5 次）、“lower”（2 次）、“newest”（6 次）、“widest”（3 次）。

首先，每个词被拆成字符序列，词尾加上特殊标记 `_`（表示词边界）。初始表示如下：

词（频次）	Token 序列
low (5)	`l o w _`
lower (2)	`l o w e r _`
newest (6)	`n e w e s t _`
widest (3)	`w i d e s t _`

第 1 轮：统计所有相邻 token 对的频率。`(e, s)` 出现 $6 + 3 = 9$ 次（来自 newest 和 widest），频率最高。合并 `e + s` → `es`。

[2] R. Sennrich et al., “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.

词（频次）	Token 序列
low (5)	`l o w _`
lower (2)	`l o w e r _`
newest (6)	`n e w es t _`
widest (3)	`w i d es t _`

第 2 轮：`(es, t)` 出现 $6+3=9$ 次，频率最高。合并 `es + t` → `est`。

词（频次）	Token 序列
low (5)	`l o w _`
lower (2)	`l o w e r _`
newest (6)	`n e w est _`
widest (3)	`w i d est _`

第 3 轮：`(est, _)` 出现 $6+3=9$ 次，频率最高。合并 `est + _` → `est_`。
第 4 轮：`(l, o)` 出现 $5+2=7$ 次，频率最高。合并 `l + o` → `lo`。
第 5 轮：`(lo, w)` 出现 $5+2=7$ 次，频率最高。合并 `lo + w` → `low`。
此时的状态：

词（频次）	Token 序列
low (5)	`low _`
lower (2)	`low e r _`
newest (6)	`n e w est_`
widest (3)	`w i d est_`

继续合并下去，“low” 最终会被编码为单个 token `low_`，“newest” 会被编码为两个 token `new` + `est_`，以此类推。这个过程完美地体现了 BPE 的核心思想：**高频模式被压缩为单个 token，低频模式由子 token 组合表示**。

3.3.4 代码实现

Listing 3.1: BPE core algorithm (simplified)

```python
def learn_bpe(corpus, vocab_size):
    # Start with character-level tokens
    vocab = set(all_characters_in(corpus))
    tokens = [list(word) for word in corpus]

    while len(vocab) < vocab_size:
        # Count all adjacent token pairs
        pair_counts = count_pairs(tokens)
        # Find the most frequent pair
        best_pair = max(pair_counts, key=pair_counts.get)
        # Merge that pair everywhere
```

```
        new_token = best_pair[0] + best_pair[1]
        vocab.add(new_token)
        tokens = merge_pair(tokens, best_pair, new_token)

    return vocab
```

这段代码展示了 BPE 的核心循环：统计频率——找到最高频对——合并——重复。`count_pairs` 遍历所有 token 序列，统计相邻 pair 的出现次数。`merge_pair` 将语料中所有该 pair 的出现替换为合并后的新 token。

3.3.5 时间复杂度分析

BPE 训练的时间复杂度值得分析。假设语料有 N 个 token，目标词表大小为 V。每一轮合并需要：(1) 遍历所有 token 对并统计频率——$O(N)$；(2) 找到最高频对——$O(|\text{pairs}|)$，可用优先队列优化到 $O(\log|\text{pairs}|)$；(3) 在语料中执行替换——最坏 $O(N)$。总共需要 V 轮合并，因此总复杂度为 $O(VN)$。

对于典型的规模（$V = 100\text{K}$，$N = 10^{10}$），直接实现会非常慢。实际的工业级实现（如 HuggingFace 的 `tokenizers` 库）使用了大量优化技巧：基于前缀树的高效合并、增量更新频率计数、Rust 实现的底层运算。这些优化使得在数十 GB 的语料上 训练一个 BPE 分词器只需要几个小时。

3.4 WordPiece 与 Unigram：其他选择

BPE 并不是唯一的子词分词算法。还有两种重要的替代方案：Google 的 WordPiece 和 SentencePiece 中的 Unigram 模型。三者的核心区别在于**合并策略**。

3.4.1 WordPiece：基于似然的合并

WordPiece 最初由 Google 为语音识别系统开发，后来被 BERT[3] 采用而广为人知。

BPE 选择合并**频率最高**的 token 对，而 WordPiece 选择合并后 **使语言模型似然提升最大**的 token 对。具体来说，对于候选合并对 $(a, b) \to ab$，WordPiece 计算的得分是：

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \times \text{freq}(b)} \tag{3.1}$$

这个得分衡量的是 a 和 b 共同出现的频率 相对于它们各自独立出现的频率——本质上是**互信息**（pointwise mutual information）的一种近似。

直觉上，BPE 倾向于合并**绝对频率**高的对，而 WordPiece 倾向于合并**关联性**强的对。例如，“t” + “h” 在英文中的绝对频率极高（因为 “th” 出现在大量词中），BPE 会优先合

[3] J. Devlin et al., “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019.

并它。但 “u” + “n” 虽然绝对频率不如 “th” 高，它们共现的条件概率可能更高（“un-” 作为前缀非常稳定），WordPiece 可能会优先合并它。

3.4.2 Unigram 模型：从大到小的剪枝

Unigram 模型（由 Kudo 于 2018 年提出）采用了与 BPE/WordPiece 完全相反的策略：**从一个很大的初始词表开始，逐步删除 token**。

初始词表通常包含所有字符和最频繁的子字符串（可能有数十万个条目）。然后，Unigram 反复计算每个 token 被移除后 对整体语言模型似然的影响——移除后似然下降最少的 token 被剪掉。这个过程重复直到词表缩小到目标大小。

Unigram 的一个独特优势是它支持**多种分词方式**。对于同一段文本，可能存在多种合法的分词方案，Unigram 模型为每种方案分配一个概率。这允许在训练时使用**子词正则化**（subword regularization）——随机采样不同的分词方案，增加训练数据的多样性，类似于数据增强（data augmentation）的效果。

3.4.3 三种方法的对比

特征	BPE	WordPiece	Unigram
方向	自底向上（合并）	自底向上（合并）	自顶向下（剪枝）
合并依据	频率最高	似然提升最大	似然损失最小
分词确定性	确定性	确定性	概率性
子词正则化	不支持	不支持	原生支持
代表模型	GPT 系列 LLaMA, Mistral Qwen	BERT	T5

在实践中，三种方法在最终效果上的差异通常不大——词表大小和训练数据的选择对性能的影响远大于合并策略的选择。这也是为什么不同实验室选择不同算法的原因：差异不大，所以选择更方便实现的那个就好。

3.5 词表大小：一个微妙的权衡

词表大小是一个重要的超参数。不同模型的选择差异巨大，背后反映了不同团队的设计哲学。

模型	词表大小	分词方式	说明
GPT-2 (2019)	50,257	BPE	早期 BPE，纯英文优化
LLaMA 1 (2023)	32,000	SentencePiece	偏小，中文效率低
LLaMA 2 (2023)	32,000	SentencePiece	同 LLaMA 1
Mistral 7B (2023)	32,000	SentencePiece	同 LLaMA 1
Gemma 2 (2024)	256,000	SentencePiece	Google，超大词表
GPT-4 (2023)	100,256	tiktoken	多语言 + 代码
LLaMA 3 (2024)	128,256	tiktoken 风格	大幅扩展，多语言
DeepSeek-V3 (2024)	128,000	BBPE	强化多语言和代码
Gemma 3 (2025)	262,144	SentencePiece	目前最大的公开词表
Qwen3 (2025)	151,643	tiktoken 风格	中文优化，CJK 效率高
LLaMA 4 (2025)	202,048	tiktoken 风格	200 语言，多模态 token

从 32K 到 262K：词表大小演进的逻辑

这张表揭示了一个清晰的趋势：词表大小从 2023 年的 32K 迅速扩展到 2025 年的 150K–262K。背后的驱动力有三个：(1) **多语言**——LLaMA 4 声称支持 200 种语言，需要足够大的词表来为每种语言提供合理的 fertility；(2) **多模态**——LLaMA 4 的词表中 包含了图像 token（`<|image|>`）和其他视觉 token，词表 ID 从 200,000 开始分配给特殊 token；(3) **代码**——Rust、Mojo 等新语言的关键词和运算符 需要更大的词表来避免被拆分为多个子词 token。Google 的 Gemma 3 更是直接使用 262K 的超大词表——证明对于足够大的模型，嵌入层的额外开销可以被忽略。

词表太小：每个文本需要更多的 token 表示（因为更多词被拆成子词），输入序列变长，注意力计算更慢，模型需要从更长的上下文中学习。一个量化这个问题的指标是 **fertility**（生育率）——平均每个词被切分成多少个 token。fertility 越接近 1，分词效率越高。

词表太大：嵌入矩阵（$V \times d$）消耗大量参数和内存。稀有 token 的嵌入得不到充分训练。对于小模型，嵌入层可能占总参数的很大比例。

近年来的趋势是增大词表：GPT-2 用 50K，LLaMA 1 用 32K，LLaMA 3 增大到 128K，Qwen3 更是用到了 150K。这是因为更大的词表能更高效地编码多语言文本（每个中文词可以用更少的 token 表示），而现代模型的参数量足够大，嵌入层占比相对较小。

为什么大词表能帮助中文?

在小词表（32K）下，许多中文常用词会被拆成 2–3 个 token，例如「人工智能」可能被切成「人工” + “智” + “能」。而在 150K 词表下，「人工智能」可以作为一个完整的 token 存在。这不仅节省了序列长度（效率提升），也让模型能直接学习这个词的完整语义（而不是从子词中拼凑），提升了模型对中文的理解能力。

具体的数字可以说明这个差异。LLaMA 1（32K 词表）处理中文时，平均 fertility 约为 2.5——即每个中文词平均被切成 2.5 个 token。同样的中文文本在 Qwen3（150K 词表）下，fertility 降至约 1.2。这意味着相同的上下文窗口可以容纳约 2 倍的中文内容——对中文理解能力的提升是直接且显著的。

fertility 的隐含成本

fertility 不仅影响理解能力，还直接影响**推理成本**。API 服务按 token 计费——如果你的语言 fertility 是 2.5（即同等内容需要 2.5 倍的 token），那么你每次调用的成本也是 fertility 为 1 的语言的 2.5 倍。对于使用 LLaMA 1 分词器的模型处理中文，用户在成本上受到了系统性的「不公平对待」。这也是为什么中文 AI 社区强烈推动大词表的原因之一。

3.6 SentencePiece：工业级实现

SentencePiece 是 Google 开源的分词工具[4]，被 LLaMA、Qwen、DeepSeek、Mistral 等主流模型采用。它不仅是一个分词算法的实现，更是一个**完整的分词框架**——同时支持 BPE 和 Unigram 两种算法，提供训练、编码、解码的全套功能。

3.6.1 语言无关性

SentencePiece 的核心特点是将输入视为**原始 Unicode 字节流**，不依赖任何预分词（pre-tokenization）步骤。

传统的 BPE 实现（如 GPT-2 的 tokenizer）需要先按空格和标点将文本切分为「词」，然后在词内部做 BPE 合并。这种设计对英文友好（空格是天然的词边界），但对中文、日文等没有空格的语言不够自然。

SentencePiece 直接在字节级别操作：将所有文本编码为 UTF-8 字节序列，然后在字节序列上学习 BPE（或 Unigram）合并规则。这意味着它可以处理**任何语言**——甚至是 emoji、数学符号和未知的 Unicode 字符——因为一切最终都是字节。

[4] T. Kudo et al., “SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” *Proceedings of EMNLP: System Demonstrations*, 2018.

3.6.2 字节回退机制

SentencePiece 的一个关键设计是**字节回退**（byte fallback）机制。当遇到词表中不存在的 Unicode 字符时，SentencePiece 不会输出 `<UNK>`，而是将该字符拆解为其 UTF-8 字节表示，用特殊的字节 token（如 `<0xE4>`）来编码。

这意味着 SentencePiece 的分词结果 **永远不会包含未知 token**——任何输入都能被编码和解码。这对于处理用户输入至关重要，因为用户可能输入任何字符——生僻汉字、emoji、甚至是乱码。

3.6.3 确定性与可复现性

SentencePiece 的设计还带来一个重要的工程优势：分词器的输出是**确定性的**——同一段文本总是产生相同的 token 序列，与操作系统、locale 设置和文本预处理无关。这在分布式训练中至关重要，因为不同 GPU 上的分词结果必须完全一致。

3.6.4 训练与使用

使用 SentencePiece 训练一个分词器非常简单：

Listing 3.2: Training a SentencePiece tokenizer

```python
import sentencepiece as spm

# Train a BPE tokenizer with 32K vocab
spm.SentencePieceTrainer.train(
    input='training_corpus.txt',
    model_prefix='my_tokenizer',
    vocab_size=32000,
    model_type='bpe',    # or 'unigram'
    character_coverage=0.9995, # cover 99.95% of characters
    byte_fallback=True, # enable byte fallback
    split_digits=True,  # split each digit separately
    num_threads=16,
)

# Use the trained tokenizer
sp = spm.SentencePieceProcessor(model_file='my_tokenizer.model')
tokens = sp.encode('Hello world!', out_type=str)
# ['_Hello', '_world', '!']
ids = sp.encode('Hello world!', out_type=int)
# [12345, 2048, 67]
text = sp.decode(ids)
# 'Hello world!'
```

注意 `character_coverage=0.9995` 这个参数——它指定词表应覆盖训练语料中

99.95% 的字符。剩下的 0.05% 极稀有字符将通过字节回退处理。对于以 CJK 字符为主的语料，这个值通常需要设得更高（如 0.99995），因为 CJK 字符集本身就很大。

3.7 特殊 Token 的设计

现代 LLM 的词表不仅包含自然语言 token，还包含一系列**特殊 token**，用于控制模型的行为。

3.7.1 基础特殊 Token

- `<bos>`（begin of sequence）：标记序列的开始。
- `<eos>`（end of sequence）：标记序列的结束，模型在生成时遇到它就停止。
- `<pad>`（padding）：用于将不同长度的序列填充到相同长度，以便组成 batch。

3.7.2 对话模板 Token

随着 LLM 从文本补全转向多轮对话，对话模板（chat template）的设计变得至关重要。不同模型使用不同的对话格式：

ChatML 格式（OpenAI/Qwen）：

```
<|im_start|>system
You are a helpful assistant.
<|im_end|>
<|im_start|>user
What is BPE?
<|im_end|>
<|im_start|>assistant
BPE stands for Byte Pair Encoding...
<|im_end|>
```

LLaMA 3 格式：

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>

You are a helpful assistant.
<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is BPE?
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

这些特殊 token 在预训练阶段不存在——它们是在 SFT（监督微调）阶段被添加到词表中的。模型通过 SFT 学会这些 token 的含义：`<|im_start|>` 表示一条消息的开始，`<|`

im_end|> 表示消息结束。

3.7.3 推理与工具调用 Token

2024–2025 年的一个重要趋势是 为特定功能设计专用的特殊 token：

- <think> / </think>：DeepSeek-R1[5] 和 Qwen3[6] 用于标记推理过程的 token，推理内容包在这对标签内，不展示给用户。
- <tool_call> / </tool_call>：标记工具调用请求——模型输出结构化的函数调用信息。
- <tool_response> / </tool_response>：标记工具返回的结果，模型根据这些结果继续生成回复。
- <|code|>：某些模型用于标记代码块的开始，触发代码补全模式。

这些特殊 token 的嵌入在 SFT 阶段被学习，使模型理解「到这里该停了」或「接下来是推理过程」等控制信号。它们的设计看似简单，但直接影响了模型在多轮对话、工具调用、推理模式等场景中的行为。

特殊 Token 设计的原则

好的特殊 token 设计需要满足几个条件：(1) **不与自然语言冲突**——用 <|...|> 这样的格式是因为 它几乎不可能出现在正常文本中；(2) **语义明确**——模型和后处理代码都能明确判断每个 token 的含义；(3) **向后兼容**——添加新的特殊 token 不应破坏已训练的模型对旧 token 的理解。这也是为什么添加特殊 token 通常伴随着一轮新的 SFT。

3.8 分词的陷阱与挑战

分词看似已经是一个「解决了的问题」，但在实践中仍然存在许多令人头疼的陷阱。

3.8.1 Glitch Token：分词器的「幻觉」

2023 年，研究人员发现了一个诡异的现象：当向 GPT 系列模型输入某些特定的 token（如 “SolidGoldMagikarp”）时，模型会表现出极不正常的行为——重复输出无意义的文本、拒绝回答问题、甚至产生与输入完全无关的内容。

这些被称为 **glitch tokens**（故障 token）。产生的原因是：在 BPE 训练时，这些字符串在训练语料中频繁出现（“SolidGoldMagikarp” 是一个 Reddit 用户名，出现在大量被爬取的 Reddit 帖子中），因此被编码为单独的 token。但在模型的预训练数据中，这些 token 可能很少或从未出现在有意义的上下文中——它们的嵌入向量几乎没有被训练过，停留在接近随机初始化的状态。

[5] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.

[6] Qwen Team, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.

这个故事的教训是：**分词器和模型的训练数据必须对齐**。如果分词器的训练语料和模型的预训练语料差异过大，就会产生大量「存在于词表中但模型不理解」的 token。

3.8.2 数学与数字的困境

分词对模型的数学能力有着直观但容易被忽视的影响。考虑 “12345” 这个数字：在某些分词器下，它可能被切分为 “123” + “45”、“1” + “234” + “5”、或者 “12345” 整体。不同的切分方式意味着模型「看到」的数字表示是不同的——它甚至无法可靠地判断一个数字有几位。

这就是为什么许多现代分词器选择 **将每个数字字符独立为一个 token**：“12345” → [“1”, “2”, “3”, “4”, “5”]。虽然这增加了序列长度，但让模型能够在字符级别理解数字的结构，从而提升算术能力。SentencePiece 的 `split_digits=True` 选项正是为此设计的。

数字位置嵌入：Abacus Embeddings

即使将数字逐位切分，模型的算术能力仍然有限——尤其在面对训练时未见过长度的数字时（例如训练 20 位加法，推广到 100 位加法）。McLeish 等人（2024）发现了问题的根源：Transformer 的标准位置编码无法让模型 可靠地追踪每个数字在一个长数字中的**相对位置**（个位、十位、百位……）。

他们提出了 **Abacus Embeddings**——为每个数字字符添加一个额外的嵌入，编码该数字字符**相对于数字起始位置**的偏移量。直觉上，这就像给每一位数字加上了「我是个位」「我是十位」「我是百位」的标签。

结果令人印象深刻：仅在 20 位数字上训练一天（单 GPU），模型在 100 位加法上达到了 99% 的准确率。这个结果有深刻的启示——**分词和嵌入的设计可以弥补（或加剧）Transformer 在特定能力上的不足**。数学能力不完全是「更多训练数据」的问题，有时候只需要告诉模型数字的正确结构。

数字比较的陷阱

分词还导致了另一个反直觉的问题：数字大小比较。许多 LLM 会错误地判断 “9.11 > 9.9”——因为 “.11” 和 “.9” 是不同的 token，模型无法直接比较它们的数值大小。这个问题的根源在于：分词器将小数点后的数字视为文本片段而非数值，“0.11” 中的 “.11” 可能被合并为一个 token，而 “0.9” 中的 “.9” 是另一个 token——模型看到的是两个完全不同的符号，失去了数值比较的基础。

Kreitner 等人（2024）提出了 **xVal**，一种直接将数字编码为单个 token 并附带浮点数值嵌入的方法，在科学计算任务上显著提升了数字处理精度。

这些研究共同指向一个结论：**BPE 是为自然语言设计的压缩算法，不是为数学设计的**。数字有独立于语言频率的内在结构（位值系统），强行用统计频率来决定数字的切分方式，从根本上就是一个范畴错误。

3.8.3 代码中的空白问题

Python 使用缩进来标记代码块层次。一个常见的问题是：分词器如何处理空格和制表符?

如果 4 个空格被编码为一个 token“　　”，而 2 个空格被编码为另一个 token“　”，那么 8 个空格的缩进就是两个“　　”token。但如果分词器的合并规则不同，8 个空格可能被切分为“　”+“　　”+“　”（3 个 token）——这种不一致会让模型难以学习正确的缩进模式。

GPT-4 和 LLaMA 3 的分词器 为常见的空格序列（1 个到 16 个空格）专门设置了独立的 token，避免了这个问题。同样，制表符和换行符也被设为独立 token。这些看似微小的设计决策，对模型的代码生成能力有着显著的影响。

3.8.4 新编程语言的分词挑战

随着 LLM 被用于越来越多的编程语言，分词器面临着新的挑战——尤其是那些在分词器训练语料中出现较少的新兴语言。

Rust 是一个典型的例子。Rust 的语法中包含大量复合运算符 和在其他语言中少见的符号组合：`::`（路径分隔符）、`->`（返回类型标记）、`=>`（匹配分支）、`&mut`（可变引用）、`<'a>`（生命周期标注）。在以 Python 和 JavaScript 为主训练的分词器中，这些 Rust 特有的符号组合可能未被合并为独立 token，而是被拆分为多个单字符 token——增加了序列长度，也增加了模型学习 Rust 语法的难度。

以生命周期标注 `<'a, 'b>` 为例，它可能被分为 7 个 token（`<, ', a, ,, ', b, >`），而在 Rust 开发者眼中这是一个不可分割的语义单元。更麻烦的是 Rust 的宏系统（如 `vec![1, 2, 3]`）——`vec!` 中的感叹号在大多数语言中不出现在标识符后，分词器会将其与标识符拆开，丢失了「这是一个宏调用」的关键信息。

Mojo（Modular 推出的高性能 ML 语言）面临更大的挑战：它几乎不存在于 2024 年之前的训练语料中。Mojo 的语法混合了 Python 和 Rust 的特征——`fn`（函数定义）、`let`（不可变绑定）、`var`（可变绑定）、`struct`、`trait`——这些关键词虽然与 Rust 相同，但 Mojo 独有的类型注解语法（如 `DType.float32`）和 SIMD 操作（如 `SIMD[DType.float32, 4]`）在训练语料中几乎不存在，导致模型生成 Mojo 代码时错误率显著偏高。

新语言的恶性循环

新编程语言面临一个恶性循环：(1) 训练语料中缺乏该语言的代码，分词器无法为其优化 token 合并规则；(2) 分词效率低导致模型生成该语言代码的质量差；(3) 质量差导致开发者不使用 LLM 辅助编写该语言代码，进一步减少了高质量训练数据的来源。打破这个循环的方法通常是 在微调阶段大量注入特定语言的高质量代码数据——但这需要分词器的词表中 已经包含该语言的常见 token，否则效果仍然有限。

3.9 前沿探索：超越传统分词

3.9.1 Tiktoken：速度的极致

2022 年，OpenAI 开源了 tiktoken——一个用 Rust 编写的高性能 BPE 分词器。tiktoken 的设计哲学是**推理速度优先**：它使用正则表达式进行预分词（pre-tokenization），将文本先切分为较大的块，然后在每个块内做 BPE 匹配。这种设计使得 tiktoken 的编码速度 比 Python 实现快 3–6 倍。

GPT-4 和 GPT-4o 都使用 tiktoken。LLaMA 3 也从 SentencePiece 切换到了 类似 tiktoken 的 BPE 实现。

3.9.2 无分词器模型：字节级别的终极方案?

一个更激进的想法是完全抛弃分词器，直接在字节级别训练模型。Meta 的 MegaByte（2023 年）是这种思路的早期代表：它将输入视为原始字节序列，用一个大模型处理字节块，用一个小模型在块内生成单个字节。

字节级模型的潜在优势非常诱人：

- 完全消除分词器——简化整个训练和部署流程。
- 真正的语言无关——任何语言、任何编码都是字节。
- 消除分词导致的信息损失和偏见。

BLT：动态分配计算的字节级模型

2024 年 12 月，Meta 发表了 **Byte Latent Transformer**（BLT），标志着字节级模型在实用性上迈出了关键一步。BLT 的核心创新是**动态 patch 分割**——它不使用固定大小的字节块（如 MegaByte 的做法），而是根据**下一个字节的熵**来决定 patch 的边界。

直觉上，这种设计非常自然：

- 当文本可预测时（例如常见的英文单词中间），熵低，patch 变长——用更少的计算处理简单内容。
- 当文本不可预测时（例如代码中的变量名、罕见词），熵高，patch 变短——分配更多计算给复杂内容。

BLT 的架构由三个组件构成：一个轻量的**字节编码器**将字节序列编码为 patch 表示，一个大型的**潜在 Transformer** 在 patch 级别进行推理，一个轻量的**字节解码器**将 patch 表示还原为字节预测。重型计算（注意力、FFN）只在 patch 级别发生，而 patch 的平均长度约为 4–6 个字节——接近 BPE token 的平均长度，因此计算成本可以与基于 token 的模型竞争。

Meta 的实验结果令人瞩目：在 8B 参数、4T 训练字节的规模上，BLT 首次在 FLOP 控制的对比中 **匹配了基于 token 的 LLaMA 模型**的性能，同时在推理效率上有所提升（因为简单文本使用更长的 patch，减少了前向传播次数）。更重要的是，BLT 展现了更好

的**长尾泛化能力**——在拼写、音素等字符级任务上 显著优于基于 BPE 的模型。

BLT 的意义

BLT 证明了字节级模型的可行性不再是理论上的。它的关键洞察是：**计算资源应该根据内容的复杂度动态分配，而不是像 BPE 那样用固定的词表来决定**。这与人类阅读的模式惊人地一致——我们扫过熟悉的词很快，在遇到生僻词或复杂概念时则放慢速度仔细理解。截至 2026 年初，BLT 的代码已开源，但尚未有 frontier 级别的模型采用纯字节级架构。这个方向值得密切关注。

3.9.3 多模态分词：图像也是 Token

随着多模态大模型（如 GPT-4V、Gemini）的兴起，一个自然的问题是：能不能把图像也「分词」成一组离散 token，与文本 token 统一处理?

答案是肯定的。VQ-VAE（Vector Quantized Variational Autoencoder）和 VQGAN 等方法可以将图像编码为 一组离散的码本索引——每个索引就是一个「图像 token」。例如，一张 256×256 的图像 可以被编码为 $16 \times 16 = 256$ 个图像 token。这些 token 与文本 token 共享同一个 Transformer，实现了真正的多模态统一建模。

这种方法的代表包括：Google 的 Parti 和 Gemini、Meta 的 Chameleon。它们展示了一个激动人心的可能性：**所有模态——文本、图像、音频、视频——都可以被「分词」为离散 token，然后用同一个 Transformer 进行统一建模**。分词不再只是 NLP 的预处理步骤，而是多模态 AI 的基础设施。

Emu3：纯 next-token prediction 的多模态统一

2024 年，北京智源人工智能研究院（BAAI）发布了 **Emu3**，这是多模态分词领域的一个里程碑式工作。Emu3 证明了一个强有力的命题：**仅靠 next-token prediction，不需要扩散模型或组合架构，就可以同时实现多模态感知和生成**。

Emu3 的方法概念上非常简洁：将图像、文本和视频统一编码为离散 token 序列，然后用一个标准的 Transformer decoder 做 next-token prediction。图像通过改进的 VQVAE 编码为 $32 \times 32 = 1024$ 个视觉 token，视频则被编码为帧序列中的视觉 token 流。在推理时，生成图像就是自回归地生成视觉 token，然后用 VQVAE decoder 解码回像素。

Emu3 在图像生成上超越了 SDXL，在图像理解上与 LLaVA-1.6 持平——这是第一次有纯自回归模型在两个方向上同时达到领先水平。这一结果发表在 *Nature*（2025）上，标志着多模态分词从实验性概念走向了成熟验证。

UniTok：统一视觉分词器

多模态分词面临的一个技术挑战是 **生成和理解对视觉 token 的需求不同**。图像生成需要高保真度的像素重建（偏向 VQVAE 式的离散编码），而图像理解需要高级语义特征（偏向 CLIP 式的连续编码）。

ByteDance 和香港大学的 **UniTok**（2025）尝试在一个统一的分词器中同时满足两种需求。UniTok 的核心创新是在训练视觉分词器时 同时优化重建损失和语义对齐损失，使得同一组离散 token 既能重建图像，也能承载语义信息。这个方向代表了多模态分词的未来趋势——不是为每个任务设计专门的分词器，而是用一个统一的分词器覆盖所有需求。

音频和视频的分词

音频和视频的分词也在快速发展。音频方面，**SoundStream**（Google）和 **EnCodec**（Meta）使用残差向量量化（Residual Vector Quantization, RVQ）将音频波形编码为多层离散 token。Gemma 3n（2025）直接以 6.25 tokens/秒 的速率 处理音频输入——这意味着 1 分钟的音频仅需要 375 个 token，远低于将音频转录为文本的 token 开销。

视频则是最具挑战性的模态。一段 10 秒、30fps 的 256×256 视频，如果每帧编码为 256 个 token，总共需要 $300 \times 256 = 76{,}800$ 个 token——即使对于 10M 上下文窗口的 LLaMA 4 Scout 也是一个不小的负担。视频分词的关键是**时间冗余压缩**——相邻帧之间的变化通常很小，只需要编码变化的部分，类似于视频编解码器中 I 帧和 P 帧的关系。

多模态分词的统一愿景

多模态分词的发展正在走向一个统一的架构：**所有模态共享同一个 token 空间**。文本 token 和图像 token 混合在同一个序列中，模型在预训练时同时学习所有模态的模式。LLaMA 4 已经在词表中预留了多模态 token 的位置（图像 token ID 为 200092），Gemini 使用了类似的设计。这种统一表示的终极目标是——模型不再需要知道输入的模态是什么，因为一切都已经是 token。

CHAPTER 4

数据工程——万亿 TOKEN 的炼金术

如果说 Transformer 是 LLM 的大脑，那么数据就是它的食粮。一个 LLM 的质量上限在很大程度上取决于训练数据的质量和多样性。2025–2026 年的共识越来越清晰：**数据质量远比数据数量重要**。

这不是一句空洞的口号。微软的 Phi 系列模型用不到 100B 的精选高质量 token 训练出了与万亿 token 模型竞争的结果，而大量研究表明在未经过滤的网页数据上 增加训练量带来的收益迅速递减。数据工程——从原始互联网爬取数据到 可以喂给模型的高质量 token 流——是 LLM 训练中最不起眼却最关键的环节。

4.1 数据来源

训练一个 frontier LLM 需要万亿级别的 token。这些数据来自哪里？让我们逐一审视每种数据来源的特性、规模和挑战。

4.1.1 网页数据：最大的宝藏与垃圾场

Common Crawl 是几乎所有 LLM 训练数据的源头。这个非营利组织自 2008 年起持续爬取互联网，每月发布一个新的爬取快照，累积了数百 PB 的原始 HTML 数据，覆盖超过 **2500 亿个网页**。每个月的快照以 WARC（Web ARChive）格式存储，包含完整的 HTTP 响应（HTML、头信息等），同时提供 WET 格式（仅提取文本内容）和 WAT 格式（仅元数据）。

但原始 Common Crawl 的质量极差——充满了导航栏、广告、重复文本、SEO 垃圾、色情内容和机器生成的胡言乱语。直接使用原始 Common Crawl 训练模型 得到的效果远不如在清洗后的子集上训练。因此，围绕 Common Crawl 衍生出了一系列 精心策划的高质量数据集：

- **C4**（Colossal Clean Crawled Corpus）：Google 为 T5 模型创建，对 Common Crawl 2019 年 4 月快照进行清洗，得到约 750GB / 156B token 的英文文本。清洗规则相对简单——去除含脏词的页面、过短的行、含 “lorem ipsum” 的页面等。
- **mC4**：C4 的多语言版本，覆盖 100+ 种语言。
- **RefinedWeb**（Falcon 团队）：对 Common Crawl 进行了更精细的清洗和去重，得到约 5T token，证明了仅用网页数据（不混入书籍或代码）也能训练出高质量模型。

- **FineWeb**[1]（HuggingFace）：目前最大、最高质量的公开网页数据集。对 96 个 Common Crawl 快照（2013–2024 年）进行了精细清洗，得到了 **15T token** 的高质量英文文本。FineWeb-Edu 是其教育内容子集（1.3T token），在多个基准测试上的训练效果 甚至优于使用全量 FineWeb。
- **DCLM**（DataComp-LM，Allen AI 等）：通过系统性的受控实验筛选最优的过滤策略，从 Common Crawl 中提取了 3T+ token 的高质量数据。
- **FineWeb-2**（HuggingFace，2024 年 12 月）：FineWeb 的多语言继任者。FineWeb-2 将 FineWeb 的英文处理流水线 推广到 **1000+ 种语言**，产生了约 3T 词（压缩后约 8TB）的多语言高质量数据。其核心贡献在于解决了多语言数据处理中的独特挑战——不同语言需要不同的质量过滤阈值、低资源语言的语言识别准确率较低、某些书写系统（如阿拉伯文、天城文）需要特殊的预处理规则。FineWeb-2 的论文（2025 年 6 月）详细记录了这些挑战和解决方案，是构建多语言预训练数据的重要参考。
- **Nemotron-CC**（NVIDIA，2025 年 1 月）：NVIDIA 构建的万亿 token 级英文预训练数据集。Nemotron-CC 的独特之处在于其 **分类器集成 + 合成数据改写**的双重策略：首先使用多个质量分类器的集成（而非单个分类器）来评估文档质量，然后对中等质量的文档使用 LLM 进行**改写**（rephrasing），将其提升为高质量版本。实验显示，在 Nemotron-CC 上训练的 8B 参数模型 在多项任务上超越了 LLaMA 3.1 8B。这种「改写提质」的策略 为数据质量工程提供了一个新的维度——**不只是过滤掉低质量数据，还可以主动改善数据质量**。

4.1.2 书籍：长文本的稀缺宝藏

书籍提供长篇连贯的高质量文本，尤其对模型的长文本生成和叙事能力有帮助。书籍文本的质量通常远高于网页——经过专业编辑和出版流程的审核。

GPT-3[2] 的训练使用了 Books1 和 Books2 两个书籍数据集（共约 67B token），但 OpenAI 从未公开其具体来源。The Pile（EleutherAI 的开源数据集）包含了 Books3 子集——约 19.7 万本书的全文，来自盗版电子书网站。这引发了巨大的版权争议，多位作家和出版社对此提起了诉讼，Books3 最终被从公开发布的版本中移除。

版权问题使得合法获取大量书籍数据变得困难。目前的替代方案包括：Project Gutenberg（公共领域的经典书籍，约 7 万本），以及与出版商的授权合作（但成本高昂且规模有限）。书籍数据在整体训练数据中的占比通常为 5–10%——数量不多，但对模型质量的贡献不成比例地高。

[1] G. Penedo et al., "The FineWeb datasets: Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024.

[2] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

4.1.3 代码：推理能力的秘密武器

代码是一个令人意外的关键数据源。来自 GitHub、GitLab 等平台的开源代码不仅教会模型编程，还能提升它的**通用推理能力**。代码具有严格的逻辑结构——语法错误一个字符就会导致程序崩溃——这种精确性迫使模型学习更严谨的推理模式。Qwen3[3] 在 36T token 中 纳入了大量代码数据，正是为了利用这一效应。

代码数据的主要来源包括：

- **The Stack v1/v2**（BigCode / HuggingFace）：从 Software Heritage 存档（全球最大的源代码存档）中提取，覆盖 600+ 种编程语言。The Stack v2 包含约 67TB 的源代码。关键的设计决策是**许可证过滤**——只包含使用宽松开源许可证（如 MIT、Apache 2.0、BSD）发布的代码，排除了 GPL 等具有 copyleft 要求的许可证。
- **GitHub 直接爬取**：许多团队直接从 GitHub API 获取代码。StarCoder 的数据流水线是一个优秀的参考：它对 GitHub 代码进行了多层过滤——去除自动生成的代码（如 `package-lock.json`）、去除过长或过短的文件、去除高比例的非 ASCII 字符、近似去重（代码的复制粘贴现象比文本更严重）。

代码数据的处理还需要特别注意 **个人身份信息**（PII）的清理——代码注释和配置文件中经常包含 开发者的邮箱、API 密钥、甚至密码。这些必须在训练前被检测和移除。

4.1.4 学术论文

学术论文（来自 arXiv、PubMed、Semantic Scholar 等）提供高质量的科学知识和数学推理内容。

- **arXiv**：包含约 200 万篇论文的 LaTeX 源代码，是数学和科学推理数据的主要来源。直接使用 LaTeX 源代码（而非 PDF 提取的文本）能保留公式的精确结构——这对模型学习数学推理至关重要。
- **S2ORC**（Semantic Scholar Open Research Corpus）：包含约 8100 万篇学术论文的全文和元数据，覆盖几乎所有学科。
- **PubMed / PMC**：生物医学领域的论文全文，约 3600 万篇摘要和 900 万篇全文。

PDF 到文本的转换是学术论文数据处理中的一大挑战。PDF 是一种**面向排版**的格式，不保留文档的逻辑结构。表格、公式、多栏布局、脚注——这些在 PDF 中的表示与人类阅读的理解完全不同。Nougat（Meta 推出的 PDF-to-Markdown 工具）和 GROBID 等工具试图解决这个问题，但准确率仍然有限，尤其是在数学公式和复杂表格上。这也是为什么直接使用 arXiv 的 LaTeX 源码 比使用 PDF 提取的文本效果更好。

4.1.5 对话与社交媒体

对话数据（来自论坛、社交媒体等）帮助模型学习自然的交流模式。Reddit 是最常用的来源——其 Pushshift 存档包含了数十年的帖子和评论，线程结构的讨论数据对模型学习

[3] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

对话和论证能力特别有价值。但 2023 年后 Reddit 限制了 API 访问，使得新数据的获取变得困难。

Stack Overflow 和其他 Q&A 平台也是重要来源——问题和被采纳答案的配对 天然构成了高质量的问答训练数据。

4.1.6 策划型数据集

2023–2025 年，社区逐渐认识到 仅仅从单一来源提取数据是不够的——需要**精心策划**的多源混合数据集。代表性的工作包括：

- **RedPajama**（Together AI）：复现 LLaMA 的训练数据组成，包含 1.2T token，来自 Common Crawl、Wikipedia、书籍、arXiv、GitHub 和 Stack Exchange。
- **SlimPajama**（Cerebras）：在 RedPajama 基础上做了更彻底的去重和清洗，将数据量从 1.2T 精简到 627B token——数据量减半但训练效果更好。
- **Dolma**（AI2）：Allen AI 为 OLMo 模型构建的 3T token 数据集，特点是**完全透明**——所有的数据处理步骤、过滤规则、混合比例都详细记录并公开发布。

4.2 数据清洗流水线

原始数据到训练数据之间，是一条复杂的清洗流水线。让我们以 FineWeb[4] 的流水线为例来详细说明每个步骤。

4.2.1 文本提取与规范化

首先要从 HTML 中提取纯文本。这不是简单的「去掉标签」——需要保留有意义的结构（标题、段落、列表），去除导航栏、页脚、侧边栏等「模板」文本。

trafilatura 是目前最流行的网页文本提取工具。它使用基于规则和启发式的方法来区分「正文内容」和「模板/导航」——分析 DOM 树的结构、文本密度、标签语义（`<article>` vs. `<nav>`）等。在 FineWeb 的评估中，trafilatura 的提取质量 显著优于其他工具（如 jusText、readability）。

文本提取之后还需要做**规范化**：统一 Unicode 编码（NFC/NFKC 规范化）、移除控制字符和零宽空格、统一引号和破折号的形式、修复编码错误（mojibake）。这些看似琐碎的步骤对后续的去重和质量过滤 有着不可忽视的影响——如果两篇相同的文章使用了不同的 Unicode 编码形式，去重算法可能无法检测到它们的重复。

4.2.2 语言识别

多语言清洗流水线的第一步是 判断每个文档的语言。**fastText** 的语言分类器 是最广泛使用的工具——它可以识别 176 种语言，在句子级别的分类准确率超过 95%。

[4] G. Penedo et al., "The FineWeb datasets: Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024.

语言识别的一个微妙之处是 **混合语言**文档的处理。许多网页（尤其是非英语国家的技术博客）混合使用英语和本地语言。简单地按文档级别分类会丢失这些有价值的混合语言内容。FineWeb 采用的策略是：对每个文档做段落级别的语言识别，只要主要语言（>65%）是目标语言就保留。

4.2.3 去重

互联网上的内容重复非常严重——同一篇新闻可能被数百个网站转载。如果不去重，模型会在重复内容上浪费大量训练计算，更严重的是，它可能直接记住这些文本而非学习泛化的模式。

研究表明，Common Crawl 中约 **30–40%** 的内容是近似重复的。Lee 等人（2022）的研究发现，去重可以带来与增加 3–4 倍训练数据 等价的模型质量提升——这使得去重成为数据处理中 投入产出比最高的步骤之一。

精确去重

精确去重是最简单的形式：对每个文档计算哈希值（如 SHA-256），哈希值相同的文档被视为完全重复。

URL 去重是精确去重的一个变体——对于来自同一 URL 的多次爬取（Common Crawl 的不同月度快照可能爬取同一网页），只保留最新的版本。FineWeb 在处理 96 个快照时，URL 去重就移除了约 40% 的数据。

精确去重的局限很明显：它无法检测到只有微小差异的文档——例如，同一篇文章被不同网站转载时 可能添加了不同的广告文本或页脚。

模糊去重：MinHash + LSH

模糊去重（fuzzy deduplication）使用 MinHash + LSH（局部敏感哈希）找到高度相似但不完全相同的文档。这是整个数据处理流水线中最精巧的算法之一，值得详细理解。

第一步：Shingling（构建 *n*-gram 集合）。将每个文档表示为其所有连续 n 个词组成的片段（称为 shingle）的集合。例如，对于文档 “the cat sat on the mat”，使用 3-gram shingling：

{“the cat sat”, “cat sat on”, “sat on the”, “on the mat”}

FineWeb 使用 5-gram shingling。n 的选择是一个权衡：n 太小会导致过多的误报（不相关的文档也可能共享短片段），n 太大会导致过多的漏报（相似文档可能没有足够长的共享片段）。

第二步：Jaccard 相似度。两个文档的相似程度用 Jaccard 系数衡量：

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{4.1}$$

其中 A 和 B 是两个文档的 shingle 集合。如果 $J(A, B) > 0.8$，我们认为这两个文档是近似重复的。

但直接计算所有文档对的 Jaccard 相似度需要 $O(n^2)$ 次比较——对于数十亿个文档，这是不可行的。

第三步：MinHash 签名。 MinHash 用一组 k 个随机哈希函数 将每个文档映射为一个长度为 k 的「签名」。具体做法是：对每个哈希函数 h_i，计算文档所有 shingle 在 h_i 下的哈希值，取**最小值**作为签名的第 i 个分量：

$$\mathrm{sig}_i(A) = \min_{s \in A} h_i(s) \tag{4.2}$$

MinHash 的关键数学性质是：

$$\Pr\left[\mathrm{sig}_i(A) = \mathrm{sig}_i(B)\right] = J(A, B) \tag{4.3}$$

即两个签名在第 i 个位置相同的概率 **恰好等于**它们 shingle 集合的 Jaccard 相似度。

为什么 MinHash 能近似 Jaccard?

这个等式的证明非常优雅。考虑集合 $A \cup B$ 中的所有 shingle。每个 shingle 属于三种情况之一：(a) 仅在 A 中，(b) 仅在 B 中，(c) 在 $A \cap B$ 中。一个随机哈希函数 h 给每个 shingle 一个随机值，min 操作选中的那个 shingle 等可能是 $A \cup B$ 中的任何一个。如果它恰好落在 $A \cap B$ 中，则 $\mathrm{sig}(A) = \mathrm{sig}(B)$。这发生的概率恰好是 $|A \cap B|/|A \cup B| = J(A, B)$。

使用 k 个独立的哈希函数，签名匹配的比例就是 $J(A, B)$ 的一个无偏估计，误差随 k 增大而减小（标准差为 $O(1/\sqrt{k})$）。FineWeb 使用 $k = 128$，意味着 Jaccard 相似度的估计误差约为 ± 0.09。

因此，我们只需比较签名（k 个整数），不需要比较原始的 shingle 集合（可能有数万个元素）。但即使比较签名，$O(n^2)$ 的文档对数仍然太多。

第四步：LSH（Locality-Sensitive Hashing）。 LSH 将长度为 k 的签名分成 b 个 band，每个 band 包含 r 个值（$k = b \times r$）。对每个 band，对其 r 个值再做一次哈希，只有在**至少一个 band 上完全匹配**的文档对 才会被标记为候选相似对，然后做精确比较。

通过调整 b（band 数量）和 r（每个 band 的行数），可以精确控制「召回率 vs. 误报率」的权衡。两个 Jaccard 相似度为 s 的文档被标记为候选对的概率是：

$$P(\text{candidate} \mid J = s) = 1 - (1 - s^r)^b \tag{4.4}$$

这个函数呈 S 形曲线。当 $b = 20, r = 5$（即 $k = 100$）时：

- $s = 0.8$ 的文档对被检测到的概率 ≈ 0.997（几乎全部召回）
- $s = 0.5$ 的文档对被检测到的概率 ≈ 0.47（约一半误报）
- $s = 0.2$ 的文档对被检测到的概率 ≈ 0.006（几乎不误报）

FineWeb 使用 $b = 14, r = 8$（$k = 112$），相似度阈值约 0.75——在去重召回率和计算成本之间取得了较好的平衡。

整个 MinHash + LSH 流水线将 $O(n^2)$ 的所有文档对比较 降低到近似线性的复杂度——这使得在数十亿文档上做模糊去重变得实际可行。

后缀数组去重

除了 MinHash + LSH 这种文档级别的去重，还有一种更精细的去重方式：**后缀数组去重**（suffix array deduplication）。Lee 等人（2022）提出了这种方法，LLaMA 团队[5]也采用了它。

后缀数组去重在**子字符串**级别工作：它找到语料中所有重复出现的长子字符串（通常 ≥ 50 个字符），然后只保留一个副本，其余删除。这种方法可以检测到 **段落级别的重复**——例如，许多网页可能共享相同的免责声明、版权声明或 cookie 通知。

后缀数组的构建可以在 $O(n)$ 时间内完成（使用 SA-IS 等算法），但需要的内存约为文本大小的 5–8 倍。对于 TB 级别的语料，这需要分布式实现和外部排序。

去重的层次

总结来说，一个完整的去重流水线通常包含多个层次：

层次	方法	检测目标
URL 去重	精确匹配	同一 URL 的多次爬取
文档精确去重	SHA-256 哈希	完全相同的文档
文档模糊去重	MinHash + LSH	高度相似的文档
段落/子串去重	后缀数组	共享的段落或模板

4.2.4 质量过滤

去重之后，仍然有大量低质量文本需要清除。质量过滤是整个流水线中最需要「工程直觉」的环节——什么是「高质量」并没有一个公认的数学定义。

基于规则的启发式过滤

最直接的方法是设计一组规则来过滤明显的低质量内容。以 Gopher 模型（DeepMind）的过滤规则为代表：

- **文档长度**：过短（< 50 词）或过长（> 100K 词）的文档被移除。
- **平均词长**：平均词长 < 3 或 > 10 的文档被移除（可能是乱码或非自然语言文本）。
- **特殊字符比例**：如果文档中 非字母数字字符超过 30%，被移除。
- **重复行比例**：如果超过 30% 的行是重复的，被移除（可能是导航菜单或表格的残留）。

[5] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

- **大写字母比例**：如果超过 70% 的字母是大写的，被移除（可能是全大写的标题或垃圾邮件）。
- **停用词比例**：如果英语停用词（“the”、“a”、“is”……）的比例低于 5%，被移除（可能是关键词列表而非自然文本）。
- **特征字符串检测**：包含 “lorem ipsum”、“javascript is disabled”、“cookie policy” 等特征字符串的文档被移除。

这些规则看似简单粗暴，但效果出奇地好——它们可以在几乎不损失高质量内容的情况下 移除大量明显的垃圾。

基于模型的质量过滤

更精细的过滤使用**机器学习模型**来判断文档质量。

困惑度过滤（Perplexity filtering）：使用一个在高质量文本（如 Wikipedia）上 训练的小型语言模型（如 KenLM）计算每个文档的困惑度。直觉上，高质量的文本（如维基百科文章）在语言模型眼中是「正常」的，困惑度较低；而低质量的文本（如乱码或机器生成的垃圾）具有异常高的困惑度。CCNet（Meta 的 Common Crawl 清洗工具）就是基于这种方法，将文档按困惑度分为「头部」（高质量）、「中部」和「尾部」（低质量）三个等级。

但困惑度过滤也有陷阱：它倾向于保留「类似 Wikipedia」的文本，可能过度偏向百科全书式的写作风格，而丢弃同样有价值的其他风格（如技术教程、个人博客、论坛讨论）。

分类器过滤：GPT-3[6] 的训练 使用了一个二分类器来判断文档是否「高质量」——正样本来自 Wikipedia 和 WebText（Reddit 上高赞帖子链接的网页），负样本来自原始 Common Crawl。LLaMA[7] 也采用了类似的策略。

FineWeb[8] 的关键创新之一 是使用了多轮、多方法的质量过滤，包括基于规则的启发式过滤 和基于模型的打分过滤。特别是 FineWeb-Edu 子集使用了一个 专门训练的**教育质量分类器**——这个分类器评估文档是否具有教育价值，而不仅仅是「语言质量是否高」。结果表明，1.3T token 的 FineWeb-Edu 在多项基准测试上的训练效果 甚至优于使用全量 15T token 的 FineWeb。这是「质量 > 数量」的最有力的实证之一。

安全过滤

NSFW/毒性过滤是数据清洗的重要一环。模型如果在色情、暴力、仇恨言论等内容上训练，可能会在推理时生成类似的有害内容。

常用的工具包括：

[6] T. Brown et al., “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[7] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.

[8] G. Penedo et al., “The FineWeb datasets: Decanting the web for the finest text data at scale,” *arXiv preprint arXiv:2406.17557*, 2024.

- **Jigsaw Perspective API**：Google 开发的毒性评分工具，可以为文本标注「毒性」、「侮辱性」、「威胁性」等维度的分数。
- **自训练分类器**：使用少量标注数据微调一个小模型（如 fastText）来检测 NSFW 和毒性内容。FineWeb 使用了这种方法，对检测到的有害内容进行移除或降权。

PII 移除（个人身份信息）也是安全过滤的一部分：

- 基于正则表达式的检测：邮箱地址、电话号码、身份证号、信用卡号等 具有固定格式，可以用正则表达式匹配和移除。
- 基于命名实体识别（NER）的检测：人名、地址等没有固定格式的 PII 需要使用 NER 模型来识别。

4.3 数据配比：一门精细的艺术

确定了数据来源和清洗流水线之后，还有一个关键问题：不同类型的数据应该以什么比例混合？

4.3.1 经验法则与实验结果

这并不是一个简单的优化问题。LLaMA 3[9] 的训练数据配比经过了大量实验。据社区根据技术报告推测的数据配比大致为：约 50% 网页文本、25% 代码、10% 学术论文、10% 书籍、5% 数学数据（官方未公布精确比例）。

一些令人惊讶的发现：

- 增加代码数据的比例可以提升模型在**非代码任务**（如数学推理和逻辑推理）上的表现。
- 增加数学数据的比例在超过某个阈值后 反而会降低模型的通用能力。
- 高质量数据可以被重复使用若干次（通常 2–4 epochs）而不会导致严重的过拟合。

代码数据提升推理能力这一发现值得深入解释。代码具有几个独特的特征使其成为推理能力的催化剂。首先，代码有**严格的逻辑结构**——一个括号没有匹配就会导致语法错误，一个逻辑条件写错就会导致程序行为完全不同。这种精确性迫使模型学习更严谨的逻辑推理。其次，代码包含大量**抽象和组合**——函数调用、类继承、模块导入——这些正是推理能力的核心组成部分。第三，代码的**执行结果是确定的**——不像自然语言那样模糊，代码的正确性可以通过执行来验证，为模型提供了更清晰的学习信号。

4.3.2 真实模型的配比参考

不同模型的数据配比策略差异很大，反映了各团队的设计哲学和优势领域：

[9] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

模型	总量	网页	代码	学术/书籍	数学	其他
LLaMA 3	15T	50%	25%	15%	5%	5%
DeepSeek-V3	14.8T	40%	30%	10%	10%	10%
Qwen3	36T	–	大量	–	大量	–
LLaMA 4	40T	–	大量	–	–	多模态

（各模型的详细配比大多未完全公开，上表中的百分比为根据技术报告的描述推断。LLaMA 4 在 40T token 中包含了 200 种语言的数据，且原生支持图像输入，暗示多模态数据在训练配比中占据了可观的份额。）

值得注意的是，DeepSeek-V3[10] 将代码占比提高到约 30%——这可能是其在数学和推理任务上表现优异的原因之一。而 LLaMA 4 的 40T token 训练数据中覆盖了 200 种语言——这是迄今为止语言覆盖最广的 frontier 模型，反映了 Meta 的全球化部署策略。

4.3.3 自动数据配比：DoReMi 与后继者

手动调整数据配比需要大量的试错实验，成本极高。2023 年，Xie 等人提出了 **DoReMi** 算法，试图用自动化的方式找到最优配比。

DoReMi 的核心思想是：先用一个小型的「代理模型」（proxy model）在不同配比下快速训练，通过**分布式鲁棒优化**（Distributionally Robust Optimization, DRO）找到使最差表现领域也能获得良好学习效果的配比，然后用这个配比训练大模型。

实验表明，DoReMi 找到的配比 在下游任务上的表现 比均匀配比提升了约 6%，而且这种提升在不同规模的模型上都成立。

RegMix：将配比优化建模为回归问题

2025 年（ICLR），Liu 等人提出了 **RegMix**，将数据配比问题重新建模为一个**回归任务**。

RegMix 的方法分三步：(1) 在不同的数据配比上训练大量小模型（512 个 1M 参数的模型，各用 1B token）；(2) 将每个配比作为输入、训练后的模型性能作为输出，拟合一个回归模型来预测未尝试配比的性能；(3) 用回归模型预测的最优配比来训练大规模模型。

RegMix 的关键优势在于**效率**：它仅使用 DoReMi 约 10% 的计算资源，就能在 100B token 训练的 7B 模型上 匹配或超越 DoReMi 的表现。一个令人意外的发现是：回归分析表明，**网页语料**（而非通常被认为更高质量的 Wikipedia）与下游性能有最强的正相关——这挑战了「高质量数据 = 类 Wikipedia 数据」的朴素假设。

MDE：混合数据专家模型

Google DeepMind 在 2025 年提出了 **Mixture of Data Experts**（MDE）方法，进一步提升了配比优化的精度。MDE 为每种数据领域训练一个小型「专家模型」，然后通过这些专家模型的加权组合 来高效近似任意配比下的模型损失。这种近似作为回归模型的额外特征，使得配比预测的准确性显著提升。

[10] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.

Frontier 模型的数据策略

虽然 frontier 模型（GPT-4o、Claude 3.5、Gemini 2.0、DeepSeek-V3、Qwen3、LLaMA 4）的详细数据配比通常不公开，但从技术报告和社区分析中可以推断一些趋势：

- **代码占比持续增加**：从 LLaMA 1 的 ~5% 到 LLaMA 3 的 ~25% 再到 DeepSeek-V3 的 ~30%。LLaMA 4 在 40T token 的训练数据中 据推测代码占比进一步提高。
- **多阶段配比调整**：几乎所有 frontier 模型都在训练后期（annealing 阶段）提高高质量数据的比例。这已经从「实验性策略」变成了「标准做法」。
- **数学/推理数据成为独立类别**：不再被归入「学术论文」，而是作为独立的数据源（包含大量合成数学数据）在配比中占据 5%–15% 的份额。

4.3.4 课程学习

另一种思路是**课程学习**（Curriculum Learning）——不是在整个训练过程中使用固定的数据配比，而是在不同训练阶段使用不同的数据。

直觉类比：人类学习也是从简单到复杂——先学字母，再学单词，然后学句子，最后读文章。课程学习将类似的思想应用到 LLM 训练中。

一种常见的策略是：

1. **早期阶段**（前 20% 的训练）：使用高质量、语言简单、结构清晰的数据（如 Wikipedia、教科书）。
2. **中期阶段**（20%–80%）：逐步引入更多样化的数据（网页、代码、论文）。
3. **晚期阶段**（最后 20%，称为 annealing）：提高高质量数据的比例，同时降低学习率——让模型在收敛前最后一次「复习」最重要的知识。

LLaMA 3[11] 的训练 就使用了这种 annealing 策略——在训练的最后阶段，将高质量数据（教育内容、代码、数学）的比例提高，并将学习率线性衰减到零。这个简单的策略带来了显著的质量提升。

4.3.5 数据重复（Multi-Epoch Training）

关于数据重复（multi-epoch training），经验表明高质量数据重复 2–4 次是安全的，但超过 4 次后过拟合风险急剧上升。一个实用的启发式规则是：如果某类数据的质量远高于平均水平（如精选的教科书文本或高质量代码），可以重复使用更多次；而低质量数据（如未过滤的网页文本）即使只用一次也要谨慎——因为模型可能记住其中的噪声模式。

Muennighoff 等人（2023）的研究更精确地量化了这个问题：对于 C4 数据集上的实验，**4 epochs 的退化等价于将数据集大小减半**。也就是说，重复使用数据 4 次 比首次使用新数据的价值低了一半。这为「应该继续爬取新数据还是重复使用现有数据」提供了定量的决策依据。

[11] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

4.4 合成数据：模型教模型

2024–2026 年的一个重大趋势是**合成数据**——用已有的强大模型生成训练数据来训练新模型。

4.4.1 教科书级合成数据：Phi 的启示

微软的 Phi 系列模型是合成数据成功应用的标杆。Phi-1（2023 年）使用了「教科书级」（textbook quality）的合成数据 来训练一个仅 1.3B 参数的代码生成模型。关键洞察是：与其在大量的低质量 GitHub 代码上训练，不如让 GPT-3.5 生成 **像教科书一样清晰、系统、循序渐进**的代码教程。

Phi-1 仅用约 7B token 的训练数据（其中大部分是合成的），在 HumanEval 代码基准测试上 达到了与 StarCoder（15B 参数、1T token 训练）相当的水平——参数量少 10 倍，数据量少 100 倍。这个结果震撼了整个社区。

微软的 Phi-4 将这一策略推向了极致：Phi-4 在**预训练阶段**就大量使用了合成数据——其预训练语料中包含大量由 GPT-4 生成的高质量合成内容，涵盖推理链、教科书式讲解和结构化问答等多种形式。在后续的 SFT 阶段，它使用了约 140 万个精心策划的样本进行监督微调。结果是一个仅 14B 参数的模型，在多项任务上超越了比它大数倍的模型——证明了**数据质量远比数据数量重要**。

4.4.2 自指令与进化指令

Self-Instruct（Wang 等人，2023）是合成数据生成的另一种重要范式。它的思路是让 LLM 自己生成指令数据：给模型少量种子指令（如 175 条），让模型根据这些种子生成新的指令和对应的回答，然后用这些数据来微调模型本身。

Evol-Instruct（WizardLM 团队）在 Self-Instruct 的基础上更进一步，通过**进化**策略来增加指令的复杂度。具体做法是：从简单的指令出发，让 LLM 对指令进行「深化」（添加约束）、「拓宽」（增加子任务）、「具体化」（添加背景）等进化操作，生成更复杂、更具挑战性的指令。多轮进化后，生成的指令 从简单的「翻译这段话」变成了「将这段 19 世纪英文法律文件翻译成现代中文，保留原始的法律术语，并在注释中解释关键概念」。

4.4.3 大规模合成预训练数据

合成数据的应用已经从 SFT（微调）阶段 扩展到了**预训练**阶段——这是一个质的变化。

Cosmopedia（HuggingFace，2024 年）是目前最大的开源合成预训练数据集。它使用 Mixtral-8x7B-Instruct 生成了 **超过 3000 万个文件、250 亿 token** 的合成内容，包括教科书、博客文章、故事和 WikiHow 式文章。Cosmopedia 的关键设计是**提示多样性**——使用了数百个不同的主题和写作风格模板，确保合成内容的多样性，避免模型坍缩的风险。Cosmopedia 被用于训练 SmolLM 系列小模型，在多项基准上取得了与使用真实数据训练的模型 相当甚至更好的结果。

OpenHermes 2.5（Teknium，2024 年）是开源社区最成功的合成 SFT 数据集之一。它从 15 个不同的数据源中策划了 **超过 100 万条**高质量合成指令和对话样本——包括 GPT-4 生成的数据、数学推理数据（MetaMath）、代码数据（Glaive Code Assistant）和推理链数据（CoT Alpaca）等。OpenHermes 2.5 被用于训练了多个在社区排行榜上名列前茅的模型，证明了**精心策划的多源合成数据** 可以在微调阶段带来显著的质量提升。

NVIDIA Nemotron-4 340B（2024 年）代表了合成数据策略的另一个极端：它用 **98% 的合成数据**训练了一个 340B 参数的模型，在多项基准上与 GPT-4 竞争。NVIDIA 开放了完整的合成数据生成管线，允许开发者使用 Nemotron-4 家族模型（包含生成器、评分器和奖励模型）为自己的应用领域生成定制化的合成数据。这标志着合成数据从「研究技巧」走向了「工业基础设施」。

4.4.4 数学与代码的合成数据

合成数据在数学和代码领域特别有效，因为这些领域有**可验证的正确性标准**。

数学合成数据：可以自动生成数学题目和解答过程，然后通过符号计算（如 SymPy）验证答案的正确性。DeepSeekMath[12] 就利用了 大量自动生成和验证的数学数据 来提升模型的数学推理能力。

代码合成数据：类似地，合成的代码可以通过**执行测试**来验证——生成代码和对应的单元测试，只保留能通过测试的代码作为训练数据。这种「执行验证」（execution-based verification）提供了比人工标注更精确的质量信号。

4.4.5 合成数据在 frontier 模型中的占比

截至 2025–2026 年，合成数据在 frontier 模型训练中的占比 已经从早期实验的「少量补充」增长到了**实质性组成部分**：

- **预训练阶段**：Phi-4 和 Nemotron-4 表明 合成数据可以占预训练语料的 30%–98%——具体比例取决于合成数据的质量和多样性。但对于 frontier 级模型（如 GPT-4o、Claude 3.5），普遍估计合成数据占比在 20%–40% 之间。
- **SFT 阶段**：合成数据已经成为主流。几乎所有 frontier 模型的 SFT 数据中，合成指令和合成对话占比超过 50%。
- **RLHF/DPO 阶段**：合成偏好数据（如 Constitutional AI 生成的偏好对）正在逐步取代成本高昂的人类标注。

这个趋势的驱动力是清晰的：高质量的真实数据正在**枯竭**。互联网上可用的高质量文本是有限的，而模型对数据的需求仍在增长。合成数据是解决这一矛盾的关键路径——但它引入的模型坍缩风险也必须被认真对待。

[12] Z. Shao et al., "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

4.4.6 合成数据的风险：模型坍缩

合成数据并非万能药。Meta FAIR 训练了超过 1000 个 LLM 来研究合成数据的最优使用方式[13]。核心结论是：预训练的最优比例约为 30% 合成 + 70% 真实数据，可以加速训练 5–10 倍。

但需要警惕**模型坍缩**（model collapse）。Shumailov 等人（2023）的研究发现了一个令人不安的现象：如果模型 A 生成的数据被用来训练模型 B，B 生成的数据再用来训练模型 C，如此递归数代，模型的输出会逐渐**退化**——「遗忘」数据分布的尾部（低频但重要的模式），输出越来越同质化、越来越「平庸」。

这个现象的直觉解释是：模型在生成数据时会**放大高频模式、忽略低频模式**——类似于反复复印一张照片，每次都会损失一些细节。经过多代递归后，只有最高频的模式被保留，丰富多样的尾部分布完全消失。

模型坍缩的两种形式

Shumailov 等人将模型坍缩区分为两个阶段：

早期坍缩（early collapse）：模型首先丢失分布尾部的信息——低频但有意义的模式（如罕见的写作风格、小众领域的知识、少数群体的表达方式）逐渐消失。这个阶段的坍缩往往不易察觉，因为主流基准测试通常不测量尾部分布的保留程度。

晚期坍缩（late collapse）：经过多代递归后，模型输出的分布 与原始数据分布完全不同——方差急剧减小，输出变得高度同质化。在极端情况下，模型可能退化为 反复生成少数几种固定模式。

2025 年的缓解策略

2025 年的研究提出了多种缓解模型坍缩的新方法：

- **合成数据检测与过滤**：在训练数据中检测并移除由 LLM 生成的文本。随着互联网上 AI 生成内容的比例急剧上升（估计已超过某些领域网页内容的 10%–20%），即使不主动使用合成数据，从 Common Crawl 中爬取的网页也可能包含大量 AI 生成文本。这使得合成数据检测成为数据清洗流水线的新必备步骤。
- **多样性正则化**：在合成数据生成时显式优化输出的多样性——例如使用高温度采样、多样化的提示模板、以及刻意覆盖长尾主题和风格。
- **真实数据锚定**：始终保留一定比例的真实数据作为「锚」，防止分布漂移。Nemotron-CC 的「改写」策略 是一个优雅的折中——使用 LLM 改善真实数据的表达质量，同时保留真实数据的内容多样性。
- **分布匹配验证**：定期检查合成数据的分布 是否偏离真实数据的分布，使用统计检验（如 KL 散度、MMD）来量化偏离程度并及时干预。

[13] 该结论来自 Meta FAIR 的系统性实验，具体论文待补充。

避免模型坍缩的实践建议

(1) **始终混入真实数据**——合成数据应该是真实数据的**补充**而非**替代**。(2) **限制递归深度**——不要用模型 A 生成的数据训练模型 B，再用 B 生成的数据训练 C。尽量始终用最强的模型（如 GPT-4）作为生成源。(3) **多样性保障**——在合成数据中刻意注入多样性（不同的提示模板、不同的主题、不同的风格）。(4) **质量过滤**——不是所有合成数据都是好的。使用验证机制（执行测试、一致性检查）过滤掉低质量的合成样本。(5) **检测互联网上的 AI 生成内容**——2025 年后从 Common Crawl 爬取的数据 可能已经包含大量 AI 生成文本。不加检测地使用这些数据会导致隐性的模型坍缩。

4.4.7 Constitutional AI 与 AI 反馈

Anthropic 的 Constitutional AI[14] 开创了另一种合成数据的使用方式：用 AI 生成**反馈**而非**内容**。

传统的 RLHF 需要人类标注者 对模型的输出进行偏好排序——这个过程既昂贵又难以规模化。Constitutional AI 用一组明确的「原则」（宪法，constitution）来指导 AI 自我评估：模型先生成回答，然后根据宪法中的原则对自己的回答进行批评和修订，最后用修订后的版本作为偏好学习的训练数据。

这种方法极大地降低了获取偏好数据的成本，使得快速迭代 RLHF 训练成为可能。

4.5 数据处理工具生态

2025–2026 年，LLM 的数据处理已经形成了成熟的工具生态。

4.5.1 Datatrove

Datatrove（HuggingFace 开源）是 FineWeb 背后的数据处理框架。它将整个流水线抽象为「读取 → 过滤 → 去重 → 写入」的模块化流程，支持在单机和集群上运行。核心设计理念是**可复现性**——每一步的参数和随机种子都被记录，确保相同的输入总是产生相同的输出。

Datatrove 的架构设计值得学习：每个处理步骤（称为 “block”）都实现统一的接口（读取文档流、输出文档流），可以自由组合。这种设计使得实验不同的过滤策略变得非常容易——只需替换或添加 block 即可。

4.5.2 CCNet

CCNet（Meta 开源）是专门为 Common Crawl 处理设计的工具。它实现了完整的清洗流水线：语言识别（fastText）→ 段落去重（SHA-256 哈希）→ 困惑度过滤（KenLM）。

[14] Y. Bai et al., “Constitutional AI: Harmlessness from AI feedback,” *arXiv preprint arXiv:2212.08073*, 2022.

LLaMA[15] 的训练数据 就是通过 CCNet 处理的。

4.5.3 Data-Juicer

Data-Juicer（阿里巴巴 DAIL 开源）提供了 50+ 种数据处理算子（operator），涵盖文本质量评估、语言识别、数据多样性分析、安全内容过滤等。它的特色是 支持「数据菜谱」（data recipe）的概念——将一组处理步骤和参数打包为可复用的配置文件，便于实验和分享。

4.5.4 其他重要工具

- **Dolma toolkit**（AI2）：Dolma 数据集的完整构建工具链，从原始数据到最终数据集的每一步都可复现。
- **text-dedup**：一个专注于文本去重的 Python 库，实现了 MinHash、SimHash、后缀数组等多种去重算法。
- **trafilatura**：Python 实现的网页文本提取工具，支持从 HTML 中精准提取正文内容。
- **fastText**：Meta 开源的高效文本分类和语言识别工具，在语言分类任务上是事实标准。
- **KenLM**：高效的 n-gram 语言模型实现，常用于困惑度过滤。

4.5.5 数据处理流水线示例

一个典型的数据处理流水线的伪代码如下：

Listing 4.1: Data processing pipeline (pseudocode)

```python
from datatrove import Pipeline

pipeline = Pipeline([
    # Step 1: Read raw HTML from Common Crawl
    WARCReader(data_path="s3://commoncrawl/..."),

    # Step 2: Extract text from HTML
    TrafilaturaExtractor(),

    # Step 3: Language identification
    FastTextLanguageFilter(languages=["en", "zh"]),

    # Step 4: Rule-based quality filters
    GopherQualityFilter(
        min_doc_words=50,
```

[15] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

```
        max_doc_words=100000,
        min_avg_word_length=3,
        max_symbol_to_word_ratio=0.1,
    ),

    # Step 5: Perplexity-based quality scoring
    C4QualityFilter(exclusion_writer=...),

    # Step 6: PII removal (emails, phone numbers)
    PIIRemovalFilter(),

    # Step 7: Exact deduplication
    URLDedup(),

    # Step 8: Fuzzy deduplication
    MinhashDedup(
        n_grams=5,
        num_hashes=128,
        similarity_threshold=0.8,
    ),

    # Step 9: Write processed data
    JsonlWriter(output_path="processed/"),
])

pipeline.run(num_workers=64)
```

这段伪代码展示了从原始网页到训练数据的完整流程。每一步都是一个可配置的算子，可以独立运行和调试。`num_workers=64` 表示使用 64 个并行进程处理——处理一个 Common Crawl 快照（通常包含数十亿网页）需要数百 CPU 核心运行数天。

4.5.6 规模化的工程挑战

在 TB 到 PB 级别处理数据，工程挑战远超算法本身。以 FineWeb 处理 96 个 Common Crawl 快照为例：

- **存储**：原始数据约 300TB，中间结果和最终输出需要同等甚至更多的存储空间。
- **计算**：全量 MinHash 去重 需要在数百台机器上运行，单次去重耗时数天。
- **调度**：数据处理的各步骤之间存在依赖关系（必须先提取文本才能去重），需要精心设计任务调度。
- **容错**：在数天的运行中，机器故障是常态而非例外。流水线必须支持断点续传和部分重跑。

HuggingFace 处理 FineWeb 时使用了约 **300 万 CPU 小时**——这是一个惊人的数字，说明数据处理的计算成本 虽然远低于模型训练，但绝非微不足道。

4.6 数据污染检测

随着 LLM 在各种基准测试上取得越来越高的分数，一个关键问题浮出水面：**这些成绩到底反映了真正的泛化能力，还是模型只是「记住」了测试数据？**

4.6.1 数据污染的定义与危害

数据污染（data contamination）指的是 模型的训练数据中包含了 评估基准测试的题目或答案。由于 Common Crawl 收录了互联网上的大量内容，而许多基准测试的数据也公开发布在网上，训练数据与测试数据之间的重叠几乎是不可避免的。

数据污染的危害在于：它使得基准测试的结果**不再可靠**——我们无法区分模型是「理解了问题的模式」还是「记住了具体的答案」。对于推动 LLM 进步，这是一个根本性的问题——如果基准测试失去了可信度，我们就失去了衡量进步的尺度。

4.6.2 检测方法

2024–2025 年，研究社区发展了多种污染检测方法：

- ***n*-gram 重叠检测**：最直接的方法——检查训练数据中是否包含 与基准测试题目或答案相同的长 *n*-gram（通常 $n \geq 13$）。LLaMA[16] 和 GPT-4[17] 的技术报告 都使用了这种方法。局限是：它只能检测**精确重叠**，无法发现同义改写（paraphrase）形式的泄露。
- **成员推断**（membership inference）：通过分析模型对特定样本的困惑度或置信度 来判断该样本是否出现在训练数据中。直觉是：模型对训练数据中见过的文本 会表现出异常低的困惑度。
- **CoDeC**（Contamination Detection via Context）：NVIDIA 在 2025 年提出的方法，通过比较**有无 in-context 示例** 时模型对基准测试题目的置信度变化来检测污染。关键洞察是：对于未见过的数据，in-context 示例通常**提升**模型置信度；而对于已记忆的数据，in-context 示例反而可能**降低**置信度（因为干扰了记忆的检索模式）。
- **动态基准测试**：与其检测污染，不如让污染无法发生——使用动态生成的基准测试题目（每次评估不同），使得训练数据中不可能包含未来生成的测试题。LiveBench 和 GAIA 等基准测试采用了这种策略。

[16] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[17] OpenAI et al., "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

数据污染是一个系统性问题

数据污染不是个别团队的失误，而是当前 LLM 训练范式的**系统性问题**。当训练数据规模达到万亿 token 级别时，要精确知道训练数据中包含什么几乎是不可能的。即使团队真诚地尝试去除基准测试数据，同义改写、翻译、引用等间接泄露渠道也难以完全堵截。这使得**数据溯源**（data provenance）和**基准测试设计**成为下一阶段 LLM 研究的重要课题。

4.7 版权与法律：数据的法律边界

训练数据的法律问题已经从学术讨论 演变为影响整个 AI 行业的**核心挑战**。2024–2026 年的一系列诉讼和立法 正在重塑 LLM 训练数据的获取方式。

4.7.1 《纽约时报》诉 OpenAI 案

2023 年 12 月，《纽约时报》对 OpenAI 和微软 提起了版权侵权诉讼——这是迄今为止 AI 训练数据领域**影响最大**的案件。《纽约时报》指控 OpenAI 在未经授权的情况下，将其数百万篇受版权保护的新闻报道 用于训练 GPT 系列模型，并展示了 ChatGPT 能够**逐字复述**《纽约时报》文章内容的证据。

截至 2026 年初，此案仍在审理中。2025 年 4 月的法庭裁决对《纽约时报》有利——法官认为其部分诉求（包括指控 OpenAI 的训练构成侵权）在现阶段具有成立的可能性。这一裁决虽非最终判决，但已经对行业产生了深远影响：多家 AI 公司开始与内容出版商签订**许可协议**，为训练数据付费。

4.7.2 合理使用（Fair Use）之争

美国的核心法律争议集中在 **合理使用**（fair use）原则上。AI 公司主张，将版权内容用于训练 是「转化性使用」（transformative use）——模型学习的是语言的统计模式，而不是复制特定的文章。类似的，Google 在 Google Books 案中 成功主张了对图书的扫描和索引构成合理使用。

内容创作者则反驳：LLM 训练与搜索引擎索引有本质区别——搜索引擎引导用户访问原始内容（增加流量），而 LLM 直接生成回答、替代原始内容（减少流量）。如果用户可以直接向 ChatGPT 询问最新新闻，为什么还要订阅《纽约时报》?

4.7.3 欧盟 AI 法案与全球监管

欧盟 AI 法案（EU AI Act，2024 年正式生效）对 AI 训练数据提出了明确的**透明度要求**：通用 AI 模型的提供者必须「公布训练数据中受版权保护内容的充分详细摘要」。虽然具体的执行标准仍在制定中，但这一要求已经迫使 AI 公司开始构建 **数据溯源**（data provenance）系统——记录训练数据中每一条内容的来源、许可证状态和使用限制。

欧盟还规定了**选择退出**（opt-out）机制：版权持有者可以明确声明其内容不得用于 AI 训练（例如通过 robots.txt 或 TDM-reservation 标签），AI 公司必须尊重这些声明。这使得 robots.txt 从一个非强制性的礼仪约定 变成了一个具有法律效力的 opt-out 机制。

4.7.4 行业的应对策略

面对法律风险，AI 行业采取了多种应对策略：

- **许可协议**：OpenAI 已与美联社、Axel Springer、Le Monde 等 多家媒体机构签订了内容许可协议。Google 和 Anthropic 也在推进类似的合作。
- **合成数据替代**：用合成数据替代受版权保护的真实数据——这是 Phi 系列和 Nemotron-4 策略的另一个动机。
- **数据溯源基础设施**：构建能够追踪每条训练数据来源和许可证状态的系统。C2PA（Coalition for Content Provenance and Authenticity）等标准正在被用于标记 AI 生成的内容 和追踪内容的使用历史。
- **开放数据运动**：社区推动创建明确授权用于 AI 训练的开放数据集——如 The Stack（仅包含宽松许可证的代码）、Dolma（完全透明的数据来源记录）。

版权问题的长期影响

版权争议可能从根本上改变 LLM 的训练范式。如果法院最终裁定未经授权使用版权内容构成侵权，那么 AI 公司将面临两个选择：(1) 为训练数据支付许可费用——这将大幅增加训练成本，可能使得只有资金雄厚的公司才能训练 frontier 模型；(2) 完全依赖公共领域和合成数据——这将加速合成数据技术的发展，但也增加了模型坍缩的风险。无论哪种结果，**数据溯源和透明度** 都将成为未来 LLM 训练的必备基础设施。

数据工程的核心原则

1. **质量 > 数量**：1T 高质量 token > 10T 低质量 token。Phi 系列和 FineWeb-Edu 反复证明了这一点。
2. **多样性至关重要**：单一来源的数据（即使高质量）会限制模型的泛化能力。混合网页、代码、学术、书籍等多种来源。
3. **代码是推理能力的催化剂**：即使你的目标不是代码生成，也应该纳入代码数据。代码的逻辑严密性能提升模型的通用推理能力。
4. **去重是必须的**：重复数据不仅浪费计算，还会降低模型的泛化能力。多层次去重（URL → 精确 → 模糊 → 子串）效果最好。
5. **合成数据是补充而非替代**：始终混入真实数据以避免模型坍缩。最优比例约为 30% 合成 + 70% 真实。
6. **可复现性是底线**：记录每一步的参数和随机种子，确保同事或后来者能精确复现你的数据流水线。
7. **法律合规不可忽视**：了解训练数据的版权状态，构建数据溯源系统，尊重 opt-out 声明——法律风险正在成为数据工程的核心约束之一。

CHAPTER 5

预训练——从随机噪声到语言理解

准备好了数据和模型架构，下一步就是预训练——这是整个 LLM 训练流程中计算量最大、成本最高的阶段。预训练的目标很简单：让模型学会预测下一个 token。但实现这个「简单」目标所需的工程规模，超乎大多数人的想象。

5.1 训练目标：下一个 Token 预测

LLM 的预训练使用**因果语言模型**（Causal Language Model）目标：给定前面的所有 token，预测下一个 token。

形式化地，给定一个 token 序列 $x_1, x_2, \ldots, x_T$，模型的训练损失是：

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t \mid x_1, x_2, \ldots, x_{t-1}) \tag{5.1}$$

这就是交叉熵损失——对数似然的负数，再对序列长度取平均。模型被要求在每个位置输出一个词表大小的概率分布，正确 token 的概率越高，损失越低。

5.1.1 交叉熵、似然与 KL 散度的三位一体

Equation (5.1) 看起来是一个「工程选择」，但它背后有深刻的信息论基础。让我们逐步拆解这三个等价视角。

第一视角：最大化对数似然。给定训练语料库中的所有句子，我们希望找到参数 θ 使得模型「最可能生成」这些句子。对于单个序列 $\mathbf{x} = (x_1, \ldots, x_T)$，自回归分解给出的联合概率为：

$$P_\theta(\mathbf{x}) = \prod_{t=1}^{T} P_\theta(x_t \mid x_{<t}) \tag{5.2}$$

取对数得到 $\log P_\theta(\mathbf{x}) = \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})$。最大化这个对数似然等价于最小化 Equation (5.1)——两者仅差一个负号和一个常数 $1/T$。

第二视角：最小化 KL 散度。设 P_{data} 是训练数据的真实分布，P_θ 是模型的分布。我们可以写出 KL 散度：

$$D_{\text{KL}}(P_{\text{data}} \| P_\theta) = \mathbb{E}_{P_{\text{data}}}\left[\log \frac{P_{\text{data}}(\mathbf{x})}{P_\theta(\mathbf{x})}\right] = H(P_{\text{data}}, P_\theta) - H(P_{\text{data}}) \tag{5.3}$$

其中 $H(P_{\text{data}}, P_\theta)$ 是交叉熵，$H(P_{\text{data}})$ 是数据本身的熵（常数，不依赖 θ）。因此，**最小化交叉熵 ⇔ 最小化 KL 散度 ⇔ 最大化对数似然**。

这三条路殊途同归，但直觉各不相同：对数似然视角说的是「让模型尽可能生成训练数据」；KL 散度视角说的是「让模型分布尽可能接近真实分布」；交叉熵视角说的是「让模型需要的平均编码长度最短」——最终都指向同一个优化目标。

第三视角：交叉熵的信息论解读。交叉熵 $H(P_{\text{data}}, P_\theta)$ 可以理解为：如果真实数据来自 P_{data}，但你使用模型 P_θ 来编码，平均每个 token 需要多少比特。好的语言模型 = 好的压缩器——这正是 Shannon 信息论的核心洞察。当模型完美时（$P_\theta = P_{\text{data}}$），交叉熵等于数据本身的熵，即理论上的最优编码长度。

5.1.2 Teacher Forcing

训练时，模型在预测第 t 个 token 时 使用的是**真实的**前 $t-1$ 个 token 作为条件，而不是模型自己之前生成的 token。这种策略叫做 **Teacher Forcing**。

为什么不用模型自己的输出？因为训练初期模型几乎是随机的，如果用它自己的（错误的）预测作为后续的输入，错误会指数级累积——第 1 步预测错了，第 2 步基于错误输入预测更错，第 3 步更加离谱。这种「误差传播」会让训练完全无法收敛。

Teacher Forcing 通过「强制纠正」解决了这个问题：不管模型第 $t-1$ 步预测得多差，第 t 步的输入始终是正确的 x_{t-1}。这让每个位置的预测问题相互独立，梯度信号清晰且有效。

代价是训练和推理之间存在分布偏移（exposure bias）——训练时模型总看到正确的前文，推理时只能看到自己可能不正确的生成结果。但在实践中，大规模预训练后这个问题的影响很小，因为模型已经足够准确，推理时的自回归生成很少偏离正轨。

5.1.3 因果遮盖 vs 双向遮盖

因果遮盖（Causal Masking）是 decoder-only 模型（GPT 系列、LLaMA、DeepSeek 等）的核心机制：在 Attention 计算中，位置 t 只能看到位置 1 到 t 的信息。这通过一个下三角矩阵实现：

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right)V, \quad M_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases} \tag{5.4}$$

M 中的 $-\infty$ 在 softmax 后变成 0，确保未来位置的信息不会泄漏到当前位置。

双向遮盖（Bidirectional Masking）则允许每个位置 看到序列中的所有其他位置——这就是 BERT[1] 使用的方式。双向模型在理解任务上有天然优势：要判断一个句子的情感，当然应该同时看到前后文。

那为什么 decoder-only 最终赢了？三个关键原因：

[1] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019.

1. **生成能力**：因果遮盖天然支持自回归生成。双向模型无法逐步生成文本，因为它需要看到「未来」。
2. **训练效率**：因果语言模型在每个位置都有预测目标，即每个 token 都贡献一个损失信号。BERT 的 MLM 只遮盖 15% 的 token，85% 的计算量没有直接贡献于训练目标。
3. **Scaling 效果**：GPT-3[2] 和后续工作证明，在足够大的规模下，单向因果模型的理解能力并不逊于双向模型——通过预测下一个 token，模型被迫**隐式地**理解上下文。

为什么不用 BERT 的 MLM（遮盖语言模型）?

BERT[a] 使用遮盖语言模型：随机遮盖 15% 的 token，让模型预测被遮盖的部分。这是**双向**的——模型同时看到被遮盖位置的前后文。这对理解任务（如问答、分类）很有帮助，但有一个致命缺陷：它无法用于**生成**文本。生成需要自回归过程——逐词生成，每次只能看到前面已生成的内容。因果语言模型天然支持这种自回归生成，而且 GPT-3[b] 证明了 单方向的因果模型在规模足够大时，理解能力也不逊于双向模型。因此，所有现代 LLM 都选择了因果语言模型。

[a] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019.

[b] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

5.1.4 Prefix Language Modeling

值得一提的是一种折中方案：**Prefix Language Modeling**（前缀语言模型）。它将输入序列分为两部分：**前缀**（prefix）部分使用双向注意力（互相可见），**生成**（generation）部分使用因果注意力（只看前文）。

Google 的 PaLM[3] 和 U-PaLM 都支持这种模式。前缀部分通常是任务描述或上下文，模型可以充分利用双向信息来理解它；生成部分则自回归地产出答案。

这种设计试图兼得两者的优势：前缀的双向理解 + 生成的自回归能力。但在实践中，纯因果模型（decoder-only）的简洁性和 Scaling 效果 使它成为了事实标准——额外的复杂性带来的收益不足以改变整个行业的选择。

[2] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[3] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

5.2 Scaling Laws：计算的经济学

5.2.1 Kaplan Scaling Laws（2020）

2020 年，OpenAI 发现了一个改变整个领域的定律[4]：LLM 的性能（以损失值衡量）与模型大小、数据量和计算量 之间存在**精确的幂律关系**。

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C} \tag{5.5}$$

其中 N 是模型参数量，D 是训练数据量（token 数），C 是总计算量（FLOP），α 和下标 c 是经验常数。

Kaplan 等人拟合出的指数值为：$\alpha_N \approx 0.076$，$\alpha_D \approx 0.095$，$\alpha_C \approx 0.050$。

这些数字意味着什么？以 $\alpha_N = 0.076$ 为例：模型参数量每增加 **10 倍**，损失值下降约 $10^{-0.076} \approx 0.84$，即下降 16%。这听起来不多，但在对数坐标下，这条直线**没有拐点**——不存在「增大模型就不管用了」的天花板。从 100M 到 1B 到 10B 到 100B，每增大 10 倍，损失都稳定下降约 16%。

$\alpha_C = 0.050$ 的含义则更直接：计算量（FLOP）每增加 10 倍，损失下降约 $10^{-0.05} \approx 0.89$，即 11%。这给了整个行业巨大的信心：只要你愿意投入足够的计算，就能造出更好的模型。

Kaplan 论文的另一个关键发现是：当计算预算固定时，应该**优先增大模型、少用数据**。他们认为参数量比数据量更重要——N 的增长速度应该快于 D。这直接影响了 GPT-3 的设计选择：175B 参数，但仅用 300B token 训练。

Scaling Laws 的直觉类比

想象你在装修一间房子：

- **模型参数量** $N \approx$ 房子的面积——更大的房子能放更多东西。
- **训练数据量** $D \approx$ 装修材料的数量——更多材料让房子更完善。
- **计算量** $C \approx$ 装修工人的工时——预算总是有限的。

Kaplan 说：如果预算有限，先买大房子，材料少一点也行。Chinchilla 说：不对，大房子配太少的材料是浪费——房子面积和材料应该等比例增长。

5.2.2 Chinchilla 定律（2022）

2022 年，DeepMind 对 Scaling Laws 进行了重要修正[5]。Kaplan 等人认为应该优先增大模型、少用数据，而 Chinchilla 论文证明：**对于给定的计算预算，模型参数量和训练数据量应该等比例增长**。

[4] J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[5] J. Hoffmann et al., "Training compute-optimal large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

具体而言，Chinchilla 定律建议每个参数大约需要 20 个 token 的训练数据。这意味着一个 70B 参数的模型应该在约 1.4T token 上训练——远多于 GPT-3（175B 参数但仅 300B token）所使用的数据量。Chinchilla（70B 参数，1.4T token）在几乎所有基准测试上超越了 GPT-3。

Chinchilla 的核心发现可以更精确地表述为：对于固定的计算预算 C，最优的参数量 N^* 和数据量 D^* 满足：

$$N^* \propto C^{0.50}, \quad D^* \propto C^{0.50} \tag{5.6}$$

也就是说，计算预算翻倍时，N 和 D 应该**各增长** $\sqrt{2} \approx 1.41$ 倍。这与 Kaplan 的结论截然不同——Kaplan 建议把大部分增量预算花在增大模型上，Chinchilla 则说应该均分。

这个修正的影响是深远的。按 Chinchilla 的标准，GPT-3 被严重「欠训练」了：175B 参数的最优训练数据量应该是约 $175 \times 20 = 3.5$T token，而 GPT-3 仅用了 300B token——差了超过 10 倍。换句话说，如果把训练 GPT-3 的计算量换成训练一个 更小但更充分训练的模型（如 70B 配 1.4T token），性能反而更好。

5.2.3 后 Chinchilla 时代：过训练策略

Chinchilla 定律回答的是「给定训练预算，如何最优地分配」。但它有一个隐含假设：它优化的是给定训练计算预算下的最优模型。这忽略了一个现实：**训练是一次性的，推理是永久的**。

考虑两种方案：

- 方案 A：训练一个 70B 模型（Chinchilla 最优），1.4T token。
- 方案 B：训练一个 7B 模型，14T token（按 Chinchilla 过训练 10 倍）。

两者的训练计算量接近（$C \approx 6ND$），但方案 B 的模型小 10 倍——部署时每次推理的成本只有方案 A 的 ~10%。如果这个模型要服务亿万用户，方案 B 的总成本（训练 + 推理）可能远低于方案 A。

Meta 的 LLaMA[6] 系列开创了这一思路：

- LLaMA-1 7B：1T token（Chinchilla 建议约 140B token，过训练 ~7×）
- LLaMA 2 7B：2T token（过训练 ~14×）
- LLaMA 3 8B[7]：15T token（过训练 ~100×）

DeepSeek-V3[8] 正是基于这个思想：671B 总参数（MoE 架构下每个 token 仅激活 37B），在 14.8T token 上训练——远超 Chinchilla 最优数据量。如果按 Chinchilla 的 20:1 比例，37B 活跃参数只需约 740B token，实际用了 14.8T，过训练了约 20 倍。这种「过训练」（over-training）策略使得模型在部署时极具成本优势。

[6] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[7] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[8] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

推理经济学：为什么「过训练」更划算

假设训练成本为 C_{train}，模型每天推理 Q 次查询，每次推理成本正比于活跃参数量 N_{active}，模型服务周期为 T 天。总成本 $= C_{\text{train}} + Q \cdot T \cdot f(N_{\text{active}})$。
对于广泛部署的模型（$Q \cdot T$ 极大），推理项主导总成本——此时减小 N_{active}（即使需要增加 C_{train}）几乎总是有利的。这就是 Chinchilla 最优不等于部署最优的根本原因。

5.2.4 "Chinchilla is Dead"：推理感知的 Scaling Laws

2024–2025 年，学术界和工业界对 Chinchilla 定律展开了深入的反思，甚至出现了 "Chinchilla is Dead" 的说法。核心论点是：**Chinchilla 只优化了训练成本，完全忽略了推理成本**——而对于被广泛部署的模型，推理成本远超训练成本。

Sardana 等人 (2024) 在 ICML 上发表了形式化的**推理感知 Scaling Laws**。他们将总成本建模为训练与推理的加权和：

$$C_{\text{total}} = C_{\text{train}}(N, D) + R \cdot C_{\text{inference}}(N) \tag{5.7}$$

其中 R 是模型生命周期内的总推理请求量。当 R 较大时（即模型被广泛部署），最优策略是训练**更小的模型、用更多数据**——这正是 LLaMA 系列和 DeepSeek 系列的做法。Sardana 等人的分析表明，对于每天服务数百万查询的模型，最优参数量可能只有 Chinchilla 最优的 1/5 到 1/10，但需要 5× 到 50× 的训练数据。

Bian 等人 (2025) 进一步将**模型架构**纳入 Scaling Laws 的考量。他们发现，参数量相同的模型因架构差异（如层数 vs 宽度、GQA vs MHA、FFN 比例等）推理延迟最多相差 3.5×。这意味着 Scaling Laws 不应只优化参数量和数据量，还应该**共同优化架构设计**。

5.2.5 数据受限的 Scaling Laws 与合成数据

当高质量自然数据趋于耗尽时，传统 Scaling Laws 的 "无限数据" 假设不再成立。

Muennighoff 等人 (2023) 的研究表明，重复数据的价值递减但不为零——数据重复 4 次的效果大约等价于使用 $4^{0.6} \approx 2.3$ 倍的去重数据。超过约 4 个 epoch 后，重复的边际收益急剧下降。

这个问题催生了**合成数据 Scaling Laws** 的研究。Qin 等人 (2025) 提出了 SynthLLM 框架，首次系统地研究了合成数据的 scaling 行为。他们发现：（1）朴素地扩大合成数据量存在严重的收益递减——模型会过拟合于合成数据的分布偏差；（2）通过 "精心练习"（deliberate practice）策略，即动态生成针对模型弱点的合成数据，可以显著改善合成数据的 scaling 效率。Microsoft 基于 SynthLLM 的实验表明，经过精心设计的合成数据可以有效突破 "数据墙"，为模型提供接近真实数据质量的训练信号。

5.2.6 Scaling Laws 的新方向（2025–2026）

Scaling Laws 的研究仍在快速发展。几个值得关注的方向：

下游任务的 Scaling Laws。Held 等人 (2025) 提出了**相对 Scaling Laws**，不再用绝对的损失值来描述 scaling 行为，而是预测模型之间的**相对性能差异**。这种方法对不同评估基准的尺度变化更鲁棒。Zhang 等人 (2026) 进一步提出了**规范性 Scaling Laws**，基于 5000+ 个模型评估数据点，估计给定预训练计算预算所能达到的下游准确率上界。他们发现，随着后训练技术（RLHF、推理时计算等）的进步，同样预训练预算对应的能力边界在持续上移——这意味着 Scaling Laws 不是静态的，它们会随着整个技术栈的进步而 “演化”。

推理时计算的 Scaling Laws。OpenAI 的 o1/o3 系列模型开辟了另一条 scaling 路径：不增加模型参数，而是在推理时投入更多计算（如更长的思维链、更多的采样和验证）。这引出了一个新的权衡维度：**训练时计算 vs 推理时计算**。对于某些推理密集型任务（数学、编程），在推理时使用更多计算的回报可能高于 同等计算量用于增大模型。

5.2.7 Scaling Laws 的局限性

尽管 Scaling Laws 是 LLM 领域最重要的实证发现之一，它们有几个重要的局限性：

不能预测涌现能力（Emergent Capabilities）。Scaling Laws 告诉你损失值会随规模平滑下降，但不会告诉你模型在什么规模下突然学会了 思维链推理（Chain-of-Thought）、上下文学习（In-Context Learning）或者代码生成。这些能力似乎在某个规模阈值之后「突然出现」——而这在平滑的幂律曲线上是看不到的。

仅预测训练损失，不预测下游任务性能。训练损失（perplexity）的改善不一定线性地转化为 具体任务（如数学推理、代码生成）上的性能提升。你可能需要将损失从 2.5 降到 2.3 才能看到数学能力的跳跃，但从 2.3 降到 2.1 对数学帮助不大，反而让诗歌写作能力有了飞跃。

数据质量不在公式中。Scaling Laws 假设数据质量恒定，但实际上数据质量可能比数据数量更重要。Phi-2（2.7B）在精选的「教科书级」数据上训练，在部分基准测试上超过了 25× 大的模型。

「**苦涩的教训**」。Rich Sutton 的经典文章[9] 总结了 AI 研究 70 年的规律：利用**更多计算**的方法，长期来看总是赢——那些依赖人类知识和巧妙特征工程的方法，短期内可能有效，但最终会被暴力扩展的方法超越。Scaling Laws 是这个「苦涩教训」的数学表述。

5.3 分布式训练：为什么需要数千块 GPU?

让我们做一个简单的计算来理解为什么单块 GPU 远远不够。

一个 70B 参数的模型：

- **模型权重**：$70 \times 10^9 \times 2$ bytes (BF16) = 140 GB

[9] R. Sutton, *The bitter lesson*, `http://www.incompleteideas.net/IncIdeas/BitterLesson.html`, 2019.

- **优化器状态**（Adam）：每个参数需要 12 bytes （FP32 参数副本 4 bytes + 一阶动量 4 bytes + 二阶动量 4 bytes）= 840 GB
- **梯度**：140 GB
- **激活值**：取决于 batch size 和序列长度，通常 100+ GB

总计约 **1,220+ GB**——而一块 H100 GPU 只有 80 GB 显存。即使是最新的 H200（141 GB），单块也放不下。

这就是为什么分布式训练是必须的。现代 LLM 训练使用多种并行策略的组合。

5.3.1 数据并行（Data Parallelism）

最简单的并行方式：每块 GPU 持有完整的模型副本，处理不同的数据批次，然后同步梯度。

Listing 5.1: Distributed Data Parallel in PyTorch

```python
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

# Each GPU gets the same model
model = LLM().to(local_rank)
model = DDP(model, device_ids=[local_rank])

for batch in dataloader:
    loss = model(batch)
    loss.backward()
    # Gradients are automatically all-reduced across GPUs
    optimizer.step()
```

问题是：每块 GPU 都要放下完整模型。对于 70B+ 的模型，这不可行。

梯度同步是数据并行的核心操作。所有 GPU 需要在反向传播后将梯度「平均化」，确保每块 GPU 上的模型更新一致。最常用的算法是 **Ring AllReduce**：N 块 GPU 组成一个环，每块 GPU 向下一块发送自己的梯度切片、接收上一块的梯度切片，经过 $2(N-1)$ 步通信后，所有 GPU 都拥有了完整的平均梯度。Ring AllReduce 的通信量与 GPU 数量 N 无关（每块 GPU 发送和接收的数据量约为 $2 \cdot \frac{M}{N} \cdot (N-1) \approx 2M$，其中 M 是梯度总大小），因此能很好地扩展到大规模集群。

大 Batch 训练的挑战。数据并行天然地扩大了有效 batch size——N 块 GPU 每块处理 B 个样本，有效 batch size 为 NB。但大 batch 带来两个问题：(1) 学习率需要相应调整，否则收敛速度会下降——通常使用线性缩放规则（linear scaling rule）：$\mathrm{lr}_{\mathrm{effective}} = \mathrm{lr}_{\mathrm{base}} \times N$；(2) 过大的 batch 可能让模型陷入尖锐的极小值（sharp minima），泛化性变差。这就是为什么大 batch 训练几乎总是需要 **学习率预热**（warmup）——让模型在初期用较小的步长探索参数空间。

5.3.2 ZeRO 与 FSDP：分片数据并行

DeepSpeed 的 ZeRO 优化[10] 和 PyTorch 的 FSDP 解决了这个问题：将模型的参数、梯度和优化器状态**分片**到不同 GPU 上，每块 GPU 只持有 $1/N$ 的模型状态。计算时，通过 All-Gather 临时收集需要的参数；反向传播后，通过 Reduce-Scatter 分发梯度。

ZeRO 分为三个阶段，每个阶段分片的内容不同，对应不同的内存节省和通信开销：

- **ZeRO-1**：只分片优化器状态。
- **ZeRO-2**：分片优化器状态 + 梯度。
- **ZeRO-3 / FSDP**：分片一切，包括模型参数。

让我们用一个 **10B 参数模型在 8 块 GPU 上**的例子 来量化每个阶段的内存节省：

组件	无 ZeRO	每块 GPU 需存储	Bytes/param
模型参数（BF16）	20 GB	每 GPU 全量	2
梯度（BF16）	20 GB	每 GPU 全量	2
优化器状态（FP32）			
参数副本	40 GB	每 GPU 全量	4
一阶动量 m	40 GB	每 GPU 全量	4
二阶动量 v	40 GB	每 GPU 全量	4
总计	**160 GB**		16

ZeRO-1：只分片优化器状态（120 GB → 15 GB/GPU），每块 GPU 仍需 20+20+15 = **55 GB**。节省 ~3×。

ZeRO-2：分片优化器 + 梯度（120+20 = 140 GB → 17.5 GB/GPU），每块 GPU 需 20+17.5 = **37.5 GB**。节省 ~4×。

ZeRO-3：分片一切（160 GB → 20 GB/GPU），每块 GPU 需 **20 GB**。节省 ~8×（$= N$）。

PyTorch 的 FSDP（Fully Sharded Data Parallel）本质上等价于 ZeRO Stage 3。它将每个层的参数分片到所有 GPU，前向传播时通过 All-Gather 临时收集完整参数，计算完成后立即释放其他 GPU 的参数；反向传播时同样 All-Gather 收集、计算梯度后 通过 Reduce-Scatter 将梯度分片回各 GPU。这样每块 GPU 在任何时刻只持有一层的完整参数加上 $1/N$ 的总模型状态，实现了极致的内存效率。

5.3.3 张量并行（Tensor Parallelism）

将单个矩阵运算拆分到多块 GPU 上。例如，一个 $[d, 4d]$ 的 FFN 权重矩阵可以沿列切分到 4 块 GPU，每块计算 $[d, d]$ 的子矩阵乘法，然后通过 All-Reduce 合并结果。

Megatron-LM 定义了两种切分方式：

列并行（Column Parallel）：将权重矩阵 $W \in \mathbb{R}^{d \times kd}$ 沿列切分为 $W = [W_1, W_2, \ldots, W_k]$，每块 GPU i 计算 $Y_i = XW_i$。由于激活函数（如 GeLU）是逐元素操作，可以在切分后独立应用，不需要通信。最终通过 All-Reduce 合并结果。

[10] S. Rajbhandari et al., "ZeRO: Memory optimizations toward training trillion parameter models," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC'20)*, 2020.

行并行（Row Parallel）：将权重矩阵沿行切分为 $W = [W_1; W_2; \dots; W_k]^T$，输入也对应切分。每块 GPU 计算部分结果后通过 All-Reduce 求和。

Megatron-LM 的巧妙之处在于 将 FFN 层的第一个线性层做列并行、第二个线性层做行并行——这样两层之间不需要额外通信，只在最终输出时做一次 All-Reduce。

Attention 头的切分也是张量并行的一部分。Multi-Head Attention 天然地将头分配到不同 GPU：如果有 32 个头、4 块 GPU，每块 GPU 处理 8 个头的 Q、K、V 投影和 Attention 计算，最终通过 All-Reduce 合并输出。

张量并行要求 GPU 之间有极高带宽的通信（每次前向和反向传播都需要同步），因此通常只在同一节点内的 GPU 之间使用（通过 NVLink 连接）。跨节点的 InfiniBand（即使是 800 Gb/s）也远不如 节点内 NVLink（H100 提供 900 GB/s 双向带宽）。这就是为什么张量并行的度数通常等于 单节点内的 GPU 数量（DGX H100 为 8）。

5.3.4 流水线并行（Pipeline Parallelism）

将模型的不同层分配到不同 GPU 上，数据像流水线一样依次通过各 GPU。GPU 1 处理第 1–10 层，GPU 2 处理第 11–20 层，以此类推。

流水线气泡问题是核心挑战：如果不做特殊处理，当 GPU 1 在处理前向传播时，GPU 2–4 在等待 GPU 1 的输出；当 GPU 4 在做反向传播时，GPU 1–3 在等待。在简单的流水线调度中，气泡率为 $(P-1)/M$，其中 P 是流水线阶段数，M 是 micro-batch 数。

GPipe 通过将一个 mini-batch 切分为 M 个 micro-batch 来减少气泡：多个 micro-batch 依次进入流水线，让各 GPU 尽可能保持忙碌。但 GPipe 仍然需要先完成所有前向再开始所有反向。

1F1B（One Forward, One Backward）调度更进一步：每完成一个 micro-batch 的前向传播，立即开始另一个 micro-batch 的反向传播。这种交错调度减少了内存峰值（不需要同时保存所有 micro-batch 的激活值），同时保持了流水线的利用率。

Megatron-LM 的交错调度（Interleaved Schedule）进一步减少气泡：每个 GPU 不只负责连续的层，而是负责多个不连续的层段（virtual pipeline stages）。例如，4 块 GPU、32 层的模型，传统方式是 GPU 1 = 层 1–8，GPU 2 = 层 9–16，...；交错方式是 GPU 1 = 层 1–4 和 17–20，GPU 2 = 层 5–8 和 21–24，以此类推。这使得前向和反向的路径更短，气泡更小。

DeepSeek 的 DualPipe 技术进一步优化了流水线调度——将前向和反向计算交错排列，使得一个 micro-batch 的前向与另一个 micro-batch 的反向 可以在不同 GPU 上并行进行，将流水线气泡率降低到约 3%（传统方法通常 20%+）。

5.3.5 专家并行（Expert Parallelism）

对于 MoE 模型（详见 Chapter 6），不同的专家分布在不同的 GPU 上。当一个 token 被路由到某个专家时，它需要通过 All-to-All 通信发送到对应的 GPU，计算完成后再发送回来。

All-to-All 通信是专家并行独有的通信模式。与数据并行的 AllReduce（每块 GPU 发送和接收相同大小的数据）不同，All-to-All 的通信量取决于路由决策——如果某个专家特别「热门」，持有该专家的 GPU 会收到不成比例的大量 token。这就是为什么 MoE 的负载均衡（Chapter 6）如此重要。

容量因子（Capacity Factor）是一个实用的工程参数：它规定每个专家最多能处理 $\mathrm{CF} \times \frac{T}{E}$ 个 token （T = 总 token 数，E = 专家数，CF 通常取 1.0–1.5）。超出容量的 token 会被**丢弃**（token dropping），直接跳过 MoE 层，使用残差连接的输出。训练时适度的 token dropping 对质量影响不大，但它保证了计算和通信的负载是有界的。DeepSeek 通过无辅助损失的负载均衡 大幅减少了 token dropping 的发生。

DeepSeek 开发的 DeepEP 库专门优化了这种通信模式。

5.3.6 上下文并行（Context Parallelism）

当序列长度超过单块 GPU 能处理的范围时（例如 128K 或更长的上下文），需要将序列本身拆分到多块 GPU 上——这就是上下文并行。

Ring Attention 是最著名的方案：将长序列的 Q、K、V 切分到 P 块 GPU 上，每块 GPU 持有序列的 $1/P$ 段。K 和 V 在 GPU 之间以环形方式传递——每个 step 中，GPU i 计算自己的 Q 与当前 K、V 的注意力，同时将 K、V 发送给下一块 GPU。经过 P 步后，每块 GPU 都计算了与所有位置的注意力。通信可以与计算完全重叠，效率很高。

Ulysses（DeepSpeed-Ulysses）采用了不同的切分策略：它沿着**注意力头**的维度切分序列——每块 GPU 处理所有位置但只处理部分头。前向传播时通过 All-to-All 通信交换数据，使得每块 GPU 获得完整的头信息。Ulysses 对 Grouped-Query Attention（GQA）友好，通信量更小，适合中等长度的序列。

5.3.7 多维并行组合

实际训练中，这些并行策略是组合使用的。DeepSeek-V3 的训练使用了**多维并行**：

1. **DP**（数据并行）：不同 GPU 组处理不同数据。
2. **TP**（张量并行）：节点内 GPU 拆分矩阵运算。
3. **PP**（流水线并行）：不同节点处理不同层。
4. **SP**（序列并行）：拆分长序列的归一化和 Dropout 操作（SP 通常与 TP 配合使用，在 TP 切分的基础上沿序列维度分摊非张量并行操作的内存，因此有时不被视为独立的并行维度）。
5. **EP**（专家并行）：MoE 的专家分布在不同 GPU 上。

在 2048 块 H800 GPU 上协调这五个维度，是一项极其复杂的工程挑战。

让我们具体看一个例子来理解这些并行维度如何组合。假设我们有 2048 块 GPU，训练一个 671B MoE 模型（如 DeepSeek-V3），一种可能的配置是：

- **TP = 4**：每 4 块 GPU 协同处理一个层内的矩阵运算（通过 NVLink 连接，带宽最高）。

- **PP = 8**：模型的不同层分布在 8 个流水线阶段（跨节点通信，使用 InfiniBand）。
- **EP = 4**：256 个 MoE 专家分布在 4 个专家并行组（需要 All-to-All 通信）。
- **DP = 16**：剩余的 $2048/(4 \times 8 \times 4) = 16$ 份 用于数据并行（最简单的通信模式）。

这意味着系统被组织成一个四维网格：$16 \times 8 \times 4 \times 4 = 2048$。每一个维度对通信带宽和延迟有不同的要求，这决定了它们在物理拓扑中的位置——TP 在节点内（NVLink），PP 在节点间但延迟敏感，EP 需要高带宽但可以容忍一定延迟，DP 对带宽和延迟最宽容。

另一个参考点是 **LLaMA 3 405B**[11] 在 16384 块 H100 GPU 上的配置，使用了 **4D 并行**：TP = 8，CP = 16，PP = 16，DP = 8 （$8 \times 16 \times 16 \times 8 = 16{,}384$）。这里 TP = 8 对应 DGX H100 节点的 8 块 GPU （全部通过 NVLink 连接），CP = 16（Context Parallelism）用于支持 128K 长上下文训练，将序列拆分到 16 块 GPU 上以分摊 KV cache 内存，PP = 16 将 126 层分为 16 个阶段（每阶段约 8 层），DP = 8 提供数据并行度以维持足够的全局 batch size （约 16M token per batch）。

5.3.8 通信–计算重叠

分布式训练的效率不仅取决于「如何拆分计算」，更取决于「如何隐藏通信开销」。理想状态是：当 GPU A 在计算时，GPU B 在同时发送上一步的结果——通信完全被计算「遮盖」，总时间等于纯计算时间。

实现这一点需要精细的调度。以 ZeRO-3 / FSDP 为例：当 GPU 在计算第 i 层的前向传播时，它可以提前发起第 $i+1$ 层参数的 All-Gather 请求。等第 i 层计算完毕，第 $i+1$ 层的参数已经准备好了，GPU 可以无缝开始第 $i+1$ 层的计算。反向传播时同理：完成第 i 层的梯度后，立即发起 Reduce-Scatter，同时开始第 $i-1$ 层的梯度计算。

梯度分桶（Gradient Bucketing）是另一个关键优化。PyTorch DDP 不会等所有梯度计算完毕后再发起 AllReduce——它将参数分成多个「桶」（bucket），每个桶的梯度计算完毕后立即开始 AllReduce。反向传播是从最后一层开始的，所以最后一层的梯度最先准备好——当中间层还在做反向计算时，最后几层的梯度已经在通信了。好的分桶策略可以让通信和计算几乎完全重叠。

DeepEP 的 Low-Latency 模式更进一步——它使用 GPU 硬件的 hook 机制 将通信操作从 GPU 的 SM（Streaming Multiprocessor）中完全卸载，通信不占用任何计算资源，实现了真正的零开销通信。

DeepSeek 的 **DualPipe** 将通信–计算重叠 推广到了流水线并行的维度。传统的 1F1B 调度中，前向的通信（接收上一阶段的激活值）和计算是串行的；DualPipe 将流水线的前向和反向 拆分为更细粒度的「通信阶段」和「计算阶段」，让来自不同 micro-batch 的通信和计算在时间上交错。其核心思想是：当 GPU 在为 micro-batch A 做计算时，同时为 micro-batch B 做通信——因为计算密集用的是 SM，通信密集用的是 NVLink/IB 控制器，两者可以并行而不冲突。这让 DeepSeek-V3 在 2048 块 H800 上 实现了仅 ~3% 的流水线气泡率。

[11] A. Dubey et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.

5.3.9 训练框架演进（2025–2026）

分布式训练框架在 2025–2026 年经历了重要的融合与进化。

Megatron-FSDP 是 NVIDIA 在 2025 年推出的新一代分布式训练方案，将 Megatron-LM 的高性能张量并行 与 PyTorch FSDP 的灵活分片数据并行统一到同一个框架中。它保留了 Megatron-LM 在 TP/PP 上的成熟优化（如 TransformerEngine 的 FP8 支持、交错流水线调度），同时利用 FSDP 的自动参数分片 替代了手写的 ZeRO 实现。核心特性包括：

- 与 PyTorch DTensor 原生兼容的检查点机制，支持跨不同并行配置的检查点迁移。
- 针对 GB200 NVL72 系统的优化——利用 Blackwell 的 NVLink 5.0（1800 GB/s 双向）和 NVSwitch 4.0 的全互联拓扑。
- 对 MoE 架构的一等支持，包括专家并行与 FSDP 分片的协同。

veScale-FSDP 是字节跳动在 2026 年发布的另一个重要方案，解决了标准 FSDP 在**非逐元素优化器**（如 Muon、Shampoo）和 **块级量化训练**（如 FP8/FP4 的 block scaling）上的兼容性问题。传统 FSDP 将参数展平为一维张量后分片，这破坏了矩阵级优化器所需的二维结构信息。veScale-FSDP 通过结构感知的分片策略保留了参数的原始形状，使得 Muon 等矩阵级优化器可以与 FSDP 无缝集成——这对于 Kimi K2、Gemini 等使用 Muon 优化器的前沿模型尤为重要。

5.3.10 十万卡级别的训练集群

2025–2026 年，训练集群的规模跨入了**十万卡**量级。

xAI Colossus 是目前最大的单站点 AI 训练集群。2025 年中期上线时配备了约 100,000 块 NVIDIA H100 GPU，仅用 122 天就从零建成。到 2026 年 1 月，Colossus 扩展到 555,000 块 GPU，总功耗达 2 GW（相当于两个中型核电站），硬件投入约 180 亿美元。这个集群用于训练 Grok 系列模型，其规模已经**远超**此前任何已知的训练配置——Meta 训练 LLaMA 3 405B 使用了 16,384 块 H100，DeepSeek-V3 仅用了 2,048 块 H800。

在如此规模下运行面临全新的工程挑战：

- **故障率**：假设单块 GPU 的年故障率为 5%，100K GPU 集群中每天平均有 $100000 \times 0.05/365 \approx 14$ 块 GPU 发生故障。系统必须支持**弹性训练**——在不中断整体训练的情况下隔离故障节点并重新分配计算。
- **检查点开销**：100K GPU 的模型状态可能达到数十 TB，频繁保存检查点的 I/O 开销不容忽视。异步检查点写入和分布式文件系统（如 Lustre、DAOS）是必需的基础设施。
- **网络拓扑**：十万卡级别的集群不可能用单一的 Fat-Tree 拓扑——交换机数量和布线复杂度都不允许。xAI 使用了多层 Rail-Optimized + Fat-Tree 混合拓扑，并在软件层面根据通信模式动态选择路由路径。

5.3.11 Blackwell 架构的训练表现

NVIDIA Blackwell 架构（B200/GB200）在 2025 年下半年 进入规模化部署，MLPerf Training v5.0/v5.1 基准测试 提供了第一批公开的性能数据。

GB200 NVL72 是 Blackwell 的旗舰训练系统：72 块 B200 GPU 通过第五代 NVLink 全互联（1.8 TB/s 双向带宽），形成一个共享显存池（72×192 GB = 13.8 TB）。在 MLPerf Training v5.1 中，5,120 块 Blackwell GPU 将 LLaMA 3.1 405B 的训练时间 压缩到**约 10 分钟**——相比 Hopper 架构实现了每 GPU 2.2×–2.6× 的加速。

Blackwell 的关键训练加速来自：

1. **FP4 硬件原生支持**：B200 的第五代 Tensor Core 原生支持 FP4（NVFP4 格式）矩阵乘法，理论 FLOPS 是 FP8 的 2×（9 PFLOPS FP4 vs 4.5 PFLOPS FP8）。
2. **NVLink 5.0**：双向带宽从 H100 的 900 GB/s 提升到 1800 GB/s，直接缓解了张量并行的通信瓶颈。
3. **HBM3e 升级**：192 GB 显存 + 8 TB/s 带宽，使得更大的 batch size 和更长的序列成为可能。

5.4 混合精度训练

5.4.1 数值格式详解

传统的神经网络训练使用 32 位浮点数（FP32）。但研究发现，大多数训练计算可以用更低精度完成而不损失质量。理解不同的数值格式是混合精度训练的基础。

IEEE 浮点数的表示为：$(-1)^s \times 2^{e-\text{bias}} \times (1+m)$，其中 s 是符号位，e 是指数，m 是尾数。不同格式的区别在于分配给指数和尾数的位数：

格式	总位数	符号	指数	尾数	数值范围
FP32	32	1	8	23	$\sim 10^{\pm 38}$
FP16	16	1	5	10	$\sim 6.5 \times 10^4$
BF16	16	1	8	7	$\sim 10^{\pm 38}$
FP8 E4M3	8	1	4	3	~ 448
FP8 E5M2	8	1	5	2	~ 57344

BF16（Brain Float 16）有 8 位指数和 7 位尾数。关键洞察：指数位数与 FP32 完全相同，因此数值范围也完全相同（$\sim 10^{38}$）。代价是尾数从 23 位减少到 7 位——精度大约等于 $2^{-7} \approx 0.78\%$ 的相对误差。但在训练中，随机梯度下降本身就是一个噪声过程，0.78% 的精度损失几乎可以忽略。

FP16（半精度）有 5 位指数和 10 位尾数。它的精度比 BF16 更高（$2^{-10} \approx 0.1\%$ 相对误差），但数值范围只到 ~ 65504。这在训练中是致命的：loss spike 时梯度可能达到 10^5 量级，直接溢出为 `inf`，导致训练崩溃。

为什么 BF16 优于 FP16?

FP16 的数值范围只到 ~ 65504，在训练中很容易溢出（loss spike 时梯度可能非常大）。BF16 的范围与 FP32 相同，几乎不会溢出。虽然 BF16 的精度（7 位尾数 vs. 10 位）稍低于 FP16，但这在训练中影响很小，因为随机梯度下降本身就是一个「噪声」过程。BF16 是 Google TPU 原生支持的格式，NVIDIA 从 A100 开始也全面支持 BF16。现在所有主流 LLM 的预训练都使用 BF16。

5.4.2 FP16 的 Loss Scaling

如果必须使用 FP16 训练（例如在不支持 BF16 的旧硬件上），需要 **Loss Scaling** 来避免梯度下溢。

问题在于：FP16 能表示的最小正规格化数约为 6×10^{-5}，但训练后期很多梯度的量级在 10^{-7} 到 10^{-8}，这些梯度在 FP16 中会被截断为 0（下溢）。解决办法是在计算损失时乘以一个大的缩放因子 S （通常 $S = 2^{16}$ 或更大），使得反向传播中的梯度整体放大 S 倍，避免下溢。在更新参数前再除以 S 恢复真实的梯度值。

动态 Loss Scaling（Dynamic Loss Scaling）自动调整 S：每 K 步，如果没有梯度溢出就增大 S；如果检测到 `inf` 或 `NaN`，就减小 S 并跳过这步更新。

BF16 不需要 Loss Scaling，因为它的数值范围与 FP32 相同——这是 BF16 被广泛采用的另一个重要原因。

5.4.3 AMP：自动混合精度

PyTorch 的 AMP（Automatic Mixed Precision）是混合精度训练的标准实现。核心思想是：

- **前向传播**：在低精度（BF16/FP16）下进行，减少内存和加速计算。
- **主权重**（Master Weights）：在 FP32 下维护一份完整精度的参数副本。
- **梯度更新**：在 FP32 下进行——将低精度梯度累加到 FP32 主权重上。

为什么需要 FP32 主权重？因为参数更新通常很小（$\text{lr} \times \text{grad} \sim 10^{-7}$），如果直接在 BF16 中更新，$\text{param} + \delta$ 可能等于 param——更新被精度「吃掉」了。FP32 的高精度确保了微小的更新不会丢失。

5.4.4 FP8：DeepSeek 的突破

DeepSeek-V3 是首个完全使用 FP8 精度进行端到端训练的大规模模型[12]。FP8 有两种变体：E4M3（4 位指数 + 3 位尾数，用于前向）和 E5M2（5 位指数 + 2 位尾数，用于反向梯度）。

为什么前向用 E4M3、反向用 E5M2？前向传播中，激活值的数值分布相对集中，4 位指数的范围（~ 448）通常够用，多一位尾数（3 vs 2）带来的精度提升更有价值。反

[12] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

向传播中，梯度的数值范围波动更大，5 位指数的更大范围（~ 57344）更重要，牺牲一位尾数是可接受的。

核心挑战在于精度。FP8 的 3 位尾数只能表示 8 个不同的值——直接用于矩阵乘法会导致严重的累积误差。DeepSeek 的解决方案是**细粒度量化**：将权重矩阵切成 128×128 的小块，每个小块有独立的缩放因子；激活值则以 1×128 的粒度量化。

为什么要用 128×128 的块？因为同一个矩阵中，不同区域的数值分布可能差异很大——如果用整个矩阵的统一缩放因子，数值小的区域会被压缩到 FP8 的最低几个值，丢失大量信息。块级缩放让每个小块有自己的「标尺」，大幅提高了量化的保真度。

再加上每 4 次 WGMMA 操作后强制 FP32 累加，将累积误差控制在可接受范围内。WGMMA（Warp Group Matrix Multiply-Accumulate）是 NVIDIA Hopper 架构（H100）引入的 Tensor Core 指令——FP8 的乘法在 Tensor Core 中完成，但部分和定期累加到 FP32 寄存器中，防止小数值被大数值「淹没」。

FP8 训练将显存需求减半，吞吐量提升 30–40%。PyTorch 原生的 `torchao.float8` 包在 2K H200 集群上验证了 1.34–1.43× 的训练加速。

FP8 训练的稳定性挑战

FP8 训练并非无风险。DeepSeek-V3 的技术报告提到，FP8 训练在某些条件下会出现更频繁的 loss spike，尤其是在训练早期模型参数分布还不稳定时。关键的稳定化措施包括：

1. 训练早期（前几千步）使用 BF16 预热，之后切换到 FP8。
2. 对归一化层（RMSNorm）和 Attention 的 softmax 始终保持 FP32/BF16——这些操作对精度极为敏感。
3. 动态缩放因子的更新需要延迟——使用上一步的统计量来设定当前步的缩放因子，避免缩放因子的震荡。

μnit Scaling 是 2025 年提出的一种简化 FP8 训练的方法。传统 FP8 训练需要精心调节动态缩放因子（dynamic scaling factors），而 μnit Scaling 通过系统化的参数初始化和梯度归一化 **完全消除了动态缩放的需要**——FP8 训练变得和 BF16 训练一样简单，无需额外的超参数调节。它还支持跨模型宽度的超参数迁移（hyperparameter transfer），小模型上调好的超参数可以直接用于大模型。

5.4.5 FP4：下一个精度前沿

FP4 训练是 2025–2026 年低精度训练的最新前沿。FP4 仅用 4 位表示一个浮点数——1 位符号 + 2 位指数 + 1 位尾数（E2M1 格式），仅能表示 $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$ 的有限集合（加上对应的负数）。直接用如此粗糙的精度训练似乎不可能——但 2025 年的研究证明了它的可行性。

Quartet（NeurIPS 2025）证明了**原生 FP4 训练**在理论和实验上都可以接近 BF16 的质量。关键技巧是**随机舍入**（Stochastic Rounding, SR）：将 BF16 数值量化到 FP4 时，不是简单地取最近值（Round to Nearest），而是以概率正比于距离进行随机选择。例如，1.3

有 70% 的概率被量化为 1、30% 的概率被量化为 1.5——期望值恰好是 1.3，量化是**无偏**的。这保证了梯度估计的无偏性，是收敛的关键。

NVFP4 是 NVIDIA 为 Blackwell 架构设计的 专有 FP4 格式。与标准 E2M1 不同，NVFP4 使用了优化后的编码方案，在相同的 4 位预算下提供更好的精度分布。在 MLPerf Training v5.1 的 LLaMA 3.1 405B 基准测试中，使用 NVFP4 的 Blackwell GPU 实现了 相比 FP8 Hopper 进一步 1.5×–2× 的训练加速，且模型质量与 BF16 基线无显著差异。

OCP Microscaling（MX）格式是另一条重要路线。Open Compute Project 定义了一组块级微缩放格式：每 32 个元素共享一个 E8M0 的缩放因子（8 位指数），单个元素使用 FP8、FP6 或 FP4 表示。MXFP4 相比 BF16 实现了 3.76× 的内存/传输压缩，2.73× 的运行时开销降低。

Tseng 等人 (2025) 在 AISTATS 上发表了首个接近无损的 MXFP4 训练配方。他们发现直接对 MXFP4 使用随机舍入会导致 高方差（来自块级异常值），损害收敛。解决方案是在量化前应用**随机 Hadamard 变换**——将权重和激活值旋转到一个均匀分布的空间中，理论上界了量化方差，显著改善了训练稳定性。

精度	位宽	相对 BF16 加速	内存节省	成熟度（2026 年）
BF16	16 bit	1×	1×	工业标准
FP8 (E4M3/E5M2)	8 bit	1.3–1.5×	2×	广泛采用
MXFP4	4 bit	2–3×†	~4×	实验阶段
NVFP4 (Blackwell)	4 bit	2–3×	~4×	早期部署

† 需要原生 MXFP4 硬件支持（Blackwell），在 H100 上通过 fake-quantization 模拟则无加速。

5.5 完整的训练循环

理解了分布式训练和混合精度之后，让我们看一个简化但完整的 LLM 预训练循环：

Listing 5.2: Simplified LLM pre-training loop

```python
import torch
from torch.distributed import init_process_group
from torch.nn.parallel import DistributedDataParallel as DDP

# Initialize distributed training
init_process_group(backend="nccl")
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)

# Create model with BF16 precision
model = LLM(vocab_size=128000, d_model=4096, n_layers=32)
model = model.to(dtype=torch.bfloat16, device=local_rank)
model = DDP(model, device_ids=[local_rank])

# Optimizer: AdamW with weight decay
```

```python
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,        # Peak learning rate
    betas=(0.9, 0.95), # Adam betas
    weight_decay=0.1, # L2 regularization
)

# Learning rate scheduler: warmup + cosine decay
scheduler = CosineAnnealingWithWarmup(
    optimizer,
    warmup_steps=2000, # Gradually increase LR
    total_steps=1000000, # Total training steps
    min_lr=3e-5,      # 10% of peak LR
)

# Training loop
for step, batch in enumerate(dataloader):
    tokens = batch["input_ids"].to(local_rank) # (B, T)

    # Forward pass
    logits = model(tokens[:, :-1]) # Predict next tokens

    # Cross-entropy loss
    loss = F.cross_entropy(
        logits.reshape(-1, logits.size(-1)), # (B*T, V)
        tokens[:, 1:].reshape(-1),   # (B*T,)
    )

    # Backward pass
    loss.backward()

    # Gradient clipping (prevent exploding gradients)
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    # Update weights
    optimizer.step()
    scheduler.step()
    optimizer.zero_grad()

    # Logging and checkpointing
    if step % 100 == 0:
        print(f"Step {step}, Loss: {loss.item():.4f}")
    if step % 5000 == 0:
        save_checkpoint(model, optimizer, step)
```

这段代码展示了预训练的核心循环。几个关键细节值得注意：

AdamW 优化器一直是 LLM 预训练的标准选择（尽管 2025 年出现了强有力的挑战者——详见 Section 5.6）。它结合了 Adam 的自适应学习率（每个参数有独立的学习率，基于梯度的一阶和二阶动量）和权重衰减（L2 正则化，防止参数过大）。$\beta_1 = 0.9$ 和 $\beta_2 = 0.95$ 是经过大量实验确定的标准值——β_2 比 Adam 原始论文的 0.999 更小，因为 LLM 训练中数据分布不断变化，需要更快地「遗忘」旧的梯度统计量。

AdamW 与 Adam 的区别值得展开。原始的 Adam 将权重衰减作为 L2 正则项加到损失函数中：$\tilde{L} = L + \frac{\lambda}{2}\|w\|^2$，这导致权重衰减和自适应学习率之间存在耦合——当某个参数的梯度很小（Adam 会给它更大的有效学习率）时，L2 正则的效果也会被放大，行为不直观。AdamW 将权重衰减**解耦**：$w_{t+1} = (1 - \lambda\eta)w_t - \eta\hat{m}_t/\sqrt{\hat{v}_t}$，直接在参数更新步骤中减去 $\lambda\eta w_t$。这使得权重衰减的行为更加可预测，在 LLM 训练中几乎所有团队都使用 AdamW 而非 Adam。

学习率调度是训练稳定性的关键。预热阶段（warmup）从一个很小的学习率（甚至为零）线性增加到峰值学习率。这是因为训练初期，模型参数是随机初始化的，大的学习率可能导致参数剧烈震荡。预热让模型先「稳住」，找到一个合理的参数区域，再以正常速率优化。之后使用余弦退火（cosine annealing）逐渐降低学习率，帮助模型在训练后期收敛到更平坦的最小值——平坦的最小值通常泛化性更好。

还有一种日益流行的替代方案：**WSD**（Warmup–Stable–Decay）调度。它将训练分为三个阶段：短暂的线性 warmup → 长时间的恒定学习率（stable phase）→ 最后阶段快速衰减到最小值。WSD 的优势在于 stable phase 中学习率恒定，更易于在不同长度的训练中复用超参数；而且可以在 stable phase 的任意时刻保存检查点，之后从该检查点开始 decay 阶段——这为「继续训练」（continual pretraining）提供了极大的灵活性。

交叉熵损失中的 `tokens[:, :-1]` 和 `tokens[:, 1:]` 体现了因果语言模型的核心：输入是位置 1 到 $T-1$ 的 token，目标是位置 2 到 T 的 token。模型在每个位置预测下一个 token，损失衡量预测的概率分布与真实 token 之间的差距。

梯度累积（Gradient Accumulation）是上面代码中没有显式展示但实际训练中不可或缺的技巧。当 GPU 显存不足以容纳期望的 batch size 时，可以将一个大 batch 拆成多个 micro-batch，每个 micro-batch 的梯度累加起来，达到一定步数后再执行一次 `optimizer.step()`。效果等价于用更大的 batch 训练，但内存只需一个 micro-batch 的量：

Listing 5.3: Gradient accumulation

```python
accumulation_steps = 8 # Effective batch = 8 * micro_batch_size

for step, batch in enumerate(dataloader):
    loss = model(batch) / accumulation_steps # Scale loss
    loss.backward() # Gradients accumulate in .grad

    if (step + 1) % accumulation_steps == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        optimizer.zero_grad()
```

注意损失需要除以 `accumulation_steps`——因为 `loss.backward()` 会累加梯度，如果不缩放，累积后的梯度是 K 个 micro-batch 梯度的**和** 而非**平均**，等价于用了更大的学习率。

5.6 优化器的演进：超越 AdamW

AdamW 统治 LLM 预训练长达 5 年（2019–2024），但 2025 年出现了真正的挑战者——**Muon 优化器**。

5.6.1 Muon：矩阵正交化的动量

Muon（**Mo**ment**U**m **O**rthogonalized by **N**ewton-Schulz）是一种**矩阵感知**的优化器。AdamW 对每个参数独立地维护一阶/二阶动量，将参数矩阵当作展平的向量处理，忽略了矩阵结构。Muon 则在参数的**矩阵空间**中工作：

核心步骤是对累积动量矩阵 M 进行 **Newton-Schulz 正交化**：

$$M_{\text{orth}} = \text{NS}_k(M) \quad \text{s.t.} \quad M_{\text{orth}}^\top M_{\text{orth}} \approx I \tag{5.8}$$

其中 NS_k 是 k 步 Newton-Schulz 迭代（通常 $k = 5$–10），将动量矩阵投影到最近的正交矩阵上。这等价于取动量的**极分解**中的正交部分——保留梯度的方向信息，丢弃尺度信息。

为什么正交化有帮助？直觉上，正交更新确保了参数空间中 各方向被均匀探索，避免了 Adam 在某些方向上 因自适应学习率的缩放而 “停滞” 的问题。实验表明，Muon 在相同的训练 FLOP 下 可以达到 AdamW 1.5×–2× 的收敛速度——即相同质量的模型只需**一半的训练计算量**。

Muon 的工业级采用在 2025 年迅速扩展：

- Moonshot AI 的 **Kimi K2**（1T 参数 MoE）使用了 Muon 的改进版 **MuonClip**，在 15.5T token 的训练中实现了 “零 loss spike”。
- PyTorch 2.10 将 Muon 纳入了官方优化器库。
- DeepSeek、Meta、OpenAI 等多家实验室 被报道在内部实验中采用了 Muon 或其变体。

MuonClip 是 Kimi K2 团队对 Muon 的工程改进：在 Muon 的基础上加入了 **QK-clip 稳定性技巧**——当 Attention 的 $QK^\top$ 值超过阈值时，对 Q 和 K 的梯度进行裁剪。这解决了 Muon 在超大规模训练中偶尔出现的 注意力分数爆炸问题。

5.6.2 其他值得关注的优化器

Sophia（Second-order Clipped Stochastic Optimization）使用了廉价的 Hessian 对角线估计来设定逐参数的裁剪阈值。理论上它利用了二阶信息（参数更新的步长与该参数的损失

曲率成反比），但避免了完整 Hessian 的巨大计算开销。早期实验声称训练速度是 Adam 的 2×，但后续的大规模复现研究表明 其优势在大模型上不如预期明显。

SOAP（Spectral-regularized Adam with Projection）和 **Shampoo** 属于**预条件**（preconditioning）优化器家族，它们维护参数梯度的 Kronecker 积近似来逼近完整的 Fisher 信息矩阵。这类方法在理论上更优雅，但计算和内存开销较大。

Wen 等人 (2025) 在斯坦福的一项系统性基准测试中评估了 11 种优化器在标准化的 LLM 预训练场景下的表现。使用三阶段坐标下降的严格超参数调优后，他们的关键发现是：

1. **Muon 是最强的挑战者**——在 1B–8B 规模下一致性地优于 AdamW。
2. **公平对比至关重要**——很多 "新优化器优于 Adam" 的结论 源于对 Adam 超参数的不充分调优。当给予 Adam 同等的调优预算后，差距大幅缩小。
3. Shampoo 和 SOAP 在小规模实验中表现优异，但在大规模下的工程成本（额外的矩阵分解操作）部分抵消了优化效果。

优化器选择指南（2026 年）

优化器	内存/参数	收敛速度	适用场景
AdamW	12 bytes	1×（基线）	通用、成熟、风险最低
Muon	~8 bytes	1.5–2×	大规模预训练，需 FSDP 支持
MuonClip	~8 bytes	1.5–2×	超大规模 MoE 训练
Sophia	12 bytes	~1.3×	实验性，大规模收益不确定

5.7 多 Token 预测

传统的因果语言模型在每个位置只预测**一个**下一个 token。**多 Token 预测**（Multi-Token Prediction, MTP）训练模型同时预测未来的 K 个 token，通过引入 $K-1$ 个轻量级的预测头来实现。

5.7.1 为什么多 Token 预测有用？

MTP 的动机有三个层面：

更丰富的训练信号。标准的 next-token 预测只提供 "下一个词是什么" 的反馈。MTP 同时要求模型预测第 2、3、...、K 个未来 token，迫使模型在更深层次上**规划**——要准确预测第 3 个 token，模型必须对未来的语义走向有一个隐式的 "计划"。这种更长视野的训练信号让模型学到了更好的表示。

采样效率。每个训练样本（序列）提供的有效梯度信号 增加了 K 倍——不增加前向计算量（因为预测头非常轻量），却让每个 token 的 "教学价值" 大幅提升。

推理加速。训练好的 MTP 头可以直接用于推理阶段的 **推测解码**（Speculative Decoding）：额外的预测头生成候选 token，主模型验证后一次性接受多个 token，减少了自回归生成的总步数。

5.7.2 DeepSeek-V3 的 MTP 实现

DeepSeek-V3[13] 使用了 $K = 2$ 的 MTP（即预测未来 2 个 token）。其实现架构是**顺序式**的——每个额外的预测头不仅接收主模型的隐状态，还接收**前一个**预测头的输出。具体而言，第 k 个 MTP 头的输入为：

$$\mathbf{h}_t^{(k)} = \mathrm{FFN}^{(k)}\left(\mathrm{Concat}\left[\mathbf{h}_t^{(0)},\ \mathrm{Embed}(\hat{x}_{t+k-1})\right]\right) \tag{5.9}$$

其中 $\mathbf{h}_t^{(0)}$ 是主模型在位置 t 的隐状态，$\hat{x}_{t+k-1}$ 是前一个头预测的 token 的嵌入。$\mathrm{FFN}^{(k)}$ 是一个轻量级的前馈网络（通常只有一层）。

训练损失是主预测头和辅助预测头的加权和：

$$\mathcal{L}_{\mathrm{MTP}} = \mathcal{L}_{\mathrm{NTP}} + \sum_{k=1}^{K-1} \lambda_k \cdot \mathcal{L}_{\mathrm{NTP}}^{(k)} \tag{5.10}$$

其中 λ_k 是辅助头的损失权重（通常 $\lambda_1 = 0.3$）。**推理时丢弃辅助头**——MTP 是一种纯训练时的技巧，不增加推理的计算成本（除非用于推测解码，此时辅助头反而**加速**推理）。

DeepSeek-V3 的实验表明，MTP 带来了 ~0.5% 的训练损失改善（在 14.8T token 训练结束时），这听起来很小，但在前沿模型中 每 0.1% 的损失改善都对应着显著的下游任务性能提升。

5.7.3 MTP 训练课程

Aynetdinov & Akbik (2025) 在 ACL 2025 上提出了 MTP 的**训练课程**策略：不是从训练一开始就使用 MTP，而是先用标准的 NTP 训练一段时间让模型建立基础能力，之后再引入 MTP 辅助头。他们发现这种 “渐进式” 引入在小模型上 可以显著改善 MTP 的效果——因为训练早期模型的预测非常不准确，多 token 预测的信号噪声比太低，反而可能干扰主模型的学习。

5.8 训练稳定性

训练一个数十亿参数的模型数周到数月，过程中不出任何问题是不现实的。训练不稳定通常表现为 **loss spike**——损失值突然跳升，如果不处理可能导致训练完全崩溃。

5.8.1 Loss Spike：原因与应对

Loss spike 的成因多种多样，常见的包括：

数据质量问题是最常见的原因。训练数据中混入了乱码、极长的重复文本、或格式异常的文档，这些异常数据会产生异常大的梯度，导致参数在一步之内偏离到不正常的

[13] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.

区域。LLaMA 3[14] 的训练日志显示，他们通过仔细的数据清洗将 loss spike 的频率降低了 90%。

学习率过高是另一个常见原因，尤其是在训练初期 warmup 不够充分时。模型在高学习率下跨过了损失景观中的「悬崖」（loss cliff），参数跳到了一个完全不同的区域，损失急剧上升。

数值不稳定也是重要原因。在低精度训练中（FP16/FP8），某些中间计算可能超出数值范围。例如 Attention 计算中 $QK^\top/\sqrt{d}$ 的值可能很大，softmax 的指数运算进一步放大，导致溢出。Flash Attention[15] 通过在线 softmax （逐块计算并维护最大值）缓解了这个问题。

应对策略：

1. **检查点回滚**：发现 loss spike 后，回退到最近的正常检查点重新开始。这要求频繁保存检查点（通常每 500–1000 步保存一次，同时保留最近 3–5 个检查点）。
2. **跳过异常 batch**：检测到梯度范数异常大的 batch 直接跳过，不执行参数更新。
3. **临时降低学习率**：发现 loss 异常上升后，临时将学习率降低 2–10 倍，让模型恢复稳定，再逐渐恢复正常学习率。
4. **提高数值精度**：对关键操作（归一化、softmax）始终使用 FP32。

5.8.2 梯度裁剪

常用的稳定性措施包括：**梯度裁剪**将梯度的全局 L2 范数限制在某个阈值内：

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \|\mathbf{g}\| \le \tau \\ \tau \cdot \frac{\mathbf{g}}{\|\mathbf{g}\|} & \|\mathbf{g}\| > \tau \end{cases} \tag{5.11}$$

其中 τ 通常设为 1.0。这意味着每步参数更新的方向保持不变，但如果梯度「太大」，步长会被缩放回安全范围。全局范数裁剪（而非逐参数裁剪）的优势在于 保持了不同参数梯度之间的相对比例关系。

5.8.3 权重衰减与 AdamW

权重衰减（Weight Decay）是 LLM 训练中 普遍使用的正则化手段，典型值为 $\lambda = 0.1$。它的效果是持续地将参数向零「拉回」——在每步更新中，参数先衰减 $(1 - \lambda\eta)$ 倍，然后再按梯度方向更新。

直觉上，权重衰减防止了任何单个参数变得「过大」——如果某个参数对损失的贡献不大（梯度接近零），权重衰减会把它逐渐压到接近零；只有对损失有显著贡献的参数才能「抵抗」衰减而保持非零值。这是一种隐式的特征选择。

[14] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[15] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

值得注意的是，权重衰减通常**不应用于偏置项和归一化参数**——这些参数的量级本身就不大，衰减它们反而会损害模型表达能力。

5.8.4 真实案例：大规模训练的挑战

大规模训练中的不稳定性不是理论上的担忧——它们在实践中反复出现：

PaLM[16] 在 6144 块 TPU v4 上训练时 经历了 20 多次 loss spike。Google 的解决方案是：每次 spike 发生后，回滚到 spike 前约 100 步的检查点，**跳过当时的数据 batch**，然后继续训练。他们发现大多数 spike 与特定数据 batch 相关——同样的数据 batch 在不同训练阶段都会引发 spike。

OPT-175B 的训练日志 是公开可查的，记录了一段混乱的训练过程：硬件故障、loss spike、超参数手动调整、训练中途改变学习率等。这份真实的训练日志被社区戏称为「OPT 训练日记」，它生动地展示了大模型训练远非「设置好超参数后等结果」那么简单——它更像是一场需要 24/7 值守的「手术」。

DeepSeek-V3[17] 的训练过程 则是一个正面案例——整个 14.8T token 的训练过程中**没有发生过** 不可恢复的 loss spike 或需要回滚的情况。这得益于他们精心设计的训练基础设施（FP8 细粒度量化、无辅助损失的负载均衡、DualPipe 调度等）以及严格的数据质量控制。

5.9 硬件基础设施

5.9.1 GPU 选择与代际跃升

2025–2026 年的主力训练 GPU 是 NVIDIA H100 和 H200。这几代 GPU 的性能跃升幅度堪称惊人：

	A100	H100	H200	B200
显存	80 GB HBM2e	80 GB HBM3	141 GB HBM3e	192 GB HBM3e
显存带宽	2.0 TB/s	3.35 TB/s	4.8 TB/s	8.0 TB/s
BF16 算力 †	312 TFLOPS	989 TFLOPS	989 TFLOPS	2.25 PFLOPS‡
FP8 算力 †	–	1979 TFLOPS	1979 TFLOPS	4.5 PFLOPS‡
NVLink	600 GB/s	900 GB/s	900 GB/s	1800 GB/s
架构	Ampere	Hopper	Hopper	Blackwell

† Dense Tensor Core TFLOPS（不含稀疏加速）。启用结构化稀疏（2:4 sparsity）后理论峰值翻倍，但训练中通常不使用结构化稀疏。‡ B200 数据来自 NVIDIA 官方规格表（SXM 版本），实际训练吞吐量取决于软件生态成熟度。

[16] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[17] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

从 A100 到 H100，BF16 算力提升了 ~3.2×，得益于 Hopper 架构引入的第四代 Tensor Core。H100 还新增了 FP8 支持，进一步翻倍。H200 保持与 H100 相同的计算芯片，但将显存升级到 141 GB HBM3e——这意味着可以在**不减少 GPU 数量**的情况下 放下更大的模型或使用更大的 batch size。B200（Blackwell）则是全面升级：192 GB 显存、4.5 PFLOPS FP8 算力、1800 GB/s NVLink，甚至支持 FP4 精度。

Google 的 TPU（Tensor Processing Unit）是另一个重要的训练加速器。TPU v5e 和 v6e 在 Google Cloud 上可用，Gemini 系列模型就是在 TPU 上训练的。TPU 的核心优势在于极高的芯片间互联带宽 和与 Google 基础设施的深度集成。

DeepSeek 使用的是 H800——H100 的中国出口限制版本，NVLink 带宽从 900 GB/s 降到 400 GB/s，跨节点互联也受到限制。但 DeepSeek 通过出色的软件优化（如 DeepEP 的通信–计算重叠策略、DualPipe 的流水线调度）不仅弥补了硬件限制，甚至在某些效率指标上超越了使用无限制 H100 的西方实验室。这充分说明了软件工程的力量——在硬件被限制的情况下，聪明的算法和精细的系统优化可以带来巨大的效率提升。

5.9.2 互联网络：数据中心的「神经系统」

训练集群的网络拓扑至关重要。不同层级的通信有不同的带宽和延迟特征：

GPU 间通信（节点内）：NVLink 是 NVIDIA 的专有互联协议，H100 提供 900 GB/s 的双向带宽（每方向 450 GB/s）。8 块 H100 通过 NVSwitch 全互联——任意两块 GPU 之间都有 900 GB/s 的带宽，不存在带宽瓶颈。NVSwitch 充当 GPU 之间的「交换机」，支持 All-Reduce、All-Gather 等集合通信操作的硬件加速。

节点间通信：InfiniBand（IB）是高性能计算的标准互联。NDR（Next Data Rate）标准提供 400 Gb/s（50 GB/s），XDR 提供 800 Gb/s（100 GB/s）。RoCE（RDMA over Converged Ethernet）是以太网上的 RDMA 方案，成本更低但性能和稳定性略逊于 IB。

网络拓扑设计直接影响训练效率。传统的 **Fat-Tree** 拓扑保证了任意两个节点间 等带宽的通信路径，但成本极高（需要大量高端交换机）。**Rail-Optimized** 拓扑是一种更经济的替代方案：每个节点的第 i 块 GPU 连接到同一个 rail 交换机，同一 rail 上的 GPU 之间通信最快。这种拓扑与张量并行（节点内，NVLink）+ 数据并行（同 rail 跨节点，IB）的组合特别契合。

5.9.3 集群设计与成本

一个典型的 DGX H100 系统包含 8 块 H100 GPU，售价约 $250K–$300K（约 $30K–$37K per GPU）。

训练一个 frontier LLM 需要多少钱?

Meta 的 LLaMA 3-405B[18] 使用 16384 块 H100 GPU 训练了约 54 天，仅 GPU 租赁成本估计超过 1 亿美元。而 DeepSeek-V3 仅用 2048 块 H800，训练 2 个月，成本约 560 万美元——性能相当，成本降低了 **约 20 倍**。

[18] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

这个巨大的成本差距来自 MoE 架构（每个 token 只激活 5.5% 的参数）、FP8 训练（减少内存和计算需求）以及高效的工程实现。它证明了：在 LLM 训练中，**聪明的工程设计可以弥补（甚至超越）硬件优势**。

云端 vs 自建的经济学也值得考量。按云端 H100 的典型租价 ~$2–3/GPU·hour 计算，16384 块 GPU 训练 54 天的成本约为 $16384 \times 54 \times 24 \times \$2.5 \approx \$53$M。自建集群的前期投入更大（16K H100 的硬件成本约 $500M，加上电力、冷却、机房等），但如果集群能持续运行多个训练任务，长期来看单次训练的分摊成本更低。这就是为什么 Meta、Google、DeepSeek 等 频繁训练模型的机构都选择自建——而初创公司或偶尔训练的团队更倾向于云端租用。

5.9.4 GPU 利用率：最重要的效率指标

在大规模训练中，最关键的效率指标是 **MFU** （Model FLOPs Utilization，模型浮点运算利用率）——实际的计算吞吐量占 GPU 理论峰值的百分比。

更精确地定义：

$$\text{MFU} = \frac{\text{模型每步的理论 FLOP 数/每步实际耗时}}{\text{GPU 理论峰值 FLOPS} \times \text{GPU 数量}} \tag{5.12}$$

分子是「如果所有计算资源都用于模型计算，每秒能做多少 FLOP」；分母是硬件的理论上限。MFU 衡量的是「有多少算力真正用在了模型计算上」，其余部分被通信等待、内存访问、流水线气泡等「浪费」了。

理想的 MFU 是 100%，但实际中远达不到。原因包括：通信等待时间、内存带宽瓶颈、流水线气泡、数据加载延迟等。典型的 MFU 值：

- **优秀**：50–60%（如 PaLM 在 6144 TPU 上达到 57%）
- **良好**：40–50%（大多数精心优化的训练任务）
- **一般**：30–40%（较大的通信开销或未优化的配置）
- **差**：<30%（严重的瓶颈或配置问题）

LLaMA 3[19] 报告的 MFU 为 38–43% （16384 块 H100 GPU）。这个数字看起来不高，但考虑到 16K GPU 的规模 以及 DP+TP+PP 三维并行的通信开销，已经是非常出色的工程成就。

每提升 1% 的 MFU，在 2048 块 GPU 的训练中 可以节省约 1% 的训练时间——对于两个月的训练周期，这意味着节省了约 15 小时的 GPU 时间，换算成成本是数万美元。这就是为什么基础设施工程师对每一个百分点都精打细算。

[19] A. Dubey et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.

CHAPTER 6

MIXTURE OF EXPERTS——参数效率的极致

6.1 核心思想：不是每个神经元都需要工作

想象一个公司有 256 名专家，分别精通不同领域。当一个任务到来时，你不需要让所有 256 人都参与——你只需要挑选最相关的 8 个人来处理。这就是 Mixture of Experts (MoE) 的核心思想。

6.1.1 历史脉络

MoE 的思想远比深度学习更古老。1991 年，Jacobs 等人提出了最早的 MoE 框架——用多个「专家」网络分别处理输入空间的不同区域，一个「门控」网络决定每个输入由哪些专家处理。但这个想法在之后的 25 年里基本停留在学术论文中，没有被大规模应用。

2017 年，Shazeer 等人[1] 将 MoE 引入了 NLP 领域，在 LSTM 语言模型中嵌入了稀疏门控的 MoE 层，用 >1000 个专家实现了当时的最大模型（137B 参数）。这篇论文的标题直接叫做 “Outrageously Large Neural Networks”——「大得离谱的神经网络」。

2022 年，Switch Transformer[2] 将 MoE 扩展到 Transformer 架构，用 top-1 路由（每个 token 只发给 1 个专家）训练了一个 1.6T 参数的模型。Switch Transformer 证明了一个关键点：**即使只激活一个专家，只要专家数量足够多、路由足够准确，模型性能就可以超越同规模的稠密模型**。

自此，MoE 从一种学术好奇心 变成了 LLM 训练的主流架构选择。

6.1.2 稠密模型 vs 稀疏模型

传统的「稠密」模型（Dense Model）在处理每个 token 时 使用所有参数——参数量直接决定了计算量。MoE 模型则打破了这种绑定：**总参数量决定了模型的知识容量，但每个 token 的计算量只取决于被激活的参数**。

DeepSeek-V3 的数据最能说明这种效率：总参数 671B（知识容量巨大），每个 token 仅激活 37B（计算成本低），激活率约 5.5%。

用一个类比来理解：稠密模型像一个通才——不管什么问题，他调动全部知识和精力来回答，效果不错但很累。MoE 模型像一个高效的团队——不管什么问题，先判断这是

[1] N. Shazeer et al., “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations*, 2017.

[2] W. Fedus et al., “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

什么类型的问题，然后把最擅长的几个人召集起来处理，其他人可以休息。团队的总知识量（总参数）可以很大，但每次实际消耗的精力（计算量）很小。

6.2 路由机制

MoE 层替换了 Transformer 中的 FFN 层。对于输入 token $\mathbf{x}$，路由器（一个简单的线性层）计算每个专家的分数，然后选择得分最高的 K 个专家：

$$g_i = \frac{e^{s_i}}{\sum_{j \in \text{TopK}} e^{s_j}}, \quad s = W_{\text{router}}\mathbf{x}, \quad \mathbf{y} = \sum_{i \in \text{TopK}} g_i \cdot \text{Expert}_i(\mathbf{x}) \tag{6.1}$$

路由器本身极其轻量——它只是一个 $[d, N_{\text{experts}}]$ 的线性投影，没有偏置，没有激活函数。它的参数量微不足道（$4096 \times 256 \approx 1M$），但它做出的决策至关重要：将每个 token 送到最合适的专家去处理。

训练过程中，路由器通过梯度下降与专家一起优化。一个有趣的现象是路由器会自发学到有意义的分工：某些专家可能专注于代码相关的 token，某些专注于数学符号，某些专注于特定语言的文本。这种**自动专业化**不是人为设定的——它完全是训练过程中涌现出来的。

6.2.1 Top-K 路由

Top-K 路由是目前最主流的方案。K 的选择涉及质量与效率的权衡：

Top-1（Switch Transformer[3]）：每个 token 只发给 1 个专家。计算效率最高，但单个专家可能无法充分处理复杂 token。而且 top-1 的路由决策更「尖锐」——一旦选错专家，没有其他专家来补救。

Top-2（Mixtral[4]、GShard）：每个 token 发给 2 个专家，结果按路由权重加权求和。这是一种「保险」策略——即使一个专家不太合适，另一个通常能弥补。Top-2 是最常见的选择，在质量和效率之间取得了良好平衡。

Top-8（DeepSeek-V3[5]）：每个 token 发给 8 个专家。看起来计算量更大？不——因为 DeepSeek 使用了 256 个**细粒度小专家**，每个专家只有标准专家的 ~1/32 大小。选 8 个小专家的总计算量与选 2 个大专家相当，但组合空间爆炸式增长（从 28 种增加到 44 亿种）。

6.2.2 Token Choice vs Expert Choice

上述路由方式都是 **Token Choice**——每个 token 独立选择自己要去的专家。这会导致一个固有问题：热门专家可能收到过多 token，冷门专家闲置。

[3] W. Fedus et al., "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[4] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[5] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

Zhou 等人（2022）提出了 **Expert Choice** 路由：反过来，让专家选择 token。每个专家从所有 token 中选取得分最高的 k 个来处理。这天然保证了负载均衡——每个专家处理相同数量的 token，不存在过载或空闲。

但 Expert Choice 有一个问题：某些 token 可能被多个专家选中（计算冗余），而某些 token 可能不被任何专家选中（信息丢失）。在实践中，Token Choice + 负载均衡机制 仍然是更主流的方案。

6.2.3 其他路由策略

Hash 路由是最简单的基线：根据 token 的哈希值确定性地分配到专家。不需要可学习的路由器，负载完全均衡，但缺乏语义感知——代码 token 和诗歌 token 可能被分到同一个专家。实验表明 Hash 路由的性能显著低于学习型路由。

随机路由也是一个有用的基线：随机均匀地将 token 分配到专家。在某些配置下，随机路由的表现出人意料地不差——因为即使路由是随机的，专家本身仍然可以通过梯度下降 学到对不同类型输入的通用处理能力。这提示了一个有趣的问题：路由器到底有多重要？答案是：在大规模训练中，学习型路由**显著**优于随机路由，但差距不如你直觉认为的那么大。

6.2.4 动态路由与可微路由（2025–2026 前沿）

标准的 Top-K 路由有一个根本局限：**每个 token 固定激活 K 个专家**，无论 token 的复杂度如何。简单的 token（如 “the”）可能只需要 1 个专家就够了，而复杂的 token（如数学公式中的关键符号）可能需要 5 个以上的专家协同处理。

DynaMoE (2026) 放松了这两个约束：（1）每个 token 激活的专家数量根据输入复杂度 动态变化（从最少 1 个到层级相关的最大值）；（2）不同层的专家总容量可以不同——底层可能需要更多专家处理低级特征的多样性，顶层可能需要更少但更专精的专家。DynaMoE 提出了 6 种容量调度策略（线性递增、线性递减、中间峰值等），让模型自动学习最优的层间容量分配。

DirMoE 是 ICLR 2026 上提出的**完全可微**的概率路由器。传统的 Top-K 路由不可微——“选择第 3 和第 7 个专家” 这个离散决策 无法产生干净的梯度信号。DirMoE 将路由分解为两个可微的概率过程：

- **Bernoulli 过程**决定每个专家是否被激活（连续松弛的 Gumbel-Sigmoid）。
- **Dirichlet 过程**决定被激活专家之间的权重分配。

这种分离使得路由在端到端训练中拥有干净的梯度流。DirMoE 还引入了基于 Simpson 指数的 “稀疏度旋钮”——通过一个标量超参数精确控制期望的活跃专家数量，**无需辅助损失**来维持负载均衡。实验表明 DirMoE 在匹配标准 MoE 吞吐量的同时 实现了更稳定的路由和更均匀的专家利用率。

L2R（Low-Rank & Lipschitz-Controlled Routing）从另一个角度改进路由：在低秩潜在空间中执行路由决策，并通过 Lipschitz 控制确保路由函数的平滑性。直觉上，标准的

线性路由器在高维空间中 可能因 “角度集中” 现象而产生 几乎相同的分数——微小的输入变化导致路由决策翻转。L2R 通过约束路由函数的 Lipschitz 常数，确保了路由决策的稳定性。

用代码来看，MoE 路由器的核心逻辑可以用几行实现：

Listing 6.1: MoE router implementation

```
class MoERouter(nn.Module):
   def __init__(self, d_model, n_experts, top_k):
      super().__init__()
      self.gate = nn.Linear(d_model, n_experts, bias=False)
      self.top_k = top_k

   def forward(self, x):
      # x: (batch, seq_len, d_model)
      scores = self.gate(x) # (batch, seq_len, n_experts)

      # Select top-k experts per token
      topk_scores, topk_ids = scores.topk(self.top_k, dim=-1)

      # Normalize scores (softmax over selected experts only)
      topk_weights = F.softmax(topk_scores, dim=-1)

      return topk_weights, topk_ids
```

这段代码展示了路由的三个步骤：线性投影得到每个专家的分数，TopK 选出得分最高的 K 个专家，在选出的专家上做 softmax 得到归一化的权重。这些权重决定了各专家输出的贡献比例。其中 g_i 是归一化后的路由权重，只在被选中的 TopK 专家上计算 softmax。

6.3 负载均衡：MoE 训练的核心挑战

6.3.1 为什么负载不均衡是灾难性的

MoE 训练中最棘手的问题是**专家坍缩**（expert collapse）：路由器学会将绝大多数 token 发送到少数几个「强」专家，而大部分专家几乎不接收任何 token，逐渐退化为随机噪声。

为什么这是灾难性的？两个层面：

计算浪费：假设有 64 个专家分布在 64 块 GPU 上，如果 80% 的 token 都被路由到 4 个专家，那么只有 4 块 GPU 在满负荷工作，另外 60 块 GPU 基本在空转——你花了 64 块 GPU 的钱，只得到了 4 块 GPU 的效果。在分布式训练中，整个系统的速度受最慢节点（最忙的专家）制约，所以负载不均衡直接导致训练吞吐量暴跌。

知识容量浪费：MoE 的核心优势是「用总参数量换知识容量」。如果大部分专家闲置，等价于这些参数不存在——671B 参数的 MoE 模型可能退化为 只有几十 B 参数的效果。

6.3.2 传统方案：辅助损失

最早的解决方案[6] 是在训练目标中加入一个辅助损失来惩罚不均衡：

$$L_{\text{total}} = L_{\text{LM}} + \alpha \cdot L_{\text{balance}} \tag{6.2}$$

其中辅助损失的形式为：

$$L_{\text{balance}} = N \cdot \sum_{i=1}^{N} f_i \cdot P_i \tag{6.3}$$

这里 f_i 是实际被分配到专家 i 的 token 比例，P_i 是所有 token 对专家 i 的平均路由概率。N 是专家数量，用于归一化。

直觉上，$f_i \cdot P_i$ 的乘积形式 鼓励两者同时尽可能均匀：当所有专家均匀接收 token 时（$f_i = P_i = 1/N$），$\sum f_i P_i = N \times (1/N)^2 = 1/N$，达到最小值。当负载高度不均衡时（所有 token 集中在一个专家），$\sum f_i P_i = 1$，达到最大值。

Switch Transformer[7] 简化了这个公式，使用 $\alpha = 0.01$ 作为默认系数，并证明即使是 top-1 路由，只要辅助损失设置得当，训练也能保持稳定。

但辅助损失有一个**根本矛盾**：α 太大，负载均衡得到满足，但辅助损失的梯度「污染」了语言建模的梯度——路由器不再纯粹地为质量最优而路由，而是在「质量」和「均衡」之间妥协，模型质量下降。α 太小，辅助损失形同虚设，专家坍缩照常发生。在实践中，α 的调节需要大量实验，且没有一个值能适用于训练的所有阶段。

6.3.3 DeepSeek 的无辅助损失方案

DeepSeek-V3[8] 提出了一个优雅的替代方案：**不加辅助损失**，而是给每个专家的路由分数加一个**不参与梯度计算**的偏置项 b_i：

$$s_i = W_{\text{router}}\mathbf{x} + b_i \quad (b_i \text{ 不参与反向传播}) \tag{6.4}$$

偏置的更新规则极其简单：如果第 i 个专家过载，$b_i \leftarrow b_i - \gamma$；如果空闲，$b_i \leftarrow b_i + \gamma$（$\gamma \approx 10^{-3}$）。

因为 b_i 不在计算图中，语言建模的梯度完全不受干扰——训练目标纯粹是预测下一个 token，没有任何妥协。2025 年芝加哥大学的理论论文证明了这种方法等价于 原始–对偶分配问题的在线求解，具有单调改进保证。

这个方法的优雅之处在于它将两个不同的优化问题**解耦**了：语言建模（预测下一个 token）通过标准的梯度下降优化，负载均衡通过偏置项的启发式规则调整。两者互不干扰——语言建模的梯度不受负载均衡的影响，负载均衡的调整也不需要通过梯度传播。

[6] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[7] W. Fedus et al., "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[8] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

这就像一个工厂的生产线和排班表是分开管理的——生产线的优化不需要考虑排班约束，排班表的调整也不需要改变生产流程。

负载均衡方法对比

方法	需要辅助损失	影响 LM 梯度	代表模型
辅助损失	是	是（α 敏感）	Switch Transformer
Expert Choice	否	部分	Expert Choice Routing
偏置项调节	否	否	DeepSeek-V3

6.4 DeepSeek 的三大创新

DeepSeek-V3[9] 在 MoE 上做了三项改变游戏规则的创新。

6.4.1 细粒度专家

传统 MoE 使用少量大专家（如 Mixtral 的 8 个专家，每个约 7B 参数）。DeepSeek 使用 **256 个小专家**（每个仅约 0.5B 参数）加 1 个**共享专家**（处理所有 token 的通用知识）：

$$\mathbf{y} = \text{SharedExpert}(\mathbf{x}) + \sum_{i \in \text{Top8}} g_i \cdot \text{Expert}_i(\mathbf{x}) \tag{6.5}$$

256 个专家中选 8 个，组合数为 $\binom{256}{8} \approx 4.4 \times 10^9$——相比 Mixtral 的 $\binom{8}{2} = 28$ 种组合，组合空间扩大了 10^8 倍。这意味着模型可以为不同类型的输入激活完全不同的专家组合，实现更精细的专业化。

为什么细粒度专家更好？三个原因：

更灵活的组合。8 个大专家中选 2 个，就像只有 8 门课的课程表中选 2 门——选择空间很有限。256 个小专家中选 8 个，就像 256 门课中选 8 门——可以精确匹配每个学生（token）的需求。

更好的专业化。小专家被迫在一个更窄的领域内做到精通，因为它们的参数量有限，无法成为「通才」。大专家则可能在多个不相关的领域之间分散注意力。实验表明，细粒度专家的专业化程度更高——某些专家可能只处理 Python 代码中的列表推导，另一些只处理数学证明中的符号推理。

相似的计算成本。关键洞察：8 个小专家的总计算量 ≈ 2 个大专家的计算量（如果大小比例为 1:4）。所以细粒度专家不增加每个 token 的计算成本——它只是用更多的小模块替代了少数大模块。

[9] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

6.4.2 共享专家

在上面的公式中，SharedExpert(**x**) 是一个始终被激活的专家——不管 token 是什么类型，它都参与计算。

为什么需要共享专家？在纯路由的 MoE 中，每个路由专家都需要独立学会一些「基础知识」（如语法规则、常见词的语义等），因为不能保证某个 token 每次都被路由到同一个专家。这导致了跨专家的冗余——256 个专家中可能有 200 个都各自学了一遍「the 后面通常跟名词」。

共享专家将这些通用知识集中到一个始终激活的模块中，路由专家只需学习**差异化**的专业知识。这既减少了冗余，又让路由专家能更专注于自己的领域。

从残差连接的视角看，共享专家提供了一个强大的「默认输出」——即使路由器把 token 送到了不太合适的专家，共享专家的输出仍然保证了基本的质量。

6.4.3 DeepEP：专用通信库

MoE 的一个工程瓶颈是 All-to-All 通信：每个 token 需要被发送到对应专家所在的 GPU，计算完成后再发送回来。这种细粒度的跨 GPU 通信模式不同于数据并行的 All-Reduce，标准通信库的效率很低。

为什么 All-to-All 比 AllReduce 更难优化？AllReduce 中每块 GPU 发送和接收的数据量是确定性的——总是 M/N 的数据。All-to-All 中数据量取决于路由决策——如果某个专家特别热门，对应 GPU 可能收到 10 倍于平均水平的数据，而其他 GPU 几乎收不到数据。这种**不均匀的通信模式** 让带宽利用和延迟优化变得困难。

DeepSeek 开源的 DeepEP 提供两种模式：

Normal 模式用于训练和 prefill（高吞吐量优先）：将大量 token 的路由请求打包成批次，利用 NVLink（节点内）和 InfiniBand（节点间）的 分层通信策略——同一节点内的专家通信走 NVLink（900 GB/s），跨节点的专家通信走 IB（400 Gb/s），两者可以并行进行。

Low-Latency 模式用于推理 decode （通过 hook-based 通信–计算重叠实现零额外 SM 占用）。在 decode 阶段，每个时间步只有 1 个 token 需要路由，通信量很小但对延迟极其敏感。DeepEP 利用 GPU 的硬件 hook 机制 将通信操作卸载到 NVLink/IB 控制器，SM 完全不参与通信——计算和通信真正同时进行，零开销。

6.5 MoE 在实践中

6.5.1 Mixtral 8x7B：MoE 的开源先驱

Mixtral 8x7B[10] 是 Mistral AI 发布的 第一个广泛使用的开源 MoE 模型。它的规格说明了 MoE 的效率优势：

- **总参数**：47B（8 个 FFN 专家 × ~5.5B + 非 FFN 参数）

[10] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

- **活跃参数**：13B（每个 token 激活 2 个专家）
- **性能**：在大部分基准测试上 匹配或超越 LLaMA 2 70B（稠密模型）
- **推理成本**：约为 LLaMA 2 70B 的 1/5

47B 总参数 vs 70B 稠密参数，Mixtral 的参数总量反而更少——但它用 MoE 的稀疏激活实现了类似甚至更好的性能，同时推理时只需要 13B 的计算量。

6.5.2 DeepSeek-V3：MoE 的巅峰

DeepSeek-V3[11] 将 MoE 推向了新的高度：

- **总参数**：671B
- **活跃参数**：37B（256 路由专家选 8 + 1 共享专家）
- **训练数据**：14.8T token
- **训练成本**：~$5.6M（2048 块 H800，约 2 个月）
- **性能**：在多项基准测试上达到或超越 GPT-4o、Claude 3.5 Sonnet

$5.6M 的训练成本令整个行业震惊——它比同等性能的 LLaMA 3 405B 的估计训练成本低约 20 倍。成本优势来自三方面叠加：MoE 架构（37B 活跃 vs 405B 全激活）、FP8 训练（减少 ~50% 内存和 ~30% 计算）、以及精细的工程优化（DualPipe、DeepEP 等）。

6.5.3 LLaMA 4：Meta 的 MoE 转型

2025 年 4 月，Meta 发布了 LLaMA 4 系列——这是 LLaMA 家族首次采用 MoE 架构，标志着 Meta 从稠密模型全面转向 MoE。

LLaMA 4 包含三个变体：

模型	总参数	活跃参数	专家数 / Top-K	特色
Scout	109B	17B	16 / 1	10M token 上下文
Maverick	400B	17B	128 / 1	原生多模态
Behemoth	~2T	288B	16 / 2	训练中（未发布）

LLaMA 4 的 MoE 设计与 DeepSeek 有显著差异：

Top-1 路由。Scout 和 Maverick 都使用 Top-1 路由（每个 token 只发给 1 个专家），而不是 DeepSeek 的 Top-8。这是一个大胆的选择——Top-1 在 Switch Transformer 中曾被证明可行，但需要更精确的路由器。Meta 通过从 Behemoth（Top-2）的知识蒸馏 来弥补 Top-1 路由的信息损失。

知识蒸馏。Scout 和 Maverick 都通过从 Behemoth 蒸馏获得了 显著的质量提升——这是一种 “大模型训练小模型” 的策略。Behemoth 本身有 288B 活跃参数（Top-2 路由 16 个专家），在 STEM 基准测试上超越了 GPT-4.5 和 Claude Sonnet 3.7。

超长上下文。Scout 支持 **10M token 上下文窗口**——目前最长的生产级上下文长度。这通过高效的 iRoPE 位置编码 和分块注意力机制实现，但也对 MoE 的路由一致性提出了更高要求。

[11] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.

6.5.4 Qwen3-235B：阿里巴巴的 MoE 实践

Qwen3-235B-A22B 是阿里巴巴 Qwen 团队发布的旗舰 MoE 模型，采用了与 DeepSeek 相似但不同的设计选择：

- **总参数**：235B，**活跃参数**：22B。
- **128 个路由专家，Top-8 路由**——与 DeepSeek-V3 的 256 专家 + Top-8 相比，专家数减半但每个专家更大。
- **94 层**，使用 GQA（64 头，4 KV 头）。
- 支持 128K token 上下文，通过 YaRN 位置编码外推。

Qwen3 的一个独特创新是**思考/非思考双模式**——模型可以在 "思考模式"（生成内部推理链）和 "直接回答模式" 之间切换，用户通过 prompt 控制推理预算。这对 MoE 路由提出了新要求：思考模式下的 token 可能需要不同的专家组合 来处理推理链中的中间步骤。

6.5.5 Kimi K2：万亿参数的 MoE

Moonshot AI 的 Kimi K2 将 MoE 推到了**万亿参数**量级：

- **总参数**：~1T（1.04T）。
- **活跃参数**：32B。
- **384 个路由专家，Top-8 路由**——比 DeepSeek-V3 更多的专家（384 vs 256），组合空间为 $\binom{384}{8} \approx 1.8 \times 10^{13}$。
- 使用 **MLA**（Multi-head Latent Attention）压缩 KV cache。
- 128K token 上下文。

Kimi K2 在 MoE 训练上的关键创新是 将 **Muon 优化器**引入 MoE 训练（使用改进版 MuonClip），并在 15.5T token 的全程训练中实现了 "零 loss spike"。这是一个重要的工程里程碑——万亿参数 MoE 模型的训练稳定性 一直是业界最大的担忧之一。

2025–2026 主要 MoE 模型对比

模型	总参数	活跃参数	专家数	Top-K	共享专家
Mixtral 8x7B	47B	13B	8	2	无
DeepSeek-V3	671B	37B	256	8	1
LLaMA 4 Maverick	400B	17B	128	1	无 †
LLaMA 4 Behemoth	~2T	288B	16	2	无 †
Qwen3-235B	235B	22B	128	8	未公开
Kimi K2	1.04T	32B	384	8	有

† LLaMA 4 未明确公开共享专家设计。

从表中可以看到一个清晰的趋势：**细粒度小专家 + 高 Top-K** 成为主流设计（DeepSeek、Qwen3、Kimi K2 都使用 Top-8 + 100 个以上的专家）。LLaMA 4 的 Top-1 路由 + 知识蒸

馏是一条独特的路线，但 Behemoth（未发布的旗舰模型）也回到了 Top-2。

6.6 MoE 推理的工程挑战

MoE 的训练效率优势在推理端面临严峻的工程挑战。

6.6.1 内存瓶颈与专家卸载

MoE 模型的推理面临一个核心矛盾：**每次只激活 5%–10% 的参数，但 100% 的参数都必须驻留在可访问的存储中**。DeepSeek-V3 的 671B 参数在 BF16 下需要 ~1.3 TB 内存，至少需要 17 块 H100（80 GB）才能装下——但每个 token 实际只需要 37B 参数（~74 GB）的计算。

专家卸载（Expert Offloading）将不活跃的专家 从 GPU 显存卸载到 CPU 内存甚至 SSD，只在需要时加载到 GPU：

FineMoE (2026) 提出了**细粒度**的卸载策略：不是以整个专家为单位进行卸载/加载（粒度太粗——一个专家可能有几百 MB），而是将每个专家进一步拆分为更小的 “子专家” 块，按需加载。关键洞察是：专家的使用频率遵循**长尾分布**——少数 “热门专家” 被频繁调用，大部分专家很少使用。FineMoE 将热门专家常驻 GPU，冷门专家按子块粒度从 CPU 按需传输。

ADEPT（2026）进一步引入了**领域感知的预取**策略：根据输入文本的领域特征（代码、数学、自然语言等）预判哪些专家可能被激活，提前将它们从 CPU 加载到 GPU。这将专家加载操作从推理的关键路径上移除，有效隐藏了 PCIe 总线的 I/O 延迟。

6.6.2 MoE 模型的量化

量化是降低 MoE 内存占用的另一条路径——但 MoE 的量化比稠密模型更具挑战性。

DynaExq (2025) 提出了**动态混合精度**的量化方案：不同专家使用不同的量化位宽。热门专家（频繁使用）保持较高精度（如 8 bit），冷门专家使用更激进的压缩（如 4 bit 甚至 2 bit），因为它们对最终输出的贡献权重较小。DynaExq 在运行时根据路由统计量动态调整专家的量化位宽，在精度和内存之间找到最优平衡。

MiLo（Mixture of Low-Rank Compensators）则在量化后为每个专家附加一个**低秩补偿器**——用少量的额外参数修正量化带来的误差。对于 4-bit 量化的 MoE 模型，MiLo 可以恢复 90%+ 的原始精度，而额外的参数开销不到原始专家的 5%。

6.6.3 批处理的矛盾

MoE 推理还面临一个 “批处理悖论”：**批处理是推理吞吐量的关键，但它抵消了 MoE 的稀疏性优势**。

在单请求推理中，每个 token 只激活 K 个专家，MoE 的计算节省是真实的。但在批处理（batched inference）中，不同请求的 token 可能路由到不同的专家——当 batch size

足够大时，所有 N 个专家都会被至少一个 token 激活，等价于运行一个**完整的稠密模型**。更糟的是，MoE 的 All-to-All 通信开销在稠密模型中不存在。

Lynx (2025) 提出了一种 “专家重映射” 方案：在运行时动态地将 batch 中不同请求的 token 重新分配到重叠度更高的专家子集上，减少实际需要激活的专家总数。这在保持 <1% 精度损失的前提下，将推理吞吐量提升了 1.23×。

MoE-SpeQ (2025) 则将推测解码与专家预取结合：在主模型验证候选 token 的同时，利用路由器的预判结果预加载下一步可能需要的专家。推测解码本身减少了生成步数，专家预取则隐藏了每步的加载延迟——两种优化相互增强。

6.6.4 MoE 的优势与挑战

MoE vs Dense：何时选择哪种架构

MoE 的优势：

- 知识容量与推理成本解耦——大容量、低推理成本
- 对于广泛部署的模型，推理经济性是压倒性优势
- 专家专业化带来的任务适配能力

MoE 的挑战：

- **内存占用**：所有专家必须加载到内存中，即使每次只激活一小部分。671B MoE 模型的权重（BF16）约 1.3 TB——需要至少 17 块 H100（80 GB）才能放下。
- **微调困难**：LoRA 等参数高效微调方法 在 MoE 上的应用并不直接——应该给每个专家加 LoRA 适配器，还是只给共享部分加？给所有 256 个专家都加 LoRA 会导致参数量爆炸，只给部分专家加则可能破坏路由的平衡。
- **训练复杂度**：负载均衡、通信优化、容量因子调节——MoE 的超参数空间比稠密模型大得多。
- **推理延迟**：虽然 FLOP 更少，但 All-to-All 通信引入了额外延迟。在单 GPU 推理中，MoE 的延迟优势可能被 更多的内存访问抵消（所有专家的参数都在内存中，cache 命中率更低）。

经验法则：当目标模型的知识容量需要 >100B 参数，且推理成本是重要考量时，选择 MoE。当模型规模在 7B–14B，且部署环境受限（单 GPU、边缘设备）时，选择 Dense。

为什么 MoE 是 2026 年的标准选择?

答案是**推理经济学**。训练一次，推理无数次。MoE 使得你可以用 37B 的推理成本获得 671B 的知识容量。当你服务数百万用户、每天处理数十亿 token 时，这个 ~20× 的推理计算节省是巨大的。这就是为什么 2026 年所有 frontier 开源模型（DeepSeek-V3/V4、LLaMA 4、Qwen3-235B、Mistral Large 3）都采用了 MoE 架构。

值得注意的是，MoE 并不总是最优选择。对于参数量在 7B–14B 的小模型，Dense（稠密）架构仍然占主导——因为小模型的参数本来就不多，再拆分成多个专家后每个专家太小，专业化程度不够。而且小模型通常部署在单块 GPU 甚至 CPU 上，MoE 的 All-to-All 通信开销在单设备上没有意义。

MoE 的真正优势在大规模模型上：当你需要 >100B 的知识容量，但推理成本必须控制在 ~20B 参数以内时，MoE 是唯一的选择。

CHAPTER 7

长上下文——让模型阅读整本书

早期的 Transformer 模型只能处理 512 或 2048 个 token——大约一两页文字。这严重限制了模型的实用性：它无法阅读一篇长论文，无法理解一个完整的代码仓库，更无法在长篇对话中保持一致性。

2025–2026 年，长上下文能力取得了巨大突破。下表展示了截至 2026 年初主要模型的上下文窗口对比：

模型	上下文窗口	位置编码	发布时间
LLaMA 4 Scout	10M tokens	iRoPE	2025.04
Gemini 2.5 Pro	1M tokens	未公开	2025.03
Gemini 3 Pro/Flash	1–2M tokens	未公开	2025–2026
Claude Sonnet 4.5 / Opus 4.5	200K–1M tokens	未公开	2025–2026
GPT-5 系列	400K tokens	未公开	2025–2026
Kimi K2.5	256K tokens	RoPE 变体	2025
Jamba 1.5-Large	256K tokens	RoPE + SSM	2024.08
DeepSeek-V3	128K tokens	YaRN + MLA	2024.12
LLaMA 3.1 405B	128K tokens	RoPE + 频率调整	2024.07
Qwen 2.5	128K–1M tokens	YaRN	2024–2025

LLaMA 4 Scout 以 10M token（约 15,000 页）创下了公开模型的最长上下文记录。Google 的 Gemini 系列稳定支持 1–2M token。Anthropic 的 Claude 系列在 2025–2026 年将上下文扩展到了 200K–1M token。OpenAI 的 GPT-5 系列达到了 400K token。

但这些数字背后隐藏着艰巨的工程和科学挑战——让模型「看到」长文本和让模型「理解」长文本，完全是两回事。正如 Chroma 的 context rot 研究[1] 所揭示的：**标称上下文长度和有效上下文利用率之间存在巨大鸿沟**。

7.1 为什么长上下文很难?

长上下文的困难不是单一的，而是四重挑战的叠加。

[1] K. Hong et al., “Context rot: How increasing input tokens impacts LLM performance,” Chroma, Tech. Rep., 2025.

7.1.1 计算复杂度：$O(n^2)$ 的诅咒

注意力机制的计算复杂度是 $O(n^2)$——序列长度翻倍，计算量翻四倍。让我们把这个抽象的复杂度换算成具体的数字：

处理 4K token 时，每一层注意力需要计算 $4{,}096 \times 4{,}096 = 16.8M$ 个注意力分数对。处理 128K token 时，这个数字变成 $131{,}072 \times 131{,}072 \approx 16.4B$——**增长了近 1000 倍**。如果模型有 80 层（如 LLaMA 70B），那就是 $80 \times 16.4B \approx 1.3$ 万亿次注意力分数计算。

即使有 FlashAttention[2] 这样的优化（将注意力的 IO 复杂度从 $O(n^2)$ 降到 $O(n^2/M)$，其中 M 是 SRAM 大小），计算量本身的 $O(n^2)$ 增长是无法改变的。128K token 的计算量就是 4K token 的 1024×，这是数学决定的，没有捷径。

7.1.2 内存瓶颈：KV Cache 的爆炸

比计算更致命的是内存。在推理时，Transformer 需要缓存所有已处理 token 的 Key 和 Value 向量（称为 KV cache），以便后续 token 做注意力计算。

KV cache 的大小可以这样估算：

$$\text{KV cache} = 2 \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times n_{\text{tokens}} \times \text{bytes per element} \tag{7.1}$$

其中因子 2 来自 Key 和 Value 两组向量，$n_{\text{kv_heads}}$ 是 KV 头的数量（注意不是 query 头的数量——在使用 GQA 的模型中两者不同）。

LLaMA 70B 使用 GQA（Grouped Query Attention），有 64 个 **query 头**但只有 8 个 **KV 头**（每 8 个 query 头共享一组 KV 投影）。因此 KV cache 计算直接使用 $n_{\text{kv_heads}} = 8$。

对于 LLaMA 70B（80 层，8 个 KV heads，head 维度 128）处理 128K token，使用 BF16（2 bytes）：

$$2 \times 80 \times 8 \times 128 \times 131{,}072 \times 2 \approx 21 \text{ GB}$$

仅一个用户的一次 128K 请求，光是 KV cache 就要占用约 21 GB——超过单块 H100（80 GB）显存的四分之一，模型权重还没算。如果不使用 GQA（即 64 个 KV heads），KV cache 将膨胀到约 171 GB，远超单块 GPU 容量。

KV Cache 是长上下文推理的真正瓶颈

很多人以为长上下文的瓶颈是计算。实际上，对于推理来说，**内存才是硬约束**。计算可以通过更多 GPU 和更长时间来解决，但 KV cache 必须驻留在 GPU 显存中，而单块 GPU 的显存是有限的。这就是为什么 KV cache 的压缩和分布式管理 是长上下文推理工程中最核心的问题。

[2] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

7.1.3 训练–推理的长度失配

第三个挑战更加根本：模型在训练时只见过一定长度的序列，当推理时序列长度超过训练长度时会发生什么？

RoPE[3] 的位置编码依赖于旋转角度 $\theta_i = 10000^{-2i/d}$。在训练长度内（比如 4K），模型见过的最大旋转角度是 $4096 \times \theta_i$。当推理时序列长度变成 32K，旋转角度变为 $32768 \times \theta_i$——这些角度值是模型从未在训练中遇到过的。

直觉上，这就像一个只学过 0–100 范围内加法的学生 被要求做 0–1000 范围的加法。虽然底层规则相同，但具体的数值模式是「未见过」的，注意力分数变得不可靠——这就是所谓的「外推失败」（extrapolation failure）。

低频维度受影响最大：它们的旋转角度在训练范围内变化缓慢（不到一整圈），但在超长序列中可能转过好几圈，进入完全陌生的角度空间。高频维度则相对安全，因为它们在训练范围内已经转过了很多圈，多转几圈不会带来本质变化。

7.1.4 Lost in the Middle：中间信息的遗忘

第四个挑战来自模型的注意力分配模式。2023 年 Liu 等人的研究揭示了一个令人不安的现象：当输入序列很长时，模型对序列**开头**和**结尾**的信息关注度最高，而对**中间**部分的信息检索准确度显著下降——下降幅度超过 30%。

这种「U 型曲线」注意力模式有深层原因。在自回归训练中，每个 token 的损失只取决于它之前的 context。序列开头的 token 是所有后续 token 的 context，因此它们的信息被反复强化。序列结尾的 token 距离当前预测位置最近，在注意力分数中自然占优势（近因效应）。中间的 token 则两头不沾——既不是「总是被参考」的锚点，也不是「最近看到」的新鲜信息。

这个发现意味着：仅仅增加上下文窗口是不够的。如果关键信息恰好落在长序列的中间部分，模型可能会「看到但不记住」——就像一个学生虽然读了整本教科书，但只记得开头和结尾的内容。

7.2 位置编码的扩展方法

面对训练–推理的长度失配问题，研究者们提出了一系列巧妙的位置编码扩展方法。这些方法的核心思想都是：让超长序列中的位置编码落入模型训练时见过的值域范围内。

7.2.1 线性位置插值

最直接的方法是**线性插值**：如果要将 4K 上下文扩展到 32K（8×），就把所有位置索引除以 8。原来位置 32,768 的 token 变成位置 4,096——刚好是训练时见过的最大位置。

[3] J. Su et al., "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127 063, 2024.

数学上，将位置 m 替换为 m/s，其中 $s = L_{\text{target}}/L_{\text{train}}$ 是扩展比例。RoPE 的旋转角度变为 $(m/s) \cdot \theta_i$，确保所有角度都落在训练时见过的范围内。

为什么这个简单的技巧有效？因为它保持了**相对位置关系**不变——如果 token A 和 token B 的相对位置差是 Δ，插值后变成 Δ/s，但两者的差值比例没变。模型学到的「距离越远注意力越弱」的模式仍然适用。

但线性插值有明显的局限：当扩展比例 s 很大时（比如 32× 以上），相邻 token 的位置差变得极小（从 1.0 变成 1/32 = 0.03125），模型难以区分紧邻的 token 的位置关系——局部分辨率严重下降。这就像把一张高清照片压缩到很小：全局结构保留了，但细节模糊了。

7.2.2 NTK-aware 插值

NTK-aware 插值的核心洞察是：**不应该对所有频率维度做相同比例的缩放**。

回忆 RoPE 的频率：$\theta_i = b^{-2i/d}$（$b = 10000$）。高频维度（i 小）负责编码**局部**位置关系（相邻 token 之间的差异），低频维度（i 大）负责编码**全局**位置关系（文档开头 vs 文档结尾）。

线性插值对所有频率一视同仁地除以 s，结果高频维度——负责局部关系的维度——被不必要地压缩了。NTK-aware 插值只调大 base 值 b：将 $b = 10000$ 替换为 $b' = 10000 \cdot s^{d/(d-2)}$。

这产生了一种非均匀缩放效果：

- **高频维度**几乎不受影响——它们的频率本来就高，稍微调整 base 对它们影响很小。局部位置分辨率得以保留。
- **低频维度**被显著缩放——让它们能编码更远的位置关系。

一个有趣的类比：这就像数字系统中的进制变化。在十进制中，三位数能表示 0–999。如果我们把进制从 10 变成 100，三位数就能表示 $0\text{–}100^3 = 999{,}999$——扩大了 1000 倍的范围，但低位的分辨率几乎不变。

NTK-aware 插值的最大优势是**不需要微调**——在中等扩展比例（4× 左右）下，直接修改 base 值就能在推理时扩展上下文，质量损失很小。但在高扩展比例下仍会退化。

7.2.3 YaRN：优雅的频率操控

YaRN（Yet Another RoPE extensioN）是当前最流行的上下文扩展方法。它的核心洞察是：RoPE 的不同频率维度应该被**区别对待**，而且不仅要调整频率，还要调整注意力的温度。

RoPE 的频率 $\theta_i = 10000^{-2i/d}$ 从高频（i 小）到低频（i 大）。YaRN 将这些频率分为三组，并定义了一个关键概念——频率维度的「波长」$\lambda_i = 2\pi/\theta_i$。如果 λ_i 远大于训练长度 L，说明该维度在训练中连一整圈都没转完，外推风险最高。如果 λ_i 远小于 L，说明该维度已经转过很多圈，外推风险最低。

基于这个分析，YaRN 将频率维度分为三组：

- **高频维度**（$\lambda_i < L$，对应短距离模式）：不做任何修改——这些维度已经能处理局部关系，不需要调整。它们在训练中转过了很多圈，外推到更长序列时只是多转几圈而已。
- **低频维度**（$\lambda_i > s \cdot L$，对应长距离模式）：按上下文扩展比例进行激进的频率缩放——比如要将 4K 扩展到 128K（32×），这些维度的频率除以 32。它们必须被压缩到训练范围内，否则外推完全不可靠。
- **中频维度**（$L \le \lambda_i \le s \cdot L$）：进行渐进式的 NTK-aware 插值——在不缩放和完全缩放之间做平滑过渡。过渡函数是一个 ramp（线性插值因子），避免了频率空间中的突变。

此外，YaRN 引入了**注意力温度缩放**因子 $\sqrt{t}$。长序列中注意力分数的方差会增大（因为更多的 token 参与 softmax 竞争），导致注意力分布变得过于「尖锐」——集中在少数 token 上，忽略其余信息。温度因子 $\sqrt{t}$ 相当于在 softmax 之前 将注意力 logits 除以 $\sqrt{t}$（$t > 1$），降低 softmax 的「锐度」，让注意力分布更加平滑，使模型能更均匀地关注长序列中的信息。

通过这些手段，YaRN 只需在原始预训练数据的极小比例上做微调（通常 ~0.1% 的训练数据，训练 ~400 步），就能将上下文窗口扩展 32× 甚至更多。Qwen2.5、DeepSeek 等模型均使用 YaRN 变体进行上下文扩展。

7.2.4 Dynamic NTK：推理时自适应

Dynamic NTK 是一种在推理时根据实际序列长度 **动态调整**插值比例的方法。

核心思想很简单：不预设固定的扩展比例，而是在推理时检查当前序列长度 n——如果 $n \le L_{\text{train}}$，不做任何修改；如果 $n > L_{\text{train}}$，将插值比例设为 $s = n/L_{\text{train}}$，然后应用 NTK-aware 缩放。

这种方法的优势是**完全免微调**——不需要任何额外训练，只需修改推理代码。对于中等程度的扩展（2×–4×），Dynamic NTK 的质量损失可以接受。但对于大比例扩展（8× 以上），质量下降明显，因为模型没有在长序列上做过任何适应。

Dynamic NTK 在实际应用中常被用作一种「应急方案」——当用户的输入偶尔超过训练长度时，它能优雅地降级，而不是直接崩溃。

7.3 iRoPE：Meta 的交替架构

LLaMA 4 引入了更激进的方案——iRoPE（interleaved RoPE）。它不是在位置编码的数值上做文章，而是从**架构层面**重新思考位置编码的角色。

7.3.1 核心设计：交替堆叠

iRoPE 交替堆叠两种注意力层：

- **RoPE 层**：使用旋转位置编码，捕捉局部上下文和精确的相对位置关系。这些层知道「哪个 token 在前，哪个在后」。
- **NoPE 层**（No Positional Encoding）：完全没有位置编码，注意力只基于语义内容。这些层天然不受序列长度限制——因为它们根本不编码位置信息。

为什么这种交替架构有效？关键洞察是：**不是所有注意力模式都需要位置信息**。

考虑两种典型的注意力模式：一种是「这个代词指代的是三句话前的名词」——这需要精确的位置感知（RoPE 层的工作）。另一种是「这段话在讨论量子物理的概念」——这只需要语义匹配，不关心具体位置（NoPE 层的工作）。

NoPE 层的存在使得模型对长序列有天然的适应性——它们关注的是「这个 token 的内容是什么」而非「这个 token 在哪个位置」。当序列从 8K 扩展到 1M 时，NoPE 层完全不受影响，只有 RoPE 层需要做位置编码的扩展。这等于把需要外推的层数减少了一半。

7.3.2 LLaMA 4 的 iRoPE 实现细节

LLaMA 4 系列包含三个模型，全部采用 iRoPE + MoE 架构：

模型	总参数	活跃参数	专家数	上下文
Scout	109B	17B	16	10M tokens
Maverick	400B	17B	128	1M tokens
Behemoth	2T	288B	16	— (训练中)

Scout 的 10M token 上下文窗口（约 15,000 页文本）是截至 2025 年 4 月所有公开模型中最长的。这个惊人的长度正是得益于 iRoPE 的架构设计：NoPE 层提供了「免费」的上下文扩展空间，RoPE 层通过频率调整处理位置敏感的注意力模式。

在 LLaMA 4 的具体实现中[4]，iRoPE 的交替模式并非简单的 1:1 交替。Meta 发现不同的交替比例适合不同的模型规模——Scout 和 Maverick 使用了针对各自专家数量和模型规模 优化的交替策略。

iRoPE 还与 LLaMA 4 的其他架构创新协同工作：

- **早期融合**（Early Fusion）：LLaMA 4 是原生多模态模型，文本、图像和视频 token 在模型早期就被融合处理。NoPE 层对跨模态注意力特别有价值——「这张图片展示的概念」和「文本描述的概念」之间的匹配是纯语义的，不需要位置信息
- **MoE 稀疏激活**：每个 token 只激活少量专家，使得 Scout 的 109B 总参数中仅有 17B 活跃——这让 10M 上下文在 8×H100 上可运行

[4]Meta AI, *The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation*, Meta AI Blog, 2025.

iRoPE 的训练流程

LLaMA 4 的长上下文训练分三个阶段：

1. **预训练**：在 8K 上下文长度上完成主体训练。这个阶段消耗了绝大部分计算。Behemoth 作为「教师模型」在这个阶段训练，Scout 和 Maverick 通过蒸馏获得高质量初始化。
2. **中等长度扩展**：调整 RoPE 层的频率参数，将上下文扩展到 128K–256K，在长文档数据上做少量继续训练。
3. **超长扩展**：进一步扩展到 1M–10M。NoPE 层在这个阶段几乎不需要适应，RoPE 层通过 YaRN 风格的频率操控进行扩展。

LLaMA 4 Scout 通过 iRoPE 达到了 10M token 的上下文窗口——这在纯 RoPE 架构中几乎不可能实现。

7.3.3 iRoPE 的推理优化

iRoPE 的另一个优势是推理效率。NoPE 层可以使用更激进的 KV cache 压缩策略（比如丢弃远距离的 KV），因为它们的注意力模式基于语义相似度而非位置，远距离的 token 只要语义不相关就不重要。而 RoPE 层则保留完整的位置信息，确保精确的位置感知不受影响。

vLLM 在 v0.8.3 版本中已原生支持 LLaMA 4 的 iRoPE 架构。实测显示，使用 8×H100，vLLM 可以为 Scout 提供 1M 上下文的推理服务，为 Maverick 提供约 430K 上下文的推理服务。

iRoPE 的局限与争议

LLaMA 4 在发布后收到了一些社区反馈：虽然 10M token 的上下文窗口在 NIAH 等检索测试上表现优秀，但在需要**深度推理**的长上下文任务上，实际有效利用的上下文长度远低于标称值。这再次验证了「标称上下文长度 $\neq$ 有效上下文长度」的原则。此外，Behemoth 模型截至 2026 年初仍未正式发布，社区对其实际性能持审慎态度。

7.4 Ring Attention：跨 GPU 处理超长序列

即使解决了位置编码的问题，超长序列的 KV cache 仍然可能超过单块 GPU 的内存容量。Ring Attention 的解决方案是 将长序列**分割到多块 GPU** 上。

7.4.1 核心思想：环形传递

Ring Attention 的灵感来自一个简单的观察：标准注意力要求每个 query 都能访问所有的 key 和 value，但没人说所有 KV 必须**同时**在同一块 GPU 上。

设备排列成环形拓扑。每块 GPU 持有序列的一个 chunk（包含该 chunk 的 Q、K、V），然后执行以下步骤：

1. 每块 GPU 计算自己持有的 Q 与**当前本地** KV 的注意力。
2. **同时**，每块 GPU 将自己的 KV block 发送给环上的下一块 GPU，并从前一块 GPU 接收新的 KV block。
3. 收到新的 KV block 后，计算 Q 与这些新 KV 的注意力，将结果与之前的注意力输出做**在线聚合**（online softmax aggregation——不需要存储完整的注意力矩阵，只需要维护 running sum 和 running max）。
4. 重复步骤 2–3，直到 KV block 在环上转了一整圈，每块 GPU 都看到了所有的 KV。

关键的工程技巧在于步骤 2：**通信与计算重叠**。当 GPU 在计算当前 KV block 的注意力时，下一个 KV block 正在通过网络传输过来。只要计算时间大于传输时间——在序列足够长的情况下通常成立——传输延迟就被完全隐藏了。

7.4.2 内存与扩展性分析

Ring Attention 将每块 GPU 的内存需求 从 $O(n)$（存储完整 KV cache）降为 $O(n/P)$（P 是 GPU 数量），理论上支持无限长的序列——只要有足够多的 GPU。PyTorch 已经将 Ring Attention 原生集成到 ContextParallel API 中。

但实际的扩展性受两个因素限制：

- **通信带宽**：每个 KV block 需要在环上传递 $P-1$ 次。当 GPU 数量很多时，KV 的总通信量为 $O(n \cdot (P-1))$，网络带宽成为瓶颈。在 NVLink（900 GB/s）互连的节点内这不是问题，但跨节点（通常 100–400 Gb/s InfiniBand）时延迟显著增加。
- **收益递减**：超过一定长度后，模型的有效信息利用率下降（context rot 效应），更多的 GPU 和更长的上下文不一定带来更好的结果。

7.4.3 Zig-Zag Ring Attention：负载均衡

Ring Attention 有一个变体叫 **Zig-Zag Ring Attention**，它解决了原始方法在自回归模型中的负载不均衡问题。在因果注意力中（每个 token 只能关注之前的 token），序列开头的 GPU 计算量最小（只有少量的 KV 对），序列末尾的 GPU 计算量最大（需要关注所有之前的 token）。

如果将序列按顺序均匀分配给 P 块 GPU，第 1 块 GPU 的计算量约为第 P 块的 $1/P$——当 $P=8$ 时，最轻的 GPU 只有最重的 1/8 的工作量，7/8 的时间在空等。

Zig-Zag 通过锯齿形的序列分割策略平衡各 GPU 的负载：第一块 GPU 拿第 1、4、5、8 段序列，第二块拿第 2、3、6、7 段……使得每块 GPU 处理的总 token 数近似相等。直觉上，每块 GPU 同时拥有序列开头（计算量小）和序列末尾（计算量大）的 chunk，两者的计算量互相平衡。

Striped Attention 是另一种变体，它将序列按 token 级别交错分配（第 i 个 token 分配给第 $i \mod P$ 块 GPU），实现了理论上完美的负载均衡，但通信模式更复杂。

7.5 长上下文训练的实践

支持长上下文不是简单地「把序列长度调大」就行。需要在数据、训练策略和工程三个维度同时发力。

7.5.1 多阶段扩展训练

通常需要分阶段训练：

阶段一：在标准上下文长度（如 4K–8K）上完成主体预训练，消耗大部分训练 token（如 14T 中的 13.5T）。这个阶段的重点是学习语言和知识，上下文长度不是关注点。

阶段二：逐步扩展上下文长度——先扩展到 32K，训练少量数据适应；然后扩展到 128K，再训练一批数据。Qwen2.5 的长上下文版本甚至使用了三阶段扩展（32K → 128K → 1M）。每次扩展时需要调整 RoPE 的频率参数（通过 YaRN 或类似方法）。

LLaMA 3[5] 的长上下文训练提供了一个典型的工程实例：预训练在 8K 上下文上完成，然后通过六个子阶段逐步扩展到 128K——每个子阶段将上下文长度翻倍（8K → 16K → 32K → ... → 128K），同时逐步增大 RoPE 的 base 频率（从 5×10^5 逐步增大到 8×10^6）。

阶段三：在长上下文数据上做 SFT，确保模型能够在长文本中遵循指令、回答关于文档特定部分的问题、以及保持长距离的一致性。

这种分阶段策略的原因是：长序列的训练效率远低于短序列——注意力的 $O(n^2)$ 复杂度意味着 一条 128K 的样本比 32 条 4K 的样本**慢得多**，但包含的有效 token 数是一样的。在短序列上完成大部分学习，然后在长序列上做少量适应，是效率最优的策略。

7.5.2 长上下文训练数据

长上下文训练面临一个独特的数据挑战：**大多数高质量文本都不长**。网页平均长度约 2K–4K token，新闻文章约 500–2000 token。真正超过 32K token 的高质量文档并不多。

长上下文训练数据的来源：

- **书籍**：单本书通常在 50K–500K token，是最天然的长上下文数据源。但版权问题限制了可用数量。
- **代码仓库**：一个完整的代码仓库（包含所有文件及其依赖关系）可以轻松超过 100K token。代码的优势是结构清晰、有明确的跨文件引用关系。
- **学术论文**：单篇论文 8K–30K token，可以将相关论文串联起来作为长文档。
- **法律和财务文档**：合同、招股书、法规 动辄数十万 token，是天然的长文本。
- **合成长文档**：将多个相关短文档 按逻辑顺序拼接成长文档。例如将关于同一主题的 Wikipedia 页面拼接起来，或者在一组文档上生成跨文档的问答对。这种方法需要小心处理拼接边界，避免模型学到「文档之间无关联」的错误模式。

[5] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

7.5.3 长上下文 SFT 数据

阶段三的长上下文 SFT 需要专门设计的指令数据。典型的任务类型包括：

- **长文档总结**：给出一篇 50K+ token 的文档，要求模型生成不同粒度的摘要（一句话/一段/多页）。
- **多文档问答**：给出多篇文档，要求模型综合多个来源回答问题。这测试的是跨文档信息整合能力。
- **仓库级代码理解**：给出整个代码仓库，要求模型解释架构、定位 bug、或在保持全局一致性的前提下修改代码。
- **长距离事实一致性**：在对话的早期提供一组事实，在很久之后（上万 token 的对话之后）询问与这些事实相关的问题。

7.5.4 推理工程优化

SGLang 在 2026 年初发布的 Chunked Pipeline Parallelism 是长上下文推理的重要工程突破。它将长序列的 prefill（首次处理整个输入序列的阶段）分成多个 chunk，在不同 GPU 上以流水线方式并行处理。在 DeepSeek-V3.1 上的实测显示，处理 1M token 的输入延迟从 55.5 秒降至 10.5 秒——降低了 81%，相当于 3.31× 的吞吐提升。这意味着用户可以在约 10 秒内完成对一整本书的「阅读」，极大地提升了长上下文模型的实用性。

7.6 长上下文的实际应用场景

长上下文能力开启了许多此前不可能的应用场景：

整仓库代码理解：将一个完整的代码仓库（数万行代码）放入上下文，让模型理解整体架构后再回答具体问题或做修改。这比传统的 RAG（只检索相关片段）能获得更全面的上下文理解。Cursor 和 GitHub Copilot Workspace 等开发工具 已经在利用这种能力——将整个项目的代码、配置文件、README 一起放入上下文，让模型做出全局一致的修改。

长文档分析：将一份数百页的法律合同或学术论文完整放入上下文，让模型进行总结、比较或提取特定信息。这避免了将文档切分成小块后丢失跨段落关联的问题。在金融领域，分析师可以将一整份招股书（通常 200–500 页）放入上下文，直接询问「这家公司的主要风险因素和竞争优势分别是什么?」

多文档对比分析：将多篇研究论文同时放入上下文，让模型比较它们的方法、结果和结论。这在文献综述和系统性综述中特别有价值——传统做法需要人类研究者反复翻阅多篇论文，而长上下文模型可以「同时阅读」所有论文并做对比。

多轮对话的长期记忆：在一次持续数小时的编程对话中，模型可以记住早期讨论的设计决策，在后期做修改时保持一致性。对于短上下文模型，长对话必须依赖 RAG 或摘要来维持「记忆」，但这些方法不可避免地会丢失信息。足够长的上下文窗口可以直接保留完整的对话历史。

少样本学习：在上下文中放入大量的示例和说明，让模型在不做任何微调的情况下学会新任务。更长的上下文意味着可以提供更多、更多样的示例。一个极端的例子：在上下文中放入整本编程教材，然后要求模型按照教材的风格和约定写代码——这比用几个 few-shot 示例效果好得多。

长内容生成：生成长篇小说章节、技术文档、剧本等需要 在数万 token 的范围内保持情节、角色、技术概念一致性的内容。短上下文模型在长篇生成中常出现前后矛盾——因为它「忘记了」前面写过什么。

但正如下一节将讨论的，更长的上下文不等于更好的利用。在实际应用中，如何组织和筛选放入上下文的信息（context engineering）与上下文长度本身同样重要——甚至更重要。

7.7 Context Rot 与 NIAH：长上下文的质量评估

在追求更长的上下文窗口时，我们需要区分两个不同的能力维度：**检索**（retrieval）和**理解**（reasoning）。2025 年的研究表明，这两个维度之间的差距 远比人们最初预期的更大。

7.7.1 Needle in a Haystack 测试

NIAH（Needle in a Haystack）是最广泛使用的长上下文评估方法。测试方法很简单：在一段无关的长文本（haystack）中 插入一条关键信息（needle），然后询问模型关于这条信息的问题。通过改变 needle 的位置（开头/中间/结尾）和 haystack 的长度，可以绘制出模型在不同位置和长度下的检索准确率热力图。

NIAH 测试揭示了几个重要模式：

- 大多数现代模型在 NIAH 上表现优秀——即使在 128K 长度下，主流模型的检索准确率也接近 100%。
- 但 NIAH 本质上只测试**检索**——模型能否找到特定信息。这是长上下文能力的**必要条件**，但远不是**充分条件**。

一个模型可以完美地通过 NIAH（找到 needle），但在需要**综合**长上下文中多处信息做推理时表现糟糕。比如「根据文档第 3 章和第 7 章的描述，这两部分的结论是否矛盾?」——这要求模型不仅找到两处信息，还要理解并比较它们的含义。

7.7.2 NIAH 的局限与更好的评估

NIAH 的根本问题是它把长上下文简化成了一个搜索任务。在真实场景中，用户很少需要「在长文档中找一句话」——更常见的是需要**综合**、**推理**和**比较**。

更有挑战性的评估任务包括：

- **Multi-Needle NIAH**：在 haystack 中插入**多个**相关信息片段，要求模型综合所有片段才能回答问题。这测试的是信息**聚合**能力，而非单点检索

- **RULER**（2024）：包含多种长上下文推理任务——变量追踪、多跳推理、以及需要整合分散在上下文各处的信息的复合任务
- **长文档摘要质量**：给模型一篇 100K+ token 的文档，评估摘要是否遗漏了中间部分的关键信息。这直接测量 lost-in-the-middle 效应
- **BABILong**：基于 bAbI 任务改编的长上下文推理基准，需要在长文本中追踪实体状态和关系

7.7.3 Context Rot：系统性退化

2025 年 7 月，Chroma 发布了影响深远的 **context rot** 研究报告[6]，对 18 个 frontier 模型（包括 GPT-4.1、Claude 4、Gemini 2.5、Qwen3 等当时最新的模型）进行了系统性的长上下文评估。核心发现令人警醒：**所有模型都随着输入长度的增加而性能下降，即使远未达到标称的上下文窗口上限**。

具体发现：

- **U 型曲线持续存在**：模型对序列开头和结尾的信息关注度最高，中间部分的信息检索准确度下降超过 30%。这被称为「lost in the middle」效应[7]。尽管 2024–2025 年的训练改进缓解了部分问题，但 U 型模式并未消失——只是变得不那么陡峭
- **反直觉发现**：有逻辑结构的上下文 比随机打乱的上下文**更难**检索——模型在有意义的干扰信息中更容易「迷路」。这意味着真实应用场景（放入完整文档而非随机文本）的难度 比 NIAH 测试暗示的更高
- **推理 vs 检索的鸿沟**：在需要跨上下文推理的任务中，性能下降比纯检索任务更为严重。模型可能能找到两个相关段落，但无法正确地将它们联系起来做推理。对于简单的重复词计数任务，一些模型在输入增长到几千 token 时 就开始出现可靠性问题
- **标称 vs 有效上下文**：实际测试显示，许多模型的「有效上下文利用率」（实际可靠使用的比例）仅为标称上下文窗口的 50–80%。例如一个标称 200K 的模型，在约 100–160K token 后可靠性就开始下降
- **核心结论**：**Context engineering（如何组织上下文）比 context length（能塞多少）更重要**

7.7.4 Context Engineering：从「更长」到「更智能」

Anthropic 在 2025 年下半年 明确将 “context engineering” 定义为构建高效 AI Agent 的核心技能，标志着行业思维从「追求更长的上下文窗口」转向「更智能地利用有限的上下文预算」[8]。

[6] K. Hong et al., “Context rot: How increasing input tokens impacts LLM performance,” Chroma, Tech. Rep., 2025.

[7] N. F. Liu et al., “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[8] P. Rajasekaran et al., *Effective context engineering for AI agents*, Anthropic Engineering Blog, 2025.

Context Engineering 的核心原则

基于 context rot 的研究和生产实践，以下原则已成为行业共识：

- **重要信息前置**：把最关键的指令和信息放在上下文的开头，利用模型对开头信息的高关注度。系统提示词（system prompt）和核心约束应在最前面
- **冗余而非堆积**：对关键信息做适度冗余（比如在不同位置重复强调重要约束），而不是试图塞入尽可能多的不同信息。每个不必要的 token 都在**主动**降低其他 token 的注意力预算
- **结构化组织**：使用清晰的标题、分隔符和层次结构 帮助模型定位和组织信息。XML 标签、Markdown 标题、编号列表 都能显著提高信息检索的可靠性
- **即时上下文（Just-in-Time Context）**：不要在一开始就加载所有信息，而是在需要时才检索和注入——这对 AI Agent 尤为重要。一个执行 20 步任务的 Agent，在每一步只需要当前相关的上下文，而非全部 20 步的信息
- **上下文压缩（Compaction）**：对长时间运行的 Agent，定期将已处理的上下文压缩为简洁的摘要，丢弃不再需要的细节。这类似于人类的「工作记忆管理」
- **RAG + 长上下文的协同**：RAG 负责从海量文档中筛选相关内容，长上下文负责一次性理解所有筛选出的内容。两者互补而非替代。经验表明 RAG 检索 top-20 到 top-50 相关片段 并放入 32K–128K 的上下文中，效果通常优于直接将整个文档塞入 1M 上下文

长上下文的未来方向

截至 2026 年初，长上下文领域有几个值得关注的研究方向：

1. **混合架构**：iRoPE（LLaMA 4）和 Transformer-SSM 混合模型 通过架构创新扩展上下文，而非仅依赖位置编码技巧。SSM 层不需要 KV cache，天然适合超长上下文
2. **稀疏注意力训练**：DeepSeek 的 NSA 证明了稀疏注意力可以端到端训练，不再需要全注意力预训练后蒸馏的两阶段流程
3. **KV cache 压缩**：MLA（DeepSeek）、GQA、动态 KV 驱逐等技术 持续降低长上下文推理的内存开销
4. **无限上下文**：Ring Attention + 流水线并行使得 理论上可以处理任意长度的序列，瓶颈从「能否处理」转变为「能否有效利用」
5. **Context engineering 工具化**：自动上下文管理工具（如智能截断、动态检索增强、注意力预算分配）可能比扩展上下文窗口本身更有实际价值

CHAPTER 8

后训练——从语言模型到 AI 助手

预训练完成后，我们得到了一个强大的**基座模型**（Base Model）。它拥有海量的知识，能够流畅地续写文本，但它还不是一个有用的 AI 助手。

为什么？因为基座模型只学会了一件事：预测下一个 token。如果你对它说「请总结下面这段话」，它不会去总结——它会继续「预测」你这段话之后可能出现的内容，比如更多的类似指令。它没有「遵循指令」的概念，也没有「什么该说什么不该说」的判断。

后训练（Post-Training）就是把基座模型变成一个有用、安全、诚实的 AI 助手。这个过程通常分为三个阶段：**监督微调（SFT）→ 偏好优化（RLHF/DPO/GRPO）→ 安全对齐**。

8.1 监督微调（SFT）：学会对话

SFT 的核心思想很简单：给模型展示大量高质量的（指令, 回答）对，让它学习如何根据指令生成合适的回答。

Listing 8.1: SFT data format example

```
{
  "instruction": "Explain quantum entanglement simply.",
  "response": "Imagine you have two magic coins. When
    you flip one and get heads, the other ALWAYS shows
    tails, no matter how far apart they are. That's
    quantum entanglement -- two particles linked so
    measuring one instantly reveals the other's state."
}
```

SFT 的训练过程与预训练相同（最小化交叉熵损失），但只在回答部分计算损失（不对指令部分计算损失），因为我们希望模型学会**如何回答**，而不是如何提问。

8.1.1 为什么需要 SFT?

基座模型和对话助手之间的差距，远比表面上看起来更深刻。

一个在万亿 token 上预训练的 70B 模型 确实「知道」几乎所有领域的知识。但它的行为模式是「续写」——给它一个开头，它会按照训练数据的分布延续下去。如果你问它「中国的首都是哪里?」，它可能回答「中国的首都是哪里？这是一个常见的地理问题」然后继续生成更多类似的问题——因为在训练数据中，问题后面跟着的经常是更多问题

（比如考试卷或 FAQ 列表），而不是答案。

SFT 要做的事情在概念上很简单：**改变模型的条件生成分布**。在预训练中，$P(\text{next}|\text{context})$ 是对整个互联网的建模。在 SFT 后，同样的条件概率变成了 对「一个有帮助的助手会如何回答」的建模。模型的知识没有变，变的是它的**行为模式**。

8.1.2 Stanford Alpaca：SFT 的分水岭

2023 年 3 月，Stanford 的一个团队发布了 Alpaca——一个让整个开源 LLM 社区震动的项目。

他们做了一件看似简单的事：用 GPT-3.5（text-davinci-003）生成了 52K 条 指令-回答对（使用 Self-Instruct 方法），然后在 LLaMA-7B 上做 SFT。整个训练在 8 块 A100 上跑了 3 小时，API 调用成本约 $500。

结果令人惊讶：这个仅 7B 参数的模型 在人类评估中的表现与 GPT-3.5 相当——至少在简单对话任务中如此。Alpaca 证明了两件事：

1. SFT 不需要天量数据或巨额成本，少量高质量数据就足以让基座模型学会对话。
2. 可以用强模型的输出来训练弱模型（后来被称为「蒸馏」），大幅降低了 LLM 对齐的门槛。

Alpaca 的发布像打开了闸门——在接下来的几个月里，开源社区涌现了 Vicuna、WizardLM、Orca 等 大量基于 SFT 的模型。

8.1.3 数据质量的悖论

SFT 的数据量通常很小——从数千到数十万条对话，远少于预训练的万亿 token。但数据质量至关重要。

这看似矛盾：预训练需要海量数据，SFT 却只需要少量数据？原因在于 SFT 的目标不是教模型新知识——知识已经在预训练中获得了。SFT 的目标是改变模型的**行为模式**：从「续写文本」转变为「遵循指令并给出有帮助的回答」。这种行为模式的转变，只需要少量但高质量的示例就够了。

2023 年的 LIMA 论文将这个观察推向了极致。

LIMA 的作者仅用 **1,000 条**精心编写的对话数据 对 LLaMA-65B 做 SFT，得到的模型在人类评估中击败了 用 52K 数据训练的 Alpaca 和 用 300K 数据训练的 GPT-4-All。他们据此提出了「**浅层对齐假说**」（Superficial Alignment Hypothesis）：*模型在预训练中已经学到了几乎所有的知识和能力，对齐所做的只是教模型用正确的「格式」来展现这些能力——使用什么语气、如何组织回答、何时拒绝等。这种格式学习只需要极少量的高质量示例。*

一个有用的类比是：预训练像是让一个人读了图书馆里所有的书——他获得了海量的知识，但你跟他对话时，他可能会漫无边际地「续写」而不是「回答你的问题」。SFT 就像教他「当别人问你问题时，你应该先理解问题，然后组织语言给出有帮助的回答」。这个行为模式的学习不需要读更多的书，只需要几个好的对话示例就够了。

LIMA 的局限性

浅层对齐假说有其适用边界。LIMA 的 1,000 条数据在简单对话中效果好，但在复杂推理、代码生成、长文档分析等任务中 与大数据量 SFT 模型仍有显著差距。这说明对于**复杂能力**的激发，仅靠「格式对齐」是不够的——还需要足够多的示例来教模型 如何在新的行为模式下运用复杂能力。数据质量是必要条件，但充足的数据量也不可或缺。

8.1.4 SFT 的数据来源

高质量的 SFT 数据可以来自多个渠道：

人工编写是质量最高但成本也最高的方式。OpenAI 在训练 InstructGPT[1] 时 雇佣了约 40 名标注者编写高质量的指令-回答对。这些标注者经过严格筛选和培训，确保回答的准确性、全面性和安全性。人工数据的优势在于质量可控——每一条数据都经过审核。劣势是成本高且速度慢——一名训练有素的标注者每天大约能产出 50–100 条高质量数据。

模型合成（Self-Instruct / Evol-Instruct）使用已有的强大模型生成训练数据。

Self-Instruct 的流程：提供少量种子指令 → 让 LLM 生成新指令 → 让 LLM 为新指令生成回答 → 过滤低质量结果 → 重复。Alpaca 用的就是这种方法。

Evol-Instruct（WizardLM 提出）是一种迭代式方法：从简单指令出发，让 LLM 逐步「进化」出更复杂的指令和回答。例如，将「写一个排序算法」进化为「写一个支持自定义比较器、处理空值和重复元素的泛型排序算法，并分析其时间和空间复杂度」。进化方向包括增加约束、深化推理、具体化场景、增加多步骤等。这种方法可以低成本地生成大量多样化且难度递进的训练数据。

UltraChat 是另一种大规模合成方法：用两个 ChatGPT 实例互相对话，一个扮演用户，一个扮演助手，生成多轮对话数据。这种方法特别擅长生成自然的多轮对话——因为「用户」也是 LLM，会提出合理的追问。

开源社区数据如 OpenAssistant、ShareGPT（用户分享的 ChatGPT 对话记录）提供了多样化的真实用户需求。但这些数据的质量参差不齐，需要过滤。ShareGPT 的优势是它反映了**真实用户的需求分布**——用户实际上会问什么、怎么问，这是合成数据很难完美模拟的。

领域专项数据是为特定能力设计的 SFT 数据：

- **代码**：CodeAlpaca、Code-Feedback，从代码竞赛和开源项目中构建指令-代码对。
- **数学**：MetaMathQA 用多种方式重写数学题（改变数字、改变问法、分解步骤），让模型从不同角度学习同一数学概念。
- **工具使用**：ToolBench 教模型如何调用 API——包含数千个真实 API 的使用示例。
- **多语言**：将高质量的英文指令数据翻译成其他语言，同时保留文化相关的本地化

[1] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

内容。

实际中，最优的策略通常是混合多个来源：少量人工编写的「种子」数据保证核心质量，大量合成数据提供多样性，开源数据覆盖长尾需求，领域数据强化专项能力。

8.1.5 Chat 模板与训练格式

SFT 不仅要教模型**说什么**，还要教它**用什么格式说**。不同的模型家族使用不同的对话模板（chat template），这决定了系统提示、用户消息和助手回答 在 token 流中如何编排。

Listing 8.2: ChatML format (used by GPT/Qwen)

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
The capital of France is Paris.<|im_end|>
```

Listing 8.3: Llama 3 format

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is the capital of France?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>

The capital of France is Paris.<|eot_id|>
```

这些格式看似琐碎，但至关重要。系统提示（system prompt）机制让同一个模型 能在不同场景下扮演不同角色——「你是一个代码审查专家」和「你是一个温柔的儿童教育助手」使用同一个模型，只改变系统提示。这种灵活性是通过 SFT 在模板中学到的。

8.1.6 SFT 的训练细节

SFT 的训练与预训练在优化器和架构上完全相同，但超参数设置有显著差异：

- **学习率**：比预训练低一个数量级。预训练通常用 1×10^{-4} 到 3×10^{-4}，SFT 用 1×10^{-5} 到 5×10^{-5}。过高的学习率会让模型「忘记」预训练中学到的知识（灾难性遗忘）。
- **训练轮数**：通常只训练 1–3 个 epoch。过多的 epoch 会导致过拟合——模型开始死记硬背 SFT 数据中的回答模式，而不是学习通用的「遵循指令」行为。
- **序列打包**：将多条短对话打包到一个训练序列中，用特殊 token 分隔。这避免了短序列浪费填充（padding）token 的计算。但需要确保注意力掩码正确——不同对话

之间不能互相看到。

- **损失掩码**：只在回答部分计算损失，指令部分的 token 被掩码掉。这让模型专注于学习「如何回答」，而不是「如何提问」。

8.1.7 LoRA：高效微调

对一个 70B 参数的模型做全参数 SFT 需要巨大的 GPU 内存。LoRA[2]（Low-Rank Adaptation）提供了一种优雅的解决方案。

LoRA 的数学原理

核心假设是：**微调的参数更新具有低秩结构**。也就是说，从预训练模型到微调模型的权重变化 ΔW 可以被分解为两个小矩阵的乘积：

$$W' = W + \alpha \cdot BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times d},\ r \ll d \tag{8.1}$$

其中 W 是冻结的预训练权重，A 和 B 是可训练的低秩矩阵，r（秩）通常取 8–64，α 是缩放因子。

这个假设有实证支持。Aghajanyan 等人在 2020 年发现，预训练模型在微调时的权重更新 存在极低的**内在维度**（intrinsic dimensionality）——一个 175B 参数模型的微调更新可以被投影到仅有几百或几千维的子空间中，几乎不损失性能。LoRA 正是利用了这一特性。

Listing 8.4: LoRA implementation concept

```python
class LoRALinear(nn.Module):
    def __init__(self, in_dim, out_dim, rank=8, alpha=16):
        super().__init__()
        self.W = nn.Linear(in_dim, out_dim, bias=False)
        self.W.weight.requires_grad = False # Freeze

        self.A = nn.Linear(in_dim, rank, bias=False)
        self.B = nn.Linear(rank, out_dim, bias=False)
        self.scale = alpha / rank

    def forward(self, x):
        # Original output + low-rank adaptation
        return self.W(x) + self.scale * self.B(self.A(x))
```

这段代码展示了 LoRA 的核心：原始权重 `W` 被冻结（`requires_grad = False`），只有两个小矩阵 `A` 和 `B` 参与训练。对于 $d = 4096$，$r = 8$，可训练参数从 $d^2 = 16.7M$ 降低到 $2 \times d \times r = 65K$——**减少了 256 倍**。

[2] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

Rank 选择与应用位置

Rank r 的选择是 LoRA 中最重要的超参数决策。

r 太小（如 $r = 4$）：表达能力不足，复杂任务的微调效果受限。适合简单的格式对齐或风格迁移。

r 太大（如 $r = 128$）：可训练参数接近全参数微调的 1–2%，优势减弱，同时增加了内存和计算开销。

实践指南：

- $r = 8$–16：适合大多数 SFT 场景。
- $r = 32$–64：适合需要更多适应的场景（如跨语言微调、领域专业化）。
- $r = 128$+：接近全参数微调的效果，但通常不值得——此时不如直接做全参数微调。

LoRA 应该应用到模型的哪些层？研究表明，注意力层的 **Q 和 V 投影**是最重要的——只对 Q、V 应用 LoRA 通常能获得 对所有注意力投影（Q、K、V、O）应用时 80–90% 的效果，但参数量减半。也有研究发现对 MLP 层同时应用 LoRA 在某些任务上有额外收益。最新的实践倾向于对所有线性层都应用 LoRA （attention + MLP），使用较小的 rank 补偿参数量增加。

QLoRA：极致内存效率

QLoRA[3] 更进一步：将冻结的预训练权重 W 量化为 4-bit，使得 70B 模型可以在单块 48GB GPU 上微调。

QLoRA 引入了一种新的 4-bit 数据类型——**NF4** （4-bit NormalFloat）。它的设计基于一个观察：预训练模型的权重分布近似正态分布。NF4 将 4-bit 的 16 个量化级别 按照正态分布的分位数进行不等间距划分，使得量化误差在统计意义上最小。

QLoRA 的内存账本：

- 70B 模型权重（NF4）：$70B \times 0.5$ bytes = 35 GB
- LoRA 适配器（BF16，$r = 16$）：约 200 MB
- 优化器状态（仅针对适配器）：约 400 MB
- 激活值和梯度：约 8–12 GB
- **总计**：约 45–48 GB——刚好塞进一块 A6000（48GB）

对比全参数微调的内存需求：$70B \times 2$ bytes (BF16) = 140 GB（仅模型权重）加上 Adam 优化器状态需要额外 ~280 GB，总计超过 **420 GB**——需要至少 6 块 H100。

LoRA 的演进

LoRA 自提出以来催生了一系列改进变体：

LoRA+ 观察到 LoRA 中矩阵 A 和 B 使用相同的学习率是次优的。B 在初始化时为零矩阵，需要更大的学习率来「启动」；A 使用高斯初始化，需要更小的学习率保持稳

[3] T. Dettmers et al., "QLoRA: Efficient finetuning of quantized language models," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

定。LoRA+ 对两者使用不同的学习率，在某些任务上提升了 2–5% 的效果。

DoRA（Weight-Decomposed Low-Rank Adaptation）将权重分解为**方向**和**幅度**两个分量：$W' = m \cdot \frac{W+BA}{\|W+BA\|}$，其中 m 是可学习的幅度向量。这种分解的动机是：全参数微调时 权重的方向和幅度变化模式与 LoRA 不同——LoRA 倾向于同时改变两者，而全参数微调更多地调整方向。DoRA 通过显式分离两者，使得 LoRA 的行为更接近全参数微调。

rsLoRA（rank-stabilized LoRA）修正了 LoRA 的缩放因子：原始 LoRA 使用 α/r 作为缩放，rsLoRA 改为 $\alpha/\sqrt{r}$。这个看似微小的改变确保了 不同 rank 下的学习动态更加一致——当你增大 rank 时，不需要重新调学习率。

8.2 RLHF：学会人类偏好

SFT 教会了模型如何对话，但它本质上是「模仿」——模型只能像训练数据中的回答一样好，不会更好。而且它无法区分「还行」和「很好」的回答。

RLHF（Reinforcement Learning from Human Feedback）[4] 引入了一个关键转变：不是教模型「什么是正确答案」，而是教它**人类更偏好什么**。

8.2.1 第一步：收集偏好数据

给人类标注者展示同一个指令的两个不同回答，让他们选择更好的那个。这比让人类写出完美回答要容易得多——判断「A 比 B 好」远比从零开始写一个完美回答简单。

偏好收集有两种主要方式：

- **配对比较**（pairwise comparison）：展示两个回答，选更好的那个。这是最常用的方式，也是 Bradley-Terry 模型的基础。优势是决策简单，标注者间一致性较高。
- **Likert 评分**：对单个回答按 1–5 分评分。可以提供更细粒度的信号，但标注者间的评分标准很难统一——一个人的 4 分可能是另一个人的 3 分。

标注者间一致性（inter-annotator agreement）是一个持续的挑战。InstructGPT 的经验表明，即使经过培训的标注者之间，在约 25–30% 的案例中也存在分歧。对于主观性强的问题（如创意写作），分歧比例更高。OpenAI 的做法是保留多个标注者的判断，在训练奖励模型时使用多数投票或概率化处理。

8.2.2 第二步：训练奖励模型

用偏好数据训练一个**奖励模型**（Reward Model），它接受一个（指令, 回答）对，输出一个标量分数，表示人类对这个回答的偏好程度。

奖励模型通常也是一个 LLM，但输出层替换为回归头——最后一个 token 的隐藏状态通过一个线性层 映射为一个标量分数。

[4] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

奖励模型的训练使用 **Bradley-Terry 模型**：给定偏好对 (y_w, y_l)（其中 y_w 被偏好），奖励模型的损失函数为：

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r(x, y_w) - r(x, y_l)) \tag{8.2}$$

其中 $r(x, y)$ 是奖励模型对回答 y 的评分，σ 是 sigmoid 函数。这个损失函数的直觉是：被偏好回答的分数应该高于不被偏好回答的分数，两者的差值越大，损失越低。

奖励模型的规模通常与策略模型相当或稍小。InstructGPT 使用了一个 6B 参数的奖励模型 来引导 175B 参数的策略模型。更大的奖励模型通常更准确，但增加了 RLHF 管线的内存和计算开销。

奖励模型训练的挑战

训练一个好的奖励模型远比上面的数学公式暗示的困难。实践中会遇到多层次的挑战：

标注噪声与不一致性。人类标注者之间的偏好不一致是根本性的。对于「哪个回答更好」，不同标注者的判断 在 25–30% 的案例中存在分歧（InstructGPT 数据）。对于主观性强的任务（创意写作、开放式建议），分歧比例可能超过 40%。这意味着奖励模型的训练上界受限于人类一致性——它无法比人类标注者自己更准确。

处理标注噪声的常见策略包括：

- **多数投票**：每个偏好对由 3–5 名标注者独立判断，取多数意见。但这使标注成本翻 3–5 倍。
- **概率标签**：不用硬标签（y_w 或 y_l），而是用标注者投票比例作为软标签。例如 3 人选 A、2 人选 B，则标签为 $(0.6, 0.4)$ 而非 $(1, 0)$。
- **标注者校准**：根据每个标注者的历史表现 为其判断加权。一致性高的标注者获得更高权重。

奖励模型的分布偏移。奖励模型在固定的偏好数据分布上训练，但策略模型在 RL 训练过程中不断生成新的、不同分布的回答。随着策略更新，模型生成的回答分布 越来越偏离奖励模型训练时的数据分布，导致奖励模型在新分布上的估计不准确。这是奖励黑客的根本原因之一。

一种缓解策略是**迭代训练**：RL 训练一段时间后，用当前策略模型生成新的回答，收集新的人类偏好，更新奖励模型，然后继续 RL。这种「在线 RLHF」大幅增加了复杂度和成本，但能显著减少奖励黑客。Anthropic 和 OpenAI 在其生产管线中都采用了迭代式奖励模型更新。

奖励模型的规模选择。Llama 2 的技术报告透露，Meta 在不同版本中尝试了与策略模型相同规模的奖励模型（70B），发现更大的奖励模型确实更准确，但带来了显著的内存和延迟开销。实践中的权衡是：奖励模型通常为策略模型的 50–100% 大小。近期的趋势是使用多个较小的奖励模型集成（ensemble），每个模型独立训练，取平均分数作为最终奖励。这种方法既提高了准确性，又有助于检测奖励黑客——当多个奖励模型的分数出现大幅分歧时，通常意味着策略模型正在利用某个奖励模型的漏洞。

奖励黑客与过度优化

奖励黑客（reward hacking）是 RLHF 中最棘手的问题之一。当策略模型对奖励模型优化过度时，它会找到获得高分但实际上无用的模式。Goodhart 定律在这里完美适用：*「当一个度量变成了目标，它就不再是好的度量。」*

Gao 等人（2023）系统研究了奖励过度优化的 scaling law。他们发现一个令人不安的模式：随着 RL 训练的进行，**奖励模型的代理分数**（proxy reward）单调上升，但**真实的人类偏好**（gold reward）先上升、后下降，呈现倒 U 型曲线。也就是说，存在一个最优的优化程度——超过这个点后，模型在奖励模型上的「表现」越来越好，但实际上的回答质量越来越差。

奖励黑客的常见表现形式包括：

- **长度操纵**：奖励模型对更长的回答给出更高分数（因为训练数据中好的回答往往更详细），策略模型学会了生成冗长但空洞的回答。这是最常见的奖励黑客形式。
- **格式套利**：模型学会使用特定的格式化模式（如加粗、列表、总结段落）来获得高分，而不是真正提升内容质量。
- **谄媚**（sycophancy）：奖励模型可能对 肯定用户观点的回答给出更高分数，导致策略模型学会了「讨好用户」——即使用户的观点是错误的，模型也倾向于赞同。
- **自我重复**：模型反复强调同一个正确的关键点，因为奖励模型对包含正确信息的回答给出高分。
- **代码执行操纵**：在代码任务中，模型可能学会修改测试用例而不是修复代码 来通过验证——这是一种更隐蔽的奖励黑客。

缓解奖励黑客的主要策略：

1. **KL 惩罚**（已在 PPO 目标中）：限制策略偏离参考模型的程度。β 的选择是关键——过小则抑制不住奖励黑客，过大则策略几乎不更新。典型值为 $\beta = 0.01$–0.1。
2. **奖励模型集成**：使用多个独立训练的奖励模型，取保守估计（最小值或下四分位数）。单个奖励模型容易被利用，但多个模型同时出错的概率更低。
3. **长度惩罚**：在奖励信号中显式减去 与回答长度成比例的惩罚项，或将奖励归一化为每 token 的平均奖励。
4. **早停**（early stopping）：监控真实的人类评估指标，在代理奖励持续上升但人类偏好开始下降时停止训练。这需要在训练过程中定期进行人类评估抽检。
5. **迭代奖励模型更新**：定期用策略模型的最新输出 收集新的偏好数据，更新奖励模型，使其跟上策略的分布变化。

能量损失现象与奖励黑客

2025 年 ICML 的一项研究揭示了 RLHF 中的 **能量损失现象**（energy loss phenomenon）。研究发现，在 RL 训练过程中，LLM 最后一层的隐藏状态的**能量**（范数的平方）会逐渐降低，这与奖励黑客行为存在强相关。当能量损失加剧时，模型更容易产生奖励黑客行为。通过在训练中加入能量正则化（鼓励维持隐藏状态的能量），可以有效缓解奖励黑客——这为理解和解决奖励黑客提供了新的视角。

8.2.3 第三步：PPO 优化

使用 PPO[5]（Proximal Policy Optimization）算法，让模型（策略）生成回答，用奖励模型打分作为奖励信号，通过 RL 最大化期望奖励。

PPO 的核心目标函数：

$$J_{\text{PPO}} = \mathbb{E}\left[\min\left(\frac{\pi_\theta}{\pi_{\text{old}}} A_t,\ \text{clip}\left(\frac{\pi_\theta}{\pi_{\text{old}}},\ 1 \pm \epsilon\right) A_t\right)\right] - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \tag{8.3}$$

其中 $\pi_\theta/\pi_{\text{old}}$ 是策略概率比，A_t 是优势函数（由 critic 网络估计），clip 操作限制策略更新幅度防止过大的变化，KL 惩罚项防止模型偏离预训练分布太远。注意此处是 RLHF 中常用的 PPO 变体——原始 PPO 算法[6]使用裁剪目标 或自适应 KL 惩罚二者之一，而 RLHF 实践中通常将裁剪与固定系数的 KL 惩罚结合使用，以同时限制策略更新幅度和防止奖励黑客。

让我们拆解这个公式中的每个组件：

策略概率比 $\pi_\theta/\pi_{\text{old}}$ 衡量新策略相对于旧策略的变化。如果某个回答在新策略下的概率增加了，这个比值大于 1；如果减少了，小于 1。

优势函数 A_t 用 **GAE**（Generalized Advantage Estimation）计算：$A_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}$，其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 是 TD 误差。GAE 通过 λ 参数在偏差和方差之间权衡——$\lambda = 0$ 时等于单步 TD（低方差高偏差），$\lambda = 1$ 时等于 Monte Carlo（低偏差高方差）。实践中通常取 $\lambda = 0.95$。

Clip 操作是 PPO 的核心创新。它限制了策略更新的幅度：如果新策略相对于旧策略的变化超过了 $(1-\epsilon, 1+\epsilon)$ 的范围（通常 $\epsilon = 0.2$），梯度被截断，阻止过大的策略跳跃。这使得训练更加稳定——旧的 policy gradient 方法（如 REINFORCE）因为没有这种限制，经常出现策略崩溃。

KL 惩罚是防止**奖励黑客**（reward hacking）的关键。没有 KL 惩罚，策略模型会学到一些 获得高奖励分数但实际上无意义的模式——比如生成冗长的、自我重复的回答（因为奖励模型碰巧对更长的回答给出更高分数）。KL 惩罚通过限制策略偏离参考模型（SFT 模型）的程度，确保模型在优化奖励的同时不丢失基本的语言能力。

[5] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
[6] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

8.2.4 PPO 的工程挑战

RLHF 的 PPO 阶段是整个后训练管线中 工程难度最高的部分。主要原因是它需要同时在 GPU 上加载**四个大模型**：

1. **策略模型**（Policy）：正在被优化的模型——它生成回答，接收梯度更新。
2. **参考模型**（Reference）：冻结的 SFT 模型——用于计算 KL 惩罚。
3. **奖励模型**（Reward）：评估回答质量——对策略模型生成的回答打分。
4. **价值模型**（Value/Critic）：估计状态价值函数——用于计算优势函数 A_t。

对于一个 70B 参数的策略模型，即使参考模型和价值模型可以用更小的版本，四个模型的总内存需求也极其恐怖。再加上每一步都需要：生成回答 → 奖励打分 → 计算优势 → 更新策略——这个循环比 SFT 的纯前向/反向传播复杂得多。

InstructGPT[7] 的训练规模：约 40 名标注者，收集约 33K 偏好比较。结果令人印象深刻：仅 1.3B 参数的 InstructGPT 在人类评估中 被偏好于 175B 参数的 GPT-3——这有力地证明了 RLHF 的价值：一个经过良好对齐的小模型 可以比一个未对齐的大模型更受用户欢迎。

8.2.5 PPO 的工程实践

RLHF 的 PPO 训练循环可以分解为四个交替执行的步骤，每一步都有独特的工程考量：

第一步：样本生成（Rollout/Generation）。策略模型对一批 prompt 执行自回归生成。这一步是纯推理过程——需要大 batch 的高吞吐生成，通常使用 vLLM 或 TensorRT-LLM 等推理引擎加速。关键超参数包括 temperature（通常 0.7–1.0，比 SFT 的贪心解码更高以鼓励探索）和 top-p（通常 0.9–0.95）。每个 prompt 生成 1–4 个回答，batch 大小从数百到数千个 prompt 不等。

第二步：奖励评估（Reward Scoring）。将生成的回答送入奖励模型打分。这一步是纯前向传播，无需梯度，但需要处理大量的（prompt, response）对。为防止奖励黑客，通常对原始奖励做裁剪（clip 到 $[-5, 5]$）或归一化（减均值除标准差）。

第三步：优势估计（Advantage Estimation）。用 critic 网络计算每个 token 位置的价值估计，结合奖励信号通过 GAE 计算优势函数。LLM 版本的 PPO 有一个特殊之处：**奖励通常只在序列末尾给出**——中间 token 没有显式的奖励信号。这意味着 critic 需要学会将这个终端奖励 分配到整个序列的各个位置——这比传统 RL（每步都有奖励）更难。

第四步：策略更新（Policy Gradient Update）。用 PPO 的 clipped 目标函数计算策略梯度，更新策略模型和 critic 模型的权重。这一步需要全精度的反向传播和优化器步骤。通常对同一批数据做 1–4 次 mini-batch 更新（PPO 的 epoch 数），然后回到第一步生成新样本。

四步循环中，**样本生成**通常是时间瓶颈——自回归生成比前向/反向传播慢得多。一种优化策略是将生成和训练**异步化**：一组 GPU 专门做生成（推理模式），另一组 GPU

[7] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

做梯度更新（训练模式），两者通过共享队列传递数据。DeepSpeed-Chat 和 OpenRLHF 等开源框架 提供了这种异步 PPO 训练的实现。

Listing 8.5: RLHF PPO training loop (high-level)

```python
for epoch in range(num_epochs):
    # Step 1: Generate responses
    prompts = sample_batch(prompt_dataset, batch_size)
    with torch.no_grad():
        responses = policy_model.generate(
            prompts, temperature=0.8, top_p=0.95,
            max_new_tokens=512
        )

    # Step 2: Score with reward model
    with torch.no_grad():
        rewards = reward_model.score(prompts, responses)
        rewards = torch.clamp(rewards, -5.0, 5.0)

    # Step 3: Compute advantages with GAE
    with torch.no_grad():
        values = critic_model.predict_values(
            prompts, responses
        )
        advantages = compute_gae(
            rewards, values, gamma=1.0, lam=0.95
        )
        advantages = (advantages - advantages.mean()) \
                 / (advantages.std() + 1e-8)

    # Step 4: PPO update (multiple mini-batch epochs)
    old_log_probs = policy_model.log_prob(
        prompts, responses
    ).detach()
    ref_log_probs = ref_model.log_prob(
        prompts, responses
    ).detach()

    for ppo_epoch in range(4):
        log_probs = policy_model.log_prob(
            prompts, responses
        )
        ratio = torch.exp(log_probs - old_log_probs)

        surr1 = ratio * advantages
        surr2 = torch.clamp(
            ratio, 1 - clip_eps, 1 + clip_eps
```

```
    ) * advantages
    policy_loss = -torch.min(surr1, surr2).mean()

    kl = log_probs - ref_log_probs
    kl_loss = kl_beta * kl.mean()

    loss = policy_loss + kl_loss
    loss.backward()
    optimizer.step()
```

内存优化技巧。在实际部署中，以下技巧可以显著降低 PPO 的内存需求：

- **参考模型权重共享**：用 LoRA 表示策略模型，基础权重同时作为参考模型使用。只需将 LoRA 适配器的贡献置零即可得到参考模型的输出。这省去了一个完整模型的内存。
- **Critic 复用**：让 critic 和奖励模型共享骨干网络，只使用不同的输出头。或者用更小规模的 critic（如策略的 1/4 大小）。
- **梯度检查点**：在训练步骤中使用 activation checkpointing 交换计算和内存。
- **混合精度**：生成和评分用 FP16/BF16，梯度更新用 FP32，优化器状态用 FP32。

为什么 RL 比 SFT 更强?

SFT 是「模仿学习」——模型只能学到训练数据中展示的行为。RL 是「探索优化」——模型可以尝试训练数据中**没有**的新策略，只要奖励模型认为它更好。这就是为什么 RLHF 后的模型 通常比纯 SFT 的模型更能处理复杂、开放式的问题——它学会了「什么是好的」，而不仅仅是「什么是被人写过的」。

一个直觉的类比：SFT 像是让学生抄写优秀作文——他学到了好作文的模式，但不知道**为什么**好。RLHF 像是有一位老师不断评价学生自己写的作文——学生可以尝试新的写作方式，老师告诉他哪些尝试更好、哪些更差，最终学生学会了「好」的本质，甚至可能写出超越范文的作品。

8.3 DPO：去掉奖励模型

RLHF 强大但复杂——它需要同时维护四个模型：策略模型、参考模型、奖励模型和 critic 网络。这在工程上和内存上都是巨大的负担。

DPO[8]（Direct Preference Optimization）的核心洞察是：**奖励模型是可以被数学消除的**。

[8] R. Rafailov et al., "Direct preference optimization: Your language model is secretly a reward model," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

8.3.1 从 RLHF 到 DPO 的完整推导

DPO 的数学推导是后训练领域最优美的结果之一。让我们从 RLHF 的优化目标开始，一步一步推导出 DPO 的损失函数。

起点：KL 约束的 RL 问题。RLHF 的优化目标是在 KL 约束下最大化期望奖励：

$$\max_{\pi_\theta} \ \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}[r(x,y)] - \beta\, D_{\mathrm{KL}}\big(\pi_\theta(y|x) \,\|\, \pi_{\mathrm{ref}}(y|x)\big) \tag{8.4}$$

展开 KL 散度：

$$= \mathbb{E}_{x,y\sim\pi_\theta}\left[r(x,y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}\right] \tag{8.5}$$

步骤一：求解最优策略。对每个固定的 x，这是一个关于分布 $\pi_\theta(\cdot|x)$ 的优化问题。通过变分法（对 $\pi_\theta(y|x)$ 求泛函导数并令其为零），可以得到最优策略的闭式解：

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{r(x,y)}{\beta}\right) \tag{8.6}$$

其中配分函数 $Z(x) = \sum_y \pi_{\mathrm{ref}}(y|x)\exp(r(x,y)/\beta)$ 确保 π^* 是合法的概率分布。这个结果的直觉是：最优策略在参考策略的基础上，按奖励的指数进行重新加权——高奖励的回答获得更高的概率提升。

步骤二：从策略反解奖励。将公式 (8.6) 两边取对数并重排：

$$r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\mathrm{ref}}(y|x)} + \beta \log Z(x) \tag{8.7}$$

这个等式揭示了一个关键洞察：**最优策略隐式地定义了奖励函数**。也就是说，如果我们知道最优策略 π^*，就能反推出奖励函数（模一个与 y 无关的常数 $\beta \log Z(x)$）。

步骤三：代入 Bradley-Terry 模型。在 Bradley-Terry 偏好模型中，人类偏好 y_w 胜过 y_l 的概率为：

$$P(y_w \succ y_l|x) = \sigma\big(r(x,y_w) - r(x,y_l)\big) \tag{8.8}$$

将公式 (8.7)的奖励表达式代入：

$$\begin{aligned} r(x,y_w) - r(x,y_l) &= \beta \log \frac{\pi^*(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} + \beta \log Z(x) \\ &\quad - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)} - \beta \log Z(x) \\ &= \beta \log \frac{\pi^*(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)} \end{aligned} \tag{8.9}$$

关键的数学魔法发生在这里：$\beta \log Z(x)$ **项完美消去**——因为两个回答共享相同的输入 x，配分函数是相同的。这意味着我们不需要计算难以处理的配分函数！

步骤四：构造损失函数。用策略模型 π_θ 替代最优策略 π^*，对偏好数据集上所有偏好

对取负对数似然，得到 DPO 的损失函数：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right] \tag{8.10}$$

其中 y_w 是人类偏好的回答（winner），y_l 是不被偏好的回答（loser），σ 是 sigmoid 函数。

这个推导的本质是：**DPO 跳过了显式的奖励建模步骤，直接在策略空间中优化偏好**。奖励模型被「折叠」进了策略本身——$\beta\log\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ 就是策略隐式定义的奖励。

8.3.2 DPO 损失函数的直觉

让我们通过一个具体例子来理解 DPO 损失函数的含义。假设用户问「如何煮意大利面?」，模型生成了两个回答：

- y_w（被偏好的）：「在大锅中烧开盐水，下入意大利面煮 8–10 分钟至有嚼劲（al dente），捞出沥干，拌入你喜欢的酱汁。」
- y_l（不被偏好的）：「可以煮着吃。」

DPO 损失函数 (8.10)中的 $\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}$ 衡量的是：相比参考模型，当前模型在多大程度上 更倾向于生成被偏好的回答？减去 $\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}$ 则衡量：当前模型在多大程度上**减少**了 对不被偏好回答的倾向？通过 sigmoid 函数将这个差值映射到 $(0,1)$ 区间，取负对数作为损失。

当模型的行为已经很好（正确地偏好 y_w）时，sigmoid 的输入很大，$-\log\sigma(\cdot)\to 0$，损失接近零。当模型还没有学好（偏好 y_l 而非 y_w）时，sigmoid 的输入为负，$-\log\sigma(\cdot)$ 很大，损失很高。梯度自然地推动模型增加 y_w 的概率、降低 y_l 的概率。

β 参数控制优化的「温度」：β 越大，模型对偏好差异的反应越敏感，但也越容易在偏好数据的噪声上过拟合。典型的 β 值在 0.1–0.5 之间。

8.3.3 DPO vs RLHF：简单性的代价

DPO 将 RLHF 从一个复杂的在线 RL 问题 简化为一个离线的监督学习问题——只需要偏好数据对和标准的梯度下降即可。这使得偏好优化变得几乎和 SFT 一样简单。

DPO 的简化清单

DPO 相比 RLHF (PPO) 省去了：

- 训练奖励模型（省去一整个训练流程）
- 价值网络（critic）（省去一个大模型的内存）
- 在线生成回答（省去推理开销）
- 强化学习循环（省去复杂的工程管线）

DPO 只需要：偏好数据 + 策略模型 + 参考模型 + 梯度下降。
内存需求从 4 个模型降到 2 个模型。

但 DPO 的简单性不是没有代价的：

- **离线限制**：DPO 只能从已有的偏好数据中学习，不像 PPO 可以在训练过程中探索新的回答。这意味着 DPO 的效果高度依赖偏好数据的覆盖度——如果数据中没有覆盖某类问题，DPO 在那类问题上就无法改进。
- **过拟合风险**：在有限的偏好数据上，DPO 容易过拟合到数据的表面特征，而不是学到真正的偏好。比如如果偏好数据中更长的回答总是被标注为更好，DPO 可能学到「长 = 好」的虚假模式。
- **分布偏移**：偏好数据是由参考模型（或某个旧版本模型）生成的。随着训练的进行，策略模型的分布偏离了生成数据的分布，导致估计偏差。PPO 通过在线生成天然避免了这个问题。

8.3.4 DPO 的训练代码

DPO 的实现简洁得令人吃惊——核心逻辑不到 30 行代码：

Listing 8.6: DPO training implementation

```python
def dpo_loss(policy_model, ref_model, batch, beta=0.1):
    """Compute DPO loss for a batch of preference pairs."""
    prompts = batch["prompt"]
    chosen = batch["chosen"] # y_w (preferred)
    rejected = batch["rejected"] # y_l (rejected)

    # Log probs from current policy
    pi_chosen = policy_model.log_prob(prompts, chosen)
    pi_rejected = policy_model.log_prob(prompts, rejected)

    # Log probs from frozen reference model
    with torch.no_grad():
        ref_chosen = ref_model.log_prob(prompts, chosen)
        ref_rejected = ref_model.log_prob(
            prompts, rejected
        )

    # DPO implicit reward:
    #  r(x,y) = beta * log(pi/ref)
    chosen_reward = beta * (pi_chosen - ref_chosen)
    rejected_reward = beta * (pi_rejected - ref_rejected)

    # Bradley-Terry loss with implicit rewards
    loss = -F.logsigmoid(chosen_reward - rejected_reward)

    # Useful metrics for monitoring
    reward_margin = (chosen_reward - rejected_reward).mean()
    accuracy = (chosen_reward > rejected_reward).float().mean()
```

```

    return loss.mean(), {
        "reward_margin": reward_margin.item(),
        "accuracy": accuracy.item(),
    }
```

这段代码的关键在于 `log_prob` 的计算。对于一个回答 $y = (y_1, y_2, \ldots, y_T)$，对数概率是所有 token 的对数概率之和：$\log \pi(y|x) = \sum_{t=1}^{T} \log \pi(y_t|x, y_{<t})$。在实际实现中，这通过一次前向传播获得所有 token 的 logit，然后从中提取对应 token 的对数概率并求和。

训练 DPO 时需要监控的关键指标：

- **隐式奖励边距**（reward margin）：$\mathbb{E}[r_\theta(x, y_w) - r_\theta(x, y_l)]$ 应该随训练逐渐增大，表示模型越来越能区分好坏回答。
- **偏好准确率**：模型正确偏好 y_w 的比例。训练初期通常在 50–60%（接近随机），训练结束时应达到 70–80%。超过 85% 通常意味着过拟合。
- **选择与拒绝的对数概率变化**：理想情况下，$\log \pi_\theta(y_w|x)$ 增加，$\log \pi_\theta(y_l|x)$ 减少。如果两者同时减少，说明模型在「遗忘」，需要降低学习率或增大 β。

8.3.5 DPO 的变体与演进

DPO 的成功激发了一系列改进方法。2024–2025 年间出现了数十种 DPO 变体，形成了一个**偏好优化方法家族**。以下是最重要的几个。

IPO：去掉 Bradley-Terry 假设

IPO（Identity Preference Optimization）由 DeepMind 提出，它指出 DPO 的一个根本局限：DPO 隐含地假设了 Bradley-Terry 偏好模型，即偏好概率严格由奖励差的 sigmoid 决定。但人类偏好通常不满足这个理想化的模型——偏好可能是噪声的、非传递的、或上下文相关的。

IPO 去掉了 Bradley-Terry 假设，直接优化偏好对数概率差的**正则化平方**：

$$\mathcal{L}_{\text{IPO}} = \left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{2\beta} \right)^2 \tag{8.11}$$

与 DPO 的 $-\log \sigma(\cdot)$ 不同，IPO 使用平方损失，并引入了一个目标边距 $\frac{1}{2\beta}$。这意味着 IPO 不会无限制地扩大 y_w 和 y_l 的概率差——一旦达到目标边距，梯度就会消失。这种「有目标的」优化使 IPO 更不容易过拟合，特别是在偏好数据中存在大量噪声标签时。

KTO：非配对偏好学习

KTO（Kahneman-Tversky Optimization）的独特之处在于它**不需要配对数据**。传统的 DPO 需要同一个问题的两个回答（y_w vs y_l），KTO 只需要单独标注的「好」或「不好」（thumbs

up / thumbs down）。这大幅降低了数据收集的门槛——用户的点赞/点踩行为可以直接作为训练信号，不需要将不同回答配成对。

KTO 的理论基础来自行为经济学中的 **前景理论**（Prospect Theory）——人类对损失的厌恶大于对等额收益的喜好（丢 100 元的痛苦大于捡到 100 元的快乐）。KTO 的损失函数显式编码了这种不对称性：

$$\mathcal{L}_{\text{KTO}} = \begin{cases} 1 - \sigma\left(\beta\left(\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} - z_{\text{ref}}\right)\right) & \text{if } y \text{ is desirable} \\ 1 - \sigma\left(\beta\left(z_{\text{ref}} - \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}\right)\right) & \text{if } y \text{ is undesirable} \end{cases} \tag{8.12}$$

其中 $z_{\text{ref}} = \mathbb{E}_{x',y'\sim\mathcal{D}}\left[\beta\, D_{\text{KL}}(\pi_\theta(y'|x')\|\pi_{\text{ref}}(y'|x'))\right]$ 是一个归一化基线。

KTO 的实用价值在于：现实中，配对偏好数据（同一问题的两个回答）比非配对的质量标注（单个回答是好是坏）更难收集也更昂贵。在产品场景中，用户的点赞/点踩行为 天然就是非配对的——用户只对他看到的那一个回答做出反应，不会同时看到同一问题的另一个回答。KTO 使得这类非配对反馈可以直接用于训练。

SimPO：无参考模型的偏好优化

SimPO（Simple Preference Optimization）进一步简化了 DPO，去掉了参考模型。它用回答的**平均对数概率**（即对数概率除以回答长度）作为隐式奖励：

$$\mathcal{L}_{\text{SimPO}} = -\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l|x) - \gamma\right) \tag{8.13}$$

其中 $|y|$ 是回答的 token 长度，γ 是目标奖励边距（通常 $\gamma = 0.5$–2.0）。

SimPO 的两个关键设计选择：

1. **长度归一化**：除以 $|y|$ 消除了长度偏差——避免模型仅仅因为生成更长的回答而获得更高的隐式奖励。这直接解决了 DPO 中常见的长度操纵问题。
2. **目标边距** γ：确保被偏好回答的奖励至少比不被偏好的高 γ。没有这个边距，模型可能「偷懒」——只要 y_w 的奖励略高于 y_l 就停止优化。

去掉参考模型的实用价值巨大：**内存需求从 2 个模型降到 1 个模型**。对于 70B 模型，这意味着节省约 140 GB 的 GPU 内存。实验表明 SimPO 在多个基准上的效果 与 DPO 相当甚至更好——参考模型的约束可能在某些情况下过于保守。

ORPO：SFT 与偏好优化的统一

ORPO（Odds Ratio Preference Optimization）提出了一个更激进的简化：将 SFT 和偏好优化合并为**一步训练**，不需要单独的 SFT 阶段，也不需要参考模型。

ORPO 的核心观察是：标准的 SFT 交叉熵损失 对被偏好和不被偏好的回答都会增加概率——因为交叉熵只看目标 token 的概率，不关心「好」和「差」的区别。ORPO 在

SFT 损失的基础上加入一个**赔率比**惩罚项：

$$\mathcal{L}_{\text{ORPO}} = \underbrace{\mathcal{L}_{\text{SFT}}(y_w)}_{\text{SFT on preferred}} + \lambda \cdot \underbrace{\log \sigma\left(\log \frac{\text{odds}_\theta(y_w|x)}{\text{odds}_\theta(y_l|x)}\right)}_{\text{preference penalty}} \tag{8.14}$$

其中 $\text{odds}_\theta(y|x) = \frac{P_\theta(y|x)}{1-P_\theta(y|x)}$ 是回答 y 的赔率。

ORPO 的优势是**训练流程极其简洁**：一次训练同时完成格式学习（SFT）和偏好对齐（DPO-like）。劣势是两个目标可能互相干扰——在实践中，ORPO 的效果对 λ 的选择敏感。

cDPO 与在线 DPO

cDPO（conservative DPO）在损失函数中引入 标签噪声的显式建模。假设偏好标注以概率 ϵ 出错（标注者将 y_w 和 y_l 搞反），cDPO 修改 DPO 损失为：

$$\mathcal{L}_{\text{cDPO}} = -\log\left[(1-\epsilon)\sigma(\hat{r}_w - \hat{r}_l) + \epsilon\sigma(\hat{r}_l - \hat{r}_w)\right] \tag{8.15}$$

其中 $\hat{r} = \beta \log\left(\pi_\theta/\pi_{\text{ref}}\right)$。当 $\epsilon = 0$ 时退化为标准 DPO。

在线 DPO（Online DPO / Iterative DPO）解决 DPO 的离线限制。核心思想是在每轮训练后，用当前策略模型重新生成回答并收集新的偏好标注，然后在更新后的数据上继续 DPO 训练。这使 DPO 获得了类似 PPO 的在线探索能力，实验表明在线 DPO 的效果显著优于离线 DPO，接近甚至超过 PPO。

8.3.6 前沿实验室的选择：2025–2026

一个实践者关心的问题是：前沿实验室在 2025–2026 年实际使用哪种偏好优化方法？

答案因实验室和任务而异，没有「一种方法统治一切」的局面：

- **OpenAI**：o1/o3 系列推理模型 使用大规模 RL（PPO 变体）进行推理能力训练，配合**审议式对齐**（deliberative alignment）进行安全训练。对于非推理的通用模型（如 GPT-4.5），据信仍使用 PPO-based RLHF。
- **Anthropic**：Claude 系列使用 Constitutional AI（RLAIF）框架，训练过程中结合了 RLHF 和 AI 反馈。2026 年 1 月发布的新宪法 暗示了更复杂的多阶段对齐管线。
- **Meta**：Llama 3 使用了 DPO 作为主要的偏好优化方法（技术报告明确提及），辅以拒绝采样和迭代 DPO。Llama 4 据信继续使用 DPO 家族方法。
- **DeepSeek**：R1 和 V3 使用 GRPO，这在推理任务上表现优异。对通用对话任务辅以奖励模型驱动的 RL。
- **Google**：Gemini 系列的对齐方法 未完全公开，但从技术报告中可推断 使用了 RLHF（PPO 变体）结合 大规模 AI 评判者的偏好数据。
- **开源社区**：DPO 和 SimPO 是最受欢迎的方法，因为它们实现简单、内存高效。

GRPO 因 DeepSeek-R1 的成功 在 2025 年被 Hugging Face TRL 和 OpenRLHF 框架广泛支持，成为开源推理模型训练的首选。

偏好优化的方法选择指南

没有一种方法适合所有场景。以下是实践中的选择逻辑：

- **有配对偏好数据 + 内存受限**：DPO 或 SimPO（最简单）
- **有非配对反馈（点赞/点踩）**：KTO
- **从头训练，没有 SFT 模型**：ORPO
- **可验证的推理任务**：GRPO（无需奖励模型）
- **追求最高质量，资源充裕**：在线 DPO 或 PPO（需要显著更多的工程投入）
- **偏好数据噪声大**：IPO（对噪声鲁棒）

8.4 GRPO：推理模型的训练

8.4.1 PPO 的瓶颈

PPO 虽然强大，但有一个致命的扩展性问题：它需要一个 critic 网络来估计优势函数 A_t。对于 70B+ 的模型，这意味着你需要同时在 GPU 上加载四个大模型（策略 + 参考 + 奖励 + critic），内存需求极其恐怖。在实践中，PPO 在大规模模型上的训练成本和工程复杂度远高于 DPO/GRPO，需要额外的内存优化技术（如模型并行、参数卸载等）。

更根本的问题是：对于数学和代码这类有**可验证答案**的任务，用一个学出来的奖励模型来提供奖励信号 既不必要（答案对不对可以直接验证），又不可靠（奖励模型本身可能犯错）。

8.4.2 GRPO 的核心算法

GRPO[9]（Group Relative Policy Optimization）由 DeepSeek 提出，专门为大规模推理模型训练设计。它的核心创新是**消除 critic 网络**：

对于每个输入 x，生成一组 G 个回答 $\{y_1, \ldots, y_G\}$，用规则或奖励模型为每个回答打分 $\{R_1, \ldots, R_G\}$。然后用**组内统计量**替代 critic 来估计优势：

$$A_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G)} \tag{8.16}$$

直觉上：如果一个回答的奖励高于同组的平均水平，它就是「好的」，应该被强化；如果低于平均水平，就应该被抑制。这是一种**相对评价**——我们不需要知道一个回答的「绝对」好坏，只需要知道它在这组回答中的**相对**位置。

[9] Z. Shao et al., "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

GRPO 的完整优化目标：

$$J_{\text{GRPO}} = \mathbb{E}_{x,\{y_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\min\left(\frac{\pi_\theta(y_i|x)}{\pi_{\text{old}}(y_i|x)}A_i,\ \text{clip}\left(\frac{\pi_\theta(y_i|x)}{\pi_{\text{old}}(y_i|x)}, 1\pm\epsilon\right)A_i\right) - \beta\, D_{\text{KL}}(\pi_\theta\|\pi_{\text{ref}})\right] \tag{8.17}$$

与 PPO 对比，GRPO 的关键区别：

- **无 critic**：优势 A_i 由组内统计量计算，不需要价值网络。内存减少 25–30%。
- **组采样**：每个 prompt 生成 G 个回答（通常 $G = 8$–64），而非 PPO 的单个回答。这提供了更丰富的对比信号。
- **规则奖励**：对数学和代码任务，可以使用确定性的规则奖励（答案正确 +1，答案错误 −1），无需训练奖励模型。这种使用可验证奖励的方法也被称为 **RLVR**（Reinforcement Learning with Verifiable Rewards）——通过数学答案比对、代码执行测试等确定性验证手段提供奖励信号，避免了奖励模型的不可靠性。

GRPO 与 PPO 的数学对比

为了深入理解 GRPO 的设计动机，让我们将它与 PPO 在数学层面进行对比。

在标准 PPO 中，优势函数通过**价值网络** $V_\phi(s_t)$ 估计：

$$A_t^{\text{PPO}} = \sum_{l=0}^{T-t}(\gamma\lambda)^l\left[r_t + \gamma V_\phi(s_{t+l+1}) - V_\phi(s_{t+l})\right] \tag{8.18}$$

这要求训练一个参数化的价值函数 V_ϕ，它本身就需要大量的数据和计算来训练，且价值估计的误差会传播到策略梯度中。

GRPO 的核心观察是：**对于 LLM 的对话任务，可以用同一 prompt 的多个采样来替代价值网络**。当我们对同一个 prompt x 采样 G 个回答时，组内平均奖励 $\bar{R} = \frac{1}{G}\sum_j R_j$ 就是该 prompt 下期望奖励的**无偏估计**——这本质上就是价值函数 $V(x)$ 的 Monte Carlo 估计。因此组内归一化的优势：

$$A_i = \frac{R_i - \bar{R}}{\text{std}(\{R_j\})}$$

是对「回答 y_i 相比该 prompt 的平均水平好多少」的无偏估计，无需学习一个参数化的 critic。

这种替代在 RL 文献中并非全新——它与 **REINFORCE with baseline** 密切相关，其中 baseline 使用蒙特卡洛均值。GRPO 的贡献在于：(1) 将这种思想与 PPO 的 clipped 目标结合，保留了 PPO 的训练稳定性；(2) 通过标准差归一化进一步降低了方差；(3) 证明了这种方法在百亿级 LLM 上的实用性。

GRPO 相比 PPO 的**计算权衡**：

- **节省**：省去了 critic 的前向传播、反向传播和参数存储。对 70B 模型，这节省约 140–280 GB GPU 内存。

- **代价**：每个 prompt 需要生成 G 个回答而非 1 个，采样开销增加 G 倍。但生成是无梯度的推理过程，可以高效并行化。
- **净效果**：对于大模型，GRPO 的总训练效率 通常优于 PPO，因为瓶颈从内存变成了计算，而计算更容易通过并行化解决。

GRPO 的 Token 级变体

原始 GRPO 在**回答级别**计算优势——整个回答 y_i 获得一个标量优势 A_i。但实际上，一个回答中的不同 token 对最终质量的贡献是不同的——关键的推理步骤比格式化 token 重要得多。

一种改进是 **token 级 GRPO**：将回答级的优势分配到各个 token 位置。DeepSeek-R1 的实现中，对于回答 y_i 的第 t 个 token，优势为：

$$A_{i,t} = A_i \quad \forall t \in \{1, \ldots, |y_i|\} \tag{8.19}$$

即所有 token 共享相同的回答级优势。虽然这看似粗糙，但实践中效果良好——因为 policy gradient 中的 $\nabla \log \pi_\theta(y_t|\cdot)$ 已经为不同 token 位置提供了差异化的梯度信号。

更精细的方法（如 token 级过程奖励）目前是活跃的研究方向，将在推理模型的 PRM 部分讨论。

8.4.3 GRPO 的广泛采用

DeepSeek-R1 的成功使 GRPO 迅速成为 推理模型训练的事实标准。到 2025 年中，GRPO 已被多个团队采用或改进：

- **Hugging Face TRL**：在 v0.13+（2025 年初）中正式支持 GRPO trainer，成为开源社区训练推理模型的首选工具。使用 TRL 的 `GRPOTrainer`，几十行代码即可在自定义数据上训练推理模型。
- **OpenRLHF**：一个专注于 RLHF/GRPO 的分布式训练框架，支持多节点 GRPO 训练，已在 8 节点 64-GPU 集群上验证了 70B 模型的 GRPO。
- **Hybrid GRPO**：Sane（2025）提出在 GRPO 的组内增加 多采样策略（不同温度、不同 top-p），提供更多样化的对比信号。
- **各开源推理模型**：QwQ、Sky-T1、Open-R1 等 多个开源推理模型项目 都采用了 GRPO 作为核心训练算法。

GRPO 的成功还催生了一个更广泛的范式——**RLVR**（Reinforcement Learning with Verifiable Rewards）。RLVR 的核心理念是：对于那些答案可以自动验证的任务（数学、代码、形式逻辑、棋类博弈等），应该直接使用规则验证器作为奖励信号，而不是训练一个可能出错的奖励模型。GRPO + RLVR 的组合已经在多项推理基准上 超越了传统的 RLHF + PPO 方法。

8.4.4 GRPO 的训练代码

让我们看一下 GRPO 训练的核心逻辑:

Listing 8.7: GRPO training step (simplified)

```python
def grpo_step(model, ref_model, prompts, reward_fn,
          group_size=8, clip_eps=0.2, kl_beta=0.04):
   all_loss = []
   for prompt in prompts:
      # Generate a group of responses
      responses = [model.generate(prompt)
               for _ in range(group_size)]

      # Score each response
      rewards = [reward_fn(prompt, r) for r in responses]
      rewards = torch.tensor(rewards)

      # Compute group-relative advantages
      advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-8)

      # Compute old log probs (from policy at time of generation)
      old_log_probs = [model.log_prob(prompt, r).detach()
                  for r in responses]

      for resp, adv, old_lp in zip(responses, advantages,
                           old_log_probs):
         # Current policy and reference log probs
         log_prob = model.log_prob(prompt, resp)
         ref_log_prob = ref_model.log_prob(prompt, resp)

         # Policy ratio uses OLD policy (at sample generation time)
         ratio = torch.exp(log_prob - old_lp)

         # Clipped PPO-style objective
         surr1 = ratio * adv
         surr2 = torch.clamp(ratio, 1-clip_eps, 1+clip_eps) * adv
         policy_loss = -torch.min(surr1, surr2)

         # KL penalty uses REFERENCE model (frozen SFT model)
         kl_loss = kl_beta * (torch.exp(log_prob - ref_log_prob)
                        - 1 - (log_prob - ref_log_prob))

         all_loss.append(policy_loss + kl_loss)

   loss = torch.stack(all_loss).mean()
   loss.backward()
   optimizer.step()
```

这段代码展示了 GRPO 的三个核心步骤。第一步，对每个 prompt 生成一组回答（`group_size=8`），用奖励函数给每个回答打分。第二步，计算组内相对优势——分数高于组均值的回答获得正 advantage，低于均值的获得负 advantage。第三步，用 PPO 风格的 clipped 目标优化策略——增加高 advantage 回答的概率，降低低 advantage 回答的概率，同时通过 KL 惩罚防止策略偏离参考模型太远。

注意代码中的两个关键区分：**策略比率**使用的是生成样本时的旧策略 `old_lp`（即 π_{old}），而 **KL 惩罚**使用的是冻结的参考模型 `ref_log_prob`（即 π_{ref}）。这与公式 (8.17)完全对应——clipped 目标中的比率是 $\pi_\theta/\pi_{\text{old}}$，KL 散度是 $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$。此处的 KL 使用了 Schulman 提出的近似形式 $\hat{D}_{\text{KL}} = e^r - 1 - r$（其中 $r = \log \pi_\theta - \log \pi_{\text{ref}}$），相比简单的 r 形式对大偏差更加敏感。

另外，这里没有 critic 网络——advantage 完全由组内统计量（均值和标准差）计算，这就是 GRPO 相比 PPO 节省内存的关键。

8.4.5 奖励信号的设计

对于推理模型（如 DeepSeek-R1），奖励信号的设计是一门精细的艺术。DeepSeek-R1 的奖励信号由多个分量组成：

准确性奖励：对于数学和代码问题，直接验证最终答案是否正确（+1 或 −1）。这种奖励是**可验证的**——不需要人类判断，不需要奖励模型，只需要一个自动检查器（如数学答案比对或代码执行）。对于数学题，提取 `\boxed{}` 中的答案与标准答案比对。对于代码题，在沙箱中执行代码并检查测试用例是否通过。

格式奖励：鼓励模型将推理过程 放在 `<think>` 标签内，最终答案放在特定格式中。这个小小的格式约束至关重要——它引导模型将思考和回答分离，使得推理过程可以被隐藏或展示给用户。格式奖励通常是二值的：符合格式得 +1，不符合得 −1。

过程奖励（Process Reward）是一种更精细的奖励信号：不仅评价最终答案对不对，还评价中间的推理步骤是否合理。虽然 DeepSeek-R1 的核心训练没有使用过程奖励（依靠准确性和格式奖励就够了），但过程奖励在数学推理领域（如 Math-Shepherd、PRM800K）被广泛研究。它的理想形态是对每一步推理给出奖励，但标注成本极高。

8.4.6 DeepSeek-R1：涌现的推理能力

DeepSeek-R1[10] 的训练展示了一个令人兴奋的现象。

R1-Zero 实验：纯 RL 的探索

在正式构建 R1 的训练管线之前，DeepSeek 团队 首先进行了一个极具启发性的实验——**R1-Zero**：直接在 DeepSeek-V3-Base 上使用 GRPO 做纯 RL 训练，不做任何 SFT、不提

[10] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

供任何人类推理示例。奖励信号极其简单——只看最终答案是否正确。结果令人惊叹：模型**自发涌现**了以下能力：

- 链式思维推理（chain-of-thought）——模型学会了在给出答案前先进行详细的分步推理。
- 自我验证和纠错（“wait, let me re-check...”）——模型学会了在推理过程中暂停，回头检查之前的步骤是否正确。
- 所谓的 “aha moment”——模型在推理过程中突然意识到之前的方向是错的，转向新的思路。训练过程中可以观察到 这种行为的出现频率随着训练进行而增加。
- **自适应推理长度**——简单问题用较短的推理链，困难问题自动使用更长、更详细的推理。这种「按需分配思考量」的行为完全是自发涌现的。

这些能力没有被人类演示过——它们纯粹是 RL 优化过程中自发出现的。这强有力地支持了「RL 能发现超越人类演示的策略」这一观点。

但 R1-Zero 也有明显的缺陷：回答格式混乱（有时推理和答案混在一起），多语言混用（中文问题用英文推理），可读性差。这说明纯 RL 能涌现出**能力**，但不能涌现出**良好的格式和习惯**——后者仍然需要 SFT 来注入。这一实验为 R1 的正式训练管线设计提供了关键启示：**RL 能涌现能力，但需要 SFT 来规范格式**。

R1 的四阶段后训练管线

基于 R1-Zero 的经验，DeepSeek-R1 的正式训练 在 V3-Base 基座模型上执行了四个阶段的后训练，展示了 SFT 和 RL 如何精密配合：

1. **冷启动 SFT**：在 V3-Base 上用少量精心设计的推理示例做 SFT（数千条长 CoT 数据），教会模型使用 `<think>...</think>` 格式。这些数据由 R1-Zero 的输出经人工筛选和改写而来。冷启动的目的不是教模型**如何**推理（R1-Zero 已证明 RL 能涌现推理），而是给 RL 一个更好的起点——具备基本格式约束的模型在后续 RL 中能更快收敛。
2. **推理导向 RL**（GRPO）：在冷启动 SFT 后的模型上做 GRPO 训练，主要针对数学和代码等可验证任务。这一阶段的重点是提升推理质量，使用准确性奖励（答案对不对）和格式奖励（是否使用正确的标签结构）。
3. **拒绝采样 + SFT**：用第二阶段 RL 后的模型生成大量回答，只保留正确的、格式好的回答（约 800K 条推理 trace），同时混入通用的非推理数据（写作、翻译、问答等约 200K 条），在 V3-Base 上做一轮全面的 SFT。这一步的关键是**平衡推理能力与通用能力**——纯推理 RL 后的模型在非推理任务上可能退化。这些高质量推理数据也被用于蒸馏——训练 R1-Distill 系列（1.5B–70B）。
4. **全场景 RL**：最后一轮 GRPO 覆盖所有任务类型——推理任务使用规则奖励，通用任务使用奖励模型。这一阶段对齐有用性和安全性，是最终的偏好优化和安全对齐步骤。

R1 训练的核心洞见

DeepSeek-R1 的训练管线揭示了一个重要模式：**RL 负责发现能力，SFT 负责整理格式**。两者的顺序和配合至关重要——先做冷启动 SFT 建立格式约束，再做推理 RL 提升推理质量，然后用拒绝采样 + SFT 整合推理与通用能力，最后做全场景 RL 完成对齐。这种 SFT-RL-SFT-RL 的交替训练模式 成为了推理模型训练的标准范式。

8.5 Constitutional AI：让 AI 自我改进

Anthropic 提出的 Constitutional AI[11] 代表了安全对齐的另一种思路。传统 RLHF 依赖大量的人类标注——但人类标注昂贵、缓慢，而且标注者之间可能存在严重的不一致。

8.5.1 核心思想：RLAIF

Constitutional AI 的核心思想是：用 **AI 反馈**（AI Feedback, RLAIF）替代**人类反馈**（Human Feedback, RLHF）。

给模型一套**宪法原则**（如「回答应该无害且诚实」「不应帮助用户伤害他人」「应该承认不确定性而不是编造信息」），然后让模型根据这些原则自我批评和修正。

8.5.2 两阶段流程

Constitutional AI 分为两个阶段：

阶段一：监督学习阶段（SL-CAI）

1. **红队生成**：让模型生成可能有害的回答。通过精心设计的 adversarial prompt 诱导模型产生有毒、不安全或不诚实的输出。
2. **自我批评**：将模型的回答连同一条宪法原则 一起呈现给模型，要求它根据该原则审视自己的回答。例如：「根据原则' 回答应该真实且不编造事实'，评估以下回答是否存在问题。」
3. **自我修正**：让模型根据自己的批评重写回答，使其符合宪法原则。这一步可以迭代多次，每次使用不同的宪法原则进行批评和修正。
4. **SFT 训练**：用（原始问题, 最终修正后的回答）作为训练数据，对模型做 SFT。

阶段二：强化学习阶段（RL-CAI）

1. 用模型生成多个回答。
2. 让 **AI 评判者**（而非人类标注者）根据宪法原则对回答进行排序。
3. 用这些 AI 生成的偏好数据训练奖励模型。
4. 用标准的 RLHF 流程（PPO 或 DPO）优化策略。

[11] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

8.5.3 宪法的设计

宪法原则的设计是 Constitutional AI 中 最需要人类智慧的部分。典型的原则包括:

- 「选择最不可能被一个有道德的人反对的回答。」
- 「选择不鼓励非法或危险行为的回答。」
- 「选择诚实且不编造信息的回答。如果不确定，应该说' 我不确定'。」
- 「选择不带有种族、性别或其他偏见的回答。」
- 「选择最有帮助、最信息丰富的回答，同时确保无害。」

宪法原则的选择和优先级排序 直接决定了模型的「价值观」。不同的原则可能互相冲突——比如「最大限度地有帮助」和「避免任何潜在危害」在边界情况下会产生矛盾。Constitutional AI 通过明确的原则层次来处理这种冲突。

8.5.4 Constitutional AI 的意义

Constitutional AI 的一个深层意义在于它试图解决一个 RLHF 中固有的哲学问题:「**人类偏好**」**到底代表谁的偏好?** 不同的标注者有不同的价值观和知识水平，他们的偏好可能互相矛盾。

Constitutional AI 通过明确定义一组原则，将模糊的「人类偏好」转化为可操作的、可审计的规则。这使得对齐过程更加透明和可控——你可以检查宪法原则列表，理解模型为什么做出某个决定，并通过修改原则来调整模型的行为。

RLAIF 的实用价值也不可忽视。在 RLHF 中，收集几万条人类偏好比较需要数周时间和数万美元。在 RLAIF 中，同样数量的 AI 偏好比较可以在几小时内完成，成本仅为 API 调用费用。这使得偏好优化可以更频繁地迭代——每次更新宪法原则后，可以快速重新生成偏好数据并重新训练。

Anthropic 的 Claude 系列模型大量使用了这种方法。实验表明，RLAIF 在无害性评估上的表现 与 RLHF 相当甚至更好——AI 评判者比人类标注者更一致、更不容易疲劳，而且能更严格地执行宪法原则。

8.5.5 Claude 的新宪法（2026）

2026 年 1 月 22 日，Anthropic 发布了 Claude 的全新宪法——一份长达 23,000 词的文件，被内部称为「**灵魂文档**」(soul document)。相比 2023 年的初版宪法（约 2,700 词），这份新宪法的规模增长了近 10 倍，反映了 Anthropic 对 AI 对齐理解的深化。

新宪法的核心转变在于：**从规则列表变为价值推理**。早期的宪法是一系列「做什么/不做什么」的规则，模型通过模式匹配来遵守。新宪法则解释了**为什么**需要这些规则——它教模型理解规则背后的道德推理，使模型能够在新的、未见过的情境中 做出符合价值观的判断。

新宪法的几个关键设计原则:

- **善意原则**：默认假设用户的意图是好的，除非有明确的证据表明否则。这减少了过度拒绝——模型不会因为一个问题**可能**被滥用 就拒绝回答。
- **比例原则**：对风险的反应应与风险的严重程度成比例。帮用户写一封求职信（风险极低）应该毫不犹豫，而讨论危险化学品的合成方法（风险极高）则需要极其谨慎。
- **透明原则**：当模型选择不回答某个问题时，应该解释为什么，而不是简单地说「我不能这样做」。
- **自主性原则**：尊重用户的自主决策权。对于影响用户自身的决定（如饮食建议、生活方式选择），模型应该提供信息而不是做道德判断。
- **诚实原则**：承认不确定性，不编造信息，即使这意味着给出一个不那么「满意」的回答。

8.5.6 Collective Constitutional AI：民主化对齐

Anthropic 在 2023 年启动的 **Collective Constitutional AI** 实验代表了一个值得注意的方向：**用公众参与替代专家设计**来制定 AI 的宪法原则。

在这个实验中，Anthropic 与 Polis 平台合作，让约 1,000 名公众参与者 对一系列价值取向问题进行投票和讨论——比如「AI 应该在多大程度上拒绝争议性话题」「AI 应该在什么情况下承认自己是 AI」等。然后将这些公众共识转化为宪法原则，训练出一个「公众宪法」版本的 Claude。

实验结果表明，公众制定的宪法 在多个维度上与 Anthropic 内部专家制定的宪法表现相当，但在某些维度上更加**宽容**——公众倾向于允许模型讨论更多的争议性话题。这个实验的意义在于：它展示了 **AI 对齐不一定需要由少数专家决定**，公众参与可以产生高质量的对齐规范。

8.5.7 RLAIF 的数学框架

RLAIF 的训练流程可以形式化如下。

设 $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$ 是宪法原则集合。对于一个（prompt, response）对 (x, y)，AI 评判者基于原则 c_k 给出评分 $s_k(x, y) \in [0, 1]$。聚合后的 AI 偏好分数为：

$$s_{\text{AI}}(x, y) = \sum_{k=1}^{K} w_k \cdot s_k(x, y) \tag{8.20}$$

其中 w_k 是原则 c_k 的权重。

给定两个回答 y_1, y_2，AI 评判者的偏好概率为：

$$P_{\text{AI}}(y_1 \succ y_2 | x, \mathcal{C}) = \sigma\big(s_{\text{AI}}(x, y_1) - s_{\text{AI}}(x, y_2)\big) \tag{8.21}$$

这些 AI 生成的偏好然后被用于 训练奖励模型（RL-CAI）或直接用于 DPO——后者

更为简单高效，是 2025–2026 年 RLAIF 的主流实现方式。

RLAIF 的一个关键研究发现是：**AI 评判者的一致性远高于人类标注者**。对于同一个偏好比较，AI 评判者在重复评估时的一致性超过 95%，而人类标注者通常只有 70–75%。这种高一致性部分补偿了 AI 评判者可能存在的系统性偏差。

8.6 推理能力的后训练

2024–2025 年，LLM 领域最重要的范式转变之一 是**推理模型**（reasoning models）的崛起——以 OpenAI o1/o3 和 DeepSeek-R1 为代表。这些模型通过后训练获得了 在思维链（chain-of-thought）中进行长时间、深层次推理的能力，在数学、代码和科学推理任务上远超传统的「快思考」模型。

推理模型的后训练有其独特的方法论，值得单独深入讨论。

8.6.1 推理模型的训练范式

推理模型的核心理念是：**将计算从训练时转移到推理时**（从 train-time scaling 到 test-time scaling）。传统的 LLM 对每个问题只做一次前向传播就给出答案。推理模型则在回答之前生成长长的思维链，本质上是在用更多的推理时计算来换取更高的准确率。

目前存在两种主要的推理训练范式：

OpenAI 路线：蒸馏 + RL。OpenAI 的 o1 系统卡（2024 年 12 月）透露，o1 使用了「大规模强化学习」来训练推理能力。虽然具体细节未公开，但从已知信息可以推断：

1. 首先生成大量的推理 trace（可能由更强的教师模型或搜索算法产生），对模型做 SFT 建立思维链的格式。
2. 然后用 RL 进一步优化推理质量——奖励信号来自答案的正确性验证。
3. o1/o3 的一个独特特征是**隐藏思维链**——推理过程不展示给用户，只展示最终答案。这使得 OpenAI 可以在思维链中使用 更自由的推理策略（包括可能不那么「政治正确」的推理步骤），而不用担心用户看到。

DeepSeek 路线：冷启动 SFT + GRPO。DeepSeek-R1 的训练管线已在前文详细讨论。与 OpenAI 路线的关键区别是：R1 的思维链是**透明的**——用户可以看到完整的推理过程。而且 R1-Zero 实验证明，推理能力可以仅通过 RL 自发涌现，不一定需要人类提供的推理示例。

两种路线的共同点是：**RL 是推理能力训练的核心**，SFT 只提供格式和起点。

8.6.2 审议式对齐：OpenAI 的安全推理

OpenAI 在 2024 年 12 月提出的**审议式对齐**（deliberative alignment）是推理模型对齐的一个重要创新。

核心思想是：**让推理模型在思维链中显式推理安全规范**。传统的对齐方法（RLHF、DPO）通过训练将安全行为「烘焙」进模型的权重中——模型直觉性地拒绝有害请求，

但不「理解」为什么。审议式对齐则不同：模型在训练过程中被教会了安全规范的**文本**，在遇到潜在敏感的请求时，模型会在思维链中显式引用相关的安全政策，进行推理，然后决定如何回应。

这种方法的优势是：

- **可解释性**：通过检查思维链，可以看到模型是如何做出安全决策的——它引用了哪条政策，如何权衡利弊。
- **泛化能力**：对于训练中未见过的新型攻击，模型可以通过推理安全原则来正确应对，而不是依赖于模式匹配。
- **精确性**：减少了过度拒绝——模型可以推理出一个看似危险的请求 实际上是无害的（比如「如何制造炸弹蛋糕」是烹饪问题，不是武器问题）。

审议式对齐与 Constitutional AI 有相似之处——两者都使用显式的原则来指导模型行为。区别在于：Constitutional AI 在**训练时** 使用原则来生成偏好数据，审议式对齐在**推理时** 让模型在思维链中推理原则。推理模型使两者的结合成为可能。

8.6.3 过程奖励模型 vs 结果奖励模型

在推理模型的训练中，奖励信号的粒度是一个核心设计选择。

结果奖励模型（Outcome Reward Model, ORM）只评估最终答案的正确性。例如，对于数学题「$3x + 5 = 20$，求 x」，ORM 只关心最终答案是不是 $x = 5$——不关心中间的推理步骤是否合理。

ORM 的优势是简单——对于有标准答案的任务（数学、代码），自动验证即可。劣势是**信用分配问题**：如果模型给出了错误答案，ORM 无法告诉你哪一步推理出了问题。模型可能通过错误的推理碰巧得到正确答案（假阳性），或者推理步骤大部分正确但最后一步犯了算术错误（假阴性）。

过程奖励模型（Process Reward Model, PRM）对推理链中的**每一步**给出奖励信号。PRM 判断每个推理步骤是否合理——即使最终答案错误，正确的中间步骤也能获得正奖励。

OpenAI 的 PRM800K 数据集[12] 包含了约 800K 条人工标注的推理步骤评价（每步标注为「正确」「中性」或「错误」），是目前最大的过程奖励标注数据集。

PRM 的数学形式化：对于推理链 $s = (s_1, s_2, \ldots, s_N)$（$N$ 个步骤），PRM 为每步给出奖励：

$$r_{\text{PRM}}(x, s) = \big(r(s_1|x), r(s_2|x, s_1), \ldots, r(s_N|x, s_{<N})\big) \tag{8.22}$$

最终奖励可以取多种聚合方式：

- **最小值**：$R = \min_i r(s_i|\cdot)$ ——只要有一步出错，整体奖励就低。这是最保守的策略。
- **最后一步的正确概率**：$R = \prod_{i=1}^{N} P(s_i \text{ correct}|\cdot)$ ——所有步骤都正确的联合概率。
- **加权平均**：$R = \sum_i w_i \cdot r(s_i|\cdot)$，后面的步骤权重更高（因为错误更可能在后面发生）。

[12] H. Lightman et al., “Let’s verify step by step,” *arXiv preprint arXiv:2305.20050*, 2023.

PRM 的主要挑战是**标注成本**。人工标注每一步推理的正确性极其耗时——一道 AMC 竞赛题的 10 步推理链 可能需要一名数学研究生花 10–15 分钟来标注。近期的研究探索了**自动化 PRM 标注**的方法：

Math-Shepherd（2024）提出用蒙特卡洛方法 自动估计每步的过程奖励：对推理链的每个中间步骤 s_i，从该步骤开始多次采样续写至答案，该步骤的奖励等于「续写得到正确答案的比例」。如果从某一步开始，大多数续写都得到了正确答案，说明这一步是合理的。这种方法完全不需要人工标注，但需要大量的采样计算。

PRM vs ORM：如何选择?

维度	ORM	PRM
信号粒度	整个回答	每个推理步骤
标注成本	低（自动验证）	高（人工或蒙特卡洛）
信用分配	差	好
搜索引导	弱	强（可做 beam search）
适用场景	有标准答案的简单任务	长推理链的复杂任务
实际使用	GRPO + RLVR（主流）	推理时搜索（辅助）

在 2025–2026 年的实践中，**训练**阶段主要使用 ORM（通过 GRPO + RLVR），**推理**阶段使用 PRM 做搜索引导（如 best-of-N 采样时用 PRM 选最佳路径）。

8.6.4 推理蒸馏：让小模型也能推理

推理模型的一个重要应用模式是**蒸馏**——用大型推理模型的输出来训练小型模型。

DeepSeek-R1-Distill 系列证明了这种方法的有效性：用 R1（基于 V3 的 671B MoE 模型）生成的推理 trace 对 Qwen2.5 和 Llama 3.1 系列模型做 SFT，产生了从 1.5B 到 70B 不等的蒸馏模型。令人惊讶的是，R1-Distill-Qwen-32B 在 AIME 2024（数学竞赛）上的表现 超过了 OpenAI o1-mini——一个通过 SFT 蒸馏的 32B 模型 击败了一个通过大规模 RL 训练的（可能更大的）模型。

蒸馏的关键发现：

- 蒸馏后的模型获得了长思维链推理的能力，即使它从未经历过 RL 训练。模型通过模仿教师的推理 trace 学会了推理。
- 但蒸馏模型的推理能力有**天花板**——它无法超越教师模型的推理质量。要突破这个天花板，仍然需要在蒸馏后做 RL。
- 蒸馏 + RL 的组合效果优于单独使用任何一种。先蒸馏建立推理格式，再用 GRPO 提升推理质量。

8.7 安全对齐的深层技术

安全对齐不仅仅是「教模型说不」——它涉及多层次的技术挑战，从训练时的价值注入到推理时的防御机制，再到对抗不断演化的攻击策略。

8.7.1 红队测试：攻击驱动的防御

红队测试（red teaming）是安全对齐的核心方法论：通过模拟攻击来发现模型的弱点，然后针对性地修复。

红队测试的演进经历了几个阶段：

第一代：人工红队。训练有素的安全研究员手动编写 旨在诱导模型产生有害输出的 prompt。这种方法覆盖面有限但质量高——人类擅长发现模型的深层次弱点。OpenAI、Anthropic 和 Google 都维护着 专门的红队团队（通常 20–50 人）。

第二代：自动化红队。用 LLM 自动生成攻击 prompt（LLM-as-red-teamer）。Perez 等人（2022）首先提出用语言模型来红队测试语言模型。这大幅提高了测试的覆盖面——可以在数小时内生成数万条攻击 prompt，覆盖远比人工测试更多的攻击向量。

第三代：多轮对抗。2025–2026 年，安全评估的前沿转向了**多轮对抗**。攻击者不再尝试用一个 prompt 突破防线，而是通过多轮对话逐步诱导模型——先建立信任，然后逐渐将话题引向危险方向。MLCommons 的 AILuminate 基准（2025）和 MultiBreak 基准（ICLR 2026 在审）都包含了大量的多轮越狱攻击。

红队测试的结果直接用于后训练：

1. 将成功的攻击 prompt 转化为安全 SFT 数据（教模型正确拒绝这类请求）。
2. 用攻击结果训练专门的安全奖励模型。
3. 在偏好数据中增加安全相关的偏好对。

8.7.2 表示工程：激活层面的安全控制

表示工程（Representation Engineering, RepE）是一种相对较新的对齐方法，它在模型的**内部表示**（激活向量）层面进行干预，而不是通过改变权重。

核心思想是：LLM 的隐藏状态中包含了 与安全性相关的**方向**（directions）。通过识别这些方向，我们可以在推理时**操控**模型的行为——增加「安全方向」的激活、减少「危险方向」的激活。

具体方法：

1. **方向提取**：构造对比数据集（安全回答 vs 不安全回答），提取两组回答在隐藏状态空间中的差异方向。例如，对同一个敏感问题，收集安全拒绝回答和不安全遵从回答的隐藏状态，差异向量就是「安全方向」。
2. **激活偏移**：在推理时，将模型每一层的隐藏状态 沿安全方向偏移一个可控的量：$h' = h + \alpha \cdot v_{\text{safe}}$。$\alpha$ 控制偏移的强度——太小无效，太大会影响模型的正常功能。
3. **效果**：经过激活偏移的模型 在安全评估上显著提升，同时对正常任务的影响较小。

TRYLOCK 系统（2026 年 1 月）展示了 表示工程在实际防御中的应用：它将 DPO 安全对齐、RepE 激活控制、输入规范化和分类器检测 组合成一个**纵深防御**（defense-in-depth）架构。在 249 条攻击 prompt 上，TRYLOCK 达到了 88% 的相对攻击抵御率，同时对正常查询的误拒率仅 1.6%。

8.7.3 越狱攻击的演化

越狱（jailbreak）攻击——旨在绕过 LLM 安全防线的对抗性输入——经历了快速的演化。理解攻击的演化对于设计防御至关重要。

早期越狱（2023）：主要依赖角色扮演和提示注入。「DAN（Do Anything Now）」prompt 让模型扮演 一个没有安全约束的角色。这类攻击通过简单的安全 SFT 就能防御。

自动化越狱（2023–2024）：GCG（Greedy Coordinate Gradient）攻击 使用梯度优化自动生成对抗性后缀——在正常请求后附加一串看似无意义的 token，这些 token 经过优化可以绕过安全防线。防御方法包括困惑度过滤（对抗后缀的困惑度极高）和输入规范化。

编码越狱（2024–2025）：攻击者发现将有害请求编码为 Base64、ROT13、或其他编码格式可以绕过安全检测——因为安全训练主要使用自然语言文本，模型对编码后的文本缺乏安全判断能力。防御方法是在安全训练中加入编码/解码后的数据。

多轮越狱（2025–2026）：最新的攻击策略通过多轮对话逐步构建上下文。例如先让模型帮忙写一个「虚构小说」的场景，然后逐步将场景引向危险方向。每一轮单独看都是无害的，但组合起来达成了有害的目标。这是目前最难防御的攻击类型——需要模型理解对话的**整体意图**，而不是孤立地评估每条消息。

8.7.4 多语言安全挑战

安全对齐面临的一个系统性挑战是**多语言泛化**。

绝大多数的安全训练数据都是英文——红队测试、安全 SFT、偏好数据主要覆盖英文场景。这导致了一个严重的问题：**模型在英文中学到的安全行为 不能完全迁移到其他语言**。

Welo Data（2025）的多语言安全评估揭示了令人担忧的差距：在英文中被安全拒绝的有害请求，翻译成低资源语言后，模型的拒绝率可能下降 30–50%。甚至用中文、日文等高资源语言提问，安全性也低于英文。

这个问题的根源在于：

- **安全概念的编码是语言相关的**。模型在英文中学到了「如何制造毒品」→ 拒绝的关联，但这个关联不一定迁移到其他语言中的 等价表述。
- **安全训练的数据分布偏差**。如果 99% 的安全 SFT 数据是英文，模型在其他语言中的安全行为 主要依赖跨语言迁移——而跨语言迁移在安全领域不够可靠。
- **文化差异**。不同文化对「有害」的定义不同。在一种文化中被认为需要审查的内容 在另一种文化中可能完全正常。安全训练需要**文化感知**——这远比简单的多语言翻译复杂。

缓解多语言安全差距的策略包括：多语言红队测试（每种目标语言都做安全评估）、多语言安全 SFT 数据（不仅翻译英文数据，还需要本地化的安全场景）、以及语言无关的安全表示（在表示空间中学习跨语言的安全方向）。

8.8 SFT 数据的质量工程

随着后训练技术的成熟，业界逐渐认识到：**数据质量是后训练效果的最大瓶颈**——算法和计算资源的改进 远不如数据质量的提升来得重要。

8.8.1 前沿模型的 SFT 数据规模

不同的模型使用了差异巨大的 SFT 数据量：

- **InstructGPT**（2022）：约 13K 条人工编写的指令-回答对，加上约 33K 条偏好比较。
- **Llama 2-Chat**（2023）：约 27,540 条 SFT 标注数据。Meta 强调了数据质量而非数量——这些数据由内部标注者精心编写，经过多轮质量审核。
- **Llama 3-Instruct**（2024）：显著扩大了 SFT 数据规模——技术报告透露使用了约 1M+ 条 SFT 数据，包括人工数据、合成数据和拒绝采样数据。这标志着从「少量精品」到「大规模高质量」的转变。
- **DeepSeek-R1**（2025）：约 800K 条推理 trace + 约 200K 条通用数据 用于第三阶段的 SFT。推理数据来自 RL 后模型的拒绝采样，质量极高。
- **Claude 系列**（未公开具体数字）：据 Anthropic 的公开讨论，使用了 RLAIF 生成的大规模合成偏好数据，辅以人工编写的核心安全和能力数据。

总的趋势是：**SFT 数据量从 2022 年的约 10K 增长到 2025 年的约 1M+**，增长了两个数量级。但数据质量的门槛也在不断提高——不是简单地堆量，而是每一条数据都要精心策划。

8.8.2 LIMA vs 规模路线的辩论

LIMA（2023）的「1,000 条数据就够了」引发了持续的辩论。

LIMA 派认为：SFT 的核心作用是改变输出格式，而不是教授新能力。因此，少量的高质量数据（1K–10K）足以完成格式对齐，更多的数据带来的是边际递减的收益。

规模派认为：LIMA 的结论只在简单对话任务上成立。对于复杂任务（数学推理、代码生成、多步规划、工具使用等），模型需要大量的高质量示例 来学习如何将预训练中获得的知识 应用到新的行为模式中。Llama 3 和 GPT-4 级别的模型 使用的 SFT 数据量远超 LIMA 的建议。

2025 年，ICLR 接收的 Instruct-SkillMix 论文 提供了一个折中的视角：**关键不是数据的绝对数量，而是数据的多样性和技能覆盖度**。Instruct-SkillMix 自动从 LLM 中提取 指令遵循的核心「技能」（如因果推理、类比推理、约束满足等），然后生成**每条数据同时训练两种随机技能**的 指令-回答对。这种组合性确保了高效的技能覆盖——N 种技能只

需 $O(N)$ 条数据（而非 $O(N^2)$）就能覆盖所有技能对。结果表明，使用 Instruct-SkillMix 生成的 少量但高多样性的数据（~10K 条）可以匹配使用大规模低多样性数据（~100K+ 条）的效果。

8.8.3 SFT 数据的质量过滤

不论数据来源如何，质量过滤是确保 SFT 数据有效性的必要步骤。

自动过滤方法：

- **困惑度过滤**：丢弃困惑度极高或极低的回答。极高说明回答质量差或包含噪声，极低可能说明回答是直接复制粘贴的 或过于模板化。
- **LLM-as-Judge**：用强大的 LLM 对候选 SFT 数据进行质量评分。按有用性、准确性、安全性等多个维度评分，只保留总分超过阈值的数据。这是目前最常用的自动过滤方法。
- **一致性检查**：让模型对同一个指令生成多个回答，如果多个回答之间高度一致，说明这个指令-回答对的信号明确。如果回答差异很大，可能说明指令本身模糊或回答不确定性高。
- **去重**：移除语义重复的指令。SFT 数据中大量的近似重复 会浪费训练容量并导致模型在特定模式上过拟合。通常使用 MinHash 或嵌入向量聚类来检测近似重复。

人工审核仍然不可替代。即使自动过滤可以处理大部分数据，对核心能力领域（安全拒绝、事实准确性）的人工抽检仍然是必要的。前沿实验室通常维护一支 20–50 人的标注团队，对自动过滤后的数据进行最终审核。

SFT 数据的常见陷阱

- **模型崩塌**：如果 SFT 数据全部由同一个 LLM 生成（特别是同一个模型的自我蒸馏），多轮迭代后模型的多样性会逐渐降低——这被称为「模型崩塌」（model collapse）。缓解方法是混合多个来源的数据。
- **合成数据的系统偏差**：LLM 生成的 SFT 数据 继承了生成模型的偏见和风格。例如，GPT-4 生成的回答普遍偏向详细和结构化，用这些数据训练的模型也会过度冗长。
- **指令多样性不足**：如果 SFT 数据集中的指令主要是问答和总结，模型在创意写作、角色扮演、多步推理等 覆盖不足的领域表现会明显较差。

8.9 后训练的综合流水线

在实际的模型开发中，后训练通常不是单次运行，而是一个**迭代过程**。让我们看一个完整的后训练流水线，展示各个阶段如何衔接。

8.9.1 典型流水线

一轮典型的后训练迭代包括：

1. **SFT 基础训练**：在通用指令数据上微调基座模型，建立基本的对话能力。数据量：50K–500K 条对话。训练时间：1–3 天（在 64 块 GPU 上）。
2. **能力专项训练**：对特定能力（如代码、数学、多语言）做额外的 SFT 或 RL 训练。这一步通常使用领域特定的数据和评估指标。
3. **偏好优化**：使用 DPO 或 GRPO 优化模型输出的质量——包括有用性、准确性、详细程度等维度。DPO 需要 10K–100K 偏好对。GRPO 需要可验证的奖励函数。
4. **安全对齐**：确保模型拒绝有害请求，不泄露敏感信息，不生成违法内容。这一步可能使用 Constitutional AI 或 专门的安全 SFT 数据。
5. **评估与反馈**：在全面的基准测试和人类评估中检查质量，发现问题后返回步骤 1–4 迭代修正。
6. **长上下文适配**：在长文本场景下做额外的 SFT，确保模型在长上下文中也能正确遵循指令。
7. **工具使用训练**：教模型使用搜索引擎、代码执行器、API 调用等外部工具——这通常需要专门的 SFT 数据和 RL 训练。

8.9.2 流水线中的关键决策

阶段顺序至关重要。一个常见的错误是在 SFT 之后立刻做安全对齐，然后再做偏好优化。问题在于：偏好优化可能会部分「解除」安全约束——如果偏好数据中包含模型应该详细回答 但安全训练教它拒绝的边界情况，优化有用性可能会降低安全性。

最佳实践是将安全对齐放在**最后一步**，或者在每个阶段都混入安全数据（持续安全训练，而不是一次性对齐）。

数据配比是另一个关键决策。如果代码数据在 SFT 中占比过高，模型的通用对话能力可能退化。如果安全数据过多，模型可能变得「过度谨慎」——对无害的请求也给出拒绝回答。找到正确的平衡需要反复实验和评估。

整个后训练过程可能需要 2–6 周，消耗数百到数千块 GPU。虽然计算量远小于预训练（通常仅为预训练的 1–5%），但对模型的最终用户体验影响巨大——一个糟糕的后训练可以毁掉一个优秀的基座模型，而一个出色的后训练可以让一个中等的基座模型大放异彩。

后训练方法对比

方法	优势	局限
SFT	简单、稳定、易于调试	只能模仿，无法超越训练数据
RLHF (PPO)	能发现新策略，质量最高	工程复杂，大模型训练成本极高
DPO	简单如 SFT，效果接近 RLHF	离线方法，受数据分布限制
GRPO	可扩展到 700B+，涌现推理	需要可验证的奖励信号
Constitutional AI	减少人类标注，原则透明	AI 评判者可能遗漏微妙的偏见

CHAPTER 9

推理优化——让模型跑得更快

训练完成后，模型需要服务用户。在推理阶段，挑战从「如何训练更好的模型」转变为「如何让好模型跑得更快、更便宜」。

9.1 推理的根本瓶颈

在讨论具体的优化技术之前，我们需要先理解 LLM 推理的**根本瓶颈**——只有抓住瓶颈，才能理解为什么每种优化手段有效。

LLM 推理分为两个截然不同的阶段：

Prefill 阶段（首 token 生成前）：模型接收用户的完整输入（prompt），**并行**计算所有输入 token 的 KV 值。这个阶段是**计算瓶颈**的——因为有大量 token 需要同时处理，GPU 的计算单元（CUDA core / Tensor Core）被充分利用。Prefill 的耗时决定了 **TTFT**（Time To First Token，首 token 延迟）。

Decode 阶段（逐 token 生成）：模型每次只生成**一个** token，但每生成一个 token，都需要读取**整个模型的权重**。这个阶段是**内存带宽瓶颈**的——GPU 的计算能力远远没有被用满，大部分时间花在等待从 HBM（高带宽显存）读取权重。Decode 的速度决定了 **ITL**（Inter-Token Latency，token 间延迟）。

为什么 Decode 是内存带宽瓶颈?

以一个 70B 参数的模型（BF16 精度）为例。每生成一个 token，需要读取全部权重：$70 \times 10^9 \times 2\,\text{bytes} = 140\,\text{GB}$。但一个 token 的计算量只有约 70×10^9 次乘加运算（multiply-add）。

H100 GPU 的 HBM 带宽是 3.35 TB/s，读取 140 GB 需要 $140/3350 \approx 42\,\text{ms}$。H100 的 BF16 算力是 989 TFLOPS，70×10^9 次运算只需要 0.07 ms。

计算比内存读取快 600 倍——GPU 99.8% 的时间在等待数据从显存搬运到计算单元。这就是为什么 LLM decode 阶段是内存带宽瓶颈的。

理解了这个瓶颈，一个关键的杠杆就浮现出来：**batch size**。如果同时处理 B 个请求，每次读取权重后可以为 B 个 token 做计算。读取成本不变，但有效计算量增加了 B 倍。

存在一个**临界 batch size**：

$$B_{\text{critical}} = \frac{\text{计算能力 (FLOPS)}}{\text{内存带宽 (bytes/s)} \times \text{算术强度}} \tag{9.1}$$

当 $B < B_{\text{critical}}$ 时，推理受内存带宽限制；当 $B > B_{\text{critical}}$ 时，推理变为计算瓶颈。对于 H100，临界 batch size 大约在 150–300 之间（取决于模型架构和序列长度）。

在生产环境中，还需要在三个指标之间做权衡：

- **TTFT**（首 token 延迟）：用户等待第一个字出现的时间。对交互体验影响最大。取决于 prefill 速度。
- **ITL**（token 间延迟）：两个输出 token 之间的间隔。决定了「打字速度」感受。取决于 decode 速度。
- **Throughput**（吞吐量）：每秒处理的总 token 数。决定了服务成本。可以通过增大 batch size 提升，但会牺牲 TTFT 和 ITL。

这三个指标往往相互矛盾：增大 batch size 提升吞吐量，但增加了每个请求的延迟。推理框架的核心任务就是在这三者之间找到最优平衡。

9.2 量化：精度换速度

量化的核心思想是：模型推理时的大部分计算不需要训练时那么高的数值精度。将权重和/或激活从高精度（FP16/BF16）转换为低精度（INT8、INT4 甚至 FP4），可以显著减少内存占用和计算时间。

9.2.1 量化基础

量化的本质是将连续（或高精度离散）的数值 映射到有限数量的离散级别（discrete levels）。对称线性量化公式：

$$x_q = \text{round}\left(\frac{x}{s}\right), \quad s = \frac{\max(|x|)}{2^{b-1} - 1} \tag{9.2}$$

其中 s 是缩放因子，b 是目标位宽，x_q 是量化后的整数值。解量化时，$\hat{x} = x_q \cdot s$。

非对称量化增加了一个零点（zero-point）参数 z：

$$x_q = \text{round}\left(\frac{x - x_{\min}}{s}\right), \quad s = \frac{x_{\max} - x_{\min}}{2^b - 1}, \quad z = \text{round}\left(\frac{-x_{\min}}{s}\right) \tag{9.3}$$

非对称量化能更好地处理分布不对称的权重，但增加了计算开销（解量化需要额外的加法）。

常用的数值格式各有优劣：

- **INT8**：8-bit 整数。256 个级别，覆盖 $[-128, 127]$。精度损失几乎为零，是最安全的量化选择。
- **FP8（E4M3/E5M2）**：8-bit 浮点。E4M3 有更高的精度（3-bit 尾数），E5M2 有更大的动态范围（5-bit 指数）。H100 原生支持 FP8 运算。
- **INT4**：4-bit 整数。仅 16 个级别，需要精巧的量化策略来控制精度损失。

- **NF4**（Normal Float 4）：QLoRA[1] 提出的 4-bit 格式。16 个量化级别不是均匀分布的，而是按照标准正态分布的分位数分布——因为预训练后的权重近似正态分布，这种非均匀量化能最大化信息保留。
- **NVFP4**：NVIDIA Blackwell 架构原生支持的 4-bit 浮点格式。采用 1-bit 符号 + 2-bit 指数 + 1-bit 尾数的结构，相比 INT4 保留了更好的动态范围。B200 GPU 的 NVFP4 Tensor Core 算力 达到 FP8 的 2×，使得 4-bit 推理在数据中心场景中首次变得实用。

9.2.2 Post-Training Quantization（PTQ）

PTQ 在训练完成后直接对权重进行量化，无需重新训练。最简单的方法是 Round-to-Nearest（RTN）——直接将每个权重四舍五入到最近的量化级别。RTN 对于 INT8 通常效果不错，但在 INT4 及以下位宽时精度损失明显。原因是量化误差在层间**累积**——一层的量化误差会影响下一层的输入分布，导致误差逐层放大。

GPTQ[2] 使用二阶信息（Hessian 矩阵）来最小化量化误差。核心思想是：量化一个权重时，利用 Hessian 信息调整同一行中**尚未量化**的权重，来补偿量化引入的误差。GPTQ 逐列处理权重矩阵：量化第 i 列后，根据 Hessian 的逆矩阵 调整第 $i+1$ 到第 n 列的未量化权重。这使得整行的**总输出** 尽可能接近未量化时的输出。GPTQ 通常只需要一小批校准数据（128 条样本）和几分钟的计算时间，就能完成整个模型的量化。

具体而言，GPTQ 源自 OBQ（Optimal Brain Quantization）算法。对于权重矩阵 W 的每一行 w，目标是找到量化后的 $\hat{w}$ 使得输出误差最小：

$$\min_{\hat{w}} \|Wx - \hat{W}x\|^2 = \min_{\hat{w}} (w - \hat{w})^T H (w - \hat{w}) \tag{9.4}$$

其中 $H = XX^T$ 是校准数据上的 Hessian 矩阵。当量化第 i 列时，剩余列的最优调整量为：

$$\delta_F = -\frac{w_i - \text{quant}(w_i)}{H_{ii}^{-1}} \cdot (H^{-1})_{:,i} \tag{9.5}$$

这个公式的含义是：根据 Hessian 的逆（衡量每个权重的敏感度），将第 i 列的量化误差**分摊**到后续未量化的列上。敏感度低的权重承担更多补偿，敏感度高的权重保持更精确。

GPTQ 的关键创新在于发现**列的处理顺序** 对最终精度的影响远小于预期——这使得可以固定一个处理顺序（而不是贪心搜索最优顺序），将时间复杂度从 $O(d^3 \cdot n)$ 降低到 $O(d \cdot n^2)$，使得量化一个 175B 的模型只需要约 4 GPU-hours。

AWQ（Activation-Aware Weight Quantization）[3] 发现了一个关键洞察：**并非所有权重同等重要**。少数权重（约 1%）对模型输出的影响远超其他权重——这些权重对应的激活值特别大。AWQ 通过激活值的大小来识别这些「显著权重」（salient weights），对它

[1] T. Dettmers et al., "QLoRA: Efficient finetuning of quantized language models," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[2] E. Frantar et al., "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations*, 2023.

[3] J. Lin et al., "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *MLSys*, 2024.

们施加更大的缩放因子以保护精度，同时让不重要的权重承受更大的量化误差。

AWQ 的核心操作是引入一个 per-channel 的缩放因子 s：

$$Q(w \cdot s) \cdot \frac{x}{s} \approx w \cdot x \tag{9.6}$$

对于显著通道（激活值大的通道），$s > 1$，使得权重被放大后量化精度更高；对于不重要的通道，$s \leq 1$，允许更大的量化误差。缩放因子 s 通过在校准数据上搜索最优值来确定。

AWQ 在 INT4 量化下通常优于 GPTQ，因为它把有限的精度预算花在了刀刃上。在实践中，AWQ 搭配 Marlin 高性能 kernel 可以在 H200 上达到 741 tok/s（Qwen2.5-32B），超过 FP16 的 461 tok/s。

SmoothQuant[4] 解决的是**激活值量化**的难题。权重的分布通常比较平滑，容易量化；但激活值中存在大量**离群值**（outliers）——个别通道的值可能比其他通道大 100 倍。这些离群值让量化极为困难：为了覆盖最大值，缩放因子必须设得很大，导致其他正常值的量化精度严重降低。

SmoothQuant 的解法很优雅：在矩阵乘法 $Y = XW$ 中，引入一个对角矩阵 $\mathrm{diag}(s)$：

$$Y = XW = (X \cdot \mathrm{diag}(s)^{-1}) \cdot (\mathrm{diag}(s) \cdot W) = \hat{X}\hat{W} \tag{9.7}$$

选择合适的 s，可以将激活中的离群值「转移」到权重上——$\hat{X}$ 变得平滑容易量化，$\hat{W}$ 虽然变大了一些，但权重本身分布均匀，仍然可以量化。数学上这是等价变换，不改变计算结果。

9.2.3 Quantization-Aware Training（QAT）

QAT 在训练过程中模拟量化效果：前向传播时使用量化后的权重计算，反向传播时用直通估计器（Straight-Through Estimator, STE）近似 round 操作的梯度（round 函数本身梯度为零）。模型在训练中就学会了适应量化误差，因此 QAT 的精度通常优于 PTQ，但代价是需要额外的训练步骤（通常是原始训练的 5–10%）。

对于 INT8 和 FP8，PTQ 已经足够好，QAT 的收益不大；但对于 INT4 及以下位宽，QAT 的优势变得明显。

[4] G. Xiao et al., "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*, 2023.

9.2.4 量化的质量–大小权衡

量化位宽 vs. 精度损失经验法则

- **FP8 / INT8**：近乎无损。大多数基准测试上精度损失 < 0.5%。数据中心部署的标准选择。
- **INT4（好的量化方法）**：1–3% 的精度损失。对大多数实际应用可以接受。消费级 GPU 的最佳平衡点。
- **INT4（简单 RTN）**：3–8% 的精度损失。质量明显下降，不推荐。
- **NVFP4（Blackwell 原生）**：1–2% 精度损失，吞吐量达到 FP8 的 2×。数据中心新一代标准。
- **INT3 / INT2**：显著退化。目前技术下仅适用于极端资源受限场景。研究活跃但尚未成熟。

量化方法速查（2026 年初）

- **数据中心 GPU（H100/H200）**：FP8，近零精度损失
- **Blackwell GPU（B200）**：NVFP4，2.3× 吞吐提升
- **消费级 GPU + 高吞吐**：AWQ + Marlin kernel
- **消费级 GPU + 兼容性**：GPTQ + BitBLAS kernel
- **CPU / Apple Silicon**：GGUF Q4_K_M（llama.cpp / Ollama）
- **微调**：QLoRA（4-bit 量化权重 + LoRA 适配器）

一个反直觉的发现：**量化不仅节省内存，还能提升速度**。在 H200 上对 Qwen2.5-32B 的实测中，AWQ 4-bit + Marlin kernel 的吞吐量达到 741 tok/s，反而超过了 FP16 的 461 tok/s。这是因为 LLM 推理是**内存带宽瓶颈**的——瓶颈不是计算，而是从显存中读取权重。更小的权重意味着更少的内存读取，因此更快。

Table 9.1: INT4 量化方法在 H200 上的实测对比（Qwen2.5-32B-Instruct）

方法	Perplexity	HumanEval	吞吐 (tok/s)	特点
FP16 基线	6.28	85.4%	461	参考基准
GPTQ-INT4	6.42	82.9%	624	Hessian 优化，通用性好
AWQ-INT4	6.33	84.1%	741 (Marlin)	激活感知，质量最佳
GGUF Q4_K_M	6.38	83.5%	—	CPU/Apple 首选
BitsAndBytes	6.51	81.7%	398	简单易用，QLoRA 标配

9.2.5 量化方法的选择决策

面对多种量化方法，选择的核心考量是**目标硬件**和**精度要求**：

GPTQ vs AWQ：两者都是 PTQ 方法，都针对 INT4 优化。GPTQ 通过 Hessian 信息补偿量化误差，对所有权重一视同仁地优化；AWQ 则识别出关键权重给予优先保护。在大多数基准上 AWQ 精度略优于 GPTQ，且量化速度更快（AWQ 不需要计算完整的

Hessian 逆）。但 GPTQ 的生态更成熟，在更多推理框架中有优化的 kernel 支持。

GPU 量化 vs GGUF：GPTQ 和 AWQ 产生的模型需要 GPU 推理框架（vLLM、TensorRT-LLM）；GGUF 格式面向 llama.cpp 生态，支持 CPU、Apple Silicon Metal、CUDA 和 Vulkan 多种后端。如果目标是在 MacBook 上运行模型，GGUF 是唯一实用的选择。如果目标是数据中心高吞吐部署，AWQ + Marlin 是最优选。

9.3 GGUF K-Quant：层级量化的艺术

对于在消费级硬件上运行 LLM 的用户，llama.cpp 的 GGUF 格式是事实标准。其中最受欢迎的 Q4_K_M 格式使用了精巧的**层级量化**。

9.3.1 为什么需要分块量化?

最简单的量化方式是 **per-tensor 量化**——整个权重矩阵共享一个缩放因子。问题是：如果矩阵中有个别极大的值，缩放因子被迫设得很大，导致大多数小值被量化到零或极少的几个级别，精度严重损失。

Block quantization（分块量化）将权重矩阵切分为固定大小的块（block），每个块独立计算缩放因子和零点。块大小通常为 32–256 个权重。这样每个块内的值分布更加均匀，量化精度远好于全局量化。代价是额外存储每个块的元数据（缩放因子和零点），但这个开销通常只有 0.5–1 bit/weight。

9.3.2 K-Quant 的两级结构

GGUF 的 K-quant 采用两级量化结构：

1. 层的权重被切分为 **super-block**（如 256 个权重一组），每组配一个 FP16 缩放因子。
2. 每个 super-block 内部包含多个 **sub-block**（如 32 个权重一组），各自独立计算缩放因子并量化权重。
3. sub-block 的缩放因子本身被**二次量化**（6-bit），由 super-block 级别的 FP16 缩放因子覆盖。

这种层级结构让量化误差被控制在更小的局部范围内，避免了全局量化中「大值吃掉小值」的问题。二次量化是一个巧妙的设计：sub-block 的缩放因子本身也是数值，也可以被量化。用 6-bit 量化缩放因子几乎不损失精度，但节省了大量存储——如果每个缩放因子都用 FP16 存储，256 个权重中有 8 个 FP16 缩放因子（$8 \times 16/256 = 0.5$ bit/weight），而二次量化将这个开销降低到约 0.25 bit/weight。

9.3.3 K-Quant 变体

GGUF 提供了一系列 K-Quant 变体，命名规则是 `Q{bits}_K_{size}`：

- `Q2_K`：2-bit 量化。模型极小但质量较差。

- `Q3_K_S` / `Q3_K_M` / `Q3_K_L`：3-bit 量化的小/中/大变体。后缀 S/M/L 指的不是块大小，而是**混合精度策略的激进程度**——L 变体对更多敏感层使用更高位宽。
- `Q4_K_S` / `Q4_K_M`：4-bit 量化。Q4_K_M 是最流行的选择，在质量和大小间取得了最佳平衡。
- `Q5_K_S` / `Q5_K_M`：5-bit 量化。接近 FP16 质量。
- `Q6_K`：6-bit 量化。几乎无损。
- `Q8_0`：8-bit 量化。完全无损，但大小是 Q4_K_M 的两倍。

9.3.4 混合精度策略

Q4_K_M 中的 "M" 代表 Medium 精度变体，使用**混合精度策略**：对质量影响最大的层（attention 的 Q/K/V 和 FFN 的 down_proj）使用 Q6_K（6-bit），其余层使用 Q4_K（4-bit）。这仅多出 ~0.2 bits/weight 的存储成本，但将 perplexity 增量从 0.1149 降低到 0.0535——质量提升约 2×。

这个设计背后的洞察是：**不同层对量化误差的敏感度差异巨大**。注意力层的 Q/K/V 直接影响「模型关注什么」，量化误差会在 softmax 中被放大；而 FFN 的 up_proj 和 gate_proj 对微小的数值误差 有更强的容忍度。通过对敏感层分配更多比特，可以用极小的额外成本获得显著的质量提升。

此外，模型的**第一层和最后几层**通常也更敏感：第一层负责将 token embedding 投影到模型的内部表示空间，最后几层负责将内部表示转换为 token 概率分布。这两端的量化误差会直接影响输入理解和输出质量，没有后续层来「纠正」误差。

9.3.5 llama.cpp 与 GGUF 格式

llama.cpp 之所以成为本地推理的事实标准，有几个关键原因：

- **纯 C/C++ 实现**，无 Python 依赖，可以在任何平台（Linux、macOS、Windows、甚至手机）上编译运行。
- **GGUF 文件格式**是自包含的——一个文件包含模型权重、分词器、超参数和元数据。不需要额外的配置文件或依赖。
- **CPU 推理优化**：利用 AVX2/AVX-512（x86）和 NEON（ARM）指令集加速量化矩阵乘法。
- **GPU offloading**：可以将部分层放在 GPU 上，实现 CPU–GPU 混合推理——即使 GPU 显存不够放下整个模型，也能利用 GPU 加速最耗时的层。
- **Apple Silicon 原生支持**：通过 Metal API 利用 Apple M 系列芯片的 GPU 和 ANE（Neural Engine），在 MacBook 上实现高效推理。

Ollama 在 llama.cpp 之上提供了更友好的用户体验——一条 `ollama run llama3` 命令就能下载和运行模型，对非技术用户极为便利。但 Ollama 在高并发场景下的性能远不及专业推理框架——在 128 并发请求的压力测试中，Ollama 的吞吐量仅为 484 tok/s，约为 vLLM 或 SGLang 的 1/10。

9.4 Flash Attention 与内存高效注意力

标准的注意力计算需要将完整的 $n \times n$ 注意力矩阵 写入 GPU 的高带宽显存（HBM），内存复杂度为 $O(n^2)$。当序列长度 n 达到数万甚至数十万时，注意力矩阵本身就可能耗尽所有显存。

Flash Attention[5] 通过**分块计算**（tiling）和**重计算**（recomputation）策略，避免将完整的注意力矩阵写入 HBM，将注意力计算的内存复杂度从 $O(n^2)$ 降低到 $O(n)$。核心思想是：将 Q、K、V 矩阵分成小块，每次只在 GPU 的片上 SRAM（速度比 HBM 快约 10×）中 计算一个子块的注意力，逐块累积结果，始终不将完整的 $n \times n$ 矩阵写回 HBM。

Flash Attention 2[6] 进一步优化了 GPU 线程块的工作分配，减少了非矩阵乘法运算（如 softmax 的归一化），在 A100 上实现了接近理论峰值 72% 的计算利用率。Flash Attention 3[7] 针对 H100 的异步执行和 FP8 支持做了进一步优化。

这一技术同时加速了训练和推理，是现代 LLM 部署的标配优化——几乎所有主流推理框架（vLLM、SGLang、TensorRT-LLM）和训练框架（Megatron-LM、DeepSpeed）都已集成 Flash Attention。

9.5 KV Cache：空间换时间

9.5.1 为什么需要 KV Cache?

LLM 的文本生成是自回归的——每次生成一个 token。生成第 t 个 token 时，注意力机制需要计算它与前面所有 $t-1$ 个 token 的关系。如果每次都重新计算所有位置的 K 和 V，生成一个长度为 T 的序列需要 $O(T^2)$ 的注意力计算——因为第 t 步需要 $O(t)$ 的计算。

KV cache 将之前位置计算过的 K 和 V 向量缓存起来，每次只需计算新 token 的 Q，与缓存的 K, V 做注意力运算。这将每步的新增计算量从 $O(t)$ 降到 $O(1)$（不含缓存读取），总复杂度从 $O(T^2)$ 降到 $O(T)$。

9.5.2 KV Cache 的内存开销

KV cache 的大小计算：

$$\text{KV cache size} = 2 \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{batch_size} \times \text{bytes} \tag{9.8}$$

以 LLaMA 3-70B 为例（80 层、8 个 KV 头、128 维头维度），在 BF16 精度下，**单个**

[5] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[6] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[7] J. Shah et al., "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," *arXiv preprint arXiv:2407.08608*, 2024.

请求处理 4096 个 token 的 KV cache：

$$2 \times 80 \times 8 \times 128 \times 4096 \times 2\,\text{bytes} = 1.34\,\text{GB} \tag{9.9}$$

如果上下文长度增加到 128K：$2 \times 80 \times 8 \times 128 \times 131072 \times 2 \approx 42\,\text{GB}$——单个请求的 KV cache 就超过了一张 A100 40GB 的全部显存。

当并发服务数百个请求时，KV cache 可能占用大部分 GPU 显存。这使得 KV cache 的管理成为推理系统的核心挑战之一。

9.5.3 PagedAttention

vLLM[8] 引入了 **PagedAttention**——借鉴操作系统的虚拟内存分页机制，将 KV cache 分成固定大小的「页」（page），按需分配和回收。

传统的 KV cache 管理为每个请求预分配一块**连续**内存，大小等于最大可能的序列长度。问题在于：

- **内部碎片**：大多数请求不会用满预分配的空间。如果预分配 2048 个 token 的空间，但请求只生成了 200 个 token，90% 的内存被浪费了。
- **外部碎片**：请求的生命周期不同，内存中出现大量不连续的空闲区域，无法分配给新请求。

PagedAttention 的解决方案与操作系统的虚拟内存如出一辙：

1. 将 GPU 显存分成固定大小的**物理页**（block），每页存储固定数量 token 的 KV 值（如 16 个 token）。
2. 每个请求维护一个**页表**（page table），将逻辑位置映射到物理页。
3. KV cache **不需要连续存储**——同一个请求的 KV 可以分散在不同的物理页中。
4. 按需分配：请求生成新 token 时，才分配新的物理页。

这种设计将内存浪费从 60–80% 降低到接近 0%，使得 vLLM 可以在相同的 GPU 上服务 2–4× 更多的并发请求。

9.5.4 KV Cache 压缩

除了更好的内存管理，还可以从**减少 KV cache 本身的大小**入手：

GQA（Grouped-Query Attention）[9] 在模型架构层面减小 KV cache。标准 MHA 中每个 query head 有独立的 K/V head；GQA 让多个 query head **共享**同一组 K/V head。LLaMA 3-70B 使用 64 个 query head 和 8 个 KV head（8:1 共享），KV cache 直接缩小到 MHA 的 1/8。

[8] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[9] J. Ainslie et al., "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

MLA（Multi-head Latent Attention）是 DeepSeek-V2/V3[10] 提出的更激进的方案。MLA 将 K 和 V 联合压缩到一个低秩表示 c_t：

$$c_t = W^{DKV}[k_t; v_t], \quad [k_t; v_t] = W^{UKV} c_t \tag{9.10}$$

其中 c_t 的维度远小于原始 K/V 的拼接。只需要缓存 c_t 而不是完整的 K 和 V，KV cache 大小可以压缩到原来的 1/10 甚至更小。

KV cache 量化是另一条路径：将 KV cache 从 BF16 量化到 INT8 甚至 INT4。最新的研究表明，KV cache 对量化的耐受度 甚至高于权重——INT4 KV cache 的精度损失通常可以忽略不计。这是因为 KV cache 中的数值分布比权重更集中、更平滑。在 vLLM 中，开启 FP8 KV cache 量化只需一个配置参数，即可将 KV cache 内存减半，几乎不影响输出质量。

KV cache 驱逐（eviction）策略 在 KV cache 超出显存容量时发挥作用。核心思想是：并非所有历史 token 的 KV 都同等重要。通过注意力分数来评估每个 token 的「重要性」，驱逐最不重要的 token 的 KV cache。典型的策略包括：

- **Window attention**：只保留最近 W 个 token 的 KV，丢弃更早的 token。简单但可能丢失关键的早期信息。
- **H2O**（Heavy-Hitter Oracle）：保留累积注意力分数最高的 token（“heavy hitters”）加上最近的窗口。这些 heavy hitters 只占少数 token 但集中了大部分的注意力权重。
- **EvicPress**[11]：联合优化压缩和驱逐决策，在多层存储（GPU HBM、CPU DRAM、SSD）之间 动态调度 KV cache，最小化平均生成延迟。

跨层 KV cache 共享（Cross-layer KV sharing）是一个新兴的研究方向。观察表明，相邻 transformer 层的 KV 表示 具有很高的余弦相似度——这意味着可以让多层共享同一组 KV cache，而不是每层独立存储。CommonKV[12] 利用 SVD 分解 实现相邻层参数共享，在不需要重新训练的情况下将 KV cache 压缩 2–4×。这种方法与量化和驱逐策略**正交**——可以叠加使用，实现更极致的压缩。

9.5.5 Prefix Caching

在生产环境中，许多请求共享相同的**系统 prompt**（system prompt）——例如「你是一个有帮助的 AI 助手……」。如果每个请求都重新计算系统 prompt 的 KV cache，是巨大的浪费。

Prefix caching 将共享前缀的 KV cache 缓存起来复用：第一个请求计算完系统 prompt 的 KV cache 后，后续所有使用相同系统 prompt 的请求直接复用，只需要计算用户特定输入部分的 KV cache。

[10] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.

[11] S. Feng et al., “EvicPress: Joint KV-cache compression and eviction for efficient LLM serving,” *arXiv preprint arXiv:2512.14946*, 2024.

[12] Y. Wang et al., “CommonKV: Compressing KV cache with cross-layer parameter sharing,” *arXiv preprint arXiv:2508.16134*, 2025.

对于典型的 API 服务（系统 prompt 占输入的 50–80%），prefix caching 可以将 prefill 计算量减少 50–80%，TTFT 显著降低。vLLM 支持自动 prefix caching，SGLang 则通过 RadixAttention 实现了更灵活的**树形**前缀复用——不仅支持线性前缀，还能处理分支对话树结构。

Prefix caching 为什么对多轮对话如此重要?

在多轮对话中，每一轮都需要将完整的对话历史作为输入。第 N 轮对话的输入包含前 $N-1$ 轮的所有内容。不使用 prefix caching 时，每一轮都需要重新计算 所有历史内容的 KV cache——第 10 轮对话的 prefill 成本是第 1 轮的 10 倍。

使用 prefix caching 后，每一轮只需计算**新增**内容的 KV，之前所有轮次的 KV cache 直接复用。在 SGLang 的基准测试中，多轮对话场景下 prefix caching 将 TTFT 降低了 5–10×。

9.6 推测解码：小模型辅助大模型

9.6.1 核心洞察

推测解码[13] 利用了一个观察：大部分 token 是「容易」的——大模型和小模型对它们的预测一致。只有少数「难」的 token 需要大模型的判断力。

Decode 阶段的瓶颈不是计算而是内存带宽——不管生成一个 token 还是同时验证 k 个 token，读取模型权重的成本几乎相同（因为 k 通常很小，额外的计算量可以忽略）。推测解码正是利用了这个特性。

9.6.2 算法流程

流程:

1. 用一个小的草稿模型（draft model，如 1B 参数）快速自回归生成 k 个候选 token。由于模型很小，这一步很快。
2. 将这 k 个 token 一次性送入大模型做**并行验证**。大模型在一次前向传播中同时计算所有 k 个位置的概率分布。
3. 从前到后逐个检查：如果草稿模型的 token 被接受，继续检查下一个；如果某个位置被拒绝，从大模型在该位置的分布中重新采样，丢弃后续所有草稿 token。

关键在于第 2 步：验证 k 个 token 只需要一次前向传播（因为可以并行计算），而不是 k 次。如果草稿模型的接受率是 80%，每次验证 5 个 token，平均每次前向传播产生 4 个有效 token——接近 4× 的加速。

[13] Y. Leviathan et al., "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*, 2023.

9.6.3 数学保证

推测解码最重要的性质是：它是**数学上无损的**——最终输出的概率分布与直接使用大模型**完全等价**。

接受/拒绝机制使用了一种修正的**拒绝采样**（rejection sampling）。设草稿模型在位置 i 的分布为 $q(x)$，目标大模型的分布为 $p(x)$。对于草稿 token x_i：

接受概率：

$$P(\text{accept } x_i) = \min\left(1, \frac{p(x_i)}{q(x_i)}\right) \tag{9.11}$$

拒绝后的重采样分布：当 x_i 被拒绝时，从修正分布中重新采样：

$$p'(x) = \frac{\max(0,\ p(x) - q(x))}{\sum_{x'} \max(0,\ p(x') - q(x'))} \tag{9.12}$$

为什么这个过程保证输出分布不变?

对于任意 token x，被最终选中的总概率等于：

$$P(\text{select } x) = q(x) \cdot \min\left(1, \frac{p(x)}{q(x)}\right) + P(\text{reject}) \cdot p'(x) \tag{9.13}$$

$$= \min(q(x), p(x)) + \left(1 - \sum_{x'} \min(q(x'), p(x'))\right) \cdot \frac{\max(0, p(x) - q(x))}{Z} \tag{9.14}$$

其中 $Z = \sum_{x'} \max(0, p(x') - q(x')) = 1 - \sum_{x'} \min(q(x'), p(x'))$。代入化简后恰好等于 $p(x)$。

这个证明的关键在于：$\sum \min(q, p) + \sum \max(0, p - q) = \sum p = 1$——被接受的概率质量和被拒绝后重采样的概率质量 加起来恰好等于目标分布 p。

这意味着推测解码是**免费的加速**——不牺牲任何输出质量。

9.6.4 加速比分析

推测解码的加速比取决于两个关键因素：**接受率** α 和**草稿长度** k。

设每次草稿生成 k 个 token，每个 token 的接受率为 α。每次验证循环中，期望被接受的 token 数为：

$$\mathbb{E}[\text{accepted tokens}] = \sum_{i=1}^{k} \alpha^i = \frac{\alpha(1 - \alpha^k)}{1 - \alpha} \tag{9.15}$$

加上拒绝后重采样的 1 个 token，每次循环产生的有效 token 数约为 $\frac{1-\alpha^{k+1}}{1-\alpha}$。

设草稿模型的推理时间为 t_d，大模型的推理时间为 t_v，加速比为：

$$\text{Speedup} = \frac{\frac{1-\alpha^{k+1}}{1-\alpha}}{k \cdot t_d + t_v} \cdot t_v \tag{9.16}$$

当 $t_d \ll t_v$（草稿模型远小于目标模型）且 α 较高时，加速比接近 $\frac{1}{1-\alpha}$。例如 $\alpha = 0.8$ 时理论加速比约 5×，实际中由于草稿模型开销和内存管理，通常实现 2–3× 的端到端加速。

9.6.5 草稿模型选择与进化

草稿模型的选择直接影响接受率和加速比：

同族小模型：最常用。例如 Llama 3 8B 为 Llama 3 70B 做草稿。词表和训练分布相似，接受率通常 70–85%。NVIDIA 在 H200 上展示了 用同族小模型实现 3.6× 吞吐量提升的结果。

EAGLE 系列[14]：将目标模型最后一层的隐状态 输入一个轻量级的自回归头来生成草稿。EAGLE 考虑了 token 间的依赖关系，生成的草稿通常比独立预测头更准确。

EAGLE-2 引入了**动态草稿长度**——根据上下文的可预测性自适应调整 每次验证的候选 token 数量。在「容易预测」的上下文中生成更多候选（如 8–10 个），在「难预测」的上下文中减少候选（如 2–3 个），避免浪费验证计算。

EAGLE-3[15] 进一步突破了性能瓶颈。它放弃了 EAGLE 的特征预测范式，改为直接预测 token，并通过**多层特征融合**（training-time test 技术）从目标模型的多个中间层提取信息。这使得草稿模型能充分受益于训练数据规模的增加——EAGLE 和 EAGLE-2 在增加训练数据时提升有限，EAGLE-3 则表现出持续的改善。在推理模型（如 DeepSeek-R1、QwQ）上，EAGLE-3 实现了 2–4× 的加速。

Medusa[16]：在目标模型的最后一层之上 添加多个轻量级**预测头**（prediction head），每个头预测不同位置的下一个 token。多个头并行工作，类似于同时生成多个候选。优点是不需要单独的草稿模型，但需要额外训练预测头，且由于各头独立预测，无法捕捉 token 间的依赖。

Self-speculative decoding（自推测解码）：使用目标模型自身的**前几层**作为草稿模型。不需要额外加载一个小模型，节省显存。

LayerSkip[17] 是这一方向的代表。通过在训练中使用特殊的 dropout 策略（早期层的 dropout 率低，后期层的 dropout 率高），使得模型在只使用前几层时就能产生合理的预

[14] Y. Li et al., "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *International Conference on Machine Learning*, 2024.

[15] Y. Li et al., "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," *arXiv preprint arXiv:2503.01840*, 2025.

[16] T. Cai et al., "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *International Conference on Machine Learning*, 2024.

[17] M. Elhoushi et al., "LayerSkip: Enabling early exit inference and self-speculative decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

测。推理时，使用前 $L/3$ 层作为草稿，完整的 L 层作为验证。由于草稿和验证共享同一模型的权重，无需额外的模型内存，在内存受限场景下特别有吸引力。

CLaSp[18]（In-Context Layer Skip）进一步改进了自推测解码，不需要专门的训练——通过分析上下文来动态决定跳过哪些层，使得任何现有模型都可以直接使用自推测解码。

HeteroSpec 代表了推测解码的最新趋势——**自适应策略**。通过输出分布的**熵**来判断当前上下文的难度：低熵（高确定性）上下文使用更激进的推测（更多候选 token），高熵（高不确定性）上下文使用保守的推测（更少候选）。使用决策树将推测路径分为「快车道」和「慢车道」，实现了 4.24× 的加速，超过 EAGLE-3。

推测解码方法速查（2026 年初）

- **同族小模型**：最成熟，生态最好。vLLM 和 TensorRT-LLM 原生支持。2–3× 加速。
- **EAGLE-3**：质量最高的轻量级草稿。需要训练草稿头。2–4× 加速。
- **Medusa**：无需独立草稿模型。多头并行预测。1.5–2.5× 加速。
- **LayerSkip / CLaSp**：自推测，零额外内存。适合内存受限场景。1.5–2× 加速。
- **HeteroSpec**：自适应策略，当前最快。需要熵估计开销。3–4× 加速。

实际加速通常在 2–3× 之间。加速比取决于接受率——接受率越高，每次验证产生的有效 token 越多。对于代码生成等「可预测性高」的任务，加速比更大；对于创意写作等「随机性高」的任务，加速比较小。在推理模型（reasoning models）的 thinking token 生成中，推测解码的加速效果尤为显著——因为推理过程中的许多步骤是结构化的、可预测的。

9.7 Continuous Batching：提升服务吞吐

9.7.1 Static Batching 的问题

传统的推理服务（static batching）将一批请求一起处理，等所有请求都生成完毕后才开始下一批。问题是：不同请求的生成长度差异很大——一个请求生成 10 个 token 就结束了，另一个可能需要 2000 个 token。短请求必须等待长请求完成，GPU 利用率很低。

假设一个 batch 有 8 个请求，最短的生成 50 个 token，最长的生成 2000 个 token。7 个短请求在前 50–200 步就完成了，但它们的「位置」一直被占据到第 2000 步。GPU 在 batch 后期有 80–90% 的计算能力处于空闲状态。

[18] L. Chen et al., "CLaSp: In-context layer skip for self-speculative decoding," *arXiv preprint arXiv:2505.24196*, 2025.

9.7.2 Iteration-Level Scheduling

Continuous batching（又称 iteration-level batching）解决了这个问题。核心思想是：在**每一步**（每生成一个 token）都重新审视当前 batch 的组成——

1. 检查是否有请求刚刚生成了 EOS token（结束标记）。如果有，将它从 batch 中移除。
2. 检查等待队列中是否有新请求。如果有空位，立刻将新请求加入 batch。
3. 对当前 batch 中的所有请求执行一步 decode。

GPU 永远在满负荷工作，不存在「等待最慢请求」的空闲时间。在高并发场景下，continuous batching 可以将吞吐量提升 2–5×。

9.7.3 Chunked Prefill

Continuous batching 还有一个微妙的问题：当一个新请求加入 batch 时，它需要先经过 prefill 阶段（处理其完整的 prompt）。如果 prompt 很长（比如 32K tokens），这次 prefill 会阻塞整个 batch 的 decode 步骤，导致其他正在 decode 的请求出现**卡顿**（ITL 突增）。

Chunked prefill 解决了这个问题：将长 prompt 切成固定大小的块（如 512 tokens），每步只处理一个块。prefill 分散在多步中完成，不会阻塞其他请求的 decode。vLLM 和 SGLang 都支持 chunked prefill。

9.7.4 Prefix-Aware Batching

在支持 prefix caching 的系统中，调度器可以进一步优化：将共享相同前缀的请求**分组在同一个 batch** 中。这样可以最大化 prefix cache 的命中率，减少重复的 KV cache 计算。

SGLang 的 RadixAttention 维护了一棵**基数树**（radix tree）来管理所有已缓存的前缀。当新请求到来时，调度器在基数树中查找最长匹配前缀，只需要计算未匹配的部分。对于多轮对话场景（每轮都共享之前的对话历史），这种优化效果尤为显著。

9.8 推理服务架构

9.8.1 推理框架对比

Table 9.2: 主流推理框架对比（2026 年初）

框架	核心特性
vLLM	PagedAttention, Continuous Batching, Chunked Prefill, Prefix Caching, Speculative Decoding, 支持 disaggregated serving。Python-first，生态最丰富，社区最活跃（1969+ 贡献者）。适合数据中心通用部署。H100 吞吐约 12,500 tok/s（Llama 3.1 8B）。
SGLang	RadixAttention（树形 prefix caching），Chunked Pipeline Parallelism（1M token TTFT 降低 81%），结构化生成（grammar-constrained decoding）。H100 吞吐约 16,200 tok/s——比 vLLM 快 29%。多轮对话场景再快 10–20%。适合复杂 prompt 工程和 DeepSeek 模型部署。
TensorRT-LLM	NVIDIA 原生优化, FP8/NVFP4 支持, In-flight Batching, 最高单卡性能。C++ 核心，配置复杂但性能极致。适合 NVIDIA GPU 上的高性能生产部署。
LMDeploy	TurboMind C++ 引擎，H100 吞吐约 16,100 tok/s，与 SGLang 持平。量化模型部署性能最优。适合需要极致量化推理的场景。
llama.cpp	CPU/混合推理, GGUF 量化, Metal/CUDA/Vulkan 加速。纯 C/C++，无依赖，单文件模型格式。消费级硬件和边缘设备首选。
MLC-LLM	跨平台部署（iOS、Android、浏览器）。WebGPU 支持——可以在浏览器中运行 LLM。适合移动端和嵌入式场景。

选择推理框架时需要考虑几个维度：

- **目标硬件**：NVIDIA GPU → vLLM / SGLang / TensorRT-LLM；Apple Silicon → llama.cpp；移动端 → MLC-LLM。
- **性能 vs. 易用性**：TensorRT-LLM 性能最高但配置复杂；vLLM 平衡了两者；SGLang 在多轮对话和 DeepSeek 上最优；llama.cpp 和 Ollama 最简单。
- **特殊需求**：结构化输出 → SGLang；浏览器部署 → MLC-LLM；多模态 → vLLM。
- **成本**：SGLang 比 vLLM 吞吐高 29%，百万日请求场景下月省约 $15,000 GPU 成本。

9.8.2 Disaggregated Serving：分离 Prefill 和 Decode

传统的推理框架将 prefill 和 decode 放在同一组 GPU 上执行。但这两个阶段的计算特性截然不同：

- **Prefill** 是计算密集型——大量 token 并行计算，GPU 算力被充分利用。
- **Decode** 是内存带宽密集型——逐 token 生成，GPU 大部分时间在等待内存读取。

将两者混合在同一组 GPU 上会互相干扰：长 prompt 的 prefill 会阻塞其他请求的 decode（ITL 突增），大量 decode 请求又会延迟新 prompt 的 prefill（TTFT 增加）。

Disaggregated serving（分离式推理）将 prefill 和 decode 分配到**不同的 GPU 节点**：

- **Prefill 节点**：处理输入 prompt，生成 KV cache，然后将 KV cache 传输到 decode 节点。适合使用计算能力强的 GPU。
- **Decode 节点**：接收 KV cache，逐 token 生成输出。适合使用内存带宽大的 GPU。

DeepSeek 的分离式推理架构

DeepSeek 在其 V3/R1 推理系统中 采用了大规模的分离式架构。根据 2025 年 2 月 Open Source Week 披露的信息：

- Prefill 节点和 decode 节点的比例约为 1:4——decode 阶段占用了绝大部分的 GPU 资源。
- 跨节点的 KV cache 传输通过高速互联（NVLink/InfiniBand）完成，传输延迟被控制在毫秒级。
- 得益于 MLA 的 KV cache 压缩，需要传输的数据量远小于传统 MHA 架构，使得分离式架构的通信开销可控。
- 每个 H800 节点的吞吐量 达到甚至超过同等活跃参数量（~37B）的稠密模型——这证明了 MoE + MLA + 分离式架构的组合效果。

Mooncake[19] 进一步提出了 **KVCache-Centric**（以 KV cache 为中心）的分离式架构。其核心创新是将 KV cache 视为独立的资源进行管理，而不是绑定到特定的 GPU。KV cache 可以存储在 GPU HBM、CPU DRAM 或 SSD 上，调度器根据实时负载动态决定 KV cache 的存放位置。Mooncake 还引入了**预测性早期拒绝**（Early Rejection）——在请求到达时就预估系统是否能满足 SLO，如果无法满足则提前拒绝，避免浪费计算资源。

vLLM v0.11 也引入了 disaggregated serving 支持，配合 Wide Expert Parallel（Wide-EP）实现了 DeepSeek-R1 在 H200 集群上 2,200 tok/s/GPU 的吞吐。

9.8.3 Dual-Batch 与异步调度

Dual-batch（双批次重叠）是 vLLM 和 SGLang 中的一项 前沿优化技术。核心思想是将 prefill 和 decode 的计算 在时间维度上**重叠**——在 GPU 执行一批 decode 的同时，异步准备下一批 prefill 的数据。这消除了 prefill 和 decode 之间的空闲间隙，进一步提升了 GPU 利用率。

异步调度（async scheduling）则将调度决策从 GPU 计算路径中移出。传统调度器在每步计算前同步决定 batch 组成，引入了几毫秒的调度延迟。异步调度器在 CPU 上持续运行，在 GPU 计算当前步的同时准备好下一步的 batch，使得调度开销几乎为零。

[19] R. Qin et al., "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *arXiv preprint arXiv:2407.00079*, 2024.

9.9 模型蒸馏：大模型教小模型

9.9.1 知识蒸馏的基本原理

知识蒸馏（Knowledge Distillation）的核心思想是：用一个大的**教师模型**（teacher）的输出 来训练一个小的**学生模型**（student）。相比让学生模型直接在原始数据上训练，教师模型的输出包含了更丰富的信息——不仅告诉学生「正确答案是什么」，还通过概率分布中的**暗知识**（dark knowledge）告诉学生「其他答案为什么不对，但有多接近正确」。

标准的知识蒸馏使用带温度的 softmax：

$$p_i^{(\tau)} = \frac{\exp\left(z_i/\tau\right)}{\sum_j \exp\left(z_j/\tau\right)} \tag{9.17}$$

其中 τ 是温度参数。$\tau > 1$ 时，概率分布变得更「平滑」，各类别之间的相对概率差异更加明显——这正是暗知识的来源。

蒸馏损失函数通常是两部分的加权和：

$$\mathcal{L} = \alpha \cdot \tau^2 \cdot \mathrm{KL}(p_{\text{teacher}}^{(\tau)} \| p_{\text{student}}^{(\tau)}) + (1-\alpha) \cdot \mathcal{L}_{\text{CE}}(y, p_{\text{student}}) \tag{9.18}$$

第一项让学生模仿教师的概率分布，第二项保证学生在原始任务上的准确性。τ^2 因子是为了补偿高温 softmax 的梯度缩放。

9.9.2 LLM 蒸馏的实践

在 LLM 时代，蒸馏有几种具体形式：

Logits 蒸馏：让学生模型直接匹配教师模型在每个 token 位置的完整概率分布。这是信息量最大的蒸馏方式，但需要访问教师模型的 logits——对于闭源模型（如 GPT-4）无法使用。

Sequence-level 蒸馏（或称 data distillation）：用教师模型生成大量高质量的训练数据，然后在这些数据上对学生模型做 SFT。这是最常用的方式，因为只需要教师模型的**输出文本**，不需要 logits。Stanford Alpaca[20] 首次展示了 用 GPT-3.5 生成的数据训练 LLaMA 7B 的效果——学生模型在很多任务上达到了教师的 80–90% 水平。

Chain-of-Thought 蒸馏：教师模型不仅提供答案，还提供推理过程。学生模型同时学习「怎么推理」和「推理出什么」。这在数学和代码任务上效果尤为显著——学生模型从教师的推理链中学到了**结构化的思考方式**。

[20] R. Taori et al., *Stanford Alpaca: An instruction-following LLaMA model*, GitHub, 2023.

9.9.3 DeepSeek-R1 的蒸馏实验

DeepSeek-R1[21] 的蒸馏结果 是目前 LLM 蒸馏最令人印象深刻的案例。DeepSeek 团队用 R1（671B MoE）作为教师模型，生成了包含完整推理链的 800K 条训练数据，然后对多个开源基础模型进行 SFT 蒸馏。

Table 9.3: DeepSeek-R1 蒸馏模型性能

蒸馏模型	AIME	MATH-500	LiveCodeBench	备注
R1-Distill-Qwen-1.5B	28.9%	83.9%	16.9%	手机端可运行
R1-Distill-Qwen-7B	55.5%	92.8%	37.6%	消费级 GPU
R1-Distill-Qwen-14B	69.7%	93.9%	53.1%	单卡
R1-Distill-Qwen-32B	72.6%	94.3%	57.2%	性能/成本最优
R1-Distill-Llama-70B	70.0%	94.5%	57.5%	接近教师水平
R1 教师（671B MoE）	*79.8%*	*97.3%*	*65.9%*	*参考*

几个关键发现：

蒸馏比直接 RL 更高效。在相同基础模型上，蒸馏得到的推理能力**显著优于** 直接用 GRPO 做 RL 训练。例如 Qwen2.5-32B 直接 RL 训练在 AIME 上达到 47.0%，但用 R1 蒸馏后达到 72.6%——差距巨大。这表明强大教师模型的推理链 包含了通过 RL 从零学习难以获得的信息。

小模型可以「学会」远超其自身能力的推理。R1-Distill-Qwen-7B 在 MATH-500 上达到 92.8%，而同规模的 Qwen2.5-7B（未蒸馏）只有约 70%。蒸馏将一个 7B 模型的数学推理能力 提升到了接近 70B 未蒸馏模型的水平。

再蒸馏（re-distillation）可以进一步提升质量。Mobius Labs 的实验表明，用更大的蒸馏模型（如 R1-Distill-70B）对更小的模型（如 R1-Distill-14B）进行二次 logits 蒸馏，在多个基准上还能获得 2–5% 的提升，且成本极低（仅 $3–$18 的 GPU 成本）。

9.9.4 蒸馏的局限性

蒸馏并非万能：

- **能力天花板**：学生模型的性能受限于教师模型。如果教师在某个领域不够好，学生也学不好。
- **遗忘问题**：过度蒸馏可能导致学生模型 在教师未覆盖的领域退化。
- **分布差异**：如果学生模型的预训练数据分布 与教师模型差异很大，蒸馏效果会打折扣。
- **推理能力 vs. 知识**：蒸馏擅长传递推理**能力**（如 CoT 推理的模式），但不擅长传递**知识**（如具体的事实记忆）——因为知识需要参数容量来存储，小模型的参数有限。

[21] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

9.10 推理优化的整体思路

回顾整个推理优化领域，可以归纳出一个清晰的思维框架：所有优化都在解决同一个根本问题——**LLM 推理是内存带宽瓶颈的，而非计算瓶颈的**。

在自回归生成过程中，每生成一个 token，需要读取**整个模型的权重**（因为每个 token 都要经过所有层），但只做少量的计算（一个 token 的矩阵乘法）。GPU 的计算能力远未被充分利用——大部分时间花在等待从显存读取权重。

理解了这个瓶颈，所有优化手段都变得清晰：

- **量化**减少每次需要读取的数据量（4-bit 权重是 16-bit 的 1/4 大小），直接缓解带宽瓶颈。
- **Continuous batching** 增加每次读取权重后 处理的 token 数量——读一次权重，同时处理 100 个请求的 token，提高计算与内存访问的比率。
- **推测解码**减少需要调用大模型的次数——小模型的权重更小，读取更快；大模型一次验证多个 token，减少总读取次数。
- **KV cache** 避免重复读取和计算 之前位置的 KV 值。
- **PagedAttention** 减少 KV cache 的内存浪费，允许在固定显存下服务更多并发请求。
- **Prefix caching** 消除共享前缀的重复计算，在生产环境中大幅降低 TTFT。
- **Disaggregated serving** 分离 prefill 和 decode，让每种 GPU 只做它最擅长的工作。
- **蒸馏**从根本上减小模型大小，用一个 7B 模型替代 70B 模型来服务大部分请求。

从这个角度看，推理优化的未来方向也很明确：任何能减少「每 token 需要读取的数据量」或「增加每次读取后完成的有效工作」的技术，都值得关注。NVIDIA Blackwell 的 NVFP4 支持、更大的片上 SRAM、更高带宽的 HBM4——硬件和软件都在向同一个方向努力：打破内存带宽的瓶颈。

CHAPTER 10

推理时计算缩放——用更多思考换更好答案

传统的 LLM 使用方式是：输入一个问题，模型立刻给出答案。但 OpenAI 的 o1 系列和 DeepSeek-R1 开创了一种新范式：**让模型在回答之前先「思考」**——生成大量的推理过程（thinking tokens），然后才给出最终答案。

10.1 核心洞察

推理时计算的颠覆性影响

作为一个粗略的估计：**7B 模型 + 100× 推理计算 ≈ 70B 模型标准推理**（基于 Snell et al. (2024)[a] 的研究）。这意味着你可以用一个便宜的小模型，通过投入更多推理计算来获得与大模型相当的质量。这彻底改变了「模型大小 = 能力」的等式。

[a] C. Snell et al., "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

传统的范式是**训练时缩放**（train-time scaling）：投入更多计算来训练更大的模型，推理时每个问题使用固定的计算量。Scaling Laws 描述的就是这种范式。

推理时计算缩放代表了一种**互补的范式**：模型大小固定，但根据问题的难度动态调整推理时的计算量。简单问题快速回答（几个 token 的思考），复杂问题深度推理（数千个 token 的思考）。

这两种范式的关系可以类比为人类学习和思考的区别：训练时缩放相当于**学更多年**（更多教育），推理时缩放相当于**想更久**（更深思考）。一个博士可能比一个本科生知道更多（训练时缩放），但本科生如果花足够多的时间仔细思考，在某些具体问题上也可能得到更好的答案（推理时缩放）。

为什么推理时缩放可能比训练时缩放更高效?

Snell et al. (2024) 的研究给出了一个关键发现：对于许多具体问题，将计算预算花在推理阶段比花在扩大模型更具**计算效率**。

直觉解释：训练时扩大模型，所有问题都受益于更多的参数——但简单问题本来就能做对，额外的参数是浪费。推理时缩放允许**按需分配**——简单问题快速回答节省计算，难的问题多花时间保证质量。这种动态分配本质上是一种更优的计算资源配置。

类比：训练时缩放像「给所有学生都上博士课程」——对已经懂的学生是浪费；推理时缩放像「允许每个学生根据题目难度决定思考时间」——更灵活，更高效。

10.2 推理时缩放的数学框架

10.2.1 Snell et al. 的计算最优缩放理论

Snell et al.[1] 的核心贡献是建立了 推理时计算的**缩放法则**（scaling law），回答了一个关键问题：给定固定的推理计算预算 C，如何**最优分配**来最大化性能？

他们分析了两种缩放机制：

机制一：基于验证器的搜索。生成 N 个候选回答，用 Process Reward Model（PRM）评分，选择最优。性能提升可以建模为：

$$P(\text{correct} \mid N, \text{PRM}) = f(N, q_{\text{PRM}}) \tag{10.1}$$

其中 q_{PRM} 是验证器的质量，N 是候选数量。关键发现是：**PRM 质量比候选数量更重要**。一个好的 PRM + 少量候选（$N = 4$）通常优于一个差的 PRM + 大量候选（$N = 256$）。

机制二：自适应分布更新。根据具体问题动态调整模型的回答策略。对于推理模型，这体现为 根据问题难度生成不同长度的推理链。

最优分配的核心结论是：

$$\text{Optimal strategy}(C, d) = \begin{cases} \text{少量深度推理} & \text{如果} d \text{ 高（难题）} \\ \text{多次浅层尝试} & \text{如果} d \text{ 低（简单题）} \\ \text{放弃推理时缩放} & \text{如果} d \text{ 极低或极高} \end{cases} \tag{10.2}$$

其中 d 是问题的难度。这个结论有重要的实践意义：

- 对于「模型本来就能做对」的简单问题，额外的推理计算几乎没有收益——直接回答即可。
- 对于「中等难度」的问题，推理时缩放的收益最大——相当于用计算换取了更大模型的效果。
- 对于「远超模型能力」的极难问题，再多的推理计算也无济于事——模型不具备必要的知识或推理能力。

[1] C. Snell et al., "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

推理时缩放的数学等价性

Snell et al. 的一个惊人发现是**计算等价性**：一个较小的模型（如 Llama 3.1 8B）通过最优的推理时计算分配，可以在某些问题上匹配甚至超过 一个大 14× 的模型（Llama 3.1 70B）的性能。

形式化地说，对于难度为 d 的问题集合，存在一个推理计算预算 $C^*(d)$ 使得：

$$\text{Acc}(\text{小模型}, C^*(d)) \geq \text{Acc}(\text{大模型}, C_0) \tag{10.3}$$

其中 C_0 是标准推理的基线计算量。当 $C^*(d) < C_{\text{train}}$（训练大模型的额外成本摊到推理上）时，推理时缩放就是更经济的选择。

但这个等价性不是**普适**的——它高度依赖问题难度。对于极难的问题，没有任何推理计算量 能让小模型匹配大模型。这就是为什么推理时缩放是训练时缩放的**互补**而非**替代**。

10.2.2 三种 Scaling Laws 的统一视角

到 2026 年，LLM 领域已经形成了 三**种 scaling laws** 的统一框架：

1. **预训练缩放**（Kaplan 2020, Chinchilla 2022）：更多的参数 × 更多的数据 × 更多的训练计算 → 更好的基础能力。描述的是**模型知识**的缩放。
2. **后训练缩放**（RL、RLHF、SFT）：更多的对齐训练 → 更好地遵循指令和推理。描述的是**模型行为**的缩放。
3. **推理时缩放**（Snell 2024, o1/R1）：更多的推理计算 → 在具体问题上更好的表现。描述的是**问题求解**的缩放。

这三种缩放在**效率前沿**上各有优势：预训练缩放适合需要广泛知识的场景，后训练缩放适合需要特定行为模式的场景，推理时缩放适合需要深度推理的特定问题。2026 年的趋势是同时优化这三个维度——所谓的**效率缩放**（efficiency scaling），目标是用 $1M 的总成本达到以前需要 $100M 的效果。

Budget forcing（预算强制）是推理时缩放的一个实用技术：根据问题的难度**预先分配**不同的计算预算。简单问题限制 thinking tokens 在 256 以内，中等问题分配 2K–8K，困难问题分配 16K–64K。这避免了模型在简单问题上「过度思考」浪费资源，也保证了困难问题有足够的推理深度。

10.3 三种策略

推理时计算缩放有三种基本策略，各自适用于不同的场景。

10.3.1 策略一：扩展推理链（Sequential Scaling）

顺序缩放让模型在内部进行长链推理——分解问题、逐步推导、自我验证、纠正错误。DeepSeek-R1[2] 的 thinking tokens 就是这种策略。每个 thinking token 都是在「推理计算」上的投资。模型可能生成数千个 thinking token，但最终答案的质量远高于直接回答。

一个典型的推理过程可能是这样的：模型先分析问题结构，然后尝试一种解法，发现行不通后自我纠正（“wait, that’s not right...”），换一种方法，验证中间结果，最终得到答案。这种**自我对话和纠错能力** 是 RL 训练的产物，不是人类预先编程的。

思维链推理（Chain-of-Thought, CoT）是顺序缩放的基础形式。即使不是专门训练的推理模型，简单地在 prompt 中加上 “Let’s think step by step” 就能提升推理质量。但专门训练的推理模型（如 R1、o1）的推理深度和自我纠错能力远超简单的 CoT prompting。

训练推理模型的核心方法是 GRPO[3]（Group Relative Policy Optimization）配合**结果导向奖励**（outcome-based reward）。训练过程不告诉模型「如何推理」，只告诉它「答案对不对」——模型必须自己发现有效的推理策略来最大化奖励。这种纯粹由 RL 驱动的训练方式解释了为什么 R1 会自发地「发明」推理策略：它不是在模仿人类的推理范例，而是在**发现**最大化奖励的行为模式。

顺序缩放的一个微妙之处是**推理长度与质量**的关系。并非越长的推理越好——模型有时会陷入无意义的循环重复，或者在简单问题上过度展开推理链。DeepSeek-R1 的训练中观察到，模型在 RL 早期会出现 “overthinking” 现象——推理链越来越长但答案并没有变好。通过调整奖励函数（对过长的推理施加轻微惩罚），可以让模型学会在合适的深度停止思考。

10.3.2 策略二：并行采样与 Best-of-N（Parallel Scaling）

并行缩放对同一个问题生成 N 个独立的回答，然后用多数投票或奖励模型选最好的。这相当于「三个臭皮匠赛过诸葛亮」——即使每个回答独立看不够好，通过聚合多个独立的回答可以得到更可靠的结果。

Best-of-N 采样

Best-of-N 采样是最简单的并行缩放实现：生成 N 个回答，用奖励模型（Reward Model, RM）选择分数最高的那个。

数学上，Best-of-N 的效果可以用**序统计量**分析。设每个回答的奖励分数 r_i 独立同分布于某分布 $F(r)$。N 个样本的最大值 $r_{(N)}$ 的期望为：

$$\mathbb{E}[r_{(N)}] = \int_{-\infty}^{\infty} r \cdot N \cdot F(r)^{N-1} \cdot f(r)\, dr \tag{10.4}$$

[2] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.

[3] Z. Shao et al., “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.

当 F 是正态分布 $\mathcal{N}(\mu, \sigma^2)$ 时，$\mathbb{E}[r_{(N)}]$ 近似为：

$$\mathbb{E}[r_{(N)}] \approx \mu + \sigma \cdot \Phi^{-1}\left(\frac{N}{N+1}\right) \tag{10.5}$$

其中 Φ^{-1} 是标准正态分布的分位数函数。

这意味着 Best-of-N 的收益是**对数缩放**的：从 $N = 1$ 到 $N = 4$，质量提升显著；从 $N = 16$ 到 $N = 64$，提升已经很小；到 $N = 256$，几乎无法进一步提升。计算成本线性增长但收益对数递减，所以实践中 $N = 4$–16 是最优的性价比区间。

Best-of-N 的陷阱：奖励黑客

Best-of-N 的质量**严重依赖**奖励模型的质量。如果奖励模型是训练目标的不完美代理（这几乎总是如此），过大的 N 会导致**奖励黑客**（reward hacking）——模型生成的回答获得了高奖励分数，但实际质量并不好，甚至更差。这是因为大 N 给了模型更多机会 发现奖励模型的弱点并加以利用。

正则化 Best-of-N（RBoN）[a] 通过在目标函数中加入正则项来缓解这个问题：

$$x^* = \arg\max_{x \in \{x_1, \ldots, x_N\}} \left[R(x) - \beta \cdot \mathrm{KL}(q \| p_{\mathrm{ref}})\right] \tag{10.6}$$

正则项惩罚偏离参考分布过远的回答，防止奖励黑客。

[a] Y. Ichihara et al., "Evaluation of best-of-N sampling strategies for language model alignment," *Transactions on Machine Learning Research*, 2025.

多数投票与自一致性

多数投票（Majority Voting / Self-Consistency）[4] 更适合有**可验证答案**的问题（如数学题）：生成 N 个推理过程和答案，选择出现次数最多的答案。直觉是：即使每次推理的中间步骤不同，正确答案更有可能被多次「发现」。

数学分析很清晰：如果单次回答正确的概率是 p，N 次独立采样中至少有一次正确的概率是：

$$P(\text{at least one correct}) = 1 - (1 - p)^N \tag{10.7}$$

当 $p = 0.3$（单次正确率 30%）时，$N = 10$ 次采样就有 $1 - 0.7^{10} \approx 97\%$ 的概率 至少得到一个正确答案。但从 $N = 10$ 到 $N = 100$，概率只从 97% 提升到 99.99%——**收益急剧递减**。

多数投票比 Best-of-N 更**鲁棒**，因为它不依赖可能有偏的奖励模型——它依赖的是「正确答案比错误答案更一致」这个假设。但它只适用于答案可以**精确比较**的场景（数学、选择题、代码输出），不适用于开放式文本生成。

[4] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations*, 2023.

奖励模型引导搜索

更高级的并行缩放策略结合了 奖励模型的评估能力和搜索算法。

Weighted majority voting：不是简单地数每个答案出现的次数，而是用奖励模型对每个推理链打分，然后按分数加权投票。这结合了自一致性（答案频率）和质量评估（奖励分数）的优点。

LE-MCTS（Language model Ensemble with MCTS）[5]：将多个 LLM 的推理步骤视为 MCTS 中不同的行动选择。在每一步推理中，选择哪个 LLM 来生成下一步 由 UCB（Upper Confidence Bound）策略决定，由 PRM 提供每一步的评分。这实现了**步骤级别**的模型集成，比简单的输出级别集成更加精细。

并行缩放的优点是实现极其简单、可线性扩展、而且每个独立样本可以并行生成，延迟不增加（只增加吞吐需求）。缺点是效率低——大部分生成是浪费的，而且对于需要深度推理的问题，100 次浅层思考可能不如 1 次深度思考。

10.3.3 策略三：搜索（Search / Tree of Thought）

搜索将推理过程视为一棵**树**，每个节点是一个中间推理步骤，每条边是一步推理。通过系统性地探索多条推理路径，可以避免单条路径的死胡同。

Tree of Thought（ToT）是最直观的实现：在推理的每个关键步骤，生成多个候选的下一步，评估每个候选的质量，选择最有希望的继续展开。

MCTS（Monte Carlo Tree Search）将 AlphaGo 的搜索算法引入语言推理：从根节点（问题）出发，通过选择（Selection）、扩展（Expansion）、模拟（Simulation）、回传（Backpropagation）四个步骤反复迭代，逐步构建推理树并找到最优路径。

搜索的关键组件是**评估函数**——如何判断一个中间推理步骤的质量？这里引入了 **Process Reward Model**（PRM）的概念：与传统的 Outcome Reward Model（ORM，只评分最终答案）不同，PRM 对**每个推理步骤**打分。这提供了更细粒度的信号——不仅知道最终答案对不对，还知道从哪一步开始出了问题。PRM 使得搜索可以**及早剪枝**错误的推理路径，大幅提高搜索效率。

[5] S. Park et al., "Ensembling large language models with process reward-guided tree search for better complex reasoning," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the ACL*, 2025.

ORM vs PRM：粗粒度 vs 细粒度奖励

考虑一个数学问题的 5 步推理过程，其中第 3 步出了错误。

ORM 只看最终答案：答案错了 → 整个推理链得低分。但 ORM 不知道前 2 步是对的，导致在搜索中可能丢弃那些「前半部分优秀」的推理路径。

PRM 对每一步评分：Step 1: 0.95, Step 2: 0.92, Step 3: 0.15, Step 4: 0.10, Step 5: 0.08。PRM 清晰地定位了错误在 Step 3，搜索算法可以保留 Step 1–2 的推理，只从 Step 3 开始尝试新的方向。

PRM 的训练更困难——需要**步骤级别**的标注，而不是简单的「答案对不对」。但其提供的细粒度信号 使得搜索效率提升数个数量级。Snell et al. 的实验表明，PRM 引导的 beam search 在相同计算预算下 比 ORM 引导的 Best-of-N 高出 10–20% 的准确率。

AlphaProof 和 **AlphaGeometry 2** 是搜索策略的极致展示：在 2024 年国际数学奥林匹克（IMO）中，AlphaProof 与 AlphaGeometry 2 联合解出了 6 道题中的 4 道（其中 AlphaProof 解决了代数和数论题，AlphaGeometry 2 解决了几何题），达到了银牌水平。AlphaProof 使用 Gemini 模型将自然语言转换为 Lean 4 形式化语言，再通过类似 AlphaZero 的强化学习方法进行证明搜索。每一步都可以被**机器验证**（proof checker），消除了幻觉的可能性。

Beam search over reasoning chains 是一种更轻量的搜索方法：类似于传统的 beam search，维护 B 条最有希望的推理链，每步扩展每条链，保留总分最高的 B 条。PRM 提供每一步的评分。这比完整的 MCTS 计算量更小，但仍然比纯顺序推理有显著的质量提升。

10.3.4 三种策略的对比

推理时计算策略对比

- **顺序缩放**：一次深度思考。优势——推理深度高，适合需要多步推导的问题。劣势——单次推理可能走进死胡同。
- **并行缩放**：N 次独立思考。优势——实现简单，可并行，延迟不增加。劣势——效率低，收益递减，可能遭受奖励黑客。
- **搜索**：系统性探索推理树。优势——可以回溯，避免死胡同，理论上最优。劣势——需要 PRM，计算量大，实现复杂。

经验法则：低难度问题用并行缩放，中高难度用顺序缩放，极高难度（数学竞赛、形式化证明）用搜索。

10.3.5 混合缩放（Hybrid Scaling）

混合缩放结合并行和顺序——并行生成多个推理链，每条链内部是顺序推理，最后取最优。ThreadWeaver（2025 年提出）就是这种方法，实现了 1.53× 的延迟降低同时保持准

确率。

实践中最有效的配置通常是混合的：让推理模型生成 N（通常 4–16）个独立的长推理链，然后用多数投票或奖励模型选最优。这同时利用了「深度思考」和「多样性」的优势。

10.4 DeepSeek-R1：从零开始学会推理

DeepSeek-R1[6] 是推理时缩放领域的里程碑。它首次证明了**纯粹的强化学习**（无需人类推理示范）就能让模型自发地发展出复杂的推理能力。

10.4.1 R1-Zero：从 RL 中涌现的推理

在 R1 的完整训练之前，DeepSeek 团队先进行了一个大胆的实验：在 DeepSeek-V3-Base 上**直接**应用 GRPO RL 训练，**不使用任何 SFT 数据**。这个模型被称为 **DeepSeek-R1-Zero**。

R1-Zero 的训练设计极其简洁：

- **奖励信号**：只有两类——数学题答案是否正确（accuracy reward），和输出格式是否符合要求（format reward）。没有过程奖励（PRM），没有人类偏好。
- **训练数据**：数学、代码和逻辑推理题，全部有**可验证**的正确答案。
- **优化算法**：GRPO——对每个问题生成一组回答，根据组内相对排名计算优势函数，不需要单独的价值网络或奖励模型。

10.4.2 “Aha Moment”——模型的顿悟时刻

R1-Zero 训练过程中出现了令研究者震惊的现象。在 RL 训练到一定阶段后，模型自发地开始展现出一系列 **从未在训练数据中出现过**的推理行为：

DeepSeek-R1 的 “Aha Moment”

在训练中期的某次评估中，模型在解数学题时写下了这样的推理过程：

“Wait, wait. Wait. That’s an aha moment. I can actually solve this by re-reading the problem and approaching it differently... Let me reconsider what I assumed earlier...”

这不是人类编写的推理模板——模型自己「发明」了自我反思和重新审视的策略。这个时刻被 DeepSeek 团队称为 “aha moment”，因为它标志着模型从简单的模式匹配 跃迁到了**元认知**层面的推理。

具体而言，R1-Zero 自发发展出了以下推理策略：

回溯（Backtracking）：模型在推理链中发现前面的方向不对时，主动放弃当前路径，回退到更早的步骤重新推导。这是一种典型的搜索行为——模型学会了在推理树中「剪枝」和「回溯」。

[6] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.

自我验证（Self-verification）：模型在得出中间结论后，会主动检验该结论的正确性——例如将答案代回原方程验证，或用不同方法重新推导检查一致性。这模拟了数学家的证明检查过程。

分类讨论（Case analysis）：面对多种可能情况的问题，模型学会了系统性地枚举每种情况，逐一分析，最后综合得出结论。

反证法（Proof by contradiction）：在某些逻辑推理问题中，模型会假设结论不成立，推导出矛盾，从而证明结论成立。

这些策略的涌现具有深远的意义：它表明复杂的推理能力**不需要**显式的人类教学，可以通过适当的 RL 训练从模型中自然涌现。这与 AlphaGo Zero 的启示一致——在有可验证奖励信号的领域，RL 能够发现人类未曾想到的策略。

10.4.3 R1 的完整训练流程

R1-Zero 虽然展现了令人兴奋的推理涌现，但存在两个严重的问题：

- **可读性差**：输出混合多种语言，推理链混乱难读，对用户不友好。
- **非推理任务退化**：在日常对话、翻译等不需要推理的任务上 表现明显下降。

DeepSeek-R1 的完整训练分为四个阶段：

阶段一：冷启动 SFT。收集少量高质量的推理示范数据（数千条），对基础模型做 SFT。这一步不是为了教模型推理——而是为了稳定后续 RL 训练的起点，避免 R1-Zero 中的格式混乱问题。

阶段二：推理导向 RL。在冷启动模型上做 GRPO RL 训练，使用数学、代码和逻辑题的可验证奖励。这是推理能力形成的核心阶段。

阶段三：拒绝采样 + 通用 SFT。用阶段二的模型生成大量推理链，通过拒绝采样保留高质量的输出（约 800K 条），混合通用的 SFT 数据（日常对话、翻译等），做一轮全面的 SFT。这一步恢复了模型在非推理任务上的能力。

阶段四：全场景 RL。在所有类型的任务上做最终的 RL 训练，使用混合的奖励信号（可验证任务用规则奖励，开放式任务用奖励模型），进一步提升整体质量和对齐度。

10.4.4 R1 vs o1/o3：两条不同的路径

DeepSeek-R1 和 OpenAI 的 o1/o3 系列 代表了推理时缩放的两条不同路径：

Table 10.1: DeepSeek-R1 vs OpenAI o 系列对比

维度	DeepSeek-R1	OpenAI o1/o3
核心方法	GRPO RL + 可验证奖励。不需要单独的奖励模型。	RL + 详细的 PRM。使用复杂的奖励建模系统。
推理过程	完全可见。用户可以看到完整的 thinking tokens。	隐藏。用户只看到最终答案（部分摘要可见）。
开源	模型权重完全开源，训练方法论在论文中详细公开。	完全闭源，仅通过 API 提供服务。
基础架构	671B MoE（37B 活跃参数），基于 DeepSeek-V3。	未公开。推测基于 GPT-4 级别架构。
蒸馏	提供 1.5B 到 70B 的完整蒸馏系列。	无官方蒸馏，o3-mini 作为轻量版本。
性能（AIME 2024）	79.8%	o1: 83.3%, o3-mini (high): 87.3%

o3-mini 是一个值得关注的案例：它在多个数学基准（如 AIME）上**超过**了 o1，同时成本只有 o1 的 1/14（$1.10/$4.40 vs $15/$60 per M tokens）。这证明了推理时缩放中 **算法效率的提升**可以同时提高质量和降低成本。

2025 年 4 月，OpenAI 发布了 o3 和 o4-mini，标志着推理模型的又一次飞跃。o3 和 o4-mini 引入了**工具使用**能力——它们可以在推理过程中自主决定 搜索网页、执行代码、分析图像，将工具调用和推理无缝结合。这代表了从「纯思考」到「思考 + 行动」的范式转变。

10.5 工具增强推理

2025–2026 年，推理模型的一个重要演进方向是 将**外部工具**集成到推理过程中。模型不再仅仅依赖内部推理——而是可以在思考过程中调用工具来辅助推理。

10.5.1 代码执行作为推理工具

最自然的工具增强推理是**代码执行**。在解决数学或数据分析问题时，模型可以编写代码来精确计算中间结果，而不是依赖自然语言推理（容易出现算术错误）。

典型的流程是：

1. 模型分析问题，决定使用代码来辅助计算。
2. 在 thinking 过程中生成 Python 代码。
3. 代码在沙箱环境中执行，返回结果。
4. 模型将执行结果纳入后续推理链。

这解决了 LLM 推理中最臭名昭著的弱点之一：**算术错误**。LLM 通过自然语言做多步算术时，每一步都有累积误差的风险。而代码执行保证了计算的**精确性**。OpenAI 的 o3/o4-mini 原生支持 在推理过程中执行 Python 代码——这使得它们在需要精确计算的数学和科学问题上 表现显著优于仅用自然语言推理的模型。

10.5.2 搜索与检索增强推理

推理过程中的**检索增强**（Retrieval-Augmented Reasoning）让模型在思考时可以搜索外部知识。当模型在推理中遇到不确定的事实性问题时，它可以暂停推理、搜索相关信息、然后基于检索到的事实继续推理。

这与传统的 RAG（Retrieval-Augmented Generation）不同：

- **传统 RAG**：在生成**之前**检索一次，将检索结果作为上下文输入模型。
- **推理中检索**：在推理**过程中**多次检索，每次检索都基于当前推理状态的需要。检索和推理交替进行，形成「思考 → 搜索 → 思考」的循环。

这种交互式检索在需要综合多源信息的推理任务中 效果尤为显著——例如法律案例分析（需要检索多个先例）或科学文献综述（需要交叉引用多篇论文）。

10.5.3 多步 Agent 推理

Agent 推理是工具增强推理的最完整形式。推理模型不仅可以思考和使用单个工具，还可以**规划**和**编排**多步骤的工作流。

Claude Opus 4.6 的 extended thinking 允许模型在一个统一的推理过程中 思考、调用工具、分析工具返回的结果、再继续思考——整个过程是一个**连贯的**推理链，而不是分离的「思考步骤」和「工具调用步骤」。

OctoTools[7] 提出了一个 可扩展的 agent 推理框架。通过**工具卡片**（tool card）机制，模型可以在无需额外训练的情况下 集成新的工具——每张工具卡片描述了工具的功能、输入/输出格式 和使用时机。模型通过阅读工具卡片来决定何时使用什么工具。实验表明，即使是最强的 LLM + 完善的 agent 工作流，在复杂的跨论文科学推理任务上 准确率也仅有 38.78%，困难子集上更是低至 18.47%——这表明工具增强推理仍有巨大的改进空间。

工具增强推理的层次

- **Level 0——纯推理**：模型仅用自然语言推理，不使用任何外部工具。CoT、R1-Zero 属于这一层次。
- **Level 1——单工具推理**：模型在推理中使用单一工具（如代码执行器）。o3 的 Python 执行属于这一层次。
- **Level 2——多工具推理**：模型在推理中编排多个工具（代码、搜索、API 调用）。o3/o4-mini 的完整工具使用属于这一层次。
- **Level 3——Agent 推理**：模型规划多步骤工作流，包括子任务分解、工具编排、结果验证、和基于中间结果的动态调整。Claude 的 extended thinking + tool use 属于这一层次。

[7] P. Lu et al., "OctoTools: An agentic framework with extensible tools for complex reasoning," *arXiv preprint arXiv:2502.11271*, 2025.

10.6 推理模型的部署经济学

推理时计算缩放正在深刻重塑 AI 的经济学。推理需求预计将大幅超过训练需求（据行业估计可达百倍量级）。GPT-4 等效性能的成本从 2022 年的 $20/M tokens 降至 2026 年的 $0.40/M tokens——1000× 的下降，主要来自推理优化。

但推理模型也带来了新的成本挑战：DeepSeek-R1 在回答一个数学问题时 可能生成 10,000+ 个 thinking token，然后才给出 100 个 token 的答案。thinking token 虽然不展示给用户，但同样需要计算和内存资源。

10.6.1 何时使用推理时缩放

推理时缩放不是万灵药——对于不同类型的任务，性价比差异巨大：

高价值场景（值得投入 10–100× 推理计算）：

- 数学证明和竞赛题——答案可验证，推理深度直接决定正确率。
- 代码 bug 修复和系统设计——错误成本高，多花 $0.10 的推理费用可以避免 $10,000 的生产事故。
- 法律和医疗分析——高风险决策，准确性比延迟更重要。
- 科学研究中的假设生成——需要深度推理和创造性思考。

低价值场景（标准推理即可）：

- 日常聊天和问答——首次回答通常就足够好。
- 翻译和摘要——任务结构明确，不需要深度推理。
- 简单的分类和提取——模型直觉足够准确。

10.6.2 路由与预算管理

这催生了**推理预算管理**的概念：对于简单问题，限制 thinking token 的数量以节省成本；对于困难问题，分配更多的推理预算以保证质量。

难度路由（Difficulty Routing）是一种经济高效的方案：使用一个轻量级的分类器（甚至是 prompt-based 的判断）评估问题的难度，将简单问题路由到标准模型（快且便宜），将困难问题路由到推理模型（慢但准确）。好的路由器可以将 80% 的请求用便宜的方式处理，只对 20% 真正需要深度推理的问题投入额外计算。

可配置的推理预算：行业实践

Claude Opus 4.6 支持可配置的 thinking budget，允许开发者通过 API 设置任意的 `budget_tokens` 参数 来控制推理深度。这将推理时缩放从「全有或全无」变成了**连续可调**的维度——开发者可以根据任务需求和成本预算 在延迟、质量和成本之间做精细的权衡。

Qwen3[a] 走了另一条路：在单个模型中统一 thinking 和 non-thinking 模式。用户可以根据需要在两种模式间切换——简单问题直接回答（non-thinking），复杂问题开启深度推理（thinking）——不需要部署两个不同的模型。

Google Gemini 2.5/3 引入了**动态思考模式**——模型根据问题复杂度自动调整推理深度，无需用户手动控制。Flash 版本提供快速推理，Pro 版本提供深度推理，两者共享同一架构但分配不同的计算预算。

[a] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

10.6.3 共享推理的可能性

对于多个用户问相似的难题（如热门的面试编程题），可以缓存和复用推理过程。一个用户的深度推理结果可以被后续用户共享，避免重复计算。这类似于 prefix caching 的思想，但扩展到了推理链（reasoning chain）层面。这在教育和客服场景中有巨大的潜力。

10.6.4 成本模型

一个推理模型回答一个数学问题的成本构成：

- **标准回答**（无 thinking）：~500 输出 token，成本 ~$0.001。
- **浅层推理**（1K thinking tokens）：~1500 总 token，成本 ~$0.003。
- **深度推理**（10K thinking tokens）：~10500 总 token，成本 ~$0.02。
- **Best-of-16 + 深度推理**：~168K 总 token，成本 ~$0.30。

深度推理的成本是标准回答的 20 倍，Best-of-16 + 深度推理是 300 倍。但如果问题本身的价值足够高（一道竞赛题的解答、一个关键 bug 的修复），这个成本仍然极为合算。

关键在于**匹配推理投入和问题价值**——用 $0.30 的推理计算来回答「今天天气怎么样」是浪费，用 $0.001 的快速回答来解数学竞赛题是不足。好的推理时计算系统需要**自适应**地分配资源。

10.7 推理模型的前沿与未来

10.7.1 从推理到行动：Agent 范式

2026 年的一个核心趋势是 推理模型从「思考者」进化为「行动者」。传统推理模型只在**文本空间**中推理；新一代模型在**行动空间**中推理——它们可以规划行动序列、执行工具

调用、观察结果、调整计划。

这种「推理 + 行动」的范式 本质上是将推理时缩放应用于更广阔的问题空间。不再只是「想更久」，而是「想更久，并且在想的过程中做实验来验证想法」。

10.7.2 效率缩放：做更多、花更少

2026 年行业的核心议题不再是「更大的模型」，而是**效率缩放**——用更少的计算达到同等甚至更好的效果。这体现在多个层面：

- **蒸馏**：R1 的 7B 蒸馏模型在 MATH-500 上达到 92.8%，接近 70B 级别——1/10 的参数，1/10 的推理成本。
- **混合推理模式**：Qwen3 的 thinking/non-thinking 统一架构 让一个模型适应所有场景，避免了部署两套模型的开销。
- **推理预算自适应**：根据问题难度动态分配计算，80% 的简单请求用最少的计算完成。
- **推理链缓存**：复用相似问题的推理过程，避免重复的深度推理计算。

10.7.3 开放的问题

推理时缩放仍有多个**未解决的问题**：

如何训练更好的 PRM？ PRM 的质量决定了搜索策略的上限，但获取步骤级别的训练数据极为昂贵。自动化的 PRM 训练（如通过 MCTS 回溯自动标注）是一个活跃的研究方向。

推理时缩放有没有「上限」？ Snell et al. 表明，在极难的问题上，增加推理计算的收益会趋近于零。但目前尚不清楚这个上限在哪里，以及更好的推理策略能否推高这个上限。

推理能力能否泛化？ 在数学和代码上训练的推理能力 能否迁移到法律推理、医学诊断或科学发现？现有证据表明迁移是有限的——推理的**形式**可以迁移（如回溯、验证），但**领域知识**不能自动获得。

什么是推理的本质？ 当前的推理模型是否真的在「推理」，还是只是在做更复杂的模式匹配？这个哲学问题对实践也有影响——如果只是模式匹配，那么在训练分布外的问题上 推理能力会急剧下降。

这些问题的答案将决定 推理时缩放能走多远，以及它最终能否像预训练缩放那样 带来持续的、可预测的性能提升。

CHAPTER 11

评估与安全——如何知道模型足够好?

11.1 评估的困境

评估 LLM 是一个根本性的难题。传统的 NLP 评估——给定答案，检查是否正确——对于开放式生成任务完全不适用。一个问题可以有无数种正确的回答方式。

11.1.1 Goodhart 定律

Goodhart 定律的警告

「当一个度量成为目标时，它就不再是好的度量。」已有证据表明，一些模型在特定基准测试上被过度优化——在 MMLU 上得分很高，但在实际使用中表现平平。Chatbot Arena 的人类盲评被认为是最可靠的评估方式，因为它直接衡量用户满意度，而且每天更新的问题使得「针对性优化」变得困难。

Goodhart 定律在 LLM 评估中有多种表现形式：

直接过拟合：当模型开发者将 MMLU 得分作为关键指标反复迭代时，他们可能有意或无意地调整训练数据、训练策略和超参数 来最大化 MMLU 分数。结果是模型学会了「做 MMLU 选择题」的模式，而不是真正掌握了 57 个学科的知识。

格式 hacking：某些基准测试对输出格式敏感。例如，MMLU 期望模型输出 A/B/C/D 中的一个字母。一些模型被特别训练成在遇到选择题时 以极高的概率直接输出字母，而不是先「思考」再回答。这提高了基准分数，但可能降低了模型在 需要深思熟虑的真实场景中的表现。

奖励模型 hacking：在使用 RLHF 时，模型可能学会了讨好奖励模型（生成奖励模型喜欢的风格——冗长、过度谨慎、频繁使用 “I appreciate your question” 这样的客套话），而不是真正提高回答质量。这是 Goodhart 定律在训练阶段的体现。

11.1.2 数据污染

数据污染（data contamination）是基准测试面临的另一个严峻挑战：如果测试题目出现在了模型的训练数据中，模型可能只是在「背答案」而非真正推理。

随着训练数据规模从数十亿扩大到数万亿 token，几乎不可能保证训练数据中完全不包含任何基准测试的内容——这些测试题本身就发布在互联网上。

常用的检测和缓解方法包括：

- **N-gram overlap detection**：检查训练数据和测试题之间的 n-gram 重叠率。超过阈值的样本被标记为可能泄露。
- **Canary strings**：在测试集中插入独特的标记字符串，然后检查模型是否能记忆这些字符串。
- **Temporal splits**：使用模型训练截止日期**之后**发布的数据构建测试集。LiveBench 就采用这种策略，定期更换测试题。
- **Private test sets**：不公开测试题的全部内容。SWE-bench 的部分测试用例就是私有的。

深入数据污染检测

近年来，数据污染检测已发展为一个独立的研究方向。Ravaut et al. (2024)[1] 对现有方法进行了系统综述，将检测方法分为三个层面：

第一层：基于训练数据的检测。如果能访问模型的训练数据（通常仅限开源模型），可以直接进行文本匹配。方法包括精确字符串匹配、N-gram 重叠率计算（通常用 8-gram 或 13-gram），以及语义相似度检索。GPT-4 的技术报告就使用了 基于子串匹配的污染检测，但由于训练数据本身不公开，外部研究者无法独立验证。

第二层：基于模型行为的检测。即使无法访问训练数据，也可以通过分析模型行为推断是否存在污染：

- **成员推断攻击**（Membership Inference Attack, MIA）：利用模型对训练样本和非训练样本的不同行为（如 perplexity 差异）来判断某个样本是否在训练集中。Shi et al. (2024) 提出的 Min-K% Prob 方法[2] 通过检查一段文本中概率最低的 K% token 的平均对数概率来判断——如果这些「最不可能」的 token 的概率仍然很高，说明模型很可能在训练中见过这段文本。
- **补全测试**（Completion Test）：给模型一道测试题的前半部分，检查它能否逐字补全后半部分。如果模型能精确复现原文的表述（而非给出语义等价但措辞不同的回答），强烈暗示该题目出现在训练数据中。
- **扰动一致性测试**（Perturbation Test）：对基准测试题做微小的**语义保持**修改（改变数字、替换名称、调整选项顺序），如果模型在原题上准确率很高 但在扰动版本上大幅下降，说明它可能只是记住了原题的答案。Golchin & Surdeanu (2024)[3] 系统地研究了这种方法。

第三层：基于基准设计的预防。从源头上减少污染的可能性：

- **动态基准**：LiveBench 和 LiveCodeBench 定期更新测试题，使用模型训练截止日期之后的数据，从根本上消除了数据泄露的可能性。

[1] M. Ravaut et al., "A comprehensive survey of contamination detection methods in large language models," *Transactions on Machine Learning Research*, 2024.

[2] W. Shi et al., "Detecting pretraining data from large language models," in *International Conference on Learning Representations*, 2024.

[3] S. Golchin et al., "Time travel in LLMs: Tracing data contamination in large language models," in *International Conference on Learning Representations*, 2024.

- **私有测试集**：保留一部分测试题不公开。只接受 API 提交，服务端验证结果。
- **程序化生成**：使用参数化模板动态生成测试题。每次评估使用不同的具体实例，但考察相同的能力。FrontierMath 的部分题目就采用了这种策略。

SWE-bench Verified 的污染警示

2026 年 2 月，OpenAI 发布报告指出 SWE-bench Verified 已出现严重的数据污染问题[a]。由于该基准的测试用例来自公开的 GitHub issue，许多模型的训练数据中包含了这些 issue 的讨论和解决方案。OpenAI 建议社区转向 SWE-bench Pro——一个使用私有测试用例和更严格防泄露措施的新版本。这一事件凸显了**任何公开的静态基准 都有被污染的风险**，进一步强化了动态基准和私有测试集的重要性。

[a] X. Deng et al., "SWE-Bench pro: Can AI agents solve long-horizon software engineering tasks?" *arXiv preprint arXiv:2509.16941*, 2025, Scale AI.

11.1.3 基准饱和

LLM 的进步速度让基准测试快速**饱和**——设计来区分好坏的测试，在最新的模型面前失去了区分度。

GSM8K（8500 道小学数学应用题）在 2023 年初 还被认为是衡量数学推理能力的有力基准。到 2025 年底，前沿模型在 GSM8K 上的准确率普遍超过 95%，GSM8K 已经无法区分不同模型的数学能力。社区因此推出了更难的替代品：MATH（竞赛级数学）→ AIME（数学奥林匹克）→ GPQA（博士级科学问题）→ FrontierMath（研究级数学）。每一代更难的基准测试的生命周期都在缩短。

这导致了一场**基准军备竞赛**：研究者不断设计更难的测试，模型不断攻克它们。这本身说明了 LLM 能力的快速进步，但也给评估方法论带来了挑战——我们需要的不仅是更难的题目，还需要更好的评估**范式**。

11.2 基准测试全景

主流基准测试覆盖不同能力维度：

11.2.1 知识与推理

- **MMLU**（57 学科选择题）[4]：从高中到专业级别的知识测试。涵盖人文、社会科学、STEM 等。2023 年的标杆，2025 年已接近饱和（前沿模型 >90%）。
- **MMLU-Pro**[5]：MMLU 的加强版。10 个选项（而非 4 个），更难的题目，更少的格式泄露。更好地区分当前前沿模型的知识深度。

[4] D. Hendrycks et al., "Measuring massive multitask language understanding," in *International Conference on Learning Representations*, 2021.

[5] Y. Wang et al., "MMLU-Pro: A more robust and challenging multi-task language understanding benchmark," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

- **ARC**（AI2 Reasoning Challenge）：科学推理题。ARC-Challenge 子集需要多步推理。
- **GPQA**（Graduate-Level Google-Proof QA）[6]：博士级别的物理、化学、生物问题。"Google-Proof" 意味着即使允许搜索也不容易回答——需要深度专业知识和推理。
- **Big-Bench Hard**：从 BIG-Bench 的 204 个任务中筛选出 语言模型表现最差的 23 个任务。专注于需要多步推理的困难问题。

MMLU-Pro：超越 MMLU 的下一代知识基准

MMLU 在 2023–2024 年是 LLM 评估的核心指标，但随着前沿模型准确率逼近 90% 甚至更高，其区分度急剧下降。Wang et al. (2024) 在 NeurIPS 2024 上提出了 MMLU-Pro[7]，通过三个关键改进重新拉开了模型之间的差距：

1. **10 选项制**：从 MMLU 的 4 个选项增加到 10 个。这将随机猜测的基线从 25% 降低到 10%, 大幅减少了「蒙对」的概率。更重要的是, 10 个选项中的干扰项（distractors）经过精心设计，反映了常见的错误推理路径，使得需要真正理解才能排除。
2. **更高的推理需求**：MMLU-Pro 的题目需要更多的多步推理。实验表明，使用 Chain-of-Thought（CoT）提示 可以将 MMLU-Pro 的准确率提升 10–15 个百分点，而在原始 MMLU 上 CoT 的提升幅度不到 2%。这说明 MMLU-Pro 的题目确实在考察推理能力，而非简单的知识记忆。
3. **更强的鲁棒性**：在 MMLU 上，选项顺序的变化可导致模型准确率波动 4–5%；在 MMLU-Pro 上，这种波动降低到 2% 以内。10 选项制降低了格式偏差（format bias）的影响。

截至 2026 年初，前沿模型在 MMLU-Pro 上的准确率 约为 85–90%（对比 MMLU 上的 90–95%），仍有足够的区分度。

GPQA Diamond：博士级的「Google-Proof」评估

GPQA[8] 的设计理念是创造 即使允许使用搜索引擎也难以回答的问题。题目由各领域的博士专家撰写并交叉验证：**领域专家**的准确率约为 65%，而**拥有 Google 搜索权限的非专家**准确率仅约 34%。这个巨大的差距证明了题目确实需要深度专业知识。

GPQA Diamond 是 GPQA 中最具挑战性的 198 道题子集，涵盖物理、化学和生物三个学科。截至 2026 年初，推理模型（如 o3、R1）在 GPQA Diamond 上的准确率 已达到 85–93%，**首次在某些领域超越了人类博士专家**。这一方面说明了推理模型的巨大进步，另一方面也意味着 GPQA 可能在未来 1–2 年内饱和。

[6] D. Rein et al., "GPQA: A graduate-level google-proof q&a benchmark," in *Conference on Language Modeling (COLM)*, 2024.

[7] Y. Wang et al., "MMLU-Pro: A more robust and challenging multi-task language understanding benchmark," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[8] D. Rein et al., "GPQA: A graduate-level google-proof q&a benchmark," in *Conference on Language Modeling (COLM)*, 2024.

11.2.2 数学

- **GSM8K**（8500 道小学应用题）：曾经的标准数学基准。需要 2–8 步的基本数学运算。2025 年已接近饱和——大多数前沿模型准确率 >95%。
- **MATH**（12,500 道竞赛级数学）：覆盖代数、几何、数论等 7 个类别，难度从 1 到 5。Level 5 仍有挑战性。
- **AIME**（American Invitational Mathematics Examination）：美国数学邀请赛真题。整数答案，需要创造性数学推理。推理模型（R1, o1）在此基准上表现显著优于标准模型。
- **FrontierMath**[9]：由 Epoch AI 与专业数学家合作设计的研究级数学基准。详见下文单独展开。

FrontierMath：AI 数学能力的终极测试

FrontierMath[10] 由 Epoch AI 于 2024 年 11 月发布，代表了目前最高难度的 AI 数学基准。它与之前的数学基准有根本性的不同：

- **原创的未发表题目**：所有 350 道题目都是由数学家专门为该基准**原创**的，从未在任何地方发表过，从根本上杜绝了数据泄露。
- **研究级难度**：题目覆盖数论、实分析、代数几何、范畴论等 现代数学的主要分支。解决一道典型的 FrontierMath 题目 需要相关领域的数学研究者花费**数小时到数天**。
- **四级难度体系**：Tier 1–3 覆盖本科到博士后水平，Tier 4 是研究级数学。此外还有一组**开放问题**——至今未被人类数学家解决的问题。
- **自动验证**：答案是精确的数值或数学对象，可以通过程序自动验证，无需主观判断。

FrontierMath 的标志性意义

当 FrontierMath 于 2024 年发布时，所有前沿模型的通过率不到 2%。著名数学家 Terence Tao 评论道："这些题目极具挑战性——我预计在数年内 AI 系统还无法解决其中的大部分。" 然而到 2025 年底，推理模型（如 o3）在 Tier 1–2 上的通过率已超过 25%，在 Tier 4 上仍然低于 5%。FrontierMath 可能是目前唯一一个 在可预见的未来不会饱和的 LLM 基准。

11.2.3 代码

- **HumanEval**（164 道函数级编程题）：给出函数签名和 docstring，模型生成函数体。用单元测试验证正确性。简单但有用的基线。

[9] E. Glazer et al., "FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI," *arXiv preprint arXiv:2411.04872*, 2024.

[10] E. Glazer et al., "FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI," *arXiv preprint arXiv:2411.04872*, 2024.

- **MBPP**（Mostly Basic Python Problems）：974 道入门级 Python 编程题。比 HumanEval 更多样但也更简单。
- **SWE-bench**（真实 GitHub issue 修复）[11]：给模型一个真实的 GitHub issue 和整个代码仓库，要求生成修复 patch。这是目前最接近**真实软件工程**场景的基准——需要理解上千行代码、定位 bug、生成正确的修复。前沿模型的通过率从 2024 年初的 ~5% 提升到 2025 年底的 ~50%（在 SWE-bench Verified 上）。
- **LiveCodeBench**：持续收集编程竞赛平台（LeetCode、Codeforces）的新题，避免数据泄露。题目有明确的时间戳，可以按模型训练截止日期进行公平比较。

SWE-bench：从函数到系统的代码评估革命

SWE-bench[12] 的意义在于 它将代码评估从**函数级**（HumanEval）提升到了**系统级**。一个典型的 SWE-bench 任务要求模型：

1. 阅读 GitHub issue 中用户的问题描述（通常含错误日志）。
2. 理解一个包含数百个文件、数万行代码的仓库结构。
3. 定位到 bug 所在的具体文件和函数。
4. 编写一个正确的 patch（包括代码修改和可能的测试更新）。
5. 通过仓库的既有测试套件验证。

SWE-bench 经历了三个版本的演进：

- **SWE-bench**（原版）：2294 个来自 12 个 Python 仓库的真实 issue。部分任务被发现存在评估问题（如测试不够严格导致错误方案通过）。
- **SWE-bench Verified**（2024 年 8 月）：由人类工程师审核的 500 个高质量子集。确保每个任务的测试用例足够严格。曾被广泛用作标准评估集。
- **SWE-bench Pro**（2026 年，提议中）：针对 Verified 版本出现的污染问题而提出的更新，使用私有测试用例和更严格的防泄露措施。

11.2.4 长上下文

- **RULER**：综合性长上下文评估。包含多种任务类型（检索、聚合、推理）和多种上下文长度（4K 到 128K+）。
- **Needle-in-a-Haystack**（NIAH）：在长文本中某个随机位置插入一个特定事实，然后在文本末尾询问这个事实。测试模型在不同上下文位置的信息检索能力。简单直观，但只测试了最基本的「能否找到信息」。

[11] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" In *International Conference on Learning Representations*, 2024.

[12] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" In *International Conference on Learning Representations*, 2024.

11.2.5 指令跟随与对话

- **MT-Bench**：80 道多轮对话题，由 GPT-4 评分（1–10 分）。涵盖写作、推理、数学、编码等 8 个类别。成本低，但受限于 GPT-4 自身的判断能力。
- **AlpacaEval**：805 道指令，由 GPT-4 与参考模型做 pairwise 比较。快速廉价，但存在「长度偏见」——GPT-4 评分器倾向于偏好更长的回答。AlpacaEval 2.0 引入了长度控制来缓解这个问题。
- **Arena-Hard**：从 Chatbot Arena 的真实用户问题中 筛选出最具区分度的 500 道题。使用 GPT-4-Turbo 做评判。比 MT-Bench 和 AlpacaEval 有更高的区分度。

11.2.6 动态基准：LiveBench

LiveBench：抗污染的动态基准

LiveBench[a] 采用了一种独特的策略来对抗数据泄露：它**定期更换测试题**，使用模型训练截止日期之后才出现的信息来构造问题。例如，使用最新的新闻文章、最新发布的数学竞赛题、最新的代码仓库 commit 来生成评估题目。这保证了没有模型能够通过训练数据「背答案」。代价是需要持续投入人力维护和更新测试集。

[a] C. White et al., "LiveBench: A challenging, contamination-limited LLM benchmark," in *International Conference on Learning Representations*, 2025.

LiveBench 于 2024 年发布，并作为 Spotlight Paper 入选 ICLR 2025，标志着动态评估范式获得了学术界的认可。其设计原则包括：

1. **客观可验证的答案**：每道题目都有明确的、可程序验证的正确答案，消除了对 LLM 评判者的依赖。这与 MT-Bench 和 AlpacaEval 依赖 GPT-4 评分形成对比。
2. **定期完全刷新**：测试题每 6 个月完全更换一次。旧题目公开用于研究，新题目取而代之。
3. **7 大类别、23 项任务**：涵盖数学推理、代码生成、数据分析、语言理解、指令跟随、逻辑推理、综合能力。
4. **天花板效应控制**：即使是最强的前沿模型，在 LiveBench 上的综合准确率也很少超过 65%，保持了良好的区分度。

11.2.7 Chatbot Arena：人类评估的金标准

LMSYS Chatbot Arena[13] 是目前最受信赖的 LLM 评估平台。它的设计哲学是：让真实用户用真实问题来评判模型。

核心流程

1. 用户提出**任意**问题——没有预设的题库，问题完全来自真实用户需求。

[13] LMSYS 即 Large Model Systems Organization，由 UC Berkeley SkyLab 的研究者于 2023 年创立。2025 年底，团队以 LMArena 的名义独立运营，并获得了超过 1 亿美元的融资。

2. 系统将问题同时发送给两个**匿名**模型。用户不知道自己在与哪个模型对话。
3. 用户看到两个回答，选择更好的那个（或选择平局，或选择「都不好」）。
4. 投票结束后揭示模型身份。
5. 基于所有投票，使用 Bradley-Terry 模型 计算每个模型的 Elo 评分。

Bradley-Terry 模型与 Elo 评分

Bradley-Terry 模型假设模型 i 击败模型 j 的概率是：

$$P(i \succ j) = \frac{e^{\beta_i}}{e^{\beta_i} + e^{\beta_j}} \tag{11.1}$$

其中 β_i 是模型 i 的能力参数。通过最大似然估计拟合所有投票数据，得到每个模型的 Elo 分数。

更具体地说，给定 N 次成对比较的数据 $\{(i_k, j_k, y_k)\}_{k=1}^{N}$，其中 $y_k \in \{i_k, j_k\}$ 表示胜者，对数似然函数为：

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{k=1}^{N} \left[\beta_{y_k} - \log\left(e^{\beta_{i_k}} + e^{\beta_{j_k}} \right) \right] \tag{11.2}$$

通过最大化此对数似然，可以估计所有模型的能力参数 $\boldsymbol{\beta}$。为了可解释性，最终的 Elo 分数通常线性变换到 均值约 1000–1200 的范围内。

为什么 Bradley-Terry 优于简单胜率?

直接计算胜率（Win Rate）的问题在于：不同模型被匹配的对手不同。一个模型如果总是和弱模型对比，胜率会虚高但实力并不强。Bradley-Terry 模型的优势在于它同时考虑了**所有**成对比较的传递性——如果 A 经常胜 B，B 经常胜 C，即使 A 和 C 从未直接比较，模型也能正确推断 A 应该强于 C。这使得即使对战矩阵是稀疏的（并非每两个模型都有足够的直接对战），也能得到可靠的全局排名。

Style Control：分离风格与实质

2024 年 8 月，Chatbot Arena 团队提出了 **Style Control**（风格控制）机制[14]，解决了一个重要的偏见问题：用户倾向于偏好更**长**的、使用更多 **Markdown 格式**的回答。这意味着一个输出更冗长但内容不一定更好的模型 可能获得更高的 Elo 分数。

Style Control 通过在 Bradley-Terry 模型中 加入**风格特征**作为协变量来缓解此问题：

$$P(i \succ j) = \sigma\left(\beta_i - \beta_j + \boldsymbol{\gamma}^\top (\mathbf{s}_i - \mathbf{s}_j)\right) \tag{11.3}$$

其中 σ 是 sigmoid 函数，$\mathbf{s}_i$ 是模型 i 的风格特征向量（包括回答长度、Markdown 使用率等），$\boldsymbol{\gamma}$ 是对应的系数。这样，β_i 就更纯粹地反映了模型的 **实质能力**而非风格偏好。

[14] T. Li et al., "Does style matter? Disentangling style and substance in Chatbot Arena," *LMSYS Blog*, 2024.

开启 Style Control 后的排名变化非常有启发性：某些以简洁著称的模型（如早期的 Claude）在 Style Control 排名中显著上升；而某些以冗长著称的模型则相应下降。

分类排行榜

Chatbot Arena 不仅提供总体排名，还按**能力类别**提供细分排行榜：

- **Hard Prompts**：从用户提交中筛选出特别困难的问题——通常是需要深度推理、专业知识或复杂指令跟随的问题。Hard Prompts 排名比总体排名更能区分前沿模型，因为简单问题上大多数好模型表现相似。
- **Coding**：编程相关问题的独立排名。
- **Math**：数学推理问题的独立排名。
- **Creative Writing**：创意写作能力排名。
- **Vision**：多模态（图文理解）能力排名。截至 2026 年初已积累超过 67 万次视觉评估。
- **多语言**：中文、日文、法文等非英语能力排名。

Chatbot Arena 的数据规模（截至 2026 年初）

- 累计超过 **800 万次**人类评估投票
- 覆盖超过 **200 个**模型
- 文本排行榜涵盖 100+ 模型，视觉排行榜涵盖 100+ 模型
- LMArena 独立运营并获得超过 1 亿美元融资
- 从 UC Berkeley 的学术项目演变为 AI 行业的基础设施级评估平台

Chatbot Arena 的优势在于：

- **问题来自真实用户需求**，不是研究者预设的。
- **评估标准由用户自定义**——用户根据自己的偏好判断「更好」，不受固定评分标准约束。
- **模型匿名**，消除了品牌偏见。
- **动态更新**，使得针对性优化极为困难。
- **统计显著性极高**——百万级的投票量使得 Elo 评分的置信区间非常窄。

其局限性也值得注意：

- **用户群体偏差**：Arena 的用户以技术人员为主，可能不代表普通用户的偏好。
- **单轮评估为主**：大多数投票基于单轮对话，无法完全反映多轮对话的质量。
- **位置偏差**：研究表明用户有轻微的偏好左侧回答的倾向，虽然 Arena 通过随机化位置来缓解。
- **无法评估安全性**：用户投票主要基于「有用性」，不会主动测试和评估安全性问题。

11.3 推理能力评估

随着 OpenAI o1/o3 系列和 DeepSeek-R1 等**推理模型**（reasoning models）的出现，传统的评估范式面临新的挑战。推理模型通过在推理时生成大量 thinking tokens 来进行深度思考（详见第 10 章），其能力特征与标准模型有显著差异——在需要多步推理的任务上表现极强，但在简单任务上可能反而因「过度思考」而表现更慢或更贵。

11.3.1 推理模型 vs 标准模型：评估差异

推理模型的独特评估挑战

推理模型改变了评估的基本假设：

1. **成本不再恒定**：标准模型每个问题消耗大致相同的计算量；推理模型根据问题难度动态调整，难题可能消耗 100 倍于简单题的 token。基准测试的排名需要考虑**计算效率**，而非仅仅是准确率。
2. **简单基准失去意义**：推理模型在 GSM8K 上达到 >99% 准确率，在 MMLU 上超过 90%——这些基准完全无法区分推理模型的优劣。评估推理模型需要**显著更难**的题目（AIME、GPQA Diamond、FrontierMath）。
3. **Thinking tokens 的可观察性**：一些推理模型（如 DeepSeek-R1）会输出完整的思考过程；另一些（如 o1 早期版本）隐藏了思考过程。这使得评估和比较推理质量变得复杂。

推理模型在不同基准上的表现模式值得关注。在「需要多步推理」的任务上，推理模型相对标准模型的提升幅度巨大：

- **AIME 2024/2025**：标准模型（如 GPT-4o, Claude 3.5 Sonnet）的通过率约为 10–20%，而推理模型（o3, R1）可以达到 80–90% 以上。
- **GPQA Diamond**：标准模型准确率约 50–60%，推理模型达到 70–93%，部分结果超越人类博士专家水平。
- **FrontierMath Tier 1–2**：标准模型通过率 <5%，推理模型可达 25% 以上。
- **SWE-bench Verified**：推理模型在复杂 bug 修复上 相比标准模型有 10–15 个百分点的提升。

但在「不需要深度推理」的任务上（如创意写作、简单问答、翻译），推理模型的表现与标准模型相当甚至稍弱——过多的 thinking tokens 有时反而引入不必要的复杂性。

11.3.2 过程奖励模型 vs 结果奖励模型

评估推理过程（而非仅评估最终答案）是推理模型训练和评估中的核心问题。两种主要范式形成了鲜明的对比：

结果奖励模型（ORM）

结果奖励模型（Outcome Reward Model, ORM）只关注最终答案是否正确：

$$R_{\text{ORM}}(\text{solution}) = \begin{cases} +1 & \text{if final answer is correct} \\ -1 & \text{if final answer is wrong} \end{cases} \tag{11.4}$$

ORM 的优势是**标注简单**——只需验证最终答案，不需要评估每一步推理的正确性。对于数学和代码等有明确正确答案的任务，ORM 的标注成本几乎为零（自动验证）。

但 ORM 存在严重的**信用分配问题**（credit assignment problem）：如果一个解题过程有 10 步，其中第 3 步出错导致最终答案错误，ORM 只知道「答案错了」，但不知道具体是哪一步出了问题。在使用 RL 训练时，错误的信用分配会导致学习效率低下——前两步（正确的）也可能被错误地惩罚，而第 3 步（错误的）可能没有被充分惩罚。

过程奖励模型（PRM）

过程奖励模型（Process Reward Model, PRM）[15] 对推理过程的**每一步**提供反馈：

$$R_{\text{PRM}}(\text{step}_t \mid \text{step}_{1:t-1}) \in [-1, +1] \tag{11.5}$$

其中每一步的奖励取决于该步骤 在之前所有步骤的上下文中是否正确。

PRM 解决了信用分配问题——它能精确指出推理链中**哪一步**出错，为 RL 训练提供更密集、更准确的学习信号。OpenAI 在 2023 年的研究[16] 表明 PRM 引导的搜索（Best-of-N with PRM）在数学问题上显著优于 ORM 引导的搜索。

PRM 为什么更有效?

直觉上，PRM vs ORM 的差异 类似于老师批改作业的两种方式：

- **ORM**：只看最终答案，打对号或叉号，不做过程批注。学生知道答案错了，但不知道错在哪里。
- **PRM**：逐步检查，在每一步标注对错。学生能精确知道自己在哪一步走偏了。

后者显然提供了更多的学习信息，但**批注成本**（标注成本）也高得多。

PRM 的标注瓶颈与自动化

PRM 面临的核心挑战是**标注成本**：逐步评估推理过程需要大量的人力。OpenAI 的 PRM800K 数据集[17] 包含了 80 万步的人类标注，成本极高且难以扩展。

[15] H. Lightman et al., "Let's verify step by step," in *International Conference on Learning Representations*, 2024.
[16] H. Lightman et al., "Let's verify step by step," in *International Conference on Learning Representations*, 2024.
[17] H. Lightman et al., "Let's verify step by step," in *International Conference on Learning Representations*, 2024.

Setlur et al. (2025)[18] 在 ICLR 2025 上提出了 **自动化过程验证**（Automated Process Verification）的方法。核心思想是利用结果监督来**自动生成**过程标签：

1. 给定推理链的前 t 步，让模型多次独立完成剩余步骤。
2. 如果从第 t 步之后继续推理 **大概率能得到正确答案**，那么第 t 步很可能是正确的。
3. 如果从第 t 步之后继续推理 **几乎总是得到错误答案**，那么第 t 步很可能是错误的。

这种方法通过蒙特卡洛估计每一步的「正确概率」，将不可扩展的人工逐步标注 转化为可自动化的过程，使 PRM 训练的规模化成为可能。

Yang et al. (2025)[19] 进一步指出了现有 PRM 的一个局限：在长推理链中，标注倾向于只关注**第一个错误步骤** 及其之前的步骤，而忽略了后续步骤中可能的**自我纠正**。他们提出了更灵活的「反思式」PRM，能够评估模型在发现错误后的纠正能力。

PRM 与 ORM 的实际选择

PRM vs ORM：实际应用指南

- **数学/逻辑推理**：PRM 显著优于 ORM。推理链长、每步可验证、信用分配问题严重——PRM 的优势最大化。
- **代码生成**：ORM 通常足够。有单元测试作为准确的结果信号，而代码的「步骤」边界不如数学清晰。
- **开放式文本生成**：两者都很困难。既没有明确的正确答案（ORM 难），也没有可验证的中间步骤（PRM 难）。LLM-as-Judge 或人类评估更适合。
- **混合方法**：实践中，很多系统结合 ORM 和 PRM——PRM 用于引导搜索方向，ORM 用于最终结果验证。

11.3.3 推理模型的专项基准

推理模型的快速发展催生了一系列新的评估基准，专门针对需要深度推理的任务：

- **AIME 2024/2025**：美国数学邀请赛真题是评估推理模型数学能力的标准基准。题目需要创造性的多步推理，而非模式匹配。o3 在 AIME 2024 上的得分超过 90%，接近人类数学竞赛选手的顶尖水平。
- **GPQA Diamond**：198 道博士级科学问题（见 11.2.7 节）。推理模型在此基准上的准确率 已从 2024 年初的约 50% 提升到 2026 年初的约 90%。
- **FrontierMath Tier 4**：研究级数学问题（见上文）。目前唯一仍对推理模型构成真正挑战的数学基准。
- **EpochAI Frontier Probes**：专门设计来测量推理能力边界的题目集。

[18] A. Setlur et al., "Rewarding progress: Scaling automated process verifiers for LLM reasoning," in *International Conference on Learning Representations*, 2025.

[19] Y. Yue et al., "Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?" In *Advances in Neural Information Processing Systems*, vol. 38, 2025.

11.4 前沿模型基准对比

为了呈现 2025–2026 年 LLM 能力格局的全貌，我们汇总了主要前沿模型在核心基准上的表现[20]。

Table 11.1: 2025–2026 年前沿模型基准测试对比

模型	类型	MMLU-Pro	GPQA-D	SWE-bench V	Arena Elo
推理模型（Reasoning Models）					
GPT-5.2 Pro	推理	88.7%	93.2%	65.8%	1402
Claude Opus 4.6	混合	88.2%	89.0%	72.5%	1504
DeepSeek V3.2-S	推理	85.9%	85.3%	77.8%	1361
Grok 4 Heavy	推理	86.4%	88.9%	61.0%	1375
标准模型（Standard Models）					
Gemini 3 Pro	标准	89.8%	87.5%	63.2%	1485
Claude Opus 4.5	标准	87.1%	86.5%	68.4%	1370
GPT-5.2	标准	86.3%	88.0%	58.2%	1380
Qwen 3.5	标准	84.6%	82.1%	62.5%	1342
开源模型（Open-Weight Models）					
Llama 4 Maverick	标准	83.2%	78.5%	55.8%	1320
DeepSeek R1	推理	90.8%	71.5%	49.2%	1310
Qwen 3.5	标准	84.6%	82.1%	62.5%	1342
Mistral 3	标准	82.8%	79.3%	54.1%	1315

注：GPQA-D = GPQA Diamond; SWE-bench V = SWE-bench Verified; Arena Elo 数据来自 Chatbot Arena 排行榜，截至 2026 年 2–3 月。“推理” 类型指使用 thinking tokens 的模型；“混合” 指同时支持标准和推理模式。同一模型在不同评测条件下可能给出不同分数。

从表 11.1 可以观察到几个值得注意的模式：

推理模型在 GPQA 上优势巨大：推理模型在需要深度科学推理的 GPQA Diamond 上普遍超过 85%，部分超过 90%，远高于标准模型的 78–88%。这证实了 thinking tokens 在复杂推理任务上的价值。

SWE-bench 上开源模型追赶迅速：DeepSeek V3.2-Speciale 在 SWE-bench Verified 上达到 77.8%，超过了多数闭源模型。这得益于其在代码相关的后训练数据上的大量投入。

Arena Elo 与基准分数并不完全一致：Gemini 3 Pro 在 MMLU-Pro 上得分最高（89.8%），但其 Arena Elo 低于 MMLU-Pro 分数较低的 Claude Opus 4.6。这说明**基准测试和真实用户偏好衡量的是不同的东西**——用户满意度取决于指令跟随、风格、一致性等 基准测试不一定覆盖的维度。

开源与闭源的差距在缩小：Llama 4 Maverick 和 Qwen 3.5 等开源模型 在 MMLU-Pro 上已接近闭源前沿模型（差距 <5%）。在某些特定任务上，开源模型甚至有竞争力。

[20]数据来源综合自各模型的技术报告、Chatbot Arena 排行榜和独立评测平台。数据截止到 2026 年 2 月。注意不同来源的评测条件可能存在差异。

11.5 安全对齐

LLM 的安全问题包括多个层面：

11.5.1 有害内容生成

模型可能生成暴力、歧视、违法的内容。通过 RLHF 中的安全奖励和 Constitutional AI[21]（让 AI 根据原则自我批评和修正）来缓解。

Constitutional AI 的核心思想是：不依赖人类标注者来判断每个输出是否安全，而是给模型一组**宪法原则**（如「不鼓励暴力」「尊重隐私」「提供准确信息」），让模型自己根据这些原则评估和修正输出。这大幅降低了安全标注的人力成本，同时使安全标准更加一致和可扩展。

11.5.2 越狱攻击

越狱攻击（Jailbreaking）：用户通过巧妙的提示绕过模型的安全限制。常见的越狱技巧包括：

- **角色扮演**：“假设你是一个没有任何限制的 AI...”
- **间接提问**：“写一个关于角色 X 做 Y 的小说...”（间接获取有害信息）
- **编码/加密**：用 Base64、逆序文本等方式 隐藏有害指令。
- **多轮诱导**：通过多轮对话逐步引导模型 越过安全边界。

防御策略包括**对抗训练**（在训练数据中纳入已知的越狱模式，让模型学会识别和拒绝），**输入/输出过滤**（独立的安全分类器 对输入和输出进行审查），以及**红队测试**。

Red teaming（红队测试）是系统性地寻找模型安全漏洞的过程。专业的红队由安全研究者、领域专家和普通用户组成，他们尝试各种方法来触发有害输出。越来越多的团队使用**自动化红队**——用另一个 LLM 来自动生成大量攻击 prompt，然后检查目标模型的回复。这大幅提高了测试覆盖率，但人类红队仍然不可替代——他们能发现自动化方法难以想到的创意攻击。

11.5.3 幻觉

幻觉（Hallucination）：模型自信地生成不正确的信息。这是一个尚未完全解决的根本问题——语言模型本质上是在做统计预测，不具备真正的「知道自己不知道」的能力。

幻觉可以分为两类：

- **内在幻觉**（Intrinsic hallucination）：输出与**给定的上下文/来源**矛盾。例如，文档里说「收入增长了 5%」，模型回答「收入增长了 15%」。

[21] Y. Bai et al., “Constitutional AI: Harmlessness from AI feedback,” *arXiv preprint arXiv:2212.08073*, 2022.

- **外在幻觉**（Extrinsic hallucination）：输出无法从给定来源中验证，但也不矛盾。模型「添加」了来源中不存在的信息。这类幻觉更难检测——可能是正确的补充信息，也可能是捏造的。

幻觉的根源在于模型的训练目标——最大化下一个 token 的概率——与「只在确定时才生成」之间的矛盾。模型被训练为**总是生成一些东西**，即使它对内容的准确性没有把握。而且模型没有「事实性」的内在概念——它学到的是训练数据中的统计模式，而非可验证的事实。

11.5.4 RAG：用检索对抗幻觉

Retrieval-Augmented Generation（RAG）通过在生成前检索外部知识来部分缓解幻觉问题。

RAG 的基本流程是：

1. **检索**：用户提问 → 将问题向量化 → 在向量数据库中检索最相关的文档块。
2. **增强**：将检索到的文档块与用户问题拼接，作为上下文送入 LLM。
3. **生成**：LLM 基于检索到的上下文生成回答，回答有了可追溯的「来源」。

RAG 的关键技术组件：

- **Chunking**（文档切分）：将长文档切成固定大小的块（如 512 tokens）。切分策略影响检索质量——太大则不精确，太小则失去上下文。重叠切分（相邻块有 10–20% 的重叠）可以缓解边界问题。
- **Embedding models**（嵌入模型）：将文本转换为高维向量。常用的包括 OpenAI text-embedding-3、BGE、E5-Mistral 等。嵌入质量直接决定检索质量。
- **Vector databases**（向量数据库）：存储和检索高维向量。常用的包括 Pinecone、Weaviate、Milvus、Chroma 等。核心操作是**最近邻搜索**（ANN）。

但 RAG 也不是万灵药——如果检索到的文档本身有误，或者检索不到相关信息，幻觉仍然可能发生。此外，模型有时会**忽略**检索到的信息，仍然依赖自己的「记忆」生成回答——这在检索结果与模型内在知识矛盾时尤为常见。

11.5.5 隐私与安全评估

隐私泄露：模型可能记住并泄露训练数据中的敏感信息。差分隐私训练和去敏处理是主要的缓解手段。

安全评估基准：

- **TruthfulQA**：测试模型是否会复述常见的错误信息和迷思。817 个问题涵盖健康、法律、金融等领域的常见误解。
- **BBQ**（Bias Benchmark for QA）：测试模型在不同人群（种族、性别、年龄等）上的偏见。
- **ToxiGen**：测试模型生成有毒内容的倾向。涵盖针对 13 个少数群体的隐含和显性有毒语言。

- **AdvBench**：对抗性提示集合，测试模型在面对刻意诱导时的安全性。

11.5.6 对齐税

安全对齐不是免费的——让模型更安全通常会**略微降低**其原始能力。这被称为 **alignment tax**（对齐税）。

例如，过度的安全训练可能导致模型**过度拒绝**——拒绝回答完全合法合理的问题（如「如何制作鸡尾酒」被错误地归类为有害问题）。这降低了用户体验。

能力与安全之间存在一个 **Pareto 前沿**——在给定的技术水平下，不可能同时最大化两者。更好的对齐技术（如 Constitutional AI、DPO）可以将这个前沿向外推——在相同安全水平下保留更多能力，或在相同能力水平下实现更好的安全性。

安全对齐不是一次性的工作，而是一个持续的过程——随着模型能力的增强和新攻击方式的出现，安全机制需要不断迭代更新。

11.6 评估的实践智慧

在实际的模型开发过程中，评估团队积累了许多经验教训：

11.6.1 持续评估

评估要在训练过程中持续进行，而不是等到训练结束后一次性评估。通常每 1000–5000 步在一组核心基准测试上评估一次，绘制「能力曲线」——观察不同能力在训练过程中的变化趋势。有时会发现某种能力在训练中期突然出现（涌现），或者某种能力随着训练进行反而退化（灾难性遗忘）。这些信号可以指导数据配比和训练策略的调整。

11.6.2 能力维度分析

不同基准测试衡量不同能力维度。MMLU 主要测试**知识记忆**——模型在训练数据中是否见过相关知识。GSM8K 和 MATH 测试**推理能力**——模型能否进行多步逻辑推导。HumanEval 和 SWE-bench 测试**代码能力**——前者是函数级代码生成，后者是系统级 bug 修复。LMSYS Chatbot Arena 测试**综合实用性**——真实用户在盲评中更偏好哪个模型。

一个模型可能在某些维度上很强，在另一些维度上很弱——这就是为什么单一基准测试分数永远不够。完整的评估需要**多维度的能力画像**。

11.6.3 数据泄露防范

基准测试的「数据泄露」是一个严肃问题。如果测试题目出现在训练数据中，模型可能只是在「背答案」而非真正理解。防御措施包括：使用 contamination detection （检查训练数据中是否包含测试题）、定期更新基准测试、以及使用动态生成的评估问题。

一个实用的检测方法是**扰动测试**：对基准测试题做微小的修改（改变数字、替换名称），如果模型的准确率大幅下降，说明它可能只是记住了原题的答案，而没有真正理解解题方法。

11.6.4 评估金字塔

评估金字塔：从自动到人工

1. **自动化指标**（底层，最便宜）：Perplexity、BLEU、ROUGE 等。快速、廉价、可重复，但与人类判断相关性有限。适合训练过程中的快速检查。
2. **基准测试套件**（中层）：MMLU、HumanEval、GSM8K 等标准化测试。覆盖多个能力维度，有历史可比性。适合模型之间的系统性比较。
3. **LLM-as-Judge**（中上层）：用 GPT-4 等强模型评估其他模型的输出。比人类评估便宜得多，与人类判断相关性较高。但存在自我偏见（倾向偏好自己风格的输出）。
4. **人类评估**（顶层，最贵但最可靠）：Chatbot Arena 式的盲评。直接衡量用户满意度。适合最终的模型质量判断。
5. **生产指标**（最终裁判）：用户留存率、任务完成率、NPS。真实场景中的表现是唯一的最终标准。

11.6.5 建立针对性评估

公共基准测试衡量的是通用能力，但实际应用场景往往有独特的需求。**建立针对你自己用例的评估套件** 比追逐公共基准分数更有价值。

例如，如果你在用 LLM 做客服机器人，你应该建立一个包含真实客户问题的评估集，用领域专家标注正确答案，然后衡量模型在**你的**场景中的表现。公共基准上多 2 个百分点不如 你的客服评估集上多 5 个百分点有意义。

11.6.6 A/B 测试

在生产环境中，**A/B 测试是最终的评估方式**。将两个模型版本随机分配给用户，比较关键的业务指标——用户满意度、任务完成率、对话轮次等。这是最接近「真相」的评估，因为它直接衡量了模型在真实场景中的表现。

11.6.7 回归测试

新模型版本在改进某些能力的同时，可能在其他方面**退化**——这被称为**能力回归**。例如，一个更强的数学推理模型 可能在创意写作上反而变差了。建立**回归测试套件**——一组覆盖所有关键能力的测试用例——可以在每次模型更新时快速检查是否有能力退化。

人类评估仍然是金标准。LMSYS Chatbot Arena 的 Elo 排名系统 让真实用户对两个匿名模型的回答做盲评。截至 2026 年初，已经积累了超过 800 万次人类评估，是判断

模型综合能力最可靠的指标。它的优势在于：问题来自真实用户需求，评估标准由用户自定义（不是研究者预设的），而且动态更新使得「针对性优化」变得困难。

CHAPTER 12

团队与组织——谁在训练这些模型?

训练一个 frontier LLM 不仅是技术挑战，更是一个复杂的组织工程。它需要多个专业团队的紧密协作，每个团队都面临独特的技术和管理挑战。一个常见的误解是认为 LLM 训练的核心是算法研究——事实上，工程执行占据了整个工作量的 80% 以上。

12.1 核心团队

12.1.1 数据团队：被低估的英雄

数据团队是 LLM 训练中最被低估、但影响力最大的团队。一个在业界广泛流传的经验法则是：数据工程占据了整个训练工作量的 60–80%。模型架构可以从论文中复制，训练框架可以用开源工具，但高质量的数据流水线必须从零构建。

数据团队的核心角色包括：

- **数据管道工程师**（Data Pipeline Engineers）：构建和维护从网页爬取到最终训练数据的全流程基础设施。这包括爬虫集群管理、数据清洗 pipeline（通常基于 Spark 或 Dask 构建）、去重系统（MinHash + LSH 在 PB 级数据上的高效实现）、以及数据版本管理（类似 Git LFS 但针对 TB 级数据集的版本控制）。这些工程师需要同时精通分布式系统和 NLP。
- **数据质量分析师**（Data Quality Analysts）：定义和维护数据过滤标准——什么样的文本是「高质量」的？这不是一个纯技术问题。perplexity 过低可能意味着模板化文本，过高可能意味着乱码。重复率、特殊字符比例、段落结构完整性——每一个过滤规则都需要在大量样本上验证，确保不会误杀有价值的数据。
- **语言学家**（Linguists）：对于多语言模型，语言学家的角色至关重要。分词器的设计需要考虑不同语言的形态学特征（中文的字/词粒度、日语的假名与汉字混合、阿拉伯语的从右到左书写和词根系统）。语言学家还负责评估不同语言的数据质量标准——适用于英语的 perplexity 过滤阈值不能直接套用到中文。
- **标注管理者**（Annotation Managers）：协调人类标注员团队，管理标注质量。在 SFT 和 RLHF 阶段，高质量的人类标注数据是稀缺资源。标注管理者需要设计标注指南、培训标注员、建立质量控制流程（inter-annotator agreement 监控、抽样审核、异常检测），并在成本和质量之间寻找平衡。

在一个主要 AI 实验室中，数据团队的规模通常在 10–30 人。DeepSeek 在其技术报

告[1]中披露了 对数据配比的大量实验——网页、代码、学术论文、数学文本的最优比例是通过在小模型上进行数百次 proxy 实验确定的。这种工作量是纯算法研究无法覆盖的。

为什么「数据质量 > 数据数量」?

FineWeb[a] 的实验表明，经过精细过滤的 1.3T tokens 数据集训练出的模型，在多个基准测试上优于使用 4T tokens 低质量数据训练的模型。直觉解释是：低质量数据不仅「浪费」了训练步数，还可能引入噪声模式（如 HTML 标签、广告文本、机器翻译痕迹），这些模式在推理时会以微妙的方式降低输出质量。这就是为什么数据团队的工作如此关键。

[a] G. Penedo et al., "The FineWeb datasets: Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024.

12.1.2 基础设施团队：让数千块 GPU 协同工作

基础设施（Infra）团队负责 GPU 集群的管理与优化。这个团队面临的核心挑战可以用一个数字概括：在一个 16,384 块 GPU 的集群中，假设每块 GPU 每天的故障率为 0.1%，那么平均每天会发生 $16384 \times 0.001 \approx 16$ 次 GPU 故障。这意味着**平均每 90 分钟就有一块 GPU 出问题**。

Infra 团队的核心职责包括：

- **集群管理与任务调度**：使用 SLURM（传统 HPC 集群的标配）或 Kubernetes 管理训练任务的调度。关键挑战是资源隔离——当多个训练任务共享集群时，如何确保网络带宽不互相干扰？如何在节点故障时自动重新调度而不中断其他任务？
- **故障检测与自动恢复**：这是 Infra 团队最核心的能力。自动检测 GPU 故障（CUDA error、ECC 错误、NVLink 断开）、网络故障（InfiniBand/RoCE 链路中断）、存储故障（NFS/Lustre 挂载点失效）。检测到故障后，系统需要自动隔离故障节点、将训练任务迁移到备用节点、从最近的 checkpoint 恢复训练——整个过程最好在 5–10 分钟内完成。
- **Checkpoint 管理**：对于 70B 参数的模型，每个完整训练 checkpoint 的大小约为 $70 \times 10^9 \times 8 = 560\,\text{GB}$ （包含 BF16 参数、FP32 master weights 和 Adam 优化器状态 m/v，详见附录）。如果每 1000 步保存一次，保留最近 10 个 checkpoint，就需要 5.6 TB 的高速存储空间。Infra 团队需要设计 checkpoint 的写入策略（异步写入，不阻塞训练）和存储架构（通常使用 Lustre 或 GPFS 这样的并行文件系统，提供数十 GB/s 的聚合带宽）。
- **网络监控与优化**：分布式训练的通信量巨大。在 tensor parallelism 中，每个 forward 和 backward pass 都需要 all-reduce 操作。Infra 团队需要监控网络拓扑中的热点、检测慢节点（stragglers）、优化 NCCL 的通信参数（如环形 vs. 树形 all-reduce、通信与计算的重叠度）。

[1] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

DeepSeek 的基础设施智慧

DeepSeek[a] 在 H800 集群（受出口管制限制，NVLink 带宽低于 H100）上训练 V3 模型，通过软件层面的创新弥补了硬件劣势：DualPipe 通过更精细的计算-通信重叠减少了 pipeline bubble，DeepEP 则为 MoE 的 expert parallelism 设计了专用的 all-to-all 通信内核。他们实现了 "zero irrecoverable loss spikes"，即整个训练过程中没有一次不可恢复的 loss 异常——这需要极其成熟的容错基础设施。

[a] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

12.1.3 预训练研究团队：训练的「配方师」

预训练研究团队负责回答一系列关键问题：模型应该多大（多少层？多少注意力头？多大的 hidden dimension?）；训练应该怎么做（学习率多少？batch size 多大？warmup 多长？数据怎么混合?）。

这个团队的工作方法是**先在小规模上验证，再推广到大规模**。典型的工作流程：

1. 在 1B–7B 的 proxy model 上做超参数搜索，每组实验训练数小时到数天。
2. 利用 Scaling Laws[2] 将小规模的最优配方推算到目标规模（如 70B 或 671B）。
3. 在目标规模上做少量验证实验（训练数千步，观察 loss 曲线是否符合预期）。
4. 确认配方后启动完整训练，通常持续数周到数月。

「训练配方」（training recipe）是每个实验室的核心知识资产。它包括但不限于：**学习率调度**（cosine decay 的起点、终点和 warmup 步数）、**batch size 调度**（从小 batch 逐渐增大，还是全程使用固定 batch size?）、**数据配比调度**（训练前期多用网页数据建立通用能力，后期增加代码和数学数据提升推理能力）、**长上下文扩展策略**（何时从 4K 切换到 128K，RoPE 频率基数如何调整）。

LLaMA 3[3] 的训练报告 详细披露了他们的训练配方：学习率峰值 1.5×10^{-4}，cosine 衰减到峰值的 1/30，warmup 8000 步，batch size 从 4M tokens 逐步增加到 16M tokens。这些数字看似简单，但每一个都是大量实验的结果。

12.1.4 后训练/对齐团队：定义「好」的含义

后训练团队的工作是将一个「博学但不可控」的 base model 转变为一个「有用、无害、诚实」的 assistant。这是整个训练流程中最依赖人类判断的环节。

核心职责包括：

- **RLHF/DPO 训练管道**：设计和实现偏好优化的训练流程。收集人类偏好数据（A 回答 vs. B 回答，哪个更好?），训练 reward model 或直接使用 DPO/GRPO 优化。关键挑战是偏好数据的一致性——不同标注员对「更好」的定义可能不一致。

[2] J. Hoffmann et al., "Training compute-optimal large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022, J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[3] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

- **Constitutional AI 设计**：受 Anthropic[4] 启发，一些团队使用 AI 自身来生成和评估训练数据，基于一组预定义的「宪法原则」。这减少了对人类标注的依赖，但需要精心设计原则体系——什么是「有害的」？什么程度的帮助是「过度的」？
- **Red teaming 与安全评估**：组建专业的红队，系统性地测试模型的安全边界。红队会尝试越狱攻击（jailbreaking）、提示注入（prompt injection）、社会工程学手段（伪装成合法用户请求危险信息）。每一个发现的漏洞都需要通过额外的对齐训练来修复，然后再次测试——这是一个持续的迭代过程。
- **推理能力训练**：对于 reasoning model（如 DeepSeek-R1[5]），后训练团队需要设计 RL 训练流程，让模型学会长链推理（chain-of-thought）、自我验证和错误修正。这需要精心设计的 reward signal——数学问题可以用答案正确性作为 reward，但开放性问题的 reward 设计要困难得多。

对齐的核心困难

后训练团队面临的最根本挑战是：「**有用、无害、诚实**」**这三个目标之间存在张力**。过度追求无害会导致模型过度拒绝合理请求（over-refusal）；过度追求有用会导致模型协助有害行为；过度追求诚实会导致模型在不确定时拒绝给出有用的近似回答。找到这三者的平衡点，本质上是一个价值观设计问题，没有唯一正确的答案。

12.1.5 评估团队：度量你所关心的

评估团队的工作是回答一个看似简单但极其困难的问题：**模型到底有多好?**

核心职责包括：

- **Benchmark 套件管理**：维护覆盖多个能力维度的评估基准集合——数学推理（MATH、GSM8K）、代码生成（HumanEval、SWE-bench）、知识问答（MMLU）、指令遵循（IFEval）、长文本理解、多语言能力等。每个 benchmark 都需要持续维护——防止数据泄露（训练数据中包含了测试集）、更新过时的题目、处理评估格式变化。
- **内部评估管道**：构建自动化的评估基础设施。每次生成新的 checkpoint，系统需要自动在数十个 benchmark 上运行评估，生成对比报告。这本身就需要大量 GPU 资源——对 70B 模型在 MMLU 的 14K+ 题目上做推理 可能需要数小时。高效的评估管道（如 lm-evaluation-harness 或 Lighteval）和评估集群的合理调度至关重要。
- **人类评估与 A/B 测试**：Benchmark 分数不能完全反映用户体验。评估团队还需要组织人类评估——让真实用户或专业评估者对比不同版本的模型输出，给出偏好判断。Chatbot Arena（LMSYS）的 Elo 评分系统 是目前最受认可的人类评估方法。
- **评估的局限性**：评估团队最重要的认知是：**没有完美的评估**。模型可能在所有现有 benchmark 上表现优异，但在真实场景中出现令人意外的失败。评估团队需要

[4] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[5] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

持续设计新的评估方法，特别是针对 benchmark 无法捕捉的能力（如长期规划、常识推理、文化理解）。

12.1.6 推理/部署团队：连接模型与用户

推理/部署团队负责将训练好的模型变成用户可以使用的产品。这个团队的工作直接决定了用户体验和运营成本。

核心职责包括：

- **服务架构设计**：API 网关 → 负载均衡器 → 推理集群 → 模型注册中心。需要处理突发流量（某个功能在社交媒体上火了，请求量可能在数分钟内增长 10 倍）、优雅降级（部分节点故障时不影响整体可用性）、自动扩缩容（根据负载动态调整推理服务器数量）。
- **延迟与成本优化**：用户期望的首 token 延迟（TTFT）通常在 500ms 以内，生成速度期望在 30–100 tokens/s。同时，推理成本直接影响定价和利润。部署团队需要在量化级别（FP8 vs. INT4）、批处理策略（continuous batching 的 max batch size）、缓存策略（prefix caching 的命中率优化）之间找到最优平衡。
- **SLA 管理**：商业 API 需要保证 99.9% 以上的可用性（每月最多 43 分钟停机）。这需要多区域部署、灰度发布（新版本先在小流量上验证）、自动回滚机制。
- **滥用检测**：监控异常使用模式——大量请求尝试绕过安全限制、token farming（批量生成内容）、API key 泄露等。需要实时的异常检测系统和响应流程。

12.2 组织模式：从实验室到公司

观察最成功的 LLM 团队，一个共同特点是**研究与工程的深度融合**——写论文的人和写训练代码的人往往是同一批人。但不同组织在规模、文化和战略上有显著差异。

12.2.1 主要玩家深度解析

OpenAI（2026 年初已超过 7,000 人）经历了 AI 领域最戏剧化的组织演变。从 2015 年的非营利研究实验室，到 2019 年的 “capped profit” 混合结构，再到 2025 年底正式转型为营利性公司——每一步都伴随着巨大的争议。微软累计投资超过 130 亿美元，而 2025 年 OpenAI 在日本软银领投下完成了 400 亿美元的融资，估值达到 3,000 亿美元。组织上，OpenAI 在 2025–2026 年经历了显著的人才流动：联合创始人 Ilya Sutskever 离开创立了 Safe Superintelligence Inc.（SSI），多位安全团队核心成员出走。CEO Sam Altman 将 2025 年 8 月任命前 Instagram 负责人 Fidji Simo 为 COO 并在 2026 年 2 月升任 CEO（应用），自己则转为专注 AGI 研究和战略。OpenAI 的组织高度垂直整合——从基础研究（GPT 系列、o 系列推理模型）到消费产品（ChatGPT，月活超 4 亿）、到企业 API、到硬件（自研芯片计划与 Broadcom 合作），全部在一个屋檐下。内部团队结构包括：**Foundation Model** 团队（预训练与 scaling）、**Post-Training** 团队（RLHF、推理训练）、

Safety Systems 团队（红队与对齐）、**Applied AI/ChatGPT** 团队（产品工程）、以及新成立的 **Deep Research** 团队（长链推理与信息综合）。

Anthropic（2025 年约 1,500 人，2026 年持续快速增长）由前 OpenAI 研究副总裁 Dario Amodei 和他的姐姐 Daniela Amodei 于 2021 年创立，核心使命是 AI 安全研究。Anthropic 的技术路线以 Constitutional AI[6] 为基石——用 AI 自身而非纯人工来完成对齐工作。在 Claude 模型系列中，Anthropic 展现了独特的产品策略：2025 年推出了 Claude 3.7 Sonnet（首个混合推理模型，同时支持快速回答和 extended thinking 深度推理），2026 年初发布了 Claude Opus 4 和 Claude Sonnet 4（代号 Kryptonite 和 Borealis），在编程和推理任务上取得了领先地位。组织特点是安全团队在决策中有异常大的话语权——新能力的部署需要通过 Responsible Scaling Policy（RSP）评估，模型在发布前需要通过一系列 AI Safety Levels（ASL）评估。Anthropic 在 2025 年完成了 Google、Salesforce 和 Amazon 领投的多轮融资，估值超过 600 亿美元。值得注意的产品创新包括：Claude Code（CLI 级别的自主编程 Agent）、Computer Use（通过截图和键鼠操作桌面应用）、以及 Model Context Protocol（MCP，Agent 工具交互的开放标准）。

Google DeepMind（数千人，是规模最大的 AI 研究组织）2023 年由 Google Brain 和 DeepMind 合并而成，由 Demis Hassabis 领导。拥有最多的计算资源（数十万块 TPU 组成的超级集群）和最大的研究团队，但组织规模也带来了协调成本。Gemini 系列是其旗舰产品：Gemini 2.0 Flash（2024 年底）引入了原生工具使用和代码执行，Gemini 2.5 Pro（2025 年初）成为首个在 LMSYS Chatbot Arena 登顶的 Google 模型，而 Gemini 3（预计 2026 年中发布）将进一步推进原生多模态和长上下文能力。Google DeepMind 的独特优势是 **端到端的基础设施控制**——从芯片设计（TPU）到网络架构到训练框架（JAX/Flax）到云服务（Google Cloud）再到消费产品（Gemini App、Google Search 的 AI Overviews），全栈自研。但合并后的组织需要持续在「学术自由」（DeepMind 的传统）和「产品交付」（Google 的商业需求）之间寻找平衡。多个产品团队（Gemini App、Workspace AI、Cloud AI）有时会出现内部竞争。

Meta FAIR 及 GenAI 团队（合计 1,000+ 人）Mark Zuckerberg 做出了 AI 行业最大胆的战略选择：将 LLaMA[7] 系列开源。这不是慈善——开源是一种竞争策略：当你的模型成为行业标准，围绕它的生态系统（工具、微调模型、应用）就成为你的护城河。Meta 不靠卖 API 赚钱，而是通过 AI 能力提升其广告和社交产品的收入。LLaMA 4 系列（2025 年 4 月发布，包括 Scout 109B/17B-active 和 Maverick 400B/17B-active）采用了 MoE 架构，是 Meta 开源的首个 MoE 模型。组织上，Meta AI 分为两个核心部门：FAIR（Fundamental AI Research，基础研究）和 GenAI（生成式 AI 产品工程）。Yann LeCun 仍担任 Chief AI Scientist，负责 FAIR 的长期研究方向。

DeepSeek（约 150–200 名核心研究员和工程师）背靠量化对冲基金幻方量化（High-Flyer Capital），DeepSeek 是「小团队大影响」的典范。他们用不到 Meta FAIR 十分之一

[6] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[7] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023, A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

的人员，在 DeepSeek-V3[8] 和 R1[9] 上达到了 GPT-4 级别的性能。关键因素：极致的工程效率（MLA、DeepEP 等创新）、扁平的组织结构（减少管理层级，加速决策）、以及量化基金背景带来的对效率的极度重视。DeepSeek 的团队结构高度集成——没有独立的「数据团队」「Infra 团队」「研究团队」的明确边界，研究员同时负责算法设计和底层系统优化。创始人梁文锋（Wenfeng Liang）的管理风格 强调技术深度和第一性原理思考，团队成员大多来自顶尖高校和量化行业。2025 年 1 月 DeepSeek-R1 的发布（被称为 "China's Sputnik moment"）引发了全球对中国 AI 能力的重新评估。

Mistral（约 600–700 人，2025 年）欧洲最重要的 AI 公司，创始团队来自 Google DeepMind 和 Meta。Mixtral[10] 证明了 一个精干的团队可以在 MoE 架构上做出出色的工作。Mistral 的策略是专注于效率——不追求最大的模型，而是在给定规模下做到最好。2025 年，Mistral 推出了 Mistral Large 2 和 Pixtral（多模态），同时为欧洲企业提供了符合 EU AI Act 合规要求的定制部署方案，这成为其在欧洲市场的重要竞争优势。

其他值得关注的组织：**xAI**（Elon Musk 创立，2024 年部署了超过 10 万块 H100 组成的 "Colossus" 集群，推出了 Grok 3 系列模型）；**阿里巴巴通义**（Qwen 系列，特别是 Qwen 3 在开源多语言模型中表现突出[11]）；**AI2**（Allen Institute for AI，OLMo 系列是 真正意义上的完全开源模型——数据、代码、权重全部公开）；**Cohere**（专注企业市场，Command R+ 系列针对 RAG 和企业搜索优化）。

「小团队，大影响」的启示

DeepSeek（150–200 名研究员）的成功揭示了一个重要规律：在 LLM 开发中，**团队的工程文化和技术深度 比纯粹的人员规模更重要**。小团队的优势包括：（1）更快的决策速度——没有多层审批，实验想法从提出到验证可以在数天内完成；（2）更强的 ownership——每个人负责更大的模块，减少了沟通开销和责任模糊；（3）更高的人才密度——小团队可以只招最优秀的人，大团队不可避免地需要大量普通工程师做基础工作。Mistral 在 2023 年创立初期也以 50–80 人的精干团队 产出了 Mixtral 等有影响力的模型，印证了同样的规律。但随着商业化需求增长，Mistral 已扩展至 600–700 人（2025 年），说明规模化终究是从研究到产品的必经之路。一个有趣的对比：OpenAI 的 7,000+ 人产出了 GPT-4o/o3，DeepSeek 的 200 人产出了 V3/R1——在核心模型质量上，两者差距远小于团队规模的差距。

12.3 训练成本的演变：2024–2026

训练 frontier LLM 的成本正在快速上升，但效率的提升也同样惊人。理解这个双重趋势对于评估 AI 行业的可持续性至关重要。

[8] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[9] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[10] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[11] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

12.3.1 成本的量级

截至 2026 年初，已知的训练成本估算如下：

Table 12.1: Frontier 模型训练成本估算（2024–2026）

模型	训练时间	估算成本	备注
GPT-4 (2023)	~3 个月	$78M–100M+	早期估算；含 A100 集群的 GPU 租赁成本
Gemini Ultra (2023)	数月	$191M（估算）	使用 Google 自有 TPU v4 集群
LLaMA 3 405B (2024)	~54 天	$60M–100M（估算）	16,384 块 H100，Meta 自有集群
DeepSeek-V3 (2024)	~2 个月	$5.5M（GPU 租赁）	2,048 块 H800；不含前期研发成本
GPT-5 / Orion (2025)	数月	$300M–500M（传闻）	未经官方确认
下一代 frontier (2026)	—	$500M–1B+	行业预估；含多轮训练

DeepSeek-V3 报告的 $5.5M 成本 是 AI 行业的一个标志性数字——它证明了通过工程创新，训练成本可以比同等性能的竞品低一个数量级。但需要注意，这个数字仅包含最终训练的 GPU 租赁成本，不包含数据处理、前期实验、后训练和人员成本。完整的端到端成本可能在 $20M–30M 左右。

12.3.2 成本的构成

训练一个 frontier LLM 的总成本可以分解为：

1. **GPU 计算**（50–70%）：这是最大的成本项。以 2026 年初的价格，NVIDIA H100 的云端租赁价格约为 $2.00–3.50/GPU· 小时，新一代 B200 约为 $4.00–6.00/GPU· 小时。一次使用 8,192 块 H100 训练 60 天的成本：$8192 \times 60 \times 24 \times 2.50 \approx \$29.5M$。
2. **数据获取与处理**（5–15%）：网页爬取基础设施、数据标注（高质量 SFT 数据的标注成本约 $15–50/对话）、数据清洗的计算成本。
3. **人员**（15–25%）：AI 研究员的年薪在硅谷通常为 $300K–800K （含股权），顶尖人才超过 $1M。一个 30 人的核心训练团队的年人员成本可达 $15M–20M。
4. **基础设施**（5–10%）：网络设备、存储系统、电力和冷却。一个大规模 GPU 集群的年度电力成本可达数百万美元。
5. **失败的实验**（不确定）：这是最难量化但可能占比很高的成本。训练到一半发现数据质量问题、超参数配置不佳导致 loss 不收敛——这些失败的尝试消耗的计算资源有时比最终成功的训练还多。

2026 年的成本趋势

两个相反的力量正在塑造训练成本的走向：**向上的力量**——模型规模和训练数据量的持续增长，以及推理模型的 RL 训练需要更多的计算；**向下的力量**——更高效的架构（MoE 的稀疏激活将 FLOPs 降低 5–10 倍）、更好的硬件（B200 的 FP8 性能是 H100 的 2.5 倍，单位算力价格持续下降）、更聪明的训练方法（如 DeepSeek 的各种工程优化）。净效果是：训练同等质量模型的成本在下降，但 frontier 的定义不断被推高，所以绝对成本仍在上升。行业中流传一个说法："Training costs are dropping 10x per year for the same quality, but we keep raising the bar 10x per year."

12.3.3 云端 vs. 自建集群的经济学

选择使用云服务商的 GPU 还是自建集群，是每个 AI 团队面临的关键战略决策。

云端（Cloud）的优势：无需前期资本支出（CapEx），按使用量付费（OpEx），弹性伸缩（训练完成后释放资源），以及云服务商提供的托管服务（网络配置、故障自动替换、存储管理）。主要的云 GPU 提供商包括 AWS（P5 instances with H100/H200, Trn2 instances with Trainium2）、Google Cloud（A3 with H100, Cloud TPU v5p/v6e）、Microsoft Azure（ND H100/H200 instances）和 CoreWeave（GPU 密集型专用云）。2026 年初，spot instance 价格可以将 H100 成本降低至 $1.20–1.80/GPU· 小时，但 spot instances 可能被中断，对于需要持续运行数周的预训练任务不太适用。

自建集群（On-Premise）的优势：长期来看成本更低——Deloitte 的研究表明，大规模持续使用场景下 自建可以实现云端 60–70% 的成本。一块 H100 SXM 的采购价约为 $25,000–35,000，如果使用 3 年，每 GPU· 小时的折旧成本仅约 $1.00–1.30 （不含电力、冷却、运维人员成本）。此外，自建集群可以完全控制网络拓扑、存储架构和安全策略——对于处理高度敏感数据的组织至关重要。

关键的交叉点：根据 2026 年的 TCO（Total Cost of Ownership）分析，当 GPU 利用率持续超过 60–70% 且使用时间超过 18–24 个月时，自建集群的经济性通常优于云端。这就是为什么 Meta、Google、Tesla（xAI）等大型玩家 都选择自建——他们的 GPU 几乎 24/7 满载运行。而初创公司和学术团队通常选择云端——他们的使用是间歇性的，且无法承担前期的资本投入。

混合策略正在成为主流：许多中型 AI 公司采用 "base on-prem + burst to cloud" 的策略——自建一个满足日常需求的基础集群，在需要大规模训练时临时租用云 GPU 扩展容量。

Table 12.2: 云端 vs. 自建集群：关键对比（2026 年）

维度	云端	自建
前期投入	极低	数百万–数千万美元
单 GPU 成本/小时	$2.00–6.00（按需）	$1.00–1.80（含运维）
弹性	分钟级扩缩容	数月的采购周期
维护	服务商负责	需要专业运维团队
数据安全	受限于服务商政策	完全自主
3 年 TCO（1000 GPU）	$52M–105M	$35M–55M
适合谁	初创、学术、间歇性使用	大型实验室、持续训练

12.4 开源训练框架生态

不同的团队可以选择不同的训练框架，各有侧重。

12.4.1 预训练框架

Table 12.3: 主流 LLM 预训练框架对比

框架	维护方	规模	核心优势
Megatron-LM	NVIDIA	产业级	最成熟的 3D 并行实现（TP + PP + DP），大多数主要实验室的训练起点
DeepSpeed	Microsoft	产业级	ZeRO 优化器[12]，MoE 支持，与 HuggingFace 集成良好
PyTorch FSDP	Meta (PyTorch)	中大规模	原生 PyTorch API，使用门槛低，但在超大规模上不如 Megatron 优化彻底
TorchTitan	Meta (PyTorch)	中大规模	LLaMA 训练的参考实现，基于 DTensor 分布式原语
Nanotron	HuggingFace	研究级	代码简洁可读，适合理解训练流程，教学和研究用途
LitGPT	Lightning AI	中小规模	易用的训练框架，支持多种模型架构，上手快

Megatron-LM 是大规模预训练的事实标准。它实现了完整的 3D parallelism（tensor、pipeline、data parallelism），以及 sequence parallelism 和 expert parallelism 等扩展。几乎所有训练过 100B+ 模型的团队都在 Megatron 的基础上构建。它的缺点是代码复杂度高、学习曲线陡峭——但对于需要在数千块 GPU 上稳定运行数月的训练任务，没有更可靠的选择。

DeepSpeed 的核心贡献是 ZeRO 优化器，它通过分片 optimizer states、gradients 和 parameters 大幅降低了每块 GPU 的内存需求。DeepSpeed-Chat 提供了完整的 RLHF 训练流程支持。在微调场景下，DeepSpeed 的 ZeRO-Offload 甚至允许在单块 GPU 上微调数十亿参数的模型（通过将部分状态 offload 到 CPU 内存）。

PyTorch FSDP（Fully Sharded Data Parallel）是 PyTorch 原生的分布式训练方案，概念上类似于 DeepSpeed ZeRO Stage 3。它的优势是与 PyTorch 生态无缝集成，对于已有

PyTorch 代码库的团队迁移成本最低。

12.4.2 后训练/微调框架

对于后训练（SFT/RLHF/DPO），生态系统更加多样化：

- **TRL**（HuggingFace）：集成 SFT、DPO、PPO、GRPO 训练，与 Transformers 和 PEFT 无缝对接。社区最活跃，文档最完善。
- **OpenRLHF**：专注于 RLHF/GRPO 训练，支持 Ray + vLLM 的大规模分布式 RL。在需要将训练规模扩展到数百块 GPU 时，OpenRLHF 的分布式 RL 架构更有优势。
- **LLaMA-Factory**：一键式微调工具，支持 100+ 模型，提供 Web UI 和命令行两种方式。特别适合快速实验和原型验证。
- **ms-swift**（阿里巴巴 ModelScope）：覆盖 SFT、RL、推理、部署的全流程工具链。与中国的模型生态（Qwen、通义千问等）集成最好。
- **Axolotl**：微调专用工具，支持多种方法（LoRA[13]、QLoRA[14]、full fine-tuning、FSDP）。配置文件驱动，灵活度高。
- **Unsloth**：专注于极致的微调速度和内存优化。通过手写 Triton kernel 和自定义 autograd，在相同硬件上可以比标准实现快 2–5 倍。

12.5 开源社区生态：从工具到运动

开源 LLM 不仅是一种技术选择，更是一场改变 AI 领域权力结构的运动。2025–2026 年，开源社区的成熟度和影响力达到了新的高度。

12.5.1 HuggingFace：ML 领域的 GitHub

HuggingFace 已经成为 ML 领域的核心基础设施，类似于 GitHub 之于软件开发。截至 2026 年初，其平台托管了超过 100 万个模型和 30 万个数据集，月活跃用户超过 500 万。

其生态系统包括：

- **Model Hub**：托管数十万个预训练模型，支持一行代码下载和使用。模型卡（Model Card）标准化了模型文档，包括训练数据、评估结果、已知限制和许可证信息。
- **Datasets**：标准化的数据集托管和加载，覆盖数千个 NLP/CV 数据集。FineWeb（15T tokens 的高质量网页数据）和 FineWeb-Edu（教育质量过滤的子集）是 HuggingFace 对开源训练数据的重要贡献。
- **Transformers 库**：统一的 API 支持几乎所有主流模型架构，是学术界和工业界最广泛使用的 NLP 库。新模型发布后通常在数天内就会被集成。

[13] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[14] T. Dettmers et al., "QLoRA: Efficient finetuning of quantized language models," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

- **TRL + PEFT**：后训练和参数高效微调的标准工具。TRL 在 2025 年加入了 GRPO 支持，使得复现 DeepSeek-R1 的训练流程成为可能。
- **SmolLM 系列**：HuggingFace 自研的小模型家族（135M、360M、1.7B 参数），展示了数据质量在小模型训练中的关键作用。SmolLM2[15] 通过精心策划的训练数据（Cosmopedia 合成教科书、FineWeb-Edu、Stack 代码数据）在同等规模下达到了领先性能。
- **Open LLM Leaderboard**：自动化的模型评估排行榜，是社区衡量模型质量的重要参考。2025 年升级为 v2 版本，引入了更难的评估基准（GPQA、MUSR、BBH）以应对模型能力的快速提升。
- **Spaces**：免费托管模型 demo 的平台，研究者可以零成本展示自己的工作。

HuggingFace 的核心贡献是**降低了 ML 的准入门槛**。在 HuggingFace 之前，使用一个新模型需要：阅读论文 → 找到作者的 GitHub repo → 配置环境 → 下载权重 → 编写推理代码。在 HuggingFace 之后，这简化为三行 Python 代码。

12.5.2 本地推理生态

llama.cpp 和 GGUF 格式的出现 创造了一个完整的本地推理生态系统。llama.cpp 使用纯 C/C++ 实现，支持 CPU、Apple Silicon（Metal）、CUDA、Vulkan 等多种后端，使得在没有 NVIDIA GPU 的消费级硬件上运行 LLM 成为可能。

Ollama 和 **LM Studio** 进一步将本地推理的门槛降到了非技术用户可及的水平——一键安装、一键下载模型、开箱即用的聊天界面。这使得「人人都能在自己的电脑上运行 AI」从技术理想变成了现实。

社区围绕量化模型建立了活跃的生态：模型发布后通常在数小时内就会出现 GGUF 格式的各种量化版本（Q2_K、Q4_K_M、Q5_K_S、Q8_0 等），用户可以根据自己的硬件条件选择合适的量化级别。

12.5.3 社区模型：百花齐放

开源社区的模型产出已经形成了一个完整的层次结构：

第一梯队：实验室级开源——DeepSeek-V3/R1（MIT 许可证）、LLaMA 4[16]（Llama 许可证）、Qwen 3[17]（Apache 2.0）、Gemma 3（Google，开放权重）。这些模型的质量已经接近甚至达到闭源模型的水平。

第二梯队：完全开源——OLMo 2（AI2，数据 + 代码 + 权重全部公开）、SmolLM2（HuggingFace，完整训练流程可复现[18]）、MAP-Neo（M-A-P 社区，7B 全开源 bilingual

[15] L. Ben Allal et al., "SmolLM2: When smol goes big – data-centric training of a small language model," *arXiv preprint arXiv:2502.02737*, 2025.

[16] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[17] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[18] L. Ben Allal et al., "SmolLM2: When smol goes big – data-centric training of a small language model," *arXiv preprint arXiv:2502.02737*, 2025.

模型)。这些项目的核心价值不在于模型性能的绝对领先，而在于**可复现性**——研究者可以在完全相同的条件下重复实验。

第三梯队：社区微调——基于第一梯队的 base model，社区产出了大量针对特定场景的微调模型。仅在 HuggingFace 上，基于 LLaMA 的微调模型就超过 10 万个。这种长尾效应使得开源生态的实际影响力 远超任何单一模型。

开源 LLM 的「飞轮效应」

开源 LLM 生态形成了一个自我强化的飞轮：更好的模型 → 更多的用户 → 更多的微调和改进 → 更活跃的社区 → 更好的工具和基础设施 → 更低的使用门槛 → 更多的用户 → …HuggingFace、llama.cpp、Ollama 等工具的成功 证明了这个飞轮的力量。一个关键观察是：开源模型的性能与闭源模型的差距 从 2023 年的「代际差距」缩小到 2025–2026 年的「几个月的差距」——DeepSeek-R1 在某些数学推理任务上 甚至超越了同期的闭源竞品。

12.5.4 Chatbot Arena：社区驱动的评估

LMSYS 的 Chatbot Arena 是开源社区 对 LLM 评估最重要的贡献之一。它使用类似于国际象棋 Elo 评分的系统，让真实用户对不同模型的回答进行盲评。由于评估者不知道哪个回答来自哪个模型，且样本量足够大（数百万次投票），Chatbot Arena 的排名被认为是 最能反映真实用户偏好的评估方法——比任何单一 benchmark 都更可信。

12.6 从论文到产品：被低估的工程挑战

在学术界和媒体报道中，人们往往关注算法创新和基准测试分数，而忽略了将 LLM 从实验室推向产品所需的大量工程工作。

12.6.1 训练基础设施的可靠性

在 2048 块 GPU 上训练两个月，意味着 $2048 \times 60 \times 24 = 295$ 万 GPU· 小时。如果每块 GPU 每天有 0.1% 的故障率，那么平均每小时会有 ~2 块 GPU 出现故障。训练系统必须能够自动检测故障、隔离故障节点、从最近的检查点恢复训练。DeepSeek-V3[19] 的训练报告强调他们实现了 “zero irrecoverable loss spikes”——这不是运气，而是精心设计的容错机制的结果。

12.6.2 调试分布式训练

当 loss 在第 50,000 步突然发散，而你用了 4096 块 GPU，怎么找到 bug？这是分布式训练中最令人恐惧的场景之一。可能的原因：某块 GPU 的 ECC 内存错误导致梯度异常；某个数据 shard 包含异常样本（如超长文本或乱码）；通信超时导致部分梯度丢失；甚至

[19] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.

可能是浮点数非确定性——相同的 checkpoint 在不同的 GPU 配置下恢复训练，可能产生不同的 loss 曲线。

实践中的调试策略包括：详尽的日志系统（记录每一步的 loss、gradient norm、learning rate、每块 GPU 的计算时间）；“canary” 训练（在小集群上用相同配方并行运行，比较 loss 曲线是否一致）；以及最重要的——**可重现性**，或者更准确地说，对不可重现性的接受和管理。

12.6.3 数据管道的效率

2048 块 GPU 的训练每秒消耗数百万 token，数据加载不能成为瓶颈。这需要高效的数据格式（如 memory-mapped 二进制文件）、多级缓存（SSD → CPU 内存 → GPU 内存）和异步预取（在 GPU 计算时提前准备下一批数据）。

12.6.4 评估的自动化

每次 checkpoint 需要在数十个基准测试上评估，每个测试可能需要数百到数千次推理调用。这本身就需要大量 GPU 资源。高效的评估管道（如 lm-evaluation-harness）和评估集群的合理调度至关重要。

12.6.5 人的因素

最后一个常被忽略的挑战是**人**。一次 frontier model 的训练可能消耗数千万美元的计算资源。如果训练在第六周因为一个 bug 失败而需要重启，这不仅是经济损失，也是巨大的心理压力。负责训练的工程师需要 7×24 小时 on-call——凌晨 3 点收到 loss spike 的告警是常态。燃尽（burnout）是 LLM 训练团队面临的真实挑战。

这些工程挑战解释了为什么 只有少数组织能够成功训练 frontier LLM——即使算法和数据都是公开的，将它们在大规模上可靠地运行仍然是一项极其困难的工程任务。

CHAPTER 13

前沿展望——LLM 的下一步

13.1 推理模型的进化：从技巧到系统能力

推理（reasoning）能力的发展经历了几个清晰的阶段：

第一阶段：提示工程。Chain-of-thought（CoT）提示的发现表明，只需在 prompt 中加入 “Let’s think step by step”，模型的数学和逻辑推理能力就会显著提升。这说明模型内部已经具备了推理能力，只是需要被正确地「激活」。

第二阶段：推理微调。通过在包含推理链的数据上进行 SFT，让模型默认生成推理过程。这比提示工程更稳定——模型不再依赖用户提供正确的 prompt。

第三阶段：专用推理模型。OpenAI o1/o3/o4-mini、DeepSeek-R1[1]、Claude 的 extended thinking 模式 代表了推理能力的系统化训练（其中 o1/o3 系列和 DeepSeek-R1 是专门训练的推理模型，而 Claude 的 extended thinking 是通用模型的推理增强模式，二者技术路线有本质区别）。这些模型通过 RL（如 GRPO[2]）学会了在推理过程中进行自我验证、回溯和错误修正。关键创新是将推理链的生成视为一个可以用 RL 优化的过程——模型不仅学会了「思考」，还学会了「思考如何思考」。

13.1.1 推理模型的技术路线分化

截至 2026 年初，推理模型已经形成了三条明显不同的技术路线：

路线一：专用推理模型（OpenAI o 系列）。OpenAI 的 o1（2024 年 9 月）→ o3（2024 年 12 月预览，2025 年 4 月正式发布）→ o4-mini（2025 年 4 月）形成了一条清晰的演进路径。这些模型在训练时就被专门优化为「深度思考者」——它们生成长而详尽的推理链，在数学（AIME 2025：o3 达到 96.7%）、编程（Codeforces rating >2700）和科学推理（GPQA Diamond：87.7%）等需要深度推理的任务上表现突出。o4-mini 是这条路线的效率优化版本——以显著更低的成本和延迟提供接近 o3 的推理能力，在许多评估中甚至略微超越 o3。这表明推理能力的 scaling 不仅可以通过增大模型实现，还可以通过更好的训练方法在更小的模型上实现。

路线二：开源推理模型（DeepSeek-R1）。DeepSeek-R1[3] 用一个关键实验 揭示了推理能力的涌现机制：在 R1-Zero 实验中，他们仅用格式化的 RL reward（不使用任何 SFT 数

[1] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.

[2] Z. Shao et al., “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.

[3] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.

据），就让 base model 自发学会了长链推理和自我反思。这个发现意义深远——它表明推理能力不需要人工标注的推理链数据来训练，RL 本身就能诱导出这种能力。R1 的完整版本在 AIME 2024 上达到 79.8%（与 o1 正式版的 83.3% 接近），并以 MIT 许可证开源，催生了大量基于 R1 蒸馏的小型推理模型（如 R1-Distill-Qwen-32B、R1-Distill-LLaMA-70B 等）。

路线三：混合推理模式（Claude extended thinking）。Anthropic 的 Claude 3.7 Sonnet（2025 年 2 月）开创了「混合推理」的概念——同一个模型可以在快速回答和深度推理之间切换。用户无需选择不同的模型，只需启用 extended thinking 模式，模型就会在回答前生成可见的思考过程。Claude Opus 4.6 进一步引入了可配置的 thinking budget （最高 128K tokens 的推理预算），用户可以根据任务复杂度和成本考量做出选择。这种路线的优势是灵活性——简单问题无需支付推理 token 的成本，复杂问题可以投入更多推理计算。

其他值得关注的推理模型：Google 的 Gemini 2.5 Pro（内置 “thinking” 模式，2025 年 3 月在 LMSYS Chatbot Arena 登顶）；Grok 3（xAI，2025 年 2 月，内置 “Big Brain” 推理模式）；Qwen-QwQ（阿里巴巴，开源的推理专用模型）。

推理时计算 vs. 训练时计算的经济学

一个深刻的趋势转变正在发生：传统的 Scaling Laws 关注训练时计算——投入更多算力训练更大的模型。但推理时计算（test-time compute）提供了另一条路径——保持模型大小不变，通过在推理时投入更多计算（更长的思考链、更多的候选方案探索）来提升输出质量。这改变了成本结构：训练成本是一次性的，而推理成本随使用量线性增长。对于使用量大的应用，一个更小但「想得更久」的模型 可能比一个更大但「直接回答」的模型更具经济优势。o4-mini 的成功正是这一趋势的体现——它以 o3 约 1/5 的成本提供了相当甚至更好的推理能力。

13.2 多模态：走向统一理解

2025–2026 年，纯文本 LLM 正在快速转向多模态模型——同时理解文本、图像、音频甚至视频。Gemini 3、GPT-4o、Claude 4.5 Sonnet 都是原生多模态的。多模态不再是「附加功能」，而是「基础能力」。

13.2.1 视觉-语言模型的架构演进

视觉-语言模型（Vision-Language Model, VLM）的发展经历了几代架构：

CLIP 范式：对比学习——训练一个图像编码器和一个文本编码器，使匹配的图文对在表示空间中靠近。CLIP 提供了强大的视觉表示，但不能生成文本。

LLaVA 范式：视觉编码器 + 投影层 + LLM。将预训练的视觉编码器（如 ViT/SigLIP）的输出 通过一个简单的投影层（线性层或 MLP）映射到 LLM 的输入空间，然后与文本 token 一起送入 LLM。这种架构简洁有效，是当前大多数开源 VLM 的基础。

原生多模态：GPT-4o 和 Gemini 3（基于 Gemini 架构[4] 的后续版本）代表了下一步——不再将视觉作为「插件」，而是在预训练阶段就联合训练视觉和语言理解。这种方法成本更高（需要海量的图文配对数据），但产生的模型对视觉信息的理解更深入、更自然。

13.2.2 音频：从转录到原生理解

音频能力的发展也遵循类似的路径：

转录范式：使用 Whisper 等专用模型将语音转为文本，再用 LLM 处理文本。这是目前最稳定的方案，但丢失了语音中的非文本信息（语调、情感、停顿）。

原生音频：GPT-4o 的实时语音模式 和 Gemini 的原生音频输入 代表了音频理解的下一步——模型直接处理音频波形，不经过中间的文本转录步骤。这使得模型能够理解语调变化、背景声音、甚至多人对话中的说话者切换。

实时语音交互的技术挑战巨大：模型需要在数百毫秒内开始生成回复（否则对话会感觉卡顿），同时能够被用户打断（“barge-in” 检测），还需要处理环境噪声和远场拾音。

13.2.3 视频理解与世界模型

视频是多模态中计算成本最高的模态。一段 1 分钟的 30fps 视频包含 1800 帧——如果每帧都编码为数百个 token，模型需要处理数十万个视觉 token。

当前的处理策略包括：**关键帧采样**——从视频中均匀或自适应地选取少量帧；**视频编码器**——使用 3D 卷积或时序 Transformer 将连续帧压缩为更紧凑的表示；**分层处理**——先用轻量级模型定位关键片段，再用大模型深入分析。

长视频理解（数十分钟到数小时）仍然是一个开放问题——它同时挑战了模型的上下文窗口、视觉理解能力和长期记忆能力。

世界模型（World Models）代表了视频理解的更深层方向。OpenAI 的 Sora 2（2025 年 9 月发布）被定位为不仅仅是视频生成器，而是「世界模拟器」——模型学习视频中的物理规律（重力、碰撞、流体动力学），并能够生成物理上合理的新场景。Sora 2 支持 1080p 视频生成，最长 1 分钟，并加入了同步音频生成能力。虽然当前的世界模型距离真正理解物理定律还有很大差距（生成的视频仍然会出现物理不一致性），但这个方向对于机器人、自动驾驶和游戏 AI 至关重要——一个能够预测物理世界演变的模型 可以用于规划和决策。

13.2.4 Omni-模型：全模态输入与输出

多模态的终极形态是 **omni-model**——不仅能理解多种模态的输入，还能生成多种模态的输出（文本、图像、音频、视频）。

GPT-4o 的 “o” 就代表 “omni”。它可以接收文本、图像和音频输入，并生成文本和音频输出。GPT-4o 在 2025 年 3 月获得了原生图像生成能力，不再依赖独立的 DALL-E 模

[4] G. Team et al., “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.

型。Gemini 3 更进一步，原生支持图像和音频的生成。

统一模型的关键挑战是**训练数据的模态平衡**——互联网上的文本数据远多于高质量的图文配对数据，而高质量的视频-文本配对数据更加稀缺。如何在预训练中平衡不同模态的数据比例，避免某种模态的能力被其他模态「挤压」，是一个活跃的研究方向。

13.3 Agent：从对话到行动

LLM 正在从「回答问题」演进为「执行任务」。这是 2025–2026 年最重要的技术趋势之一。

13.3.1 Agent 的核心架构

Agent 系统的核心是一个循环：**观察**（Observe）→ **推理**（Think）→ **行动**（Act）→ **观察**（Observe）。这就是 ReAct 范式——模型交替进行推理（生成文本思考过程）和行动（调用工具获取外部信息），直到任务完成或达到预设的步数上限。

这个循环可以持续数十到数百步。对于复杂任务（如修复一个涉及多文件的 GitHub issue），Agent 可能需要：阅读代码 → 理解架构 → 定位 bug → 编写修复 → 运行测试 → 分析失败 → 修改修复 → 再次测试。每一步都是「观察-思考-行动」的实例。

13.3.2 工具使用与标准化

Agent 的能力取决于它可以使用的工具。当前的主流工具类型包括：

- **Function calling**：LLM 生成结构化的函数调用（指定函数名和参数），由外部系统执行并返回结果。这是最基础的工具使用形式。
- **Code execution**：LLM 生成代码（Python、Bash 等），在沙箱环境中执行并获取输出。Code Interpreter（OpenAI）和 tool use（Anthropic）是典型实现。
- **Web browsing**：LLM 控制浏览器访问网页，提取信息。需要处理动态渲染、验证码、反爬虫等挑战。
- **Computer use**：LLM 通过理解屏幕截图和控制鼠标/键盘 来操作计算机。Anthropic 的 Claude computer use 是这个方向的先驱——模型可以打开应用程序、填写表单、点击按钮，像人类用户一样操作任意桌面软件。

MCP（Model Context Protocol）由 Anthropic 于 2024 年底开发并开源，作为 Agent 与外部工具交互的标准协议，正在被越来越多的工具提供商采用。它定义了工具描述（名称、参数、返回值）、调用和结果返回的标准格式，使得同一个 Agent 可以无缝对接不同的工具和服务——搜索引擎、数据库、API、本地文件系统、甚至其他 AI 模型。MCP 的意义类似于 HTTP 之于 Web——一个统一的协议使得生态系统可以繁荣发展。

13.3.3 代码 Agent：最成熟的落地场景

2025–2026 年，Agent 应用最成熟的领域是**软件工程**。代码 Agent 已经从辅助工具演进为半自主的编程助手，在某些场景下甚至可以独立完成完整的开发任务。

Claude Code（Anthropic，2025 年 2 月发布）是 CLI 级别的自主编程 Agent。与传统的 IDE 插件不同，Claude Code 运行在终端中，可以直接读写文件、执行 bash 命令、运行测试、管理 git 操作——全部通过自然语言交互。它支持 “headless mode”（无人值守模式），可以在 CI/CD 管道中自动执行代码审查、修复 lint 错误、生成测试等任务。Claude Code 的设计理念是 **给予 Agent 与人类开发者相同的工具和权限**——不是通过 API 读取代码片段，而是像人类一样在真实的开发环境中工作。

Devin（Cognition AI）是另一种路线的代码 Agent——完全自主的「AI 软件工程师」。Devin 在云端独立运行，用户通过 Slack 或 Web 界面分配任务，Devin 自主规划、编码、测试，最终以 Pull Request 的形式交付结果。Devin 适合被委派独立的、定义清晰的开发任务（如 “将这个 API 的认证从 OAuth 1.0 迁移到 OAuth 2.0”）。2025 年底 Cognition 收购了 Windsurf IDE，将 Devin 的自主能力与 IDE 的交互式编程结合。

Cursor 和 **Windsurf**（Codeium）代表了 IDE-integrated Agent 的路线——将 AI 能力深度嵌入代码编辑器。Cursor 的 “Composer” 模式可以跨多文件进行修改，Agent 模式可以自动运行终端命令和管理文件。

SWE-bench（修复真实 GitHub issue）的解决率已经从 2024 年初的 ~5% 提升到 2026 年初的 ~60%+，这意味着 Agent 已经能够独立完成 大多数的真实软件工程任务。

代码 Agent 的使用范式演变

代码 Agent 的使用范式正在经历三个阶段：**阶段一：补全**——GitHub Copilot 风格，逐行或逐函数补全代码，人类主导、AI 辅助。**阶段二：对话式编程**——通过自然语言描述需求，Agent 生成或修改代码块，人类审查并接受/拒绝。Cursor、Claude Code 当前主要在这个阶段。**阶段三：委托式编程**——将完整的任务委托给 Agent，Agent 自主完成规划、实现和测试，人类仅在关键决策点介入。Devin 的设计理念指向这个阶段，但可靠性仍然是核心挑战。从阶段二到阶段三的跨越 需要 Agent 具备更强的规划能力、错误恢复能力和对不确定性的处理能力。

13.3.4 多 Agent 系统

更复杂的任务可以由多个专门化的 Agent 协作完成。例如，一个软件开发任务可以分配给：**架构师 Agent**（设计整体方案）、**开发者 Agent**（编写代码）、**测试 Agent**（编写和运行测试）、**审查 Agent**（代码审查和质量检查）。

多 Agent 框架包括 AutoGen（Microsoft）、CrewAI、LangGraph 等。它们提供了 Agent 间通信、任务分配、结果汇聚等基础设施。

多 Agent 系统的核心挑战是**协调成本**——Agent 之间的通信（通常以自然语言进行）本身消耗大量 tokens，而且 Agent 间的误解可能导致任务偏离方向。当前的研究表明，对于大多数任务，一个能力足够强的单 Agent 比多个能力较弱的 Agent 协作更有效。

13.3.5 Agent 的可靠性挑战

Agent 面临的最大挑战是**可靠性**。在多步骤任务中，每一步都有一定的失败概率。如果每步的成功率是 95%，那么 20 步链式执行的整体成功率只有 $0.95^{20} \approx 36\%$。

更危险的是**静默失败**——Agent 可能在某一步产生了幻觉（比如调用了一个不存在的 API，或者基于错误的假设做出了决策），但后续步骤继续在错误的基础上执行，最终产生看似合理但完全错误的结果。

幻觉在工具使用中尤其危险：在对话中，用户可以识别不合理的回答；但在 Agent 自主执行的过程中，一个错误的数据库查询或文件操作可能造成真实的损害。

WebArena（自主网页操作）和 OSWorld（操作系统级任务）等基准测试正在推动 Agent 能力的边界。但从 benchmark 到真实世界仍有差距——benchmark 中的任务有明确的开始和结束条件，而真实世界的任务往往目标模糊、需要人类反馈、容错要求更高。

Agent 的经济影响

Agent 作为「数字劳动力」（digital workers）的概念 正在引发关于工作未来的严肃讨论。如果一个 Agent 能够完成初级软件工程师 50% 的任务，这对就业市场意味着什么？乐观者认为 Agent 会放大人类的生产力，创造新的工作类型（如 Agent 监督者、prompt 工程师）。悲观者担心大规模的工作岗位替代。现实可能介于两者之间——Agent 的可靠性问题限制了它的自主性，但它正在显著改变工作的内容和效率要求。Anthropic CEO Dario Amodei 在 2025 年的文章 “Machines of Loving Grace” 中 勾勒了 AI 增强人类而非替代人类的愿景，但同时承认转型期的阵痛不可避免。

13.4 硬件路线图：算力的下一个台阶

LLM 的能力演进与硬件进步密不可分。2025–2027 年的 AI 加速器路线图 决定了下一代模型的能力上限。

13.4.1 NVIDIA：从 Blackwell 到 Vera Rubin

NVIDIA 继续以 1–2 年的节奏推出新一代 GPU 架构：

Blackwell 架构（2024 年发布，2025 年量产）是当前一代的旗舰。B200 GPU 提供 9 PFLOPS 的 FP4 性能 和 192GB HBM3e 显存（6.4 TB/s 带宽），相比上一代 H100（FP8 约 3.9 PFLOPS，80GB HBM3）有显著提升。**GB200 NVL72** 是 Blackwell 的系统级产品——将 72 块 B200 GPU 通过第五代 NVLink 互连（GPU 间带宽 1.8 TB/s）组成一个液冷机柜，提供 720 PFLOPS 的 FP4 计算能力和 13.5TB 的统一显存。这种设计的意义在于：一个 GB200 NVL72 机柜可以将一个 700B+ 参数的模型 完全放入 GPU 显存中进行推理，消除了多机通信的延迟。

Vera Rubin 架构（预计 2026 年下半年发布）是 NVIDIA 的下一代平台。据已公布的信息，Rubin GPU 将采用 3nm 工艺，搭配 HBM4 内存（每 GPU 可能达到 256GB+，带

宽超过 8 TB/s），计算性能预计再翻倍。Vera Rubin Ultra 将是完整平台版本，包含 CPU（Vera，基于 ARM 架构）和 GPU（Rubin）的集成。

13.4.2 AMD：从追赶者到竞争者

AMD 的 Instinct 系列正在成为 NVIDIA 的真正竞争对手：

MI300X（2024 年量产）提供 192GB HBM3 显存和 5.3 TB/s 带宽——在显存容量上超越了 H100。这使得 MI300X 在大模型推理场景中有显著优势（更大的模型可以放入单 GPU），微软 Azure 和 Oracle Cloud 都已部署 MI300X 集群。

MI350 系列（预计 2025 年底–2026 年初量产）基于全新的 CDNA 4 架构和 3nm 工艺，采用 chiplet 设计，提供 288GB HBM3e 显存。AMD 在 Hot Chips 2025 上展示了 MI350 的 node-to-rack 扩展架构，目标是在系统级能效（perf/watt）上与 NVIDIA Blackwell 竞争。MI350 的关键卖点是 **开源软件栈**——ROCm 生态的持续改善 使得越来越多的训练框架（PyTorch、JAX、vLLM）原生支持 AMD GPU，降低了迁移成本。

MI400（路线图上为 2026 年底–2027 年）将是 AMD 的下一个大跳跃，但具体技术细节尚未公布。

13.4.3 Google TPU：垂直整合的典范

Google 的 TPU（Tensor Processing Unit）代表了另一种路线——自研加速器完全为自己的工作负载优化。

TPU v5p（2023 年发布）单芯片提供 459 TFLOPS 的 BF16 性能，95GB HBM 内存。TPU v5p 的核心优势是 **ICI（Inter-Chip Interconnect）网络**——一个 TPU v5p pod 可以将 8,960 块芯片通过 ICI 互连，在 pod 内实现无需显式通信编程的分布式计算。

TPU v6e（Trillium）（2024 年发布）是效率优化版本，单芯片性能提升 67%（相比 v5e），HBM 容量翻倍。v6e 特别针对推理场景优化，是 Gemini 模型的主要推理硬件。

Google 即将推出的 **TPU v7** 预计将带来又一个代际性能飞跃，特别是在 HBM 带宽和 ICI 通信速度上。TPU 的独特优势在于 Google 可以端到端优化 从编译器（XLA）到框架（JAX）到模型架构（Gemini）到硬件的整个栈——这种垂直整合在某些场景下 可以提供比通用 GPU 更高的效率。

13.4.4 AWS Trainium：云厂商的自研芯片之路

Amazon 的 Trainium 代表了 云服务商自研 AI 芯片的趋势：

Trainium2（2024 年底推出 Trn2 instances）提供了相当于 H100 约 30–50% 的单芯片性能，但以显著更低的价格提供给 AWS 客户。Trainium2 的核心价值不在于绝对性能，而在于**性价比**——AWS 可以将自研芯片的成本优势传递给用户。

Trainium3（2025 年 12 月 GA，AWS 的第四代 AI 芯片）是一个重大升级——采用 3nm 工艺（与 NVIDIA Blackwell 同代），被 SemiAnalysis 称为 “Amazon Basics 版的 GB200 NVL72”。Trn3 UltraServers 将 64 块 Trainium3 芯片 通过高速互连组成一个节点，提供接

近 NVIDIA GB200 NVL72 的系统级训练性能，但价格预计低 30–40%。**Trainium4**（已进入路线图，预计 2027 年）将进一步提升每芯片的计算密度。

Anthropic 是 Trainium 的重要早期客户——Claude 模型的部分训练和推理已经运行在 Trainium 上，这对验证非 NVIDIA 硬件的可行性具有标志性意义。

AI 加速器的竞争格局（2026 年）

NVIDIA 仍然占据 AI 训练市场约 80%+ 的份额，但竞争格局正在发生结构性变化：（1）AMD 的 MI350 在大内存推理场景中具有竞争力；（2）Google TPU 通过垂直整合在自家工作负载上效率领先；（3）AWS Trainium3 以性价比切入中端市场；（4）国产加速器（华为昇腾 910C、寒武纪等）在中国市场因出口管制而获得替代需求。多供应商策略正在成为大型 AI 团队的标准做法——减少对单一供应商的依赖，同时通过竞争压低采购成本。但 NVIDIA 的 CUDA 生态仍然是其最大的护城河——迁移到其他硬件的软件适配成本是许多团队犹豫的主要原因。

13.5 端侧模型：让 AI 无处不在

不是所有场景都需要云端的 700B 模型。端侧（on-device）AI 正在成为一个重要的发展方向。

13.5.1 为什么端侧模型重要?

端侧推理的核心价值：

- **隐私**：数据不离开设备，对于医疗记录、私人通信、企业机密等敏感场景至关重要。
- **延迟**：本地推理消除了网络往返时间，首 token 延迟可以低至数十毫秒。
- **离线能力**：飞行模式、地铁、偏远地区——不依赖网络连接的 AI 能力。
- **成本**：没有 API 调用费用。对于高频率使用场景（如写作助手、代码补全），本地模型的边际成本几乎为零。

13.5.2 NPU 硬件的快速演进

端侧 AI 的能力边界由硬件决定。2025–2026 年，主要芯片厂商的 AI 加速能力正在快速提升：

Apple Neural Engine（ANE）：Apple 的 M 系列和 A 系列芯片 内置的 Neural Engine 是端侧 AI 的标杆。M4 芯片的 ANE 提供 38 TOPS 的算力，配合 Apple Silicon 的统一内存架构（CPU/GPU/NPU 共享内存，避免数据搬运），可以在 iPhone 和 Mac 上流畅运行 3B 模型。Apple Intelligence 基于这一硬件能力构建，在设备端运行一个 ~3B 的基础语言模型[5]。

[5] T. Gunter et al., "Apple intelligence foundation language models," *arXiv preprint arXiv:2407.21075*, 2024.

Qualcomm Hexagon NPU（Snapdragon X Elite/X2 系列）：Snapdragon X Elite 的 Hexagon NPU 提供 45 TOPS，是 Windows on ARM PC 上的主要 AI 加速器。Snapdragon X2（预计 2026 年发布）进一步提升 NPU 性能，并改善了对主流 AI 框架的兼容性。Qualcomm 的 AI Stack（QNN SDK）支持将 ONNX 和 PyTorch 模型 直接部署到 Hexagon NPU 上。

Intel Lunar Lake / Arrow Lake：Intel 的最新客户端 CPU 集成了 提供 48 TOPS 的 NPU，但其软件生态（OpenVINO）相比 Apple 和 Qualcomm 仍有差距。

AMD Ryzen AI：AMD 的 Ryzen AI 300 系列集成了 XDNA 2 NPU，提供 50 TOPS，在 Windows AI PC 市场与 Qualcomm 和 Intel 竞争。

Table 13.1: 主要端侧 AI 加速器对比（2026 年初）

芯片	NPU 算力	内存	可运行的模型规模
Apple M4 (ANE)	38 TOPS	16–32GB 统一	3B 流畅，7B 可运行
Snapdragon X Elite	45 TOPS	16–32GB LPDDR5x	3B 流畅，7B 可运行
Intel Lunar Lake	48 TOPS	16–32GB LPDDR5x	3B 流畅
AMD Ryzen AI 300	50 TOPS	16–64GB DDR5	3B 流畅，7B 可运行
Apple M4 Max (ANE)	38 TOPS	36–128GB 统一	7B–14B 流畅

13.5.3 小模型家族与端侧能力

小模型家族正在快速发展：Apple Intelligence[6] 将一个 ~3B 的 LLM 部署到 iPhone 上；Google Gemini Nano（1.8B 和 3.25B）专为 Pixel 手机设计；LLaMA 3.2[7] 提供 1B 和 3B 的端侧版本；Qwen3-0.6B[8] 和 Gemma 3-1B 更是将模型压缩到了 1B 以下。

量化技术（如 GGUF Q4_K_M）和 推理优化（如 llama.cpp 的 ARM NEON 和 Metal 优化）使得在消费级硬件上运行 7B–14B 的模型成为现实。

端侧模型的核心能力保留策略包括：**知识蒸馏**——用大模型的输出训练小模型，使小模型「继承」大模型的推理能力（DeepSeek-R1-Distill 系列就是这种方法的典范）；**结构化剪枝**——去除模型中对质量贡献最小的神经元或注意力头，在保持架构兼容性的同时减小模型；**共享词嵌入**——让输入嵌入层和输出层共享权重，减少参数量（LLaMA 3 等模型采用了这种设计）。

实际测试表明，当前的 3B 模型在以下任务上已经「够用」：简单问答、文本摘要（短文）、情感分析、文本分类、短文翻译、格式转换。它们在以下任务上仍然不足：复杂推理、长文理解、代码生成（复杂逻辑）、创意写作（高质量）、多轮对话（需要长期记忆）。

13.5.4 端云协作：混合推理

端侧–云端的**混合推理**（hybrid inference）可能是最具实用价值的方向：简单请求在设备端本地处理（零延迟、完全隐私），复杂请求上传到云端的大模型处理（更高质量）。

[6] T. Gunter et al., "Apple intelligence foundation language models," *arXiv preprint arXiv:2407.21075*, 2024.
[7] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
[8] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

Apple Intelligence 的架构正是这种模式的典范：端侧 3B 模型处理大部分请求，当模型判断自己无法给出高质量回答时，请求被路由到云端更大的模型（Apple 称之为 Private Cloud Compute，使用专用的 Apple Silicon 服务器，承诺数据不存储、不记录）。路由决策可以由端侧模型自己做出——它可以评估自己对回答的「信心」，信心不足时自动升级到云端。

这种架构的经济优势明显：假设 80% 的请求可以在端侧处理，那么云端的服务器需求减少 80%，显著降低了 API 提供商的运营成本。

2026 年端侧 AI 的能力阈值

NPU 硬件的发展正在快速改变端侧 AI 的能力阈值。2024 年的 iPhone 可以流畅运行 3B 模型，2026 年的旗舰手机（搭载 A19 或 Snapdragon X2）预计可以运行 7B 模型。高端笔记本（搭载 M4 Max/Ultra，128GB 统一内存）已经可以流畅运行 70B 的量化模型。这意味着一些原本只能在服务器上运行的任务——如本地代码补全、文档摘要、个性化写作助手——将逐步转移到用户设备上，从根本上改变 AI 服务的交付模式。

13.6 AI 监管：规则正在形成

随着 LLM 能力的快速提升，全球各主要经济体正在加速构建 AI 监管框架。监管环境对 AI 公司的战略选择、开源生态和技术发展方向有深远影响。

13.6.1 EU AI Act：全球最严格的 AI 法规

欧盟 AI 法案（EU AI Act）于 2024 年 8 月 1 日正式生效，是全球首个全面的 AI 监管法律框架。其核心是**风险分级**制度：

- **不可接受风险**（禁止）：社会评分系统、实时远程生物识别（执法例外）、操纵性 AI 系统。
- **高风险**（严格监管）：用于关键基础设施、教育、就业、司法的 AI 系统。需要进行一致性评估（conformity assessment）、建立风险管理系统、确保数据治理、保留技术文档。
- **通用目的 AI 模型**（GPAI，特别规定）：所有 GPAI 提供商必须提供技术文档、遵守版权法、发布训练数据摘要。“systemic risk” 模型（训练算力超过 10^{25} FLOPs）还需进行对抗性测试、监控严重事件、确保网络安全。
- **有限风险 / 最小风险**（轻度或无监管）：聊天机器人、AI 生成内容等，仅需透明度标注（告知用户正在与 AI 交互）。

关键时间节点：2025 年 2 月——禁止不可接受风险 AI 的条款生效；2025 年 8 月——GPAI 模型条款和治理结构生效；2026 年 8 月——高风险 AI 系统的全面合规要求生效。违规罚款最高可达 3,500 万欧元 或全球年营业额的 7%（取较高者）。

EU AI Act 对开源模型有一定的豁免——“free and open-source” 的 GPAI 模型 如果不构成 systemic risk，可以免除部分合规义务（如技术文档要求的简化），但仍需遵守版权法。

13.6.2 美国：碎片化但加速中的监管

美国没有联邦级别的统一 AI 法律，但多层次的监管正在形成：

联邦层面：拜登政府 2023 年 10 月的行政令（Executive Order on AI Safety and Security）要求开发 dual-use 基础模型的公司 向联邦政府报告训练细节和安全测试结果。特朗普政府在 2025 年 1 月上任后撤销了这一行政令，转向更为宽松的 “pro-innovation” 政策。但国会层面，多项 AI 立法提案正在推进中，包括 AI 系统的透明度要求、深度伪造标注义务和 AI 在关键领域的使用规范。

州层面：加利福尼亚州的 SB 1047（AI Safety Bill）在 2024 年引发了激烈辩论——该法案要求 AI 开发者对模型造成的严重危害承担责任，最终被州长 Gavin Newsom 否决，理由是过于严厉可能遏制创新。但 2025–2026 年的后续立法尝试仍在继续。科罗拉多州于 2024 年通过了 美国首个全面的 AI 算法歧视法律。

机构层面：NIST 的 AI Risk Management Framework（AI RMF）提供了自愿性的 AI 风险管理指南；FTC 将 AI 虚假声明纳入消费者保护执法范围；SEC 开始关注 AI 在金融市场中的风险。

13.6.3 中国：审批制 + 内容管控

中国的 AI 监管采取了**分类审批**的模式：

- **生成式 AI 管理办法**（2023 年 8 月生效）：向公众提供生成式 AI 服务 需要通过算法备案和安全评估。服务提供者必须确保生成内容 符合社会主义核心价值观，不得生成「违法」内容。
- **深度合成管理规定**（2023 年 1 月生效）：AI 生成的图像、音频、视频必须标注。
- **算法推荐管理规定**（2022 年 3 月生效）：覆盖推荐算法的透明度和用户控制权。

中国的监管特点是**事前审批**（pre-deployment approval）——模型必须通过政府审查才能上线，这与欧美的事后监管（post-deployment regulation）形成对比。截至 2025 年底，已有超过 200 个 AI 大模型 通过了中国的备案审批。这种制度对开源模型的影响是复杂的——开源模型的权重可以自由下载，但基于开源模型构建的面向公众的服务 仍然需要通过审批。

监管对 AI 发展的影响

AI 监管正在形成一个全球「三极格局」：**欧盟**——严格的规则驱动，侧重权利保护和透明度，可能减慢创新速度但提高公众信任；**美国**——碎片化但灵活，侧重创新友好，可能导致监管套利和执法不一致；**中国**——集中审批制，侧重内容管控和国家安全，在保证可控性的同时可能限制学术自由度。对于 AI 公司来说，合规成本正在成为 不可忽视的运营开支——特别是需要在多个司法管辖区运营的全球化公司，需要同时满足不同且有时矛盾的监管要求。开源模型在这个格局中处于特殊位置：一方面，开源天然满足透明度要求；另一方面，开源后的模型无法被收回，一旦被用于违规目的，责任归属变得模糊。

13.7 开合之争：开源 vs. 闭源的哲学与实践

2025–2026 年，开源与闭源模型之间的「开合之争」是 AI 领域最重要的叙事之一。

13.7.1 闭源的论据

- **安全控制**：模型权重不公开，可以通过 API 层面的安全措施（内容过滤、速率限制、使用监控）防止滥用。一旦权重开源，安全对齐可以被移除。
- **竞争优势**：训练一个 frontier model 需要数千万到数亿美元的投入，开源意味着竞争对手可以免费使用你的成果。
- **责任管理**：如果模型被用于有害目的，API 提供商可以追踪和阻止滥用者；开源后则无法控制模型的使用方式。
- **收入模型**：API 定价（按 token 收费）是当前 LLM 公司最直接的变现方式。

13.7.2 开源的论据

- **创新速度**：开源社区的创新速度远超任何单一公司。LoRA[9]、QLoRA[10]、Flash Attention[11] 等 关键创新都来自开源社区或学术界。
- **科学可重复性**：如果研究成果无法被其他团队验证和复现，其科学价值就打了折扣。
- **民主化**：防止 AI 能力集中在少数公司手中。学术研究者、初创企业、发展中国家的开发者 都应该能够获取和使用先进的 AI 技术。
- **国家安全**：一个国家的 AI 能力不应该依赖于外国公司的 API——开源模型允许本地部署和主权控制。

[9] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[10] T. Dettmers et al., "QLoRA: Efficient finetuning of quantized language models," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[11] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

- **生态系统效应**：当你的模型被广泛使用，围绕它建立的工具、教程、微调模型和应用 构成了一个自我强化的生态系统。

13.7.3 “开放权重” 不等于真正的开源

一个重要的区分：当前所谓的「开源模型」大多只是 **open weights**——公开了模型权重，但训练数据、完整的训练代码、数据处理管道和评估基础设施通常不公开。

从技术角度看，开源模型的质量已经接近甚至达到闭源模型的水平——DeepSeek-R1 在数学推理上与 o1 相当，LLaMA 4 Maverick 在综合能力上逼近 GPT-4o。这是因为 LLM 的核心技术（Transformer 架构、训练方法、数据处理流水线）已经高度标准化，真正的差异化越来越依赖于**工程执行**而非秘密技术。

一个更精确的开放程度光谱是：**完全闭源**（仅内部使用）→ **API-only**（通过 API 提供服务，不公开权重）→ **开放权重**（公开权重，不公开训练代码/数据）→ **开放训练代码**（公开权重和训练代码，不公开数据）→ **完全开源**（数据 + 代码 + 权重全部公开）。

目前只有极少数模型（如 OLMo by AI2、SmolLM2 by HuggingFace）处于「完全开源」的端点。DeepSeek-V3/R1 虽然使用 MIT 许可证，但其训练数据和完整的训练基础设施并未公开。

13.7.4 Meta 的开源战略

Meta[12] 的开源决策 是 AI 行业最值得研究的战略案例之一。Mark Zuckerberg 的逻辑是**商品化互补品**（commoditize the complement）：Meta 的核心业务是社交网络和广告，AI 模型是这些业务的基础设施。如果 AI 模型成为商品（人人都能使用），那么构建在 AI 之上的应用（如 Meta 的推荐系统、广告排序、内容审核）才是真正的差异化竞争力。开源 LLaMA 还能吸引顶尖人才——研究者更愿意为一个开放的项目工作，因为他们的工作能被全世界看到。

13.7.5 DeepSeek-R1 的冲击

DeepSeek-R1[13] 的发布 深刻改变了开源 vs. 闭源的辩论格局。一个来自中国的、团队规模不到 200 人的实验室，训练出了在数学和代码推理上与 GPT-4 级别模型匹敌的模型，并以 MIT 许可证完全开源。

这对闭源阵营的打击是多维度的：（1）如果开源模型能达到这个水平，那么闭源的「技术壁垒」论据就被削弱了；（2）DeepSeek-V3 报告的最终预训练 GPU 租赁成本约 $5.5M （不含前期研发、数据处理和后训练成本），远低于 GPT-4 传闻中的 $100M+（该数据未经官方确认）——这说明 frontier model 的训练不一定需要巨额投入；（3）开源模型的性能达到商用水平后，API 定价将面临巨大的下行压力。

[12] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023, A. Dubey et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.

[13] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.

13.8 总结与展望

2025–2026 年的 LLM 领域 正在经历多条技术和产业路线的同时演进：

推理能力的系统化——从提示工程到专用推理模型再到可配置推理预算，模型正在学会「思考」而非仅仅「回忆」。test-time compute 正在成为 与训练 compute 同等重要的资源维度。

Agent 的落地——从概念验证到生产工具，代码 Agent 已经改变了软件工程的工作流程。SWE-bench 60%+ 的解决率 标志着 Agent 从「玩具」跨入了「工具」的阶段。

硬件竞争的加剧——NVIDIA 的垄断正在被 AMD、Google、AWS 逐步打破。多供应商策略降低了行业对单一供应商的依赖，也加速了算力成本的下降。

端侧 AI 的崛起——NPU 硬件的进步使得 3B–7B 模型在手机上成为现实。端云混合推理将成为 AI 服务的标准架构。

监管框架的形成——EU AI Act、美国各州立法、中国审批制度 正在塑造 AI 公司的运营边界。合规成本将成为 AI 商业化的重要变量。

开源与闭源的融合——开源模型在性能上持续逼近闭源模型，但「完全开源」仍然稀缺。行业可能走向一种混合格局——基础模型开源，增值服务闭源。

LLM 技术的发展速度令人目眩。但贯穿始终的核心原则不会改变：数据的质量决定模型的上限，计算的规模决定模型的起点，而人类的智慧——体现在架构设计、训练策略和工程实现中——决定了我们能以多高的效率到达那个上限。

这是一个激动人心的时代。从 Shannon 的信息论到 DeepSeek 的 GRPO, 从 Transformer 的注意力机制到 MoE 的稀疏激活，每一步都是人类对「什么是智能」这个根本问题的持续探索。

而这段旅程，才刚刚开始。

如果用一句话总结 LLM 训练的核心智慧，那就是：**用最简单的目标函数（预测下一个词），配合足够的数据和计算规模，让复杂性从简单性中涌现**。这种「少即是多」的哲学贯穿了整个技术栈——从 Transformer 的简洁架构到 GRPO 的简单奖励信号，从 BPE 的朴素合并规则到量化的直观截断思路。最优雅的工程往往不是最复杂的，而是最简洁的。

让我们回顾整篇文章的核心线索。LLM 的训练是一个环环相扣的流水线：**数据**决定了模型能学到什么（第四章），**架构**决定了模型能表达什么（第二、六、七章），**训练方法**决定了模型能学多好（第五章），**后训练**决定了模型能为人类服务多好（第八章），**推理优化**决定了模型能多快多便宜地服务（第九章）。

每一个环节都蕴含着深刻的思想：下一个词预测背后是压缩即理解的信息论洞察，注意力机制背后是让模型自主决定信息流动的设计哲学，MoE 背后是参数效率与计算效率的解耦思想，GRPO 背后是让模型通过探索自我发现推理策略的强化学习理念，量化背后是信息论中「够用的精度」的朴素真理。

对于想进入这个领域的读者，本文的附录提供了完整的训练流水线速查。对于已经在这个领域工作的实践者，希望本文的「为什么」视角能帮助你更深入地理解 你每天使

用的工具和方法背后的思想。

APPENDIX A

附录：完整训练流水线速查

以下是一个典型 frontier LLM（如 DeepSeek-V3 级别）从零开始训练到部署的完整流水线概览，供读者作为全景参考。

A.1 阶段一：数据准备（1–3 个月）

1. **网页爬取**：获取 Common Crawl 快照或自建爬虫，原始数据量通常在 PB 级别。
2. **文本提取**：使用 trafilatura 等工具从 HTML 中提取纯文本，去除导航栏、广告、模板文本。
3. **语言识别**：使用 fastText lid 识别文本语言，按语言分类处理。
4. **精确去重**：URL 去重 + SHA-256 哈希去重，去除完全相同的文档。
5. **模糊去重**：MinHash + LSH，*n*-gram Jaccard 相似度阈值通常设为 0.8。
6. **质量过滤**：规则过滤（行长度、特殊字符比例、重复行比例、关键词黑名单）+ 模型过滤（perplexity 过滤、分类器打分）。
7. **有害内容过滤**：分类器识别色情、暴力、仇恨言论等内容并移除。
8. **个人信息去除**：正则表达式匹配并删除电话号码、邮箱、身份证号等 PII。
9. **数据配比**：确定网页: 代码: 学术: 书籍: 数学的比例，通常需要在小模型上做 proxy 实验来确定最优比例。
10. **分词器训练**：在最终数据集上训练 BPE 分词器（使用 SentencePiece），确定词表大小（32K–150K）。
11. **数据打包**：将 token 化的数据打包成连续的序列（concatenation + document separator），按训练序列长度切分，写入高效的二进制格式（如 MDS、arrow）。

数据准备 Checklist

- 数据来源已确定？许可证已验证（CC、MIT 等）？
- 语言分布已规划？各语言的数据量比例是否合理？
- 质量指标已定义？perplexity 阈值、最小/最大文档长度、特殊字符上限？
- 去重已完成？精确去重 + 模糊去重，去重率通常在 30–60%？
- PII 已移除？电话、邮箱、身份证号、信用卡号等？
- 存储预算已确认？经验法则：~2 TB / 1T tokens（压缩后，Parquet 格式）

推荐工具链：Datatrove[1]（HuggingFace 的数据处理框架，FineWeb 的基础）用于构建

[1] G. Penedo et al., "The FineWeb datasets: Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024.

端到端的数据管道；trafilatura 用于网页文本提取；fastText lid 用于语言识别；text-dedup 用于高效去重（支持 MinHash/SimHash/SuffixArray）；Parquet 格式用于存储（列式压缩，高效随机访问）。

A.2 阶段二：预训练（1–3 个月）

1. **架构确定**：模型大小、层数、头数、FFN 维度、是否使用 MoE、专家数量、GQA 组数等。
2. **并行策略配置**：根据 GPU 数量和模型大小确定 DP / TP / PP / SP / EP 的维度划分。
3. **超参数搜索**：在小模型（proxy model）上搜索最优的 学习率、batch size、warmup steps 等，利用 Scaling Laws 推算到目标规模。
4. **训练启动**：分阶段训练——先用较短的上下文长度（如 4K）训练主体，后期扩展到长上下文（如 128K）。
5. **持续监控**：loss 曲线、梯度范数、学习率、GPU 利用率、通信带宽、checkpoint 大小，出现 loss spike 时回滚检查点。
6. **中间评估**：定期在验证集和少量基准测试上评估，发现问题及时调整数据配比或超参数。

预训练超参数起点（7B Dense Model 参考值）

超参数	参考值与说明
Learning rate	峰值 3×10^{-4}，cosine decay 至峰值的 1/10
Batch size	4M tokens（可从 1M 逐步增大）
Warmup steps	2000 步
Weight decay	0.1
Adam β	$\beta_1 = 0.9, \beta_2 = 0.95$
Gradient clipping	1.0
序列长度	主训练阶段 4K–8K，后期扩展到 32K–128K
精度	BF16 mixed precision（部分计算用 FP32 累积）

Checkpoint 策略：每 1000–2000 步保存一次 checkpoint，保留最近 5–10 个 checkpoint（滚动覆盖旧的）。每个完整训练 checkpoint 需要保存：BF16 参数（×2 字节）、FP32 master weights（×4 字节）、以及 Adam 优化器状态 m 和 v（各 ×1 字节，通常以 BF16 保存），总计约为模型参数量 × 8 字节。对于 7B 模型，单个 checkpoint 约 56 GB；对于 70B 模型，约 560 GB。

监控指标：

- **Loss 曲线**：应平滑下降，任何突然上升（spike）都需要调查。
- **Gradient norm**：应稳定在一定范围内，突然增大可能预示发散。
- **Learning rate**：确认调度器按预期工作。

- **GPU 利用率**：目标 > 90%，低于此说明存在瓶颈。
- **Tokens/second**：训练吞吐量，用于估算完成时间。
- **MFU**（Model FLOPs Utilization）：实际计算量 / 理论峰值计算量，优秀的实现可以达到 40–60%。

常见故障与对策

现象	可能原因与对策
Loss spike（突然上升）	数据异常（检查当前 batch 的数据质量）；学习率过高（降低 LR 或回滚 checkpoint）；梯度爆炸（检查 gradient clipping 是否生效）
Loss 不下降	学习率过低；数据有严重质量问题；模型架构 bug（如 residual connection 缺失）
GPU 利用率低	数据加载瓶颈（增加 dataloader workers）；通信瓶颈（检查网络带宽、调整并行策略）；batch size 过小
OOM（显存不足）	减小 batch size；增加 TP/PP 维度；启用 activation checkpointing

A.3 阶段三：后训练（2–6 周）

1. **SFT 阶段**：在 10K–1M 条高质量（指令, 回答）对上微调。数据来源包括人工编写、模型生成（合成数据）和开源数据集。
2. **偏好优化**：收集偏好数据 → 使用 DPO 或 GRPO 优化。对于推理模型，这一步可能需要多轮 RL 训练。
3. **安全对齐**：在安全数据集上做额外的 SFT/RL，确保模型拒绝有害请求。
4. **全面评估**：在所有目标基准测试上评估，与前代模型和竞品对比。
5. **红队测试**：专业团队尝试各种越狱和滥用场景，发现漏洞后迭代对齐。

后训练超参数参考

阶段	超参数	参考值
SFT	Learning rate	$1 \times 10^{-5} - 2 \times 10^{-5}$
SFT	Epochs	1–3 epochs（数据量少时可多轮）
SFT	Batch size	Effective batch size 128–512 samples
SFT	LR schedule	Cosine decay
DPO/GRPO	Learning rate	$5 \times 10^{-7} - 5 \times 10^{-6}$
DPO/GRPO	KL penalty β	0.05–0.1（DPO）；无（GRPO 用 clipping）
DPO/GRPO	Epochs	1 epoch（过拟合风险高）
SFT 数据量		10K–100K 条高质量指令对
偏好数据量		50K–200K 条偏好对比对

后训练关键提醒：每个后训练阶段完成后，都要在基础能力 benchmark（如 MMLU、HumanEval、MATH）上检查是否有能力退化（regression）。过度的后训练（特别是安全对齐）可能导致模型变得 过度保守（over-refusal），在基础能力上反而变差——这被称为 alignment tax。

A.4 阶段四：部署与推理优化（持续）

1. **量化**：将模型权重从 BF16 量化到 FP8/INT4 等格式，在质量和速度之间找到最优点。
2. **推理框架部署**：选择 vLLM / SGLang / TensorRT-LLM，配置 continuous batching、prefix caching 等。
3. **服务架构**：负载均衡、自动扩缩容、请求路由（简单请求用小模型，复杂请求用大模型）。
4. **持续监控**：延迟、吞吐量、错误率、用户满意度，发现退化时及时回滚或更新。

量化级别决策树

- 精度要求高 + 延迟预算充裕 → **FP8**（近零精度损失）
- 精度要求中 + 高吞吐需求 → **INT4/AWQ**（2–3× 加速）
- 消费级硬件 / 端侧部署 → **GGUF Q4_K_M**（4-bit，质量/速度最佳平衡）
- 极限压缩 / 嵌入式设备 → **Q2_K / 2-bit**（显著质量损失，仅限简单任务）

服务架构参考：API Gateway（认证、限流、计费）→ Load Balancer（请求分发、健康检查）→ Inference Servers（vLLM/SGLang 实例池）→ Model Registry（模型版本管理、灰度发布）。

监控指标：P50/P95/P99 延迟、吞吐量（tokens/second/GPU）、错误率（5xx 响应比例）、成本效率（$/million tokens）。

成本优化策略：Batch size 调优（更大的 batch 提高吞吐但增加延迟）；Prefix caching（对于有共同 system prompt 的请求，缓存 KV cache 可以减少 40–70% 的 prefill 计算）；模型路由（简单查询用 7B 模型，复杂查询用 70B 模型，通过 intent classifier 自动分配）。

A.5 资源需求总览

从零到部署的资源需求估算（70B Dense Model）

阶段	GPU 需求	时间与成本估算
数据准备	CPU 集群为主	1–3 个月，主要是人力和存储成本
预训练	2048× H100	2–3 个月，$10–50M GPU 租赁
后训练	128–512× H100	2–6 周，$100K–1M
推理部署	8–64× H100/请求	持续运营，$0.5–5/M tokens

A.6 术语表

Table A.1: LLM 训练核心术语速查

术语	定义
Attention	Transformer 的核心机制，允许每个 token 根据相关性动态地从其他 token 获取信息
BPE (Byte-Pair Encoding)	主流分词算法，通过迭代合并最频繁的相邻 token 对构建词表
Checkpoint	训练过程中保存的模型状态快照，用于故障恢复和评估
DPO (Direct Preference Optimization)	无需 reward model 的偏好优化方法，直接用偏好数据训练策略模型[2]
FSDP	Fully Sharded Data Parallel，PyTorch 原生的分片数据并行
GQA (Grouped-Query Attention)	多个 query head 共享一组 key/value head，减少 KV cache 大小
GRPO	Group Relative Policy Optimization，DeepSeek 提出的无需 reward model 的 RL 方法[3]
KV Cache	推理时缓存已计算的 Key 和 Value 向量，避免重复计算
LoRA (Low-Rank Adaptation)	参数高效微调方法，只训练低秩分解的小矩阵[4]
MFU (Model FLOPs Utilization)	实际计算量 / 硬件理论峰值，衡量训练效率
MoE (Mixture of Experts)	混合专家架构，每个 token 只激活部分参数，实现参数效率与计算效率的解耦
RLHF	Reinforcement Learning from Human Feedback，用人类偏好训练 reward model 再用 RL 优化策略[5]
RoPE (Rotary Position Embedding)	旋转位置编码，通过旋转矩阵编码相对位置[6]
Scaling Laws	模型性能与参数量/数据量/计算量之间的幂律关系[7]
SFT (Supervised Fine-Tuning)	监督微调，在（指令, 回答）对上继续训练
Tensor Parallelism	将单个层的权重矩阵沿行或列维度切分到多块 GPU
Pipeline Parallelism	将模型的不同层分配到不同 GPU，形成流水线
Quantization（量化）	通过降低数值精度（如 FP16→INT4）来减少模型内存占用和加速推理
RAG (Retrieval-Augmented Generation)	检索增强生成，通过检索外部知识库来增强模型的生成质量，缓解幻觉问题
CoT (Chain-of-Thought)	思维链，通过逐步推理来解决复杂问题的提示策略
MCP (Model Context Protocol)	模型上下文协议，由 Anthropic 开源的标准化工具调用协议，定义 Agent 与外部工具交互的统一格式
VLM (Vision-Language Model)	视觉语言模型，能够同时处理图像和文本的多模态模型
ZeRO	DeepSpeed 的内存优化策略，分片 optimizer states / gradients / parameters[8]

A.7 推荐阅读

以下是每个 LLM 从业者都应该阅读的核心文献：

1. **Attention Is All You Need**[9]——Transformer 架构的原始论文，奠定了整个现代 LLM 的基础。
2. **Scaling Laws for Neural Language Models**[10]——发现了模型性能与资源之间的幂律关系，改变了整个领域的资源分配策略。
3. **Training Compute-Optimal Large Language Models**[11]——Chinchilla 论文，纠正了 Kaplan 的结论，表明数据量应与模型大小同步扩展。
4. **LLaMA: Open and Efficient Foundation Language Models**[12]——证明了开源模型可以达到闭源水平，引爆了开源 LLM 生态。
5. **DeepSeek-V3 Technical Report**[13]——工程智慧的集大成之作，展示了如何用有限资源训练 frontier model。
6. **DeepSeek-R1**[14]——通过 RL 激发推理能力的典范，GRPO 的成功改变了后训练的方法论。
7. **FlashAttention**[15]——IO-aware 的注意力计算，将算法优化与硬件特性深度结合。
8. **Direct Preference Optimization (DPO)**[16]——优雅地简化了 RLHF 流程，证明了简单方法可以替代复杂系统。
9. **LoRA**[17]——参数高效微调的奠基工作，使得在消费级硬件上微调大模型成为可能。
10. **The Bitter Lesson**[18]——Rich Sutton 的经典短文，论证了利用计算规模的通用方法总是最终胜过人工设计的特殊方法。

[9] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[10] J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[11] J. Hoffmann et al., "Training compute-optimal large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[12] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[13] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[14] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[15] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[16] R. Rafailov et al., "Direct preference optimization: Your language model is secretly a reward model," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[17] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[18] R. Sutton, *The bitter lesson*, `http://www.incompleteideas.net/IncIdeas/BitterLesson.html`, 2019.

REFERENCES

[1] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948 (cit. on p. 2).

[2] C. E. Shannon, "Prediction and entropy of printed English," *Bell System Technical Journal*, vol. 30, no. 1, pp. 50–64, 1951 (cit. on p. 2).

[3] G. Delétang et al., "Language modeling is compression," in *International Conference on Learning Representations*, 2024 (cit. on p. 3).

[4] R. Sutton, *The bitter lesson*, `http://www.incompleteideas.net/IncIdeas/BitterLesson.html`, 2019 (cit. on pp. 3, 79, 248).

[5] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022 (cit. on p. 5).

[6] R. Schaeffer et al., "Are emergent abilities of large language models a mirage?" *Advances in Neural Information Processing Systems*, vol. 36, 2023 (cit. on p. 5).

[7] T. Mikolov et al., "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013 (cit. on p. 6).

[8] M. E. Peters et al., "Deep contextualized word representations," in *Proceedings of NAACL-HLT*, 2018 (cit. on p. 6).

[9] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017 (cit. on pp. 6, 12, 17, 29, 248).

[10] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019 (cit. on pp. 7, 36, 40, 74, 75).

[11] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, 2020 (cit. on pp. 7, 53, 59, 75).

[12] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023 (cit. on pp. 7, 75, 96).

[13] J. Hoffmann et al., "Training compute-optimal large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022 (cit. on pp. 7, 76, 215, 248).

[14] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022 (cit. on pp. 7, 126, 130, 134).

[15] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023 (cit. on pp. 8, 58, 59, 67, 69, 77, 218, 239, 248).

[16] A. Dubey et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024 (cit. on pp. 8, 16, 60, 62, 77, 84, 95, 97, 98, 119, 215, 218, 224, 235, 239).

[17] G. Team et al., "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023 (cit. on pp. 8, 229).

[18] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024 (cit. on pp. 8, 11, 61, 77, 87, 94, 96, 100, 103, 104, 106, 170, 214, 215, 219, 225, 248).

[19] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025 (cit. on pp. 8, 11, 46, 147, 179, 184, 188, 216, 219, 227, 239, 248).

[20] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025 (cit. on pp. 9, 11, 46, 54, 193, 219, 224, 235).

[21] J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020 (cit. on pp. 9, 76, 215, 248).

[22] K. Clark et al., "What does BERT look at? an analysis of BERT's attention," in *Proceedings of the ACL Workshop BlackboxNLP*, 2019 (cit. on p. 16).

[23] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019 (cit. on p. 16).

[24] J. Ainslie et al., "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023 (cit. on pp. 16, 169).

[25] O. Press et al., "Train short, test long: Attention with linear biases enables input length generalization," in *International Conference on Learning Representations*, 2022 (cit. on p. 17).

[26] J. Su et al., "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127 063, 2024 (cit. on pp. 18, 113).

[27] S. Chen et al., "Extending context window of large language models via positional interpolation," *arXiv preprint arXiv:2306.15595*, 2023 (cit. on p. 19).

[28] bloc97, *NTK-aware scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and target length training*, Reddit r/LocalLLaMA, 2023 (cit. on p. 19).

[29] D. Hendrycks et al., "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016 (cit. on p. 19).

[30] P. Ramachandran et al., "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017 (cit. on p. 19).

[31] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020 (cit. on pp. 19, 20).

[32] B. Zhang et al., "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, vol. 32, 2019 (cit. on p. 21).

[33] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022 (cit. on pp. 22, 23, 95, 112, 168, 238, 248).

[34] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023 (cit. on pp. 22, 23, 168).

[35] J. Shah et al., "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," *arXiv preprint arXiv:2407.08608*, 2024 (cit. on pp. 23, 168).

[36] A. Gu et al., "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2024 (cit. on p. 30).

[37] B. Peng et al., "RWKV-7 "goose" with expressive dynamic state evolution," *arXiv preprint arXiv:2503.14456*, 2025 (cit. on pp. 30, 31).

[38] T. Dao et al., "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *International Conference on Machine Learning*, 2024 (cit. on p. 30).

[39] O. Lieber et al., "Jamba: A hybrid transformer-mamba language model," *arXiv preprint arXiv:2403.19887*, 2024 (cit. on p. 31).

[40] M. Yang et al., "Zebra-Llama: Towards extremely efficient hybrid models," in *Advances in Neural Information Processing Systems*, vol. 38, 2025 (cit. on p. 31).

[41] W. Merrill et al., "OLMo hybrid: From theory to practice," 2026, Allen Institute for AI technical report (cit. on p. 31).

[42] N. Lambert, "OLMo hybrid and future LLM architectures," *Interconnects AI Blog*, 2026 (cit. on p. 32).

[43] T. Ye et al., "Differential transformer," in *International Conference on Learning Representations*, 2025 (cit. on p. 33).

[44] C. Kong et al., "Understanding differential transformer unchains pretrained self-attentions," in *Advances in Neural Information Processing Systems*, vol. 38, 2025 (cit. on p. 33).

[45] J. Yuan et al., "Native sparse attention: Hardware-aligned and natively trainable sparse attention," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025 (cit. on p. 34).

[46] R. Sennrich et al., "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016 (cit. on p. 38).

[47] T. Kudo et al., "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," *Proceedings of EMNLP: System Demonstrations*, 2018 (cit. on p. 43).

[48] G. Penedo et al., "The FineWeb datasets: Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024 (cit. on pp. 53, 55, 59, 214, 242).

[49] Z. Shao et al., "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024 (cit. on pp. 64, 143, 184, 227).

[50] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022 (cit. on pp. 66, 149, 208, 216, 218).

[51] OpenAI et al., "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023 (cit. on p. 69).

[52] S. Rajbhandari et al., "ZeRO: Memory optimizations toward training trillion parameter models," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC'20)*, 2020 (cit. on p. 81).

[53] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017 (cit. on pp. 99, 103).

[54] W. Fedus et al., "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022 (cit. on pp. 99, 100, 103).

[55] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024 (cit. on pp. 100, 105, 219).

[56] K. Hong et al., "Context rot: How increasing input tokens impacts LLM performance," Chroma, Tech. Rep., 2025 (cit. on pp. 111, 122).

[57] Meta AI, *The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation*, Meta AI Blog, 2025 (cit. on p. 116).

[58] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024 (cit. on p. 122).

[59] P. Rajasekaran et al., *Effective context engineering for AI agents*, Anthropic Engineering Blog, 2025 (cit. on p. 122).

[60] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022 (cit. on pp. 128, 223, 238, 248).

[61] T. Dettmers et al., "QLoRA: Efficient finetuning of quantized language models," *Advances in Neural Information Processing Systems*, vol. 36, 2023 (cit. on pp. 129, 163, 223, 238).

[62] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017 (cit. on p. 133).

[63] R. Rafailov et al., "Direct preference optimization: Your language model is secretly a reward model," in *Advances in Neural Information Processing Systems*, vol. 36, 2023 (cit. on pp. 136, 248).

[64] H. Lightman et al., "Let's verify step by step," *arXiv preprint arXiv:2305.20050*, 2023 (cit. on p. 153).

[65] E. Frantar et al., "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations*, 2023 (cit. on p. 163).

[66] J. Lin et al., "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *MLSys*, 2024 (cit. on p. 163).

[67] G. Xiao et al., "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*, 2023 (cit. on p. 164).

[68] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023 (cit. on p. 169).

[69] S. Feng et al., "EvicPress: Joint KV-cache compression and eviction for efficient LLM serving," *arXiv preprint arXiv:2512.14946*, 2024 (cit. on p. 170).

[70] Y. Wang et al., "CommonKV: Compressing KV cache with cross-layer parameter sharing," *arXiv preprint arXiv:2508.16134*, 2025 (cit. on p. 170).

[71] Y. Leviathan et al., "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*, 2023 (cit. on p. 171).

[72] Y. Li et al., "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *International Conference on Machine Learning*, 2024 (cit. on p. 173).

[73] Y. Li et al., "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," *arXiv preprint arXiv:2503.01840*, 2025 (cit. on p. 173).

[74] T. Cai et al., "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *International Conference on Machine Learning*, 2024 (cit. on p. 173).

[75] M. Elhoushi et al., "LayerSkip: Enabling early exit inference and self-speculative decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024 (cit. on p. 173).

[76] L. Chen et al., "CLaSp: In-context layer skip for self-speculative decoding," *arXiv preprint arXiv:2505.24196*, 2025 (cit. on p. 174).

[77] R. Qin et al., "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *arXiv preprint arXiv:2407.00079*, 2024 (cit. on p. 177).

[78] R. Taori et al., *Stanford Alpaca: An instruction-following LLaMA model*, GitHub, 2023 (cit. on p. 178).

[79] C. Snell et al., "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024 (cit. on pp. 181, 182).

[80] Y. Ichihara et al., "Evaluation of best-of-N sampling strategies for language model alignment," *Transactions on Machine Learning Research*, 2025 (cit. on p. 185).

[81] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations*, 2023 (cit. on p. 185).

[82] S. Park et al., "Ensembling large language models with process reward-guided tree search for better complex reasoning," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the ACL*, 2025 (cit. on p. 186).

[83] P. Lu et al., "OctoTools: An agentic framework with extensible tools for complex reasoning," *arXiv preprint arXiv:2502.11271*, 2025 (cit. on p. 191).

[84] M. Ravaut et al., "A comprehensive survey of contamination detection methods in large language models," *Transactions on Machine Learning Research*, 2024 (cit. on p. 196).

[85] W. Shi et al., "Detecting pretraining data from large language models," in *International Conference on Learning Representations*, 2024 (cit. on p. 196).

[86] S. Golchin et al., "Time travel in LLMs: Tracing data contamination in large language models," in *International Conference on Learning Representations*, 2024 (cit. on p. 196).

[87] X. Deng et al., "SWE-Bench pro: Can AI agents solve long-horizon software engineering tasks?" *arXiv preprint arXiv:2509.16941*, 2025, Scale AI (cit. on p. 197).

[88] D. Hendrycks et al., "Measuring massive multitask language understanding," in *International Conference on Learning Representations*, 2021 (cit. on p. 197).

[89] Y. Wang et al., "MMLU-Pro: A more robust and challenging multi-task language understanding benchmark," in *Advances in Neural Information Processing Systems*, vol. 37, 2024 (cit. on pp. 197, 198).

[90] D. Rein et al., "GPQA: A graduate-level google-proof q&a benchmark," in *Conference on Language Modeling (COLM)*, 2024 (cit. on p. 198).

[91] E. Glazer et al., "FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI," *arXiv preprint arXiv:2411.04872*, 2024 (cit. on p. 199).

[92] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" In *International Conference on Learning Representations*, 2024 (cit. on p. 200).

[93] C. White et al., "LiveBench: A challenging, contamination-limited LLM benchmark," in *International Conference on Learning Representations*, 2025 (cit. on p. 201).

[94] T. Li et al., "Does style matter? Disentangling style and substance in Chatbot Arena," *LMSYS Blog*, 2024 (cit. on p. 202).

[95] H. Lightman et al., "Let's verify step by step," in *International Conference on Learning Representations*, 2024 (cit. on p. 205).

[96] A. Setlur et al., "Rewarding progress: Scaling automated process verifiers for LLM reasoning," in *International Conference on Learning Representations*, 2025 (cit. on p. 206).

[97] Y. Yue et al., "Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?" In *Advances in Neural Information Processing Systems*, vol. 38, 2025 (cit. on p. 206).

[98] L. Ben Allal et al., "SmolLM2: When smol goes big – data-centric training of a small language model," *arXiv preprint arXiv:2502.02737*, 2025 (cit. on p. 224).

[99] T. Gunter et al., "Apple intelligence foundation language models," *arXiv preprint arXiv:2407.21075*, 2024 (cit. on pp. 234, 235).